Информатика снизу вверх

Иэн Виенанд

Русский перевод учебных материалов

Оригинал: *Computer Science from the Bottom Up*, издание 2013 года.

© Ian Wienand. CC BY-SA 3.0. Перевод, примечания и сокращение неучебных материалов.

Содержание

1. 1. Unix и продвинутый C
2. 2. Двоичная система и представление чисел
3. 3. Архитектура компьютера
4. 4. Операционная система
5. 5. Процесс
6. 6. Виртуальная память
7. 7. Инструменты разработки
8. 8. Что происходит за процессом
9. 9. Динамическое связывание
10. 10. Основы ввода-вывода
11. Глоссарий

Глава 1. Общие сведения о Unix и продвинутый C

Всё есть файл!

Часто цитируемый принцип UNIX-подобных систем, таких как Linux и BSD, гласит: всё есть файл.

Представьте файл в привычном контексте, например в текстовом редакторе. С таким файлом можно выполнить две основные операции:

1. Прочитать его: получить ранее сохранённые данные редактора.
2. Записать в него новые данные, введённые пользователем.

Рассмотрим некоторые распространённые устройства, подключаемые к компьютеру, и их связь с этими операциями:

1. Экран.
2. Клавиатура.
3. Принтер.
4. CD-ROM.

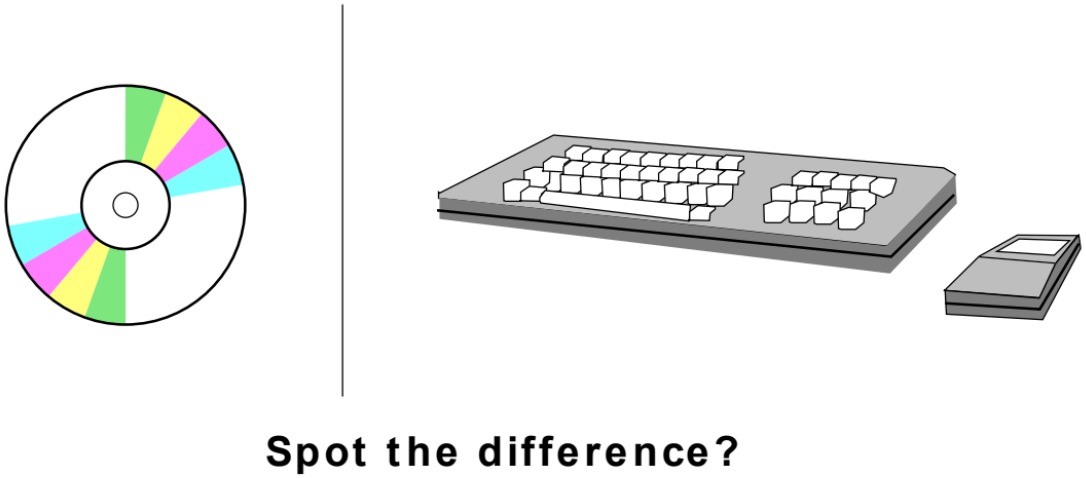
Экран и принтер похожи на файлы, доступные только для записи: вместо хранения в виде битов на диске информация отображается точками на экране или строками на бумаге. Клавиатура похожа на файл только для чтения, а данные поступают из нажатий клавиш пользователем. CD-ROM устроен похожим образом, но данные не поступают от пользователя в произвольный момент, а хранятся непосредственно на диске.

Таким образом, понятие файла хорошо абстрагирует как приёмник, так и источник данных. Поэтому оно отлично подходит для представления любых устройств, которые можно подключить к компьютеру. Это осознание — источник огромной силы UNIX, проявляющийся во всей архитектуре платформы. Одна из основных задач операционной системы — предоставлять программисту такую абстракцию оборудования.

Вероятно, не будет большим преувеличением назвать абстракцию главным понятием, лежащим в основе всех современных вычислений. Один человек не способен понять всё — от проектирования современного пользовательского интерфейса до внутренней работы современного процессора, не говоря уже о том, чтобы создать всё это самостоятельно. Для программистов абстракции служат общим языком, позволяющим сотрудничать и изобретать новое.

Научившись переходить между уровнями абстракции, можно лучше понять, как использовать их наиболее эффективно и находчиво. В этой книге нас интересуют абстракции самых нижних уровней: между приложениями и операционной системой, а также между операционной системой и оборудованием. Выше лежит множество других уровней, каждый

из которых достоин отдельной книги. Надеюсь, последующие главы помогут вам понять абстракции, предоставляемые современной операционной системой.



Реализация абстракции

Как правило, абстракция реализуется через то, что принято называть интерфейсом прикладного программирования, или API. Это довольно расплывчатый термин, который означает разные вещи в разных областях программирования. По сути, программист проектирует набор функций и документирует их интерфейс и назначение, исходя из того, что фактическая реализация API скрыта от пользователя.

Например, многие крупные веб-приложения предоставляют API, доступный по HTTP. Доступ к данным таким способом наверняка запускает сложные цепочки удалённых вызовов процедур, запросов к базам данных и передачи данных. Всё это скрыто от конечного пользователя, который просто получает оговорённые данные.

Тем, кто знаком с объектно-ориентированными языками, такими как Java, Python или C++, известна абстракция, предоставляемая классами. Методы служат интерфейсом класса, скрывая его реализацию.

Реализация абстракции на C

В ядре Linux и других крупных кодовых базах на C, в котором нет встроенного понятия объектной ориентации, распространённый приём — указатели на функции. Умение читать такой код необходимо для работы с большинством крупных проектов на C. Понимание содержащихся в коде абстракций помогает разобраться в устройстве внутренних API.

Пример 1.1. Абстракция с помощью указателей на функции

```
#include <stdio.h>
/* The API to implement */
struct greet_api
{
 int (*say_hello)(char *name);
 int (*say_goodbye)(void);
```

```
};
/* Our implementation of the hello function */
int say_hello_fn(char *name)
{
 printf("Hello %s\n", name);
 return 0;
}
/* Our implementation of the goodbye function */
int say_goodbye_fn(void)
{
 printf("Goodbye\n");
 return 0;
}
/* A struct implementing the API */
struct greet_api greet_api =
{
 .say_hello = say_hello_fn,
 .say_goodbye = say_goodbye_fn
};
/* main() doesn't need to know anything about how the
 * say_hello/goodbye works, it just knows that it does */
int main(int argc, char *argv[])
{
 greet_api.say_hello(argv[1]);
 greet_api.say_goodbye();
 printf("%p, %p, %p\n", greet_api.say_hello, say_hello_fn, &say_hello_fn);
 exit(0);
}
```

Это простейший пример конструкций, которые постоянно встречаются в ядре Linux и других программах на C. Рассмотрим отдельные элементы.

Сначала идёт структура, определяющая API (`struct greet_api`). Имена функций в скобках со знаком указателя обозначают указатели на функции.[^1] Указатель задаёт прототип функции, на которую он должен указывать. Если присвоить ему функцию с неподходящим возвращаемым типом или параметрами, компилятор как минимум выдаст предупреждение; если оставить такой код, вероятны ошибки работы или аварийное завершение.

Затем следует наша реализация API. В более сложных случаях функции реализации API часто служат лишь обёртками над другими функциями, имена которых по соглашению начинаются с одного или двух символов подчёркивания[^2] (например, `say_hello_fn()` вызывает `_say_hello_function()`). У этого приёма несколько назначений. Обычно он позволяет отделить небольшие и простые части API, например упаковку или проверку аргументов, от более сложной реализации. Это часто облегчает значительные изменения внутреннего устройства при сохранении неизменного API. В нашем случае реализация очень проста и даже не требует вспомогательных функций. В разных проектах одинарный, двойной или даже тройной префикс подчёркивания может означать разное, но в целом он визуально предупреждает, что функцию не следует вызывать напрямую «из-за пределов» API.

[^1]: Часто имена параметров опускают, указывая только их типы. Это позволяет реализующей стороне выбрать собственные имена и избежать предупреждений компилятора. [^2]: Функцию с двойным подчёркиванием, например `__foo`, в разговорной речи могут называть «dunder foo».

На предпоследнем шаге мы заполняем указатели на функции в `struct greet_api greet_api`. Имя функции является указателем, поэтому нет необходимости отдельно брать её адрес,

например `&say_hello_fn`. Наконец, в `main` мы вызываем функции API через структуру.

Этот приём постоянно встречается при изучении исходного кода. Для иллюстрации приведём небольшой пример из `include/linux/virtio.h` ядра Linux.

Пример 1.2. Абстракция в include/linux/virtio.h

```
/**
 * virtio_driver - operations for a virtio I/O driver
 * @driver: underlying device driver (populate name and owner).
 * @id_table: the ids serviced by this driver.
 * @feature_table: an array of feature numbers supported by this driver.
 * @feature_table_size: number of entries in the feature table array.
 * @probe: the function to call when a device is found.  Returns 0 or -errno.
 * @remove: the function to call when a device is removed.
 * @config_changed: optional function to call when the device configuration
 *    changes; may be called in interrupt context.
 */
struct virtio_driver {
        struct device_driver driver;
        const struct virtio_device_id *id_table;
        const unsigned int *feature_table;
        unsigned int feature_table_size;
        int (*probe)(struct virtio_device *dev);
        void (*scan)(struct virtio_device *dev);
        void (*remove)(struct virtio_device *dev);
        void (*config_changed)(struct virtio_device *dev);
#ifdef CONFIG_PM
        int (*freeze)(struct virtio_device *dev);
        int (*restore)(struct virtio_device *dev);
#endif
};
```

Достаточно в общих чертах понимать, что эта структура описывает виртуальное устройство ввода-вывода. От пользователя API, автора драйвера устройства, ожидается предоставление ряда функций, которые будут вызываться при различных событиях работы системы: обнаружении нового оборудования, его удалении и так далее. Структура также содержит данные, которые необходимо заполнить подходящими значениями. Изучение подобных описаний обычно помогает начать разбираться в различных слоях кода ядра.

Библиотеки

Библиотеки выполняют две роли, иллюстрирующие абстракцию:

- Позволяют программистам повторно использовать часто нужный код.
- Работают как чёрный ящик, предоставляющий программисту определённую функциональность.

Например, библиотека для доступа к исходным данным файлов JPEG полезна тем, что множество программ, которым нужны изображения, могут использовать одну и ту же библиотеку. При этом их разработчикам не требуется беспокоиться о деталях формата JPEG: они могут сосредоточиться на том, что программа должна делать с изображением.

Стандартную библиотеку платформы UNIX обычно называют libc. Она предоставляет основной интерфейс к системе: базовые вызовы вроде `read()`, `write()` и `printf()`. Этот API

целиком описан спецификацией POSIX. Она свободно доступна в интернете и описывает многочисленные вызовы стандартного API UNIX.

Большинство платформ UNIX в основном следуют стандарту POSIX, но часто имеют небольшие, а иногда и важные отличия. Отсюда сложность различных инструментов GNU autotools, которые пытаются скрыть эти различия. В Linux есть множество интерфейсов, не определённых POSIX; приложения, зависящие от них, будут менее переносимыми.

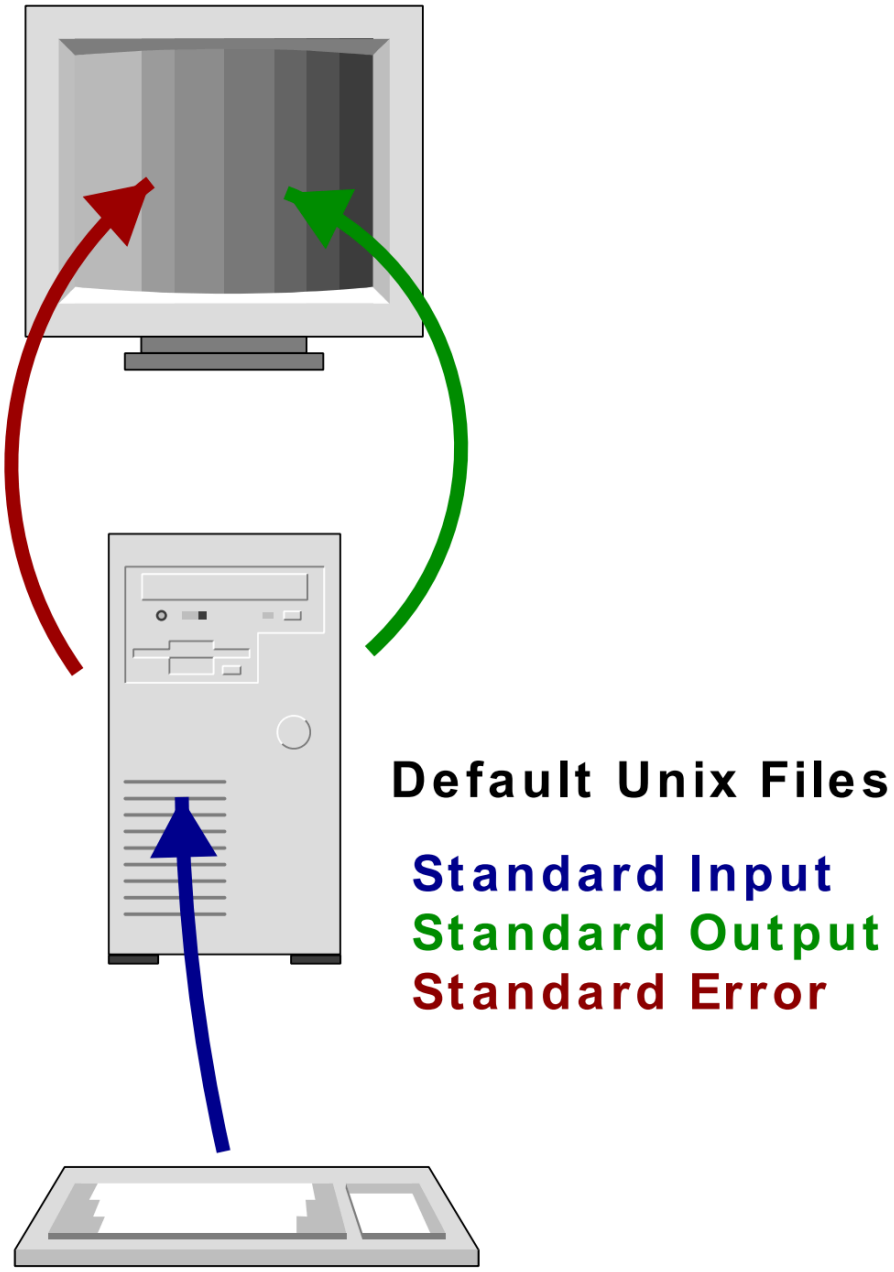
Библиотеки — фундаментальная абстракция со множеством деталей. В последующих главах их устройство будет рассмотрено гораздо подробнее.

Файловые дескрипторы

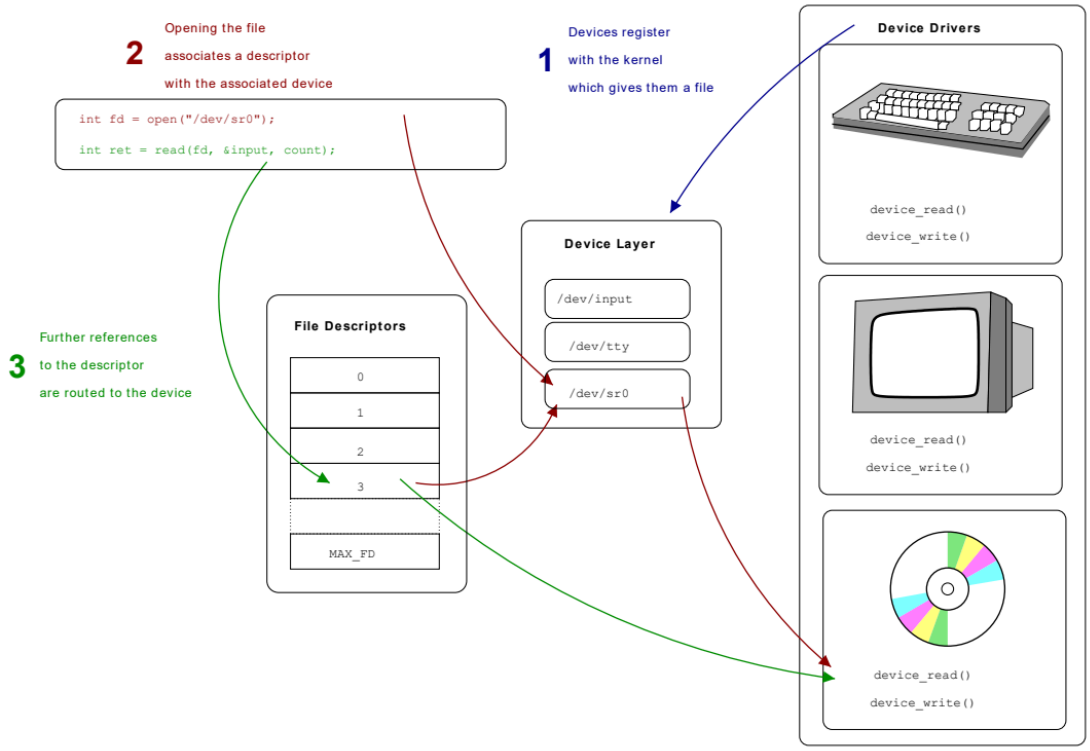
Одно из первых знаний программиста UNIX: каждая запускаемая программа начинает работу с тремя уже открытыми файлами.

Таблица 1.1. Стандартные файлы, предоставляемые Unix

Название	Номер файла	Описание
Стандартный ввод	0	Ввод с клавиатуры
Стандартный вывод	1	Вывод на консоль
Стандартный вывод ошибок	2	Вывод ошибок на консоль



Возникает вопрос: что представляет собой открытый файл? Значение, возвращаемое вызовом `open`, называется файловым дескриптором и по сути является индексом в массиве открытых файлов, который поддерживает ядро.



Файловые дескрипторы служат индексами в таблице, хранящейся в ядре. В ответ на вызов `open` ядро создаёт дескриптор и связывает его с некоторой абстракцией лежащего ниже объекта, похожего на файл: физического устройства, файловой системы или чего-то совершенно иного. После этого вызовы `read` и `write` процесса, использующие дескриптор, направляются ядром в нужное место и выполняют полезную работу.

Иными словами, файловый дескриптор — это вход в абстракции оборудования, предоставляемые ядром. Общая схема абстракции физических устройств показана на рисунке 1.3.

На самом нижнем уровне для связи с аппаратным устройством операционной системе нужен драйвер, написанный программистом. Драйвер создаётся в соответствии с API ядра, как в примере 1.2, и предоставляет набор функций, вызываемых ядром при различных запросах. В упрощённом примере выше драйверы предоставляют функции чтения и записи, вызываемые при соответствующих операциях над файловым дескриптором. Драйвер знает, как преобразовать эти общие запросы в команды конкретному устройству.

Чтобы предоставить абстракцию пользовательскому пространству, ядро предоставляет файловый интерфейс через так называемый слой устройств. Физические устройства представлены файлами в специальной файловой системе, например `/dev`. В UNIX-подобных системах узлы устройств имеют старший и младший номера, позволяющие ядру связать конкретный узел с его драйвером. Их можно увидеть командой `ls`, как в примере 1.3.

Пример 1.3. Старший и младший номера

```
$ ls -l /dev/null /dev/zero /dev/tty
crw-rw-rw- 1 root root 1, 3 Aug 26 13:12 /dev/null
```

```
crw-rw-rw- 1 root root 5, 0 Sep  2 15:06 /dev/tty
crw-rw-rw- 1 root root 1, 5 Aug 26 13:12 /dev/zero
```

Итак, файловый дескриптор — идентификатор, которым пользовательское пространство пользуется для связи с устройством. В общем случае при открытии файла ядро использует путь, чтобы связать дескриптор с объектом, предоставляющим подходящий API чтения, записи и других операций. При открытии устройства (выше это `/dev/sr0`) старший и младший номера его узла дают ядру информацию для поиска нужного драйвера и завершения связи. После этого ядро знает, как направлять последующие вызовы, например `read`, в функции драйвера.

Обычный файл работает похоже, хотя между ним и ядром больше промежуточных уровней. Здесь абстракцией служит точка монтирования. Монтирование файловой системы одновременно устанавливает связь, благодаря которой файловая система знает, какое устройство предоставляет хранилище, а ядро знает, что файлы под данной точкой монтирования нужно направлять драйверу файловой системы. Как и драйверы устройств, файловые системы пишутся в соответствии с общим API ядра.

На практике существует ещё много усложняющих картину уровней. Например, ядро старается кэшировать как можно больше данных диска в свободной памяти, что значительно повышает скорость. Оно также пытается организовать доступ к устройствам наиболее эффективно: например, упорядочить обращения к диску так, чтобы данные, физически расположенные рядом, извлекались вместе, даже если запросы поступили в ином порядке. Кроме того, многие устройства относятся к общим классам, например USB или SCSI, предоставляющим собственные уровни абстракции. Поэтому файловые системы обращаются к устройствам через множество таких уровней, а не напрямую. Понять ядро — значит понять, как эти многочисленные API взаимодействуют и сосуществуют.

Командная оболочка

Оболочка — средство взаимодействия с операционной системой. Будь то `bash`, `zsh`, `csh` или одна из многих других оболочек, у всех них есть одна основная задача: позволить вам запускать программы. Как именно оболочка это делает, станет понятнее, когда мы позже рассмотрим внутреннее устройство операционной системы.

Но оболочки умеют гораздо больше, чем просто запускать программы. Они обладают мощными средствами перенаправления файлов, позволяют запускать несколько программ одновременно и создавать полноценные сценарии. Всё это восходит к принципу «всё есть файл».

Перенаправление

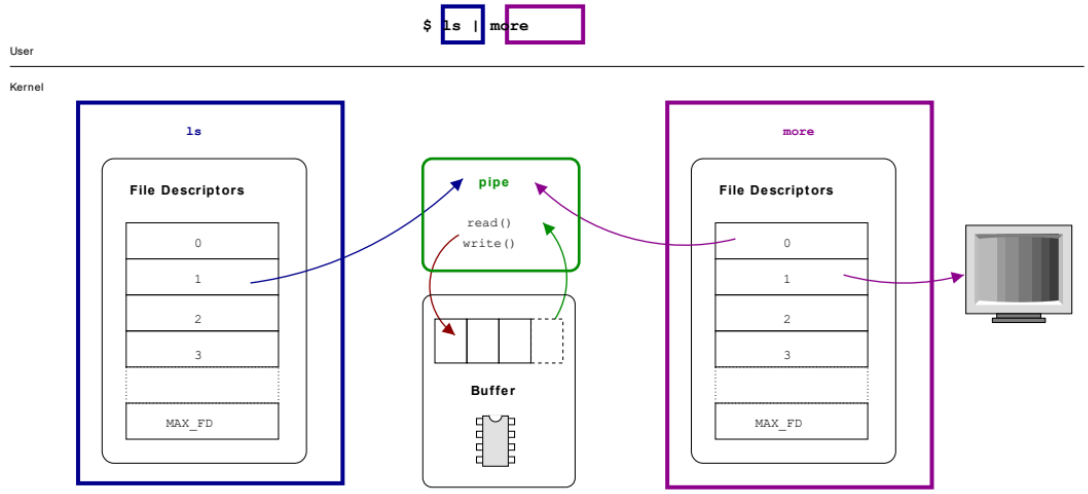
Часто нам нужно, чтобы стандартные файловые дескрипторы указывали не на свои обычные места назначения. Например, вы можете захотеть сохранить весь вывод программы в файл на диске или, наоборот, читать команды из заранее подготовленного файла. Другая полезная задача — передать вывод одной программы на вход другой. Оболочка совместно с операционной системой обеспечивает всё это и многое другое.

Таблица 1.2. Стандартные средства перенаправления оболочки

Название	Команда	Описание	Пример
Перенаправление в файл	`> filename`	Записывает весь стандартный вывод в `filename`. Оператор `>>` дописывает данные, а не перезаписывает файл.	`ls > filename`
Чтение из файла	`< filename`	Передаёт все данные файла на стандартный ввод программы.	`echo < filename`
Конвейер	`program1 | program2`	Передаёт весь стандартный вывод `program1` на стандартный ввод `program2`.	`ls | more`

Как устроен конвейер

Реализация `ls | more` — ещё один пример силы абстракции. Вместо того чтобы связывать дескриптор стандартного вывода с устройством, например консолью для вывода в терминал, его направляют в буфер в памяти, предоставляемый ядром и обычно называемый каналом (*pipe*). Другой процесс может связать свой стандартный ввод с противоположной стороной того же буфера и получать вывод первого процесса. Это показано на рисунке 1.4.



Канал — буфер в памяти, соединяющий два процесса. Файловые дескрипторы указывают на объект канала, который буферизует отправленные через `write` данные до их извлечения через `read`.

Ядро хранит записанные в канал данные, пока чтение с другой стороны не извлечёт их из буфера. Это мощная концепция и одна из основных форм межпроцессного взаимодействия, или IPC, в UNIX-подобных системах. Канал позволяет не только передавать данные, но и использовать их как сигнал. Если процесс читает пустой канал, он по умолчанию блокируется, то есть приостанавливается до появления данных. Подробнее это обсуждается в главе 5 «Процесс». Таким образом, два процесса могут сообщать о выполнении действия, записав всего один байт: важно не само значение, а наличие данных в канале.

Допустим, один процесс просит другой напечатать файл, что займёт некоторое время. Процессы могут создать между собой канал: запрашивающий процесс выполняет чтение

пустого канала и блокируется. Когда печать завершится, второй процесс записывает сообщение в канал, тем самым пробуждая первый и сигнализируя об окончании работы.

Возможность передавать данные между процессами порождает ещё один распространённый принцип UNIX: небольшие инструменты, каждый из которых выполняет одну конкретную задачу. Объединение таких инструментов в цепочки даёт гибкость, которой часто не хватает одной монолитной программе.

Глава 2. Двоичная система и представление чисел

Двоичная система — основа вычислений

Теория двоичной системы

Введение

Двоичная система счисления строит числа из элементов, называемых битами. Каждый бит может быть представлен любыми двумя взаимоисключающими состояниями. Обычно при записи и кодировании битов мы обозначаем их цифрами 1 и 0. Мы также называем их истинным и ложным значениями, а компьютер внутри представляет биты высоким и низким напряжением.

Мы строим двоичные числа так же, как числа в привычной десятичной системе. Но вместо разрядов единиц, десятков, сотен и так далее у нас есть разряды единиц, двоек, четвёрок, восьмёрок и так далее, как показано ниже.

Таблица 2.1. Двоичная система

…	2^6	2^5	2^4	2^3	2^2	2^1	2^0
…	64	32	16	8	4	2	1

Например, чтобы представить число 203 в десятичной системе, мы помещаем 3 в разряд единиц, 0 — в разряд десятков и 2 — в разряд сотен. В следующей таблице это выражено с помощью степеней.

Таблица 2.2. Число 203 в десятичной системе

10^2	10^1	10^0
2	0	3

Иными словами, $2 \times 10^2 + 3 \times 10^0 = 200 + 3 = 203$. Для представления того же числа в двоичной системе получим следующую таблицу.

Таблица 2.3. Число 203 в двоичной системе

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
1	1	0	0	1	0	1	1

Это равно $2^7 + 2^6 + 2^3 + 2^1 + 2^0 = 128 + 64 + 8 + 2 + 1 = 203$.

Перевод между системами

Проще всего переводить числа между системами счисления с помощью компьютера — в конце концов, он хорошо с этим справляется! Однако часто полезно уметь делать это вручную.

Простой метод — последовательное деление. Повторно делите частное на основание новой системы, пока частное не станет равно нулю, записывая остаток на каждом шаге. Затем запишите остатки в обратном порядке, начиная снизу и добавляя каждый следующий справа. Проиллюстрируем это примером: поскольку мы переводим число в двоичную систему, основание равно 2.

Таблица 2.4. Перевод числа 203 в двоичную систему

Частное	Остаток
203 ÷ 2 = 101	1
101 ÷ 2 = 50	1 ↑
50 ÷ 2 = 25	0 ↑
25 ÷ 2 = 12	1 ↑
12 ÷ 2 = 6	0 ↑
6 ÷ 2 = 3	0 ↑
3 ÷ 2 = 1	1 ↑
1 ÷ 2 = 0	1 ↑

Читая остатки снизу вверх и каждый раз добавляя очередной справа, получаем 11001011 — как мы уже видели, это двоичное представление числа 203.

Биты и байты

Чтобы представить все буквы алфавита, нам нужно достаточно комбинаций для строчных и прописных букв, цифр и знаков препинания, а также ещё нескольких символов. Всего потребуется, вероятно, около 80 различных комбинаций.

Два бита позволяют представить четыре уникальные комбинации: `00`, `01`, `10`, `11`. Три бита дают восемь комбинаций. Как мы видели выше, n бит позволяют представить 2^n уникальных комбинаций.

Восемь бит дают $2^8 = 256$ уникальных значений — более чем достаточно для перечисленных символов. Группу из восьми бит называют байтом. Угадайте, сколько занимает переменная `char` в C? Один байт.

ASCII

Поскольку байт может представлять 256 значений, кто угодно мог бы произвольно сопоставить символы числам. Например, производитель видеокарт мог бы решить, что

числу 10 соответствует буква A, и тогда при передаче значения 10 видеокарте на экране появлялась бы прописная «A».

Чтобы этого избежать, был создан американский стандартный код обмена информацией — ASCII. Это семибитный код, то есть он содержит $2^7 = 128$ возможных кодовых значений.

Коды делятся на две основные группы: печатаемые и непечатаемые. К печатаемым относятся прописные и строчные буквы, цифры и знаки препинания. Непечатаемые коды предназначены для управления: например, они вызывают возврат каретки или звонок терминала; специальный код NULL обозначает отсутствие символа.

Такого набора достаточно для американского английского, но он становится очень ограниченным, когда нужно представить символы других языков, особенно азиатских, где могут быть многие тысячи различных знаков.

Чтобы облегчить эту проблему, современные системы переходят от ASCII к Unicode. Для представления символа может использоваться до четырёх байтов, что даёт гораздо больше места.

> **Примечание переводчика.** Один байт из восьми бит содержит значения от 0 до 255 включительно; в исходном тексте ошибочно указан диапазон до 256. Unicode — стандарт кодовых точек, а длина в байтах зависит от кодировки; до четырёх байтов на кодовую точку использует, например, UTF-8.

Контроль чётности

Поскольку ASCII использует только семь бит, один бит байта остаётся свободным. Его можно использовать для контроля чётности — простого способа обнаружения ошибок. Представим компьютер, получающий данные с перфокарт: отверстие обозначает 1, а его отсутствие — 0. Если отверстие случайно закроется, будет считано неверное значение, что приведёт к непредсказуемому поведению.

Контроль чётности позволяет просто проверить, правильно ли считаны биты байта. Дополнительный бит служит битом чётности; можно использовать контроль как на нечётность, так и на чётность.

При контроле на нечётность общее число единиц, включая контрольный бит, должно быть нечётным; при контроле на чётность — чётным. Изменение одного бита нарушит это условие и позволит обнаружить ошибку.

> **Примечание переводчика.** В оригинале правила установки бита чётности перепутаны; выше приведено исправленное описание. Контроль чётности не обнаруживает все возможные ошибки, например изменение двух битов.

16-, 32- и 64-битные компьютеры

Числа не всегда помещаются в байт: хочется надеяться, что сумма на вашем банковском счёте требует большего диапазона! Большинство современных автору архитектур — 32-

битные компьютеры. Это означает, что при обработке данных и чтении или записи памяти они работают с четырьмя байтами за раз. Четыре байта называют словом; по аналогии с естественным языком буквы — биты — составляют слова, но в вычислительной технике все слова имеют одинаковый размер. Переменная C типа `int` занимает 32 бита. В более новых 64-битных архитектурах размер обрабатываемых процессором данных удваивается и составляет восемь байтов.

> **Примечание переводчика.** Размер машинного слова и `int` зависит от архитектуры и реализации; стандарт C не требует, чтобы `int` всегда занимал 32 бита. Утверждение автора о распространённости 32-битных архитектур отражает время написания книги.

Килобайты, мегабайты и гигабайты

Компьютеры работают с большим количеством байтов — именно поэтому они так мощны!

Нам нужен способ говорить о больших объёмах данных. Естественно воспользоваться приставками Международной системы единиц (СИ), принятыми во многих научных областях. Например, «кило» означает 10^3, или 1000 единиц: в килограмме 1000 граммов.

Число 1000 выглядит круглым в десятичной системе, но в двоичной оно записывается как `1111101000` и не выглядит особенно «круглым». Зато 1024, или 2^{10}, записывается как `10000000000` и довольно близко к десятичному значению приставки «кило».

Поэтому 1024 байта стали называть килобайтом. Первый массовый компьютер Commodore 64 получил своё имя, поскольку имел 64 килобайта памяти.

Сегодня объём в килобайтах мал даже для наручных часов, не говоря уже о персональном компьютере. Следующая приставка СИ — «мега», означающая 10^6. Как оказалось, 2^{20} также близко к десятичному значению: 1 048 576 вместо 1 000 000.

Единицы продолжают увеличиваться; с каждым шагом двоичное значение всё сильнее расходится с десятичным значением приставки СИ.

Таблица 2.5. Байты

Количество байтов	Название в историческом употреблении
2^{10}	килобайт
2^{20}	мегабайт
2^{30}	гигабайт
2^{40}	терабайт
2^{50}	петабайт
2^{60}	эксабайт

Поэтому 32-битный компьютер может адресовать до четырёх гигабайтов памяти: два дополнительных бита позволяют представить четыре группы по 2^{30} байтов. Автор указывает, что 64-битный компьютер может адресовать до восьми эксабайтов; возможно, вам будет

интересно представить себе величину этого числа. Чтобы почувствовать масштаб, вычислите, сколько времени потребуется, чтобы досчитать до 2^{64}, увеличивая счётчик раз в секунду.

> **Примечание переводчика.** Адресное пространство из 2^{64} байтов составляет 16 эксбибайтов; фактически доступный объём определяется архитектурой и операционной системой.

Килобиты, мегабиты и гигабиты

Помимо путаницы из-за двойного употребления приставок СИ для двоичных и десятичных величин, объём или скорость часто указывают в битах, а не в байтах.

Обычно это относится к сетям и устройствам хранения. Возможно, вы замечали, что скорость ADSL-соединения указывается, например, как 1500 килобит в секунду. Расчёт прост: умножьте на 1000 из-за приставки «кило», разделите на 8, чтобы получить байты, а затем на 1024, чтобы получить килобайты. Итого 1500 килобит/с ≈ 183 килобайта/с.

Организации, отвечающие за стандартизацию, признали двойное употребление и определили отдельные двоичные приставки. По стандарту 1024 байта — это кибибайт, сокращённо КиБ. Аналогично образованы и другие названия, например мебибайт. По традиции эти термины используются не всегда, но их можно встретить в литературе.

Булевы операции

Джордж Буль был математиком, открывшим целую область математики — булеву алгебру. Хотя его открытия относятся к середине XIX века, эта математика лежит в основе информатики. Булева алгебра — обширная тема; здесь мы приведём лишь минимум для начала работы.

Булевы операции принимают входные значения и по определённому правилу дают результат. Например, простейшая операция NOT просто инвертирует входной операнд. Другие операции обычно принимают два входа и возвращают один результат.

Основные операции, применяемые в информатике, перечислены ниже. Они представлены таблицами истинности, показывающими все возможные входы и выходы. Истина соответствует двоичной единице.

НЕ (NOT)

Обычно NOT обозначают `!`. Операция инвертирует значение: 0 превращается в 1, а 1 — в 0.

Таблица 2.6. Таблица истинности NOT

Вход	Выход
1	0
0	1

И (AND)

Запомнить AND можно так: результат истинен, если истинен и один вход, и другой.

Таблица 2.7. Таблица истинности AND

Вход 1	Вход 2	Выход
0	0	0
1	0	0
0	1	0
1	1	1

ИЛИ (OR)

Операция OR даёт истину, если истинен один вход или другой.

Таблица 2.8. Таблица истинности OR

Вход 1	Вход 2	Выход
0	0	0
1	0	1
0	1	1
1	1	1

Исключающее ИЛИ (XOR)

Исключающее ИЛИ, или XOR, — особый случай OR: результат истинен, если истинен один и только один вход. С помощью этой операции можно выполнять удивительно много интересных приёмов, хотя в ядре она встречается не очень часто.

Таблица 2.9. Таблица истинности XOR

Вход 1	Вход 2	Выход
0	0	0
1	0	1
0	1	1
1	1	0

Как компьютеры используют булевы операции

Хотите верьте, хотите нет, но практически всё, что делает компьютер, в конечном счёте сводится к этим операциям. Например, полусумматор — схема из булевых элементов, которая складывает биты. Она называется полусумматором, поскольку не обрабатывает входной бит переноса. Объединив несколько таких схем, можно построить устройство для сложения длинных двоичных чисел. Добавьте внешнюю память — и получите компьютер.

На электронном уровне булевы операции реализуются логическими элементами из транзисторов. Поэтому вы, возможно, слышали о количестве транзисторов и законе Мура. Чем больше транзисторов, тем больше логических элементов и тем больше операций сложения можно выполнить. Современному компьютеру требуется огромное количество и тех и других. Некоторые из новейших на момент написания книги процессоров Itanium содержали около 460 миллионов транзисторов.

Работа с двоичными данными в C

В C есть прямой интерфейс к перечисленным операциям. В таблице приведены операторы.

Таблица 2.10. Булевы операции в C

Операция	Запись в C
НЕ	`!`
И	`&`
ИЛИ	`|`
Исключающее ИЛИ	`^`

Мы используем эти операции с переменными, чтобы изменять содержащиеся в них биты. Но прежде чем перейти к примерам, необходимо отвлечься и описать шестнадцатеричную систему.

> **Примечание переводчика.** В приведённой автором таблице `!` — логическое отрицание всего значения. Для побитового отрицания в C применяется `~`.

Шестнадцатеричная система

Шестнадцатеричная система — это система счисления с основанием 16. В информатике она используется потому, что человеку в ней проще работать с двоичными числами. Компьютер оперирует двоичными данными, а шестнадцатеричная запись — удобное сокращение для нас.

Почему именно основание 16? Естественный выбор — основание 10, к которому мы привыкли в повседневной жизни. Но десятичная система плохо сочетается с двоичной: для десяти разных значений нужны четыре бита, а четыре бита дают шестнадцать комбинаций. Поэтому вместо трудного перевода между десятичной и двоичной системами можно использовать основание 16.

В шестнадцатеричной записи применяются обычные десятичные цифры и буквы A, B, C, D, E, F для значений 10, 11, 12, 13, 14 и 15 соответственно. Обратите внимание, что отсчёт начинается с нуля. Обычно префикс `0x` указывает, что число записано в шестнадцатеричной системе.

Для шестнадцати разных комбинаций требуется ровно четыре бита, поэтому каждая шестнадцатеричная цифра соответствует четырём битам. Полезно выучить следующую таблицу наизусть.

Таблица 2.11. Шестнадцатеричные, двоичные и десятичные значения

Шестнадцатеричное	Двоичное	Десятичное
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
A	1010	10
B	1011	11
C	1100	12
D	1101	13
E	1110	14
F	1111	15

Конечно, ничто не мешает продолжить ряд, например назначить G значение 16, но шестнадцать значений — отличный компромисс между возможностями человеческой памяти и количеством битов компьютера. Иногда также используется восьмеричная система, например для прав доступа к файлам в UNIX. Для представления большего числа битов мы просто используем больше цифр. Например, шестнадцатибитную переменную можно записать как `0xAB12`. Чтобы получить двоичную запись, переведите каждую цифру по таблице и соедините результаты: получится `1010101100010010`. Для обратного перевода действуйте наоборот.

Чтобы перевести число в шестнадцатеричную систему, можно воспользоваться тем же последовательным делением. Например, для числа 203:

Таблица 2.12. Перевод 203 в шестнадцатеричную систему

Частное	Остаток
203 ÷ 16 = 12	11 (`0xB`)
12 ÷ 16 = 0	12 (`0xC`) ↑

Следовательно, 203 в шестнадцатеричной системе — `0xCB`.

Практическое применение

Двоичные данные в коде

Хотя двоичная система лежит в основе работы любого компьютера, вполне можно программировать на высокоуровневом языке, совсем её не зная. Однако в интересующем нас низкоуровневом коде несколько основных принципов используются постоянно.

Маски и флаги

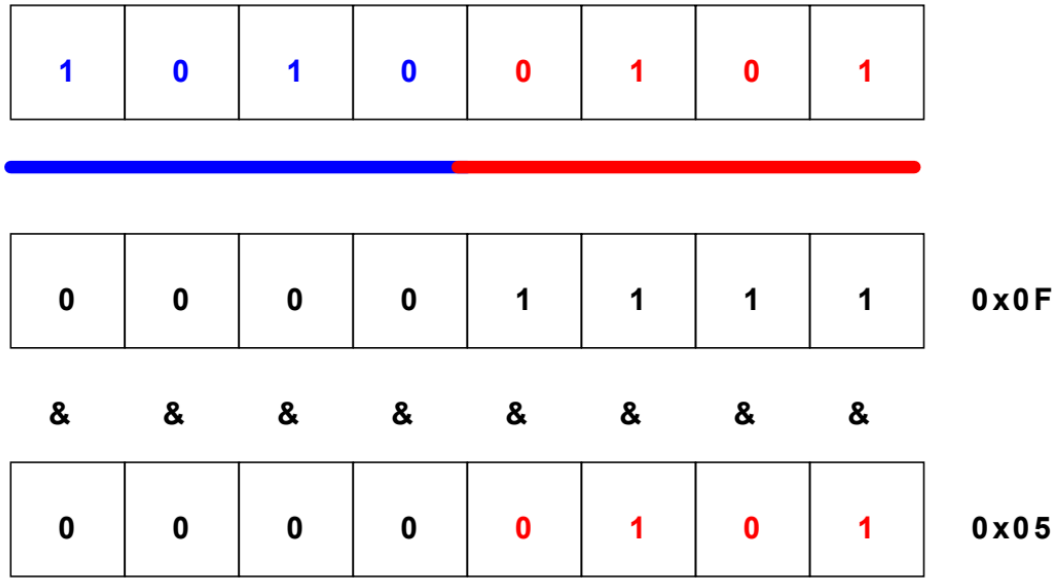
Маскирование

В низкоуровневом коде часто важно, чтобы структуры и переменные занимали как можно меньше места. Иногда для этого в одну переменную упаковывают две, обычно связанные между собой, величины.

Помните: каждый бит представляет два состояния. Поэтому если переменная имеет, скажем, только 16 возможных состояний, её можно представить четырьмя битами: $2^4 = 16$. Но наименьший тип, объявляемый в C, занимает восемь бит (`char`), поэтому можно либо потратить четыре бита впустую, либо найти способ использовать их.

Это легко сделать с помощью маскирования. Если вспомнить правила логических операций, станет понятно, как извлечь нужные значения.

Процесс показан на рисунке. Нас интересуют младшие четыре бита, поэтому соответствующие биты маски установлены в 1. Поскольку операция AND устанавливает бит результата только тогда, когда оба входных бита равны 1, нули маски скрывают ненужные биты.



Чтобы получить старшие четыре бита, нужно инвертировать маску. Обратите внимание: получится `0x90`, хотя нам нужно `0x09`. Для переноса битов в нужную позицию используется сдвиг вправо.

Для установки битов нужна операция OR. При этом вместо единиц в маске используются нули. Нарисуйте схему, подобную приведённой выше, и самостоятельно проследите установку битов операцией OR.

> **Примечание переводчика.** При установке отдельных битов операцией OR в позициях устанавливаемых битов маска содержит единицы; нули оставляют остальные позиции без изменения.

Флаги

Часто программа содержит множество переменных, которые служат лишь флагами определённых условий. Например, конечный автомат — алгоритм, переходящий между разными состояниями, но находящийся одновременно только в одном из них. Допустим, у него восемь состояний. Можно объявить восемь отдельных переменных, но во многих случаях лучше объявить одну восьмибитную переменную и назначить каждый бит флагом состояния.

Флаги — частный случай маскирования: каждый бит представляет отдельное булево состояние, включённое или выключенное. Переменная из n бит может содержать n различных флагов. Ниже приведён типичный пример; его разновидности встречаются очень часто.

Пример 2.1. Использование флагов

```
#include <stdio.h>

// define all 8 possible flags for an 8 bit variable
//      name  hex     binary
#define FLAG1 0x01 // 00000001
#define FLAG2 0x02 // 00000010
```

```
#define FLAG3 0x04 // 00000100
#define FLAG4 0x08 // 00001000
// ... and so on
#define FLAG8 0x80 // 10000000

int main(int argc, char *argv[])
{
 char flags = 0; //an 8 bit variable

 // set flags with a logical or
 flags = flags | FLAG1; //set flag 1
 flags = flags | FLAG3; //set flag 3

 // check flags with a logical and.  If the flag is set (1)
 // then the logical and will return 1, causing the if
 // condition to be true.
 if (flags & FLAG1)
  printf("FLAG1 set!\n");

 // this of course will be untrue.
 if (flags & FLAG8)
  printf("FLAG8 set!\n");

 // check multiple flags by using a logical or
 // this will pass as FLAG1 is set
 if (flags & (FLAG1|FLAG4))
  printf("FLAG1 or FLAG4 set!\n");

 return 0;
}
```

Типы и представление чисел

Стандарты C

Хотя это небольшое отступление, важно немного познакомиться с историей языка C.

C — общий язык мира системного программирования. Операционные системы и связанные с ними системные библиотеки, широко используемые на момент написания книги, написаны на C, и каждая система предоставляет компилятор C. Чтобы язык не разошёлся на несовместимые варианты в разных системах, для него был создан строгий стандарт.

Официально он называется ISO/IEC 9899:1999(E), но чаще используется сокращённое название C99. Стандарт поддерживается Международной организацией по стандартизации ISO; полный текст можно приобрести в интернете. Предыдущие версии, такие как C89 и ANSI C, охватываются более поздним стандартом. Его документация очень техническая и подробно описывает почти все части языка: например, синтаксис в форме Бэкуса — Наура, стандартные значения `#define` и поведение операций.

Важно также понимать, чего стандарт C не определяет. Прежде всего он должен подходить для всех существующих и будущих архитектур, поэтому намеренно не фиксирует зависящие от архитектуры детали. Связующим звеном между стандартом C и оборудованием служит двоичный интерфейс приложений, или ABI, который мы рассмотрим ниже. В некоторых местах стандарт указывает, что результат операции не определён однозначно или зависит от

реализации. Если программист хочет получить переносимый код, полагаться на такие результаты нельзя.

> **Примечание переводчика.** После C99 были опубликованы более новые редакции стандарта C. Здесь и далее исторические сведения приведены в контексте времени написания книги.

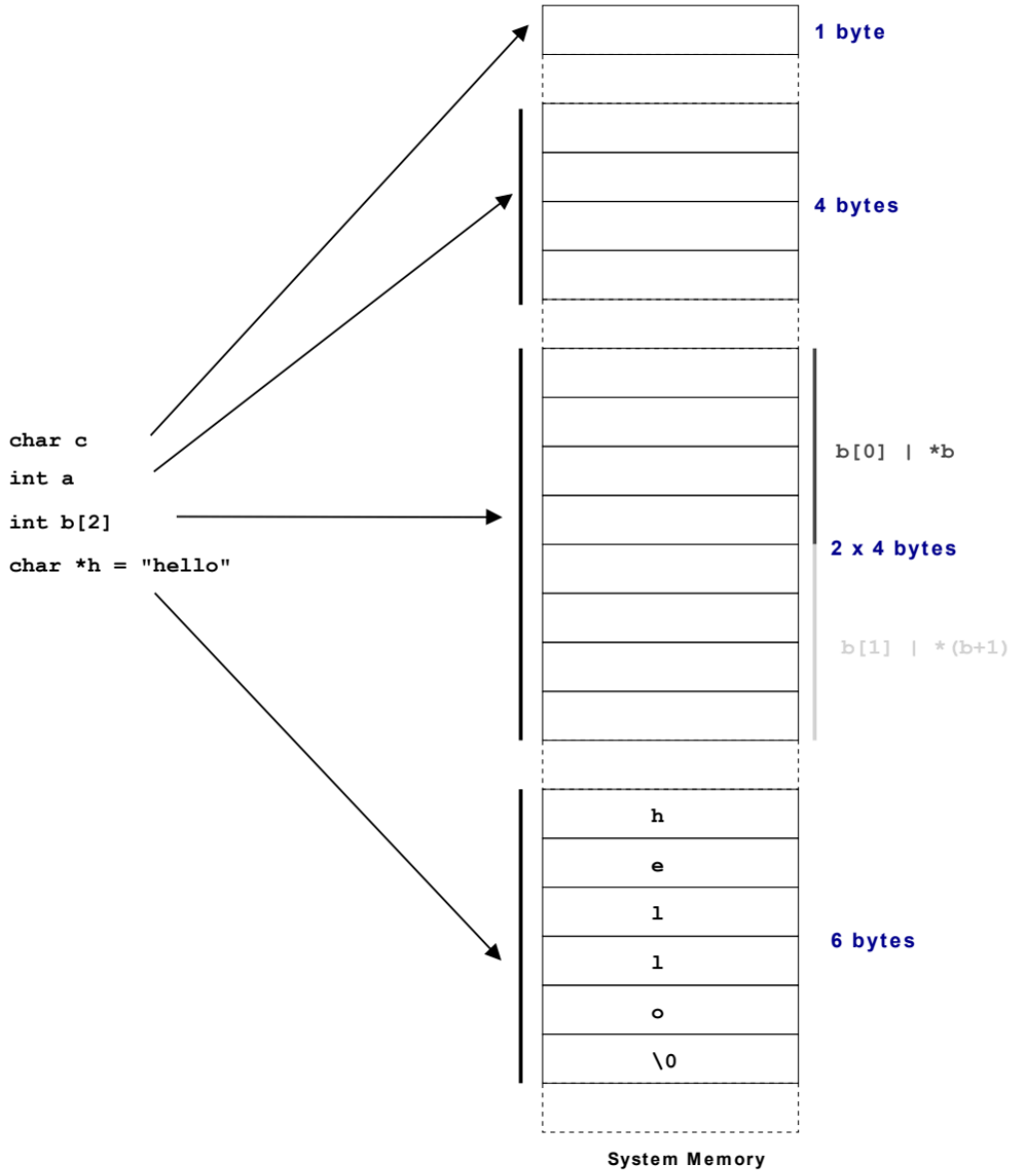
GNU C

Компилятор GNU C, чаще называемый `gcc`, почти полностью реализует стандарт C99. Кроме того, он предоставляет различные расширения, которые программисты используют для дополнительных возможностей ценой переносимости на другие компиляторы. Обычно расширения связаны с низкоуровневым кодом и чаще встречаются в системном программировании; самый распространённый пример — встроенный ассемблер. Программистам стоит читать документацию GCC и понимать, когда они используют возможности за пределами стандарта.

Компилятору можно указать придерживаться стандарта, например флагом `-std=c99`, и выдавать предупреждения или ошибки для некоторых нестандартных конструкций. Это полезно, если код должен легко переноситься на другой компилятор.

Типы

Программистам знакомо использование переменных для обозначения области памяти, в которой хранится значение. В типизированном языке, таком как C, каждая переменная должна быть объявлена с указанием типа. Тип сообщает компилятору, что мы собираемся хранить; компилятор может выделить достаточно места и проверить, не нарушает ли программист правила данного типа. На рисунке показано, сколько места выделяется для некоторых распространённых переменных.



Стандарт C99 намеренно указывает лишь минимальный размер каждого типа, поскольку на разных архитектурах и в разных операционных системах оптимальные размеры могут сильно различаться.

Для полной переносимости программистам не следует предполагать конкретный размер переменной. Однако работающей системе необходимо соглашение о размерах типов. Каждая комбинация архитектуры и операционной системы следует определённому ABI — двоичному интерфейсу приложений. ABI заполняет пробел между стандартом C и требованиями оборудования и операционной системы; он создаётся для конкретного сочетания процессора и ОС.

Таблица 2.13. Стандартные целочисленные типы и размеры

Тип	Минимум C99, бит	Типичный размер на 32-битной архитектуре, бит
`char`	8	8
`short`	16	16

Тип	Минимум C99, бит	Типичный размер на 32-битной архитектуре, бит
`int`	16	32
`long`	32	32
`long long`	64	64
Указатели	Зависит от реализации	32

Здесь видно, что `int` обычно занимает 32 бита — вдвое больше строгого минимума в 16 бит, требуемого C99.

Указатель по сути содержит адрес, то есть указывает на другое место в памяти, поэтому его размер должен позволять представлять подходящие адреса системы.

64-битные системы

Переход к 64-битным вычислениям нередко вызывает путаницу. Это означает, что процессор может обрабатывать 64-битные адреса; точнее, его регистры имеют ширину 64 бита. О регистрах мы поговорим в главе 3 «Архитектура компьютера».

В рассматриваемых моделях указатели имеют ширину 64 бита, чтобы представлять адреса системы. Но разработчикам системы необходимо решить и вопрос размеров остальных типов. Ниже показаны две распространённые модели.

Таблица 2.14. Стандартные скалярные типы и размеры

Тип	Минимум C99, бит	LP64, бит	Windows, бит
`char`	8	8	8
`short`	16	16	16
`int`	16	32	32
`long`	32	64	32
`long long`	64	64	64
Указатели	Зависит от реализации	64	64

В модели LP64 значения `long` и указатели занимают 64 бита. Это отличается от показанной ранее 32-битной модели. LP64 широко используется в системах UNIX.

В другой модели `long` остаётся 32-битным, что обеспечивает максимальную совместимость с 32-битным кодом. Эта модель используется в 64-битной Windows.

Есть веские причины, по которым размер `int` в обеих моделях не увеличили до 64 бит. Если бы его увеличили, программистам было бы негде взять 32-битную переменную, кроме как переопределить `short` как более крупный 32-битный тип.

64-битная переменная настолько велика, что для многих значений не нужна. Например, циклы очень редко повторяются больше числа раз, помещающегося в 32-битную

переменную (4 294 967 296 раз). Изображения обычно представляют восемью битами для каждого из красного, зелёного и синего компонентов и ещё восемью для дополнительной информации — альфа-канала; всего 32 бита. Поэтому во многих случаях у 64-битной переменной как минимум старшие 32 бита тратятся впустую. Вдобавок размер массива целых чисел удваивается. Программы занимают больше памяти и кэша, который подробно рассматривается в главе 3, без реального улучшения. По той же причине в Windows решили сохранить 32-битный `long`: большая часть API Windows изначально использовала `long` в 32-битной системе и не нуждалась в дополнительных битах. Это экономит место без переписывания API.

Рассмотрим альтернативу, при которой `short` переопределён как 32-битный тип. Программисты 64-битной системы могли бы использовать его для заведомо небольших значений. Но при переносе обратно на 32-битную систему та же переменная `short` занимала бы только 16 бит и гораздо легче переполнялась бы — уже на уровне 65 536 значений.

Требование явно запрашивать более крупные переменные, когда они нужны, позволяет соблюсти баланс между переносимостью и расходом места.

> **Примечание переводчика.** Разрядность процессора сама по себе не гарантирует конкретный размер всех указателей: он определяется реализацией и ABI. Кроме того, максимальное значение зависит от знаковости типа.

Квалификаторы типов

Стандарт C также описывает дополнительные характеристики типов переменных. Например, `const` указывает на запрет изменения объекта через соответствующее выражение, а `volatile` сообщает компилятору, что значение может изменяться вне обычного потока выполнения, поэтому обращения к нему требуют особого обращения.

Особенно важны `signed` и `unsigned`: они определяют, может ли целочисленная переменная представлять отрицательные значения. Подробнее мы рассмотрим это ниже.

Эти обозначения передают компилятору дополнительную информацию об использовании переменной. Благодаря этому он может обнаружить нарушение заданных правил, например попытку записи в `const`, и выполнять оптимизации с учётом полученных сведений.

Стандартные типы

В C99 учтено, что все эти правила, размеры и вопросы переносимости быстро становятся запутанными. Поэтому стандарт предоставляет специальные типы, позволяющие задать точные свойства переменной. Они определены в `<stdint.h>` и имеют форму `qtypes_t`, где `q` — уточнение, `type` — базовый тип, `s` — ширина в битах, а `_t` указывает на определённый C99 тип.

Например, `uint8_t` — беззнаковое целое ровно из восьми битов. Определено множество других типов; полный список приведён в стандарте C99 и в заголовочном файле.[^ch2-pri] Реализация C99 должна сопоставить предоставляемые типы с подходящими типами

целевой системы; в Linux соответствующие заголовки поставляются системными библиотеками.

[^ch2-pri]: C99 также предоставляет средства переносимого форматирования для `printf`: макросы `PRI` из `<inttypes.h>` можно применять как спецификаторы типов с заданным размером. Подробнее см. стандарт или заголовочные файлы.

> **Примечание переводчика.** Точные типы фиксированной ширины, такие как `uint8_t`, определяются реализацией только при наличии подходящего типа; стандарт также предусматривает типы минимальной и быстрой ширины.

Типы на практике

Ниже показано, как типы ограничивают допустимые операции над переменной и как компилятор может использовать эту информацию для предупреждений при неправильном применении. Автор описывает присваивание целого значения переменной `char`, которая из-за меньшего размера теряет правильное значение. Далее указатель на `char` присваивается переменной, объявленной как целое. Такая операция возможна, но небезопасна. Первый пример выполняется на 32-битном Pentium, и правильное значение адреса сохраняется. Во втором примере на 64-битном Itanium указатель занимает 64 бита (8 байтов), а целое — только 4 байта. Очевидно, восемь байтов не помещаются в четыре! Можно попытаться «обмануть» компилятор явным приведением, но этим мы сами себе навредим: меньшая переменная не может сохранить весь указатель, и получится неверный адрес.

Пример 2.2. Предупреждения при несовпадении типов

```
$ cat types.c
#include <stdio.h>
#include <stdint.h>

int main(void)
{
        char *c;
        int i;

        i = c;
        i = (int)c;

        return 0;
}

$ uname -m
i686
$ gcc -Wall -o types types.c
types.c: In function 'main':
types.c:19: warning: assignment makes integer from pointer without a cast
$ ./types
i is 52
p is 0x80484e8
p is 0x80484e8

$ uname -m
ia64
$ gcc -Wall  -o types types.c
types.c: In function 'main':
types.c:19: warning: assignment makes integer from pointer without a cast
```

```
types.c:21: warning: cast from pointer to integer of different size
types.c:22: warning: cast to pointer from integer of different size
$ ./types
i is 52
p is 0x40000000000009e0
p is 0x9e0
```

> **Примечание переводчика.** В оригинале показанный текст `types.c` не соответствует приведённому выводу: программа не выводит показанные значения, а указатель `c` не инициализирован. Вывод сохранён как исторический пример автора; рассчитывать на воспроизведение такого результата нельзя.

Представление чисел

Отрицательные значения

В современной десятичной записи отрицательное число обозначается знаком минус (`-`) перед ним. В двоичной системе для отрицательных значений нужен другой способ.

На современном оборудовании широко используется одна схема, хотя C99 допускает три способа представления отрицательных значений.

Знаковый бит

Самый простой способ — выделить один бит числа, значение которого указывает на положительное или отрицательное число. Это аналогично математическим знакам `+` и `-`. Такой подход вполне логичен, и некоторые ранние компьютеры действительно использовали его. Но двоичная система открывает и другие возможности, облегчающие жизнь разработчикам оборудования.

Обратите внимание: теперь ноль имеет два эквивалентных представления — с установленным и сброшенным знаковым битом. Их иногда называют соответственно −0 и +0.

Обратный код

В обратном коде для получения отрицательного числа к положительному применяется побитовое НЕ. Например, −90 (`-0x5A`) представляется как `~01011010 = 10100101`.[^ch2-complement]

Главное преимущество схемы в том, что для сложения отрицательного числа с положительным не требуется особой логики, за исключением того, что оставшийся бит переноса нужно добавить к результату.

Таблица 2.15. Сложение в обратном коде

```
Десятичное   Двоичное     Операция
-90          10100101     +
100          01100100
             --------
```

```
            100001001
10          00001010
```

При сложении бит за битом в конце появляется перенос. Добавив его к младшим восьми битам, получаем правильное значение 10. Здесь снова возникает проблема двух представлений нуля. Современные компьютеры обычно не используют обратный код, поскольку есть лучшая схема.

[^ch2-complement]: Оператор `~` в C применяет побитовое НЕ. По понятным теперь причинам его также называют оператором обратного кода.

Дополнительный код

Дополнительный код похож на обратный, но к отрицательному представлению добавляется единица, а оставшийся перенос отбрасывается. Продолжая предыдущий пример, получим для −90: `~01011010 + 1 = 10100101 + 1 = 10100110`.

Это даёт небольшую асимметрию диапазона. В восьмибитном целом есть $2^8 = 256$ возможных значений. При представлении знаком и величиной доступны числа от −127 до 127, а в дополнительном коде — от −128 до 127. Проблема двух нулей исчезает: «отрицательный ноль» равен `~00000000 + 1 = 11111111 + 1 = 00000000` с отбрасыванием переноса.

> **Примечание переводчика.** В оригинале диапазон дополнительного кода ошибочно указан как −127…128; выше он исправлен.

Таблица 2.16. Сложение в дополнительном коде

```
Десятичное   Двоичное   Операция
-90          10100110   +
100          01100100
             --------
10           00001010
```

Как видно, при использовании дополнительного кода разработчикам оборудования достаточно предусмотреть схемы сложения: вычитание выполняется изменением знака вычитаемого числа и сложением результата.

Аналогично умножение можно реализовать повторным сложением, а деление — повторным вычитанием. Таким образом, дополнительный код позволяет свести простые арифметические операции к сложению! Современные компьютеры используют дополнительный код.

Расширение знака

Из-за формата дополнительного кода при увеличении размера знакового значения важно расширять знак: дополнительные биты копируются из старшего бита исходного значения.

Например, 32-битное `int` со значением −10 представляется в дополнительном коде как `11111111111111111111111111110110`. При приведении к 64-битному `long long int` необходимо

установить дополнительные 32 бита в 1, чтобы сохранить знак и значение.

Благодаря дополнительному коду достаточно взять старший бит исходного числа и заполнить им все добавленные биты. Этот процесс называется расширением знака. Обычно его выполняет компилятор в ситуациях, определённых стандартом языка; процессор, как правило, предоставляет специальные инструкции для расширения значения до большего размера.

Числа с плавающей точкой

До сих пор мы обсуждали только целые числа. Числа, способные представлять дробные значения, относятся к классу чисел с плавающей точкой.

Чтобы представить дробное число, нужен способ выразить положение десятичной точки в двоичной системе. Самая распространённая схема — стандарт IEEE 754 для чисел с плавающей точкой, опубликованный Институтом инженеров электротехники и электроники. В основе схема довольно проста и несколько похожа на научную запись чисел.

В научной записи значение 123,45 обычно представляют как 1,2345 × 10². Здесь 1,2345 — мантисса, или значащая часть, 10 — основание, а 2 — показатель степени.

В модели IEEE доступные биты делятся между знаком, значащей частью и показателем степени. Число представляется как знак × значащая часть × 2^показатель. Знак равен 1 или −1. Поскольку используется двоичная система, основание 2 подразумевается.

Существуют разные размеры значений с плавающей точкой; ниже рассмотрено только 32-битное значение. Большее число битов позволяет повысить точность.

Таблица 2.17. Число IEEE с плавающей точкой

Знак	Показатель	Значащая часть / мантисса
S	EEEEEEEE	MMMMMMMMMMMMMMMMMMMMMMM

Ещё одна важная величина — смещение показателя степени. Он должен представлять как положительные, так и отрицательные значения, поэтому из записанного поля вычитается подразумеваемое значение 127. Например, показателю 0 соответствует поле 127; значение 128 обозначает 1, а 126 — −1.

Каждый бит значащей части добавляет точности. Рассмотрим научную запись числа 198765: $1{,}98765 \times 10^5$. Ей соответствует следующая таблица.

Таблица 2.18. Научная запись $1{,}98765 \times 10^5$

10^0	.	10^{-1}	10^{-2}	10^{-3}	10^{-4}	10^{-5}
1	.	9	8	7	6	5

> **Примечание переводчика.** В оригинале показатель в этом примере ошибочно указан как 6; для числа 198765 он равен 5. Описанная формула поля экспоненты относится к

> нормализованным значениям; нули, субнормальные значения, бесконечности и NaN имеют особые правила.

Каждая дополнительная цифра расширяет диапазон представимых дробных значений. В десятичной системе каждый знак после запятой увеличивает точность в десять раз: одним знаком можно представить значения от 0,0 до 0,9, двумя — от 0,00 до 0,99 и так далее. В двоичной системе каждый дополнительный разряд увеличивает точность только вдвое, как показано в таблице. Поэтому двоичное представление не всегда прямо соответствует десятичному.

Таблица 2.19. Двоичная значащая часть

2^0	.	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}
1	.	1/2	1/4	1/8	1/16	1/32
1	.	0,5	0,25	0,125	0,0625	0,03125

> **Примечание переводчика.** В оригинальной таблице для 1/16 ошибочно указано 0,625; правильное значение — 0,0625.

При одном дробном бите можно представить только 0 или 0,5. Добавив ещё один бит, получаем 0, 0,25, 0,5 и 0,75. С третьим битом доступны 0, 0,125, 0,25, 0,375, 0,5, 0,625, 0,75 и 0,875.

Увеличение числа битов даёт всё большую точность. Но возможных чисел бесконечно много, поэтому битов никогда не хватит для точного представления любого значения.

Например, имея два дробных бита и желая представить 0,3, мы можем лишь выбрать ближайшее значение 0,25, чего очевидно недостаточно для большинства задач. Большее число битов значащей части даёт намного более тонкий шаг. Тип `double` увеличивает число хранимых битов значащей части до 52 и расширяет диапазон показателя. Некоторые аппаратные платформы предоставляют расширенный формат с 64-битной значащей частью, обеспечивающий очень высокую точность.

> **Примечание переводчика.** Для IEEE 754 binary32 хранятся 23 бита дробной части плюс подразумеваемый ведущий бит для нормализованных значений; binary64 хранит 52 плюс ведущий бит. Десятичные дроби могут оставаться неточными и в более широком формате.

Пример 2.3. Сравнение float и double

```
$ cat float.c
#include <stdio.h>
int main(void)
{
        float a = 0.45;
        float b = 8.0;
    double ad = 0.45;
        double bd = 8.0;

        printf("float+float, 6dp    : %f\n", a+b);
```

```
        printf("double+double, 6dp  : %f\n", ad+bd);
    printf("float+float, 20dp   : %10.20f\n", a+b);
        printf("dobule+double, 20dp : %10.20f\n", ad+bd);

        return 0;
}

$ gcc -o float float.c

$ ./float
float+float, 6dp    : 8.450000
double+double, 6dp  : 8.450000
float+float, 20dp   : 8.44999998807907104492
dobule+double, 20dp : 8.44999999999999928946

$ python
Python 2.4.4 (#2, Oct 20 2006, 00:23:25)
[GCC 4.1.2 20061015 (prerelease) (Debian 4.1.1-16.1)] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> 8.0 + 0.45
8.4499999999999993
```

Практический пример приведён выше. Обратите внимание: при выводе шести знаков после запятой по умолчанию `printf` показывает одинаковый результат благодаря округлению. Но если запросить большую точность, например 20 знаков, результаты начинают различаться. Код с `double` даёт более точный результат, но он всё равно не абсолютно точен. Программисты, которые явно не работают с `float`, тоже сталкиваются с проблемой точности переменных!

Нормализованные значения

В научной записи одно значение можно представить несколькими способами: например, `10023 × 10⁰ = 1002,3 × 10¹ = 100,23 × 10²`. Автор определяет нормализованный вариант как представление, где `1/основание ≤ значащая часть < 1`. В двоичной системе первый значащий бит тогда всегда равен единице. Зная это, можно получить дополнительный бит точности, договорившись считать ведущую единицу подразумеваемой.

Таблица 2.20. Пример нормализации 0,375

2^0	.	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	Множитель	Расчёт
0	.	0	1	1	0	0	2^0	(0,25 + 0,125) × 1 = 0,375
0	.	1	1	0	0	0	2^{-1}	(0,5 + 0,25) × 0,5 = 0,375
1	.	1	0	0	0	0	2^{-2}	(1 + 0,5) × 0,25 = 0,375

Как видно, значение можно нормализовать, перемещая биты и компенсируя это изменением показателя степени.

> **Примечание переводчика.** В приведённых сдвигах влево показатель уменьшается, а не увеличивается, как сказано в исходном тексте. Для нормализованного IEEE 754 binary используется форма `1 ≤ значащая часть < 2`; ведущая единица подразумевается для нормализованных конечных ненулевых чисел.

Приёмы с нормализацией

Программистам часто нужно найти первый установленный бит в битовом поле. Рассмотрим `0100`: первый установленный бит справа имеет номер 2, если, как обычно, нумерация начинается с нуля.

Обычный способ — сдвигать число вправо, проверять очередной бит и либо завершать поиск, либо повторять его. Это медленный процесс: если поле занимает 64 бита и установлен только самый дальний бит, придётся пройти все предыдущие 63 бита.

Но если считать это поле значащей частью числа с плавающей точкой и нормализовать его, показатель степени сообщит, на сколько позиций произведён сдвиг. Нормализация обычно встроена в аппаратный блок операций с плавающей точкой и работает очень быстро — чаще всего быстрее многократных сдвигов и проверок.

Следующая программа показывает два способа найти первый установленный бит на процессоре Itanium. В Itanium, как и во многих серверных процессорах, поддерживается расширенный 80-битный формат с 64-битной значащей частью. Поэтому `unsigned long` помещается в значащую часть `long double`. При загрузке число нормализуется; прочитав показатель и вычтя смещение, можно определить величину сдвига.

Пример 2.4. Программа для поиска первого установленного бита

```
#include <stdio.h>

int main(void)
{
 //  in binary = 1000 0000 0000 0000
 //  bit num     5432 1098 7654 3210
 int i = 0x8000;
 int count = 0;
 while ( !(i & 0x1) ) {
  count ++;
  i = i >> 1;
 }
 printf("First non-zero (slow) is %d\n", count);

 // this value is normalised when it is loaded
 long double d = 0x8000UL;
 long exp;
 // Itanium "get floating point exponent" instruction
 asm ("getf.exp %0=%1" : "=r"(exp) : "f"(d));

 // note exponent include bias
 printf("The first non-zero (fast) is %d\n", exp - 65535);


}
```

> **Примечание переводчика.** Это пример для Itanium с использованием специфичной для него инструкции, а не переносимое решение на C. При поиске младшего установленного бита авторская программа проверяет именно младший бит, а не старший.

Собираем всё вместе

В следующем примере извлекаются компоненты числа с плавающей точкой и выводится представляемое им значение. Программа работает только с 32-битным значением IEEE, хотя такое представление типа `float` распространено на многих архитектурах.

Пример 2.5. Исследование float

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

/* return 2^n */
int two_to_pos(int n)
{
 if (n == 0)
  return 1;
 return 2 * two_to_pos(n - 1);
}

double two_to_neg(int n)
{
 if (n == 0)
  return 1;
 return 1.0 / (two_to_pos(abs(n)));
}

double two_to(int n)
{
 if (n >= 0)
  return two_to_pos(n);
 if (n < 0)
  return two_to_neg(n);
 return 0;
}
/* Go through some memory "m" which is the 24 bit significand of a
   floating point number.  We work "backwards" from the bits
   furthest on the right, for no particular reason. */
double calc_float(int m, int bit)
{
 /* 23 bits; this terminates recursion */
 if (bit > 23)
  return 0;

 /* if the bit is set, it represents the value 1/2^bit */
 if ((m >> bit) & 1)
  return 1.0L/two_to(23 - bit) + calc_float(m, bit + 1);


 /* otherwise go to the next bit */
 return calc_float(m, bit + 1);
}

int main(int argc, char *argv[])
{
 float f;
 int m,i,sign,exponent,significand;

 if (argc != 2)
 {
  printf("usage: float 123.456\n");
  exit(1);
 }
```

```
 if (sscanf(argv[1], "%f", &f) != 1)
 {
  printf("invalid input\n");
  exit(1);
 }

 /* We need to "fool" the compiler, as if we start to use casts
(e.g. (int)f) it will actually do a conversion for us.  We
    want access to the raw bits, so we just copy it into a same
    sized variable. */
 memcpy(&m, &f, 4);
 /* The sign bit is the first bit */
 sign = (m >> 31) & 0x1;

 /* Exponent is 8 bits following the sign bit */
 exponent = ((m >> 23) & 0xFF) - 127;

 /* Significand fills out the float, the first bit is implied
    to be 1, hence the 24 bit OR value below. */
 significand = (m & 0x7FFFFF) | 0x800000;
 /* print out a power representation */
 printf("%f = %d * (", f, sign ? -1 : 1);
 for(i = 23 ; i >= 0 ; i--)
 {
  if ((significand >> i) & 1)
   printf("%s1/2^%d", (i == 23) ? "" : " + ",
          23-i);
 }
 printf(") * 2^%d\n", exponent);
 /* print out a fractional representation */
 printf("%f = %d * (", f, sign ? -1 : 1);
 for(i = 23 ; i >= 0 ; i--)
 {
  if ((significand >> i) & 1)
   printf("%s1/%d", (i == 23) ? "" : " + ",
          (int)two_to(23-i));

 }
 printf(") * 2^%d\n", exponent);
 /* convert this into decimal and print it out */
 printf("%f = %d * %.12g * %f\n",
        f,
        (sign ? -1 : 1),
        calc_float(significand, 0),
    two_to(exponent));

 /* do the math this time */
 printf("%f = %.12g\n",
        f,
    (sign ? -1 : 1) *
        calc_float(significand, 0) *
        two_to(exponent)
  );
 return 0;
}
```

Ниже показан пример вывода для значения `8.45`, которое мы уже рассматривали.

Пример 2.6. Анализ числа 8.45

```
$ ./float 8.45
8.450000 = 1 * (1/2^0 + 1/2^5 + 1/2^6 + 1/2^7 + 1/2^10 + 1/2^11 + 1/2^14 + 1/2^15 + 1/2^18 + 1/2^19 +
1/2^22 + 1/2^23) * 2^3
8.450000 = 1 * (1/1 + 1/32 + 1/64 + 1/128 + 1/1024 + 1/2048 + 1/16384 + 1/32768 + 1/262144 + 1/524288 +
1/4194304 + 1/8388608) * 2^3
```

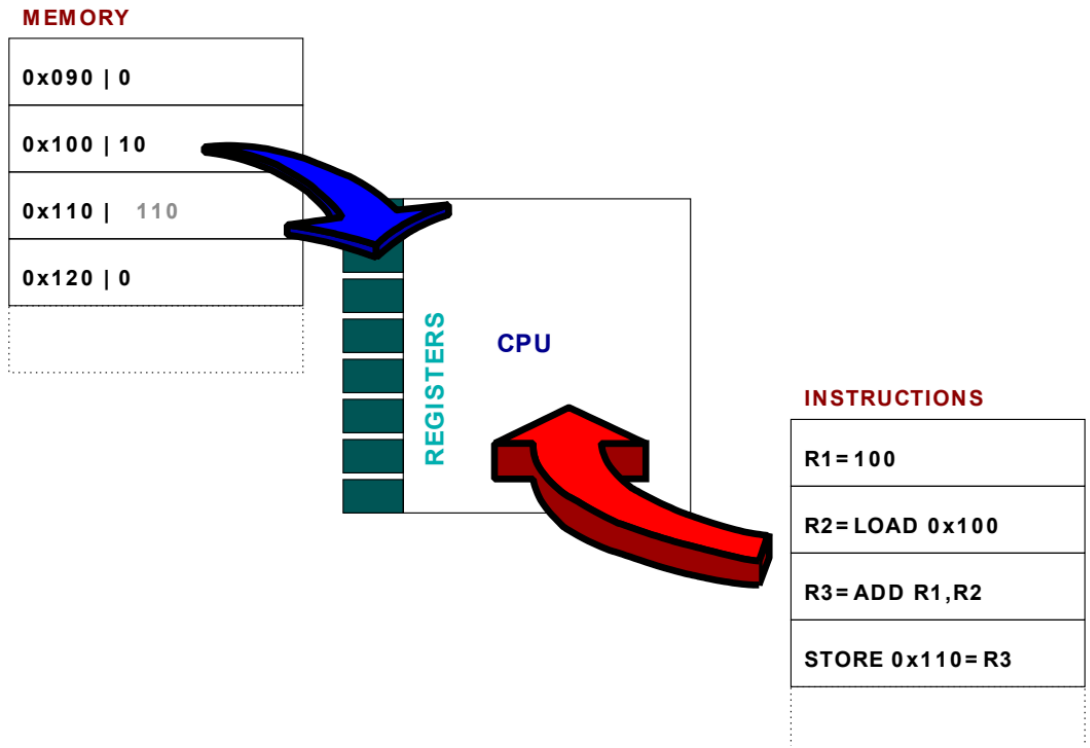
```
8.450000 = 1 * 1.05624997616 * 8.000000
8.450000 = 8.44999980927
```

Этот пример даёт представление о том, как возникают неточности в числах с плавающей точкой.

> **Примечание переводчика.** Пример рассчитан на типичное представление нормализованных чисел и не обрабатывает отдельно ноль, субнормальные значения, бесконечности и NaN. Его также не следует считать полностью переносимым между реализациями C.

Глава 3. Архитектура компьютера

Центральный процессор



Процессор выполняет инструкции над значениями в регистрах. В этом примере сначала регистру R1 присваивается значение 100, затем из памяти по адресу `0x100` в R2 загружается значение, два значения складываются, результат помещается в R3 и сохраняется для дальнейшего использования.

> **Примечание переводчика.** Подпись к исходному рисунку упоминает сохранение в R4, однако на самой схеме результат R3 записывается по адресу памяти `0x110`.

Сильно упрощая, компьютер можно представить как центральный процессор (CPU), подключённый к памяти. Рисунок выше иллюстрирует общий принцип всех операций компьютера.

Процессор выполняет инструкции, прочитанные из памяти. Их можно разделить на две категории:

1. Инструкции, загружающие значения из памяти в регистры и сохраняющие значения регистров в память.
2. Инструкции, выполняющие операции над значениями регистров: например, сложение, вычитание, умножение и деление, побитовые операции AND, OR, XOR и другие математические операции, такие как квадратный корень, синус, косинус и тангенс.

Итак, в примере мы просто прибавляем 100 к значению в памяти и сохраняем новый результат обратно в память.

Переходы

Помимо загрузки и сохранения важная операция процессора — переход. Внутри процессора адрес следующей инструкции хранится в указателе инструкций. Обычно указатель увеличивается так, чтобы перейти к следующей инструкции по порядку. Инструкция перехода, как правило, проверяет, равен ли определённый регистр нулю или установлен ли флаг, и при выполнении условия изменяет указатель на другой адрес. Тогда следующая инструкция берётся из другой части программы. Именно так работают циклы и условные операторы.

Например, условие `if (x==0)` можно реализовать с помощью операции OR над двумя регистрами: один содержит `x`, а другой — ноль. Если результат равен нулю, сравнение истинно, то есть все биты `x` равны нулю, и следует выполнить тело условия. Иначе переход пропускает его.

Такты

Мы привыкли видеть скорость компьютера в мегагерцах или гигагерцах — миллионах или миллиардах тактов в секунду. Это тактовая частота: частота импульсов внутреннего генератора компьютера.

Импульсы используются процессором для внутренней синхронизации. На каждом такте может начинаться очередная операция. Можно сравнить генератор с человеком, который бьёт в барабан, задавая ритм гребцам.

Выборка, декодирование, выполнение, сохранение

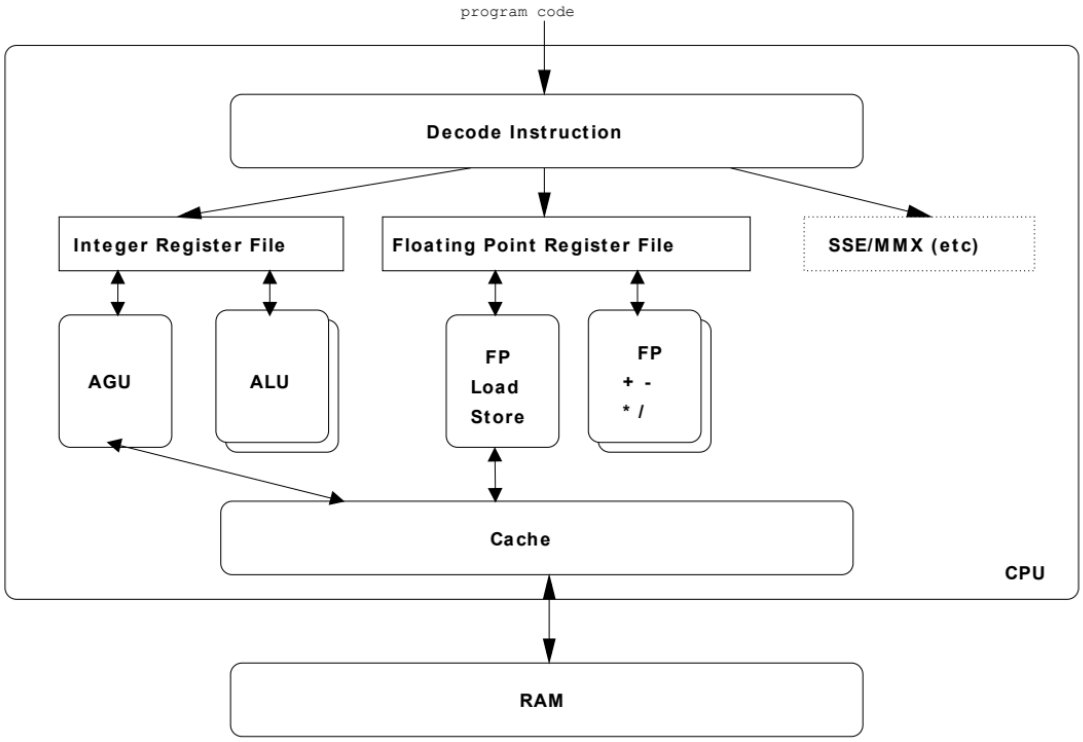
Выполнение одной инструкции состоит из цикла событий: выборки, декодирования, выполнения и сохранения.

Например, для приведённой выше инструкции сложения процессор должен:

1. **Выбрать** инструкцию из памяти и загрузить её в процессор.
2. **Декодировать** её, определив, что требуется сделать — в данном случае сложение.
3. **Выполнить** операцию: взять значения регистров и сложить их.
4. **Сохранить** результат в другом регистре. Также можно встретить термин «завершение инструкции» (*retirement*).

Внутри процессора

В процессоре множество компонентов, выполняющих перечисленные этапы, и обычно они могут работать независимо друг от друга. Это похоже на производственную линию с несколькими станциями: каждая выполняет свою задачу, передаёт результат следующей и берёт новые входные данные.



Процессор состоит из множества компонентов, каждый из которых выполняет отдельную задачу. На рисунке 3.2 показана очень упрощённая блок-схема некоторых основных частей современного процессора.

Инструкции поступают в процессор и декодируются. У процессора есть два основных вида регистров: для целочисленных вычислений и для вычислений с плавающей точкой. Плавающая точка — способ двоичного представления дробных чисел; внутри процессора они обрабатываются иначе. Регистры MMX (мультимедийное расширение), SSE (потоковые операции над несколькими данными одной инструкцией) и AltiVec похожи на регистры плавающей точки.

Регистровый файл — общее название совокупности регистров процессора. Ниже находятся компоненты, которые фактически выполняют работу.

Как уже говорилось, процессор либо загружает значение в регистр или сохраняет его из регистра в память, либо выполняет операцию над значениями регистров.

Арифметико-логическое устройство (ALU) — сердце работы процессора. Оно берёт значения из регистров и выполняет одну из многочисленных доступных операций. В современных процессорах несколько ALU, способных работать независимо. Например, в Pentium есть быстрые и медленные ALU: быстрые меньше по размеру и поэтому их можно разместить больше, но они выполняют только самые распространённые операции; медленные крупнее, зато поддерживают все операции.

Устройство генерации адресов (AGU) взаимодействует с кэшем и основной памятью, загружая значения в регистры для обработки ALU и сохраняя их из регистров обратно в память. Для регистров плавающей точки действуют те же общие принципы, хотя терминология компонентов немного отличается.

Конвейеризация

Как видно выше, сложение в ALU не зависит от записи значений в память устройством AGU, поэтому процессор может делать и то и другое одновременно. Кроме того, несколько ALU могут работать над разными инструкциями. Наконец, блок операций с плавающей точкой может работать параллельно с выполняющимися целочисленными инструкциями. Автор называет этот процесс конвейеризацией[^ch3-pipeline], а процессор с такой возможностью — суперскалярным. Современные процессоры суперскалярны.

[^ch3-pipeline]: На самом деле современный процессор обычно имеет значительно больше четырёх показанных здесь стадий. Чем больше стадий может работать одновременно, тем глубже конвейер.

> **Примечание переводчика.** Конвейеризация — перекрытие стадий обработки разных инструкций, а суперскалярность — возможность выдавать или выполнять несколько инструкций за такт. Эти понятия связаны, но не тождественны.

Конвейер можно сравнить со шлангом, наполняемым шариками, где шарики — инструкции процессора. В идеале вы кладёте их в один конец друг за другом, по одному на такт. Когда шланг заполнен, каждый новый шарик сдвигает остальные на одну позицию, а с другого конца выпадает готовый результат.

Однако инструкции перехода нарушают эту модель: они могут заставить процессор продолжить выполнение с другого места. При конвейеризации приходится угадывать направление перехода, чтобы знать, какие инструкции загружать. Если предсказание верно, всё работает хорошо.[^ch3-branch] Если нет, время потрачено зря: конвейер нужно очистить и начать заново. Этот процесс называется сбросом конвейера и похож на необходимость остановиться и вытряхнуть все шарики из шланга.

[^ch3-branch]: Процессоры вроде Pentium используют механизмы отслеживания предыдущих переходов. Например, если запомнить предыдущее направление перехода в цикле из 100 повторений, предсказание часто окажется верным вплоть до последнего прохода, когда выполнение выйдет из цикла.

Предсказание переходов

Сброс конвейера, предсказание выполнения и невыполнения перехода, слоты задержки перехода.

> **Примечание переводчика.** В исходном тексте этот раздел содержит только перечисление тем и не развит дальше.

Переупорядочивание

Если продолжить аналогию со шлангом, процессор может менять порядок шариков внутри, пока они появляются на выходе в порядке, совместимом с исходной программой. Этот исходный порядок называется программным порядком инструкций.

```
1: r3 = r1 * r2
2: r4 = r2 + r3
3: r7 = r5 * r6
4: r8 = r1 + r7
```


```
1: r3 = r1 * r2
2: r4 = r2 + r3
3: r7 = r5 * r6
4: r8 = r1 + r7
```


В последовательности на рисунке 3.3 инструкция 2 должна дождаться полного завершения инструкции 1. Поэтому конвейер простаивает в ожидании результата. Аналогично инструкции 3 и 4 связаны зависимостью через `r7`. Однако инструкции 2 и 3 не зависят друг от друга: они работают с разными регистрами. Если поменять их местами, порядок окажется лучше для конвейера: процессор сможет выполнять полезную работу вместо ожидания предыдущего результата.

При написании очень низкоуровневого кода некоторым инструкциям могут понадобиться гарантии порядка операций. Это называется семантикой памяти. Автор описывает семантику приобретения (*acquire*) как гарантию завершения предыдущих инструкций, а семантику освобождения (*release*) — как требование, чтобы последующие инструкции видели текущий результат. Ещё более строгим средством является барьер памяти, требующий сохранения операций в памяти перед продолжением.

На одних архитектурах подобные гарантии обеспечивает процессор, на других их нужно задавать явно. Большинству программистов не приходится работать с ними напрямую, но эти термины могут встретиться.

> **Примечание переводчика.** Современные модели памяти определяют acquire, release и барьеры через ограничения порядка наблюдаемых операций и синхронизацию; это не универсальная гарантия физической записи всех данных в оперативную память.

CISC и RISC

Архитектуры компьютеров часто делят на CISC — компьютеры со сложным набором инструкций — и RISC — компьютеры с сокращённым набором инструкций.

В первом примере мы явно загрузили значения в регистры, выполнили сложение и сохранили результат из регистра в память. Это пример RISC-подхода: операции выполняются над регистрами, а загрузка и сохранение в память задаются явно.

При подходе CISC одна инструкция может взять значения из памяти, сложить их внутри процессора и записать результат обратно. Такая инструкция может занимать много тактов, но в конечном итоге оба подхода достигают одной цели.

Автор отмечает, что современные архитектуры используют многие принципы RISC.[^ch3-cisc] Этому способствует несколько причин:

- Хотя RISC усложняет программирование на ассемблере, почти все программисты используют высокоуровневые языки и поручают генерацию ассемблерного кода компилятору. Другие преимущества перевешивают этот недостаток.
- Более простые инструкции оставляют больше места на кристалле для регистров. Регистры — самый быстрый вид памяти, и при прочих равных их большее число может повысить производительность.
- Простые инструкции облегчают конвейеризацию: конвейеру нужен постоянный поток инструкций, а большой разброс времени их выполнения усложняет его работу.

[^ch3-cisc]: Например, Intel Pentium с набором инструкций CISC внутри разбивает инструкции на более простые микрооперации перед выполнением.

> **Примечание переводчика.** Деление современных процессоров на RISC и CISC условно: внешняя архитектура инструкций и внутренняя реализация могут сочетать разные подходы; инструкции не обязаны выполняться за одинаковое время.

EPIC

Процессор Itanium, который часто используется в примерах книги, представляет модифицированную архитектуру EPIC — вычисления с явно выраженным параллелизмом инструкций.

Мы уже говорили, что в конвейерах суперскалярных процессоров одновременно выполняется множество инструкций в разных частях процессора. Для наилучшего результата инструкции должны поступать в порядке, позволяющем максимально использовать доступные устройства CPU.

Традиционно организацией входящего потока занимается аппаратная часть. Программа выдаёт инструкции последовательно, а процессор заглядывает вперёд и решает, как их организовать.

Идея EPIC состоит в том, что на более высоких уровнях доступно больше информации, позволяющей принимать эти решения лучше процессора. Анализ последовательности ассемблерных инструкций теряет часть сведений, которые программист мог выразить в исходном коде. Это похоже на разницу между изучением пьесы Шекспира и чтением её краткого пересказа: сюжет тот же, но оригинал содержит дополнительные сведения о ситуации и персонажах.

Таким образом, логику упорядочивания можно перенести из процессора в компилятор. Его разработчикам приходится тщательнее искать лучший порядок кода, зато процессор существенно упрощается, поскольку часть работы передана компилятору.[^ch3-epic]

[^ch3-epic]: С EPIC часто связывают VLIW — архитектуру с очень длинным словом инструкции, в которой инструкция указывает, какие внутренние устройства должны её выполнять. Жёсткая привязка к модели процессора осложняет обновление оборудования и потребовала бы множества вариантов бинарных файлов. EPIC добавляет уровень абстракции: вместо точного указания конкретного устройства описывается упрощённый

набор основных устройств, таких как память, целочисленные операции и операции с плавающей точкой.

Память

Иерархия памяти

Центральный процессор быстро получает инструкции и данные из кэша, расположенного на кристалле процессора. Кэш, в свою очередь, загружается из основной оперативной памяти — RAM. Оперативная память сохраняет содержимое только при наличии питания, поэтому данные, которые нужно сохранить надолго, записываются в постоянное хранилище.

Эти уровни образуют иерархию памяти.

Таблица 3.1. Иерархия памяти

Скорость	Память	Описание
Самая высокая	Кэш	Встроенная в процессор память. Она очень быстрая, но из-за размещения на кристалле её объём ограничен. Обычно существует несколько уровней — L1, L2 и L3 — с разными скоростями и объёмами.
Промежуточная	Оперативная память	Инструкции и данные программы хранятся в RAM. Она быстрая, но доступ процессора к ней всё же занимает ощутимое время, называемое задержкой. RAM расположена в отдельных микросхемах на материнской плате и гораздо больше кэша.
Самая низкая	Диск	Программы и файлы могут храниться на дисках. Механические устройства с вращающимися пластинами и движущимися головками работают медленнее, но предоставляют гораздо больший объём.

Главное в иерархии памяти — компромисс между скоростью и объёмом: чем быстрее память, тем она обычно меньше. Разумеется, если вы найдёте способ изменить это соотношение, то станете миллиардером!

Кэши эффективны благодаря двум видам локальности, характерным для компьютерных программ:

1. **Пространственная локальность:** данные, расположенные рядом, вероятно, будут использоваться вместе.
2. **Временная локальность:** недавно использованные данные, вероятно, вскоре понадобятся снова.

Поэтому выгодно хранить как можно больше быстро доступных недавно использованных данных, загружая небольшие блоки связанных данных.

> **Примечание переводчика.** Автор описывает типичную иерархию своего времени; современные системы также применяют твердотельные накопители и разнообразные конфигурации кэша. Данные могут поступать в процессор и при промахе кэша, а оперативная память используется не только для адресов и инструкций.

Подробнее о кэше

Кэш — один из важнейших элементов архитектуры процессора. Чтобы писать эффективный код, разработчику полезно понимать, как устроен кэш его системы.

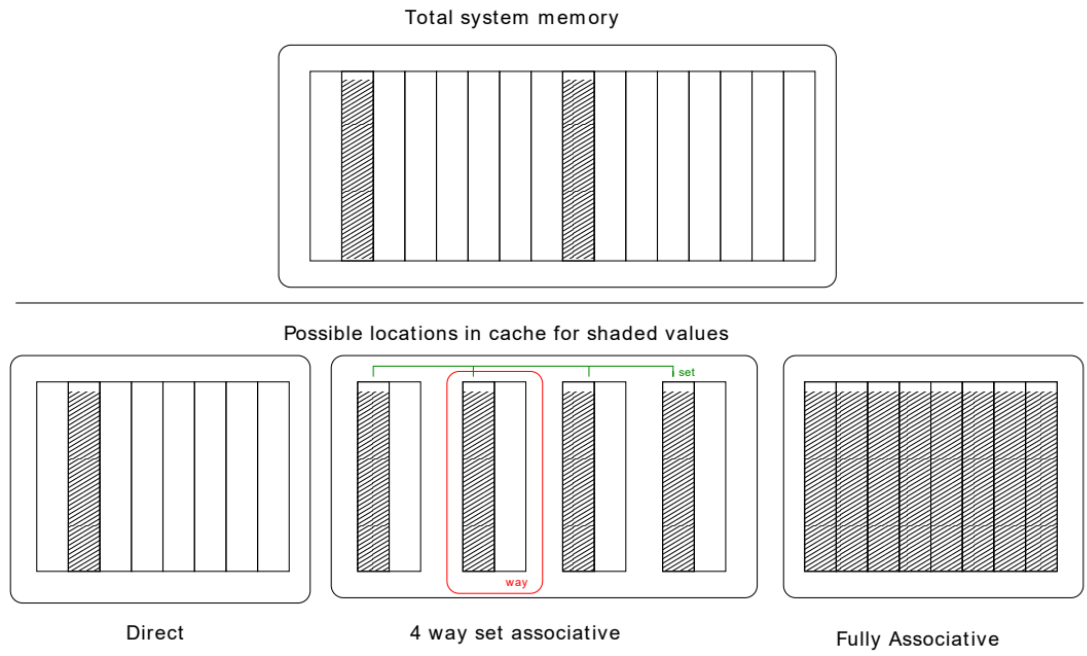
Кэш — очень быстрая копия части более медленной основной памяти. Он значительно меньше RAM, поскольку находится на кристалле рядом с регистрами и логикой процессора. Это очень ценное место, и максимальный размер кэша ограничен экономическими и физическими факторами. По мере роста числа транзисторов размер кэша увеличивается, но он всё равно обычно измеряется мегабайтами, тогда как основная память — гигабайтами, а накопители — терабайтами.

Кэш состоит из небольших фрагментов, отражающих данные основной памяти. Размер фрагмента называется размером строки кэша и обычно составляет, например, 32 или 64 байта. Под строкой кэша понимают такой блок данных. Данные обычно загружаются в кэш и вытесняются из него целыми строками.

Кэши имеют собственную иерархию — обычно L1, L2 и L3. L1 самый быстрый и маленький, L2 больше и медленнее, L3 ещё больше и медленнее.

Кэш L1 обычно разделён на кэш инструкций и кэш данных. Такой подход называют гарвардской архитектурой по имени релейного компьютера Harvard Mark I, в котором он появился. Раздельные кэши уменьшают конкуренцию в конвейере: ранние стадии чаще обращаются к инструкциям, а поздние — к данным.

Помимо снижения конкуренции за общий ресурс, отдельный кэш инструкций допускает решения, учитывающие характер потока инструкций: он используется только для чтения и потому не нуждается в некоторых дорогих возможностях кристалла, например нескольких портах, или обработке чтений части блока в той же мере, поскольку обращения к инструкциям обычно регулярнее.



A given cache line may find a valid home in one of the shaded entries.

При обычной работе процессор постоянно проверяет, находится ли определённый адрес в кэше. Поэтому нужен очень быстрый способ определить наличие нужной действительной строки.

Если адрес можно поместить куда угодно, при каждом обращении придётся проверять все строки, определяя попадание или промах. Для скорости проверки выполняются параллельно аппаратными средствами, однако поиск по всем записям слишком дорог для кэша разумного размера. Поэтому конструкцию упрощают, ограничивая места, где может находиться конкретный адрес. Это компромисс: кэш намного меньше системной памяти, поэтому некоторым адресам приходится претендовать на одни и те же места. Если два таких адреса постоянно обновляются, они «борются» за строку кэша. Различают три основных типа:

- **Кэш прямого отображения** разрешает хранить строку только в одной определённой записи. Он проще всего аппаратно, но два адреса, отображаемые в одну запись, неизбежно конфликтуют.
- **Полностью ассоциативный кэш** позволяет разместить строку в любой записи. Конфликтов из-за фиксированного места нет, но одновременная проверка всех позиций очень дорога.
- **Множественно-ассоциативный кэш** сочетает оба подхода: определённый блок может находиться в одной из нескольких строк своего множества. Кэш разделён на равные пути, или *ways*. В n-канальном кэше для каждого множества доступны n возможных мест. На рисунке показан кэш из восьми элементов с четырьмя путями: у двух адресов есть четыре возможных положения, поэтому при поиске проверяется только половина кэша. Чем больше путей, тем больше возможных мест и меньше конфликтов.

> **Примечание переводчика.** Размер множества — число путей ассоциативности; число множеств равно общему числу строк, делённому на число путей. Формулировка исходного текста с операцией `mod` неточна.

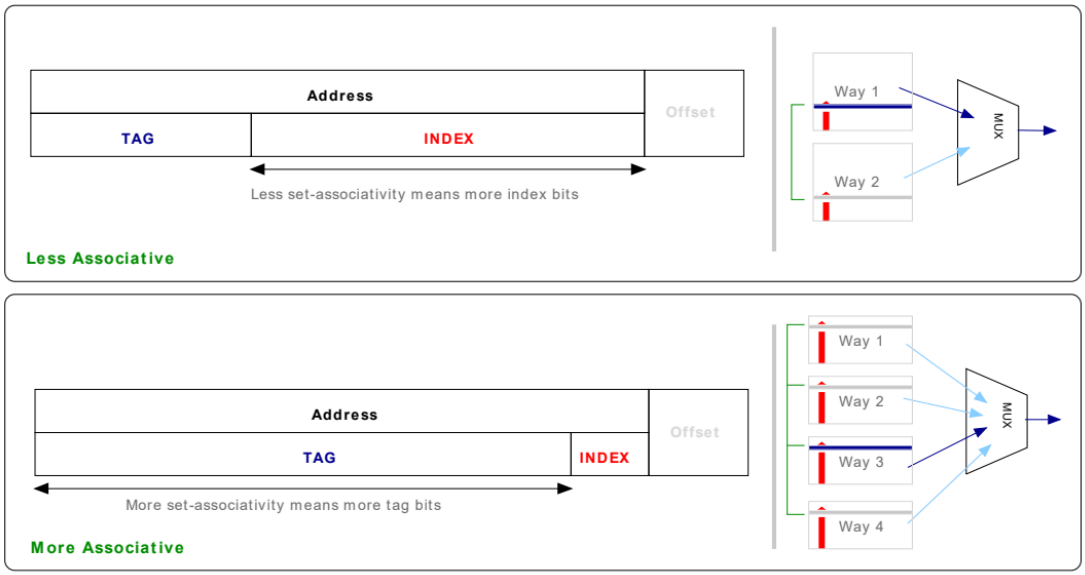
Когда кэш заполняется, процессору нужно вытеснить строку, чтобы освободить место. Существует много алгоритмов выбора. Например, LRU вытесняет строку, которая дольше остальных не использовалась.

Если данные только читаются из кэша, обновлять основную память не требуется. Но при записи в строку кэша нужно решить, когда обновлять RAM. При **сквозной записи** (*write-through*) изменения сразу передаются в основную память. Это медленнее, поскольку запись в RAM занимает больше времени. При **обратной записи** (*write-back*) обновление RAM откладывается до необходимости. Преимущество — меньше обращений к основной памяти. Строки, изменённые в кэше, но ещё не записанные в память, называются грязными. Недостаток в том, что при вытеснении могут понадобиться два обращения: сначала записать грязные данные, затем загрузить новые.

Если данные одновременно присутствуют на двух уровнях иерархии, кэш может быть инклюзивным. В эксклюзивной схеме наличие строки на одном уровне исключает её присутствие на другом. Этот выбор подробнее обсуждается в разделе об эксклюзивности кэша в SMP-системах.

Адресация кэша

Мы ещё не обсудили, как кэш определяет, находится ли в нём конкретный адрес. Очевидно, ему нужен каталог данных, содержащихся в строках. Каталог и сами данные могут находиться рядом на процессоре, но могут быть и разделены. Например, в POWER5 каталог L3 расположен в ядре, а для доступа к данным нужно пройти по шине L3 к памяти за пределами ядра. Такая организация ускоряет определение попадания или промаха без затрат на размещение всего кэша внутри ядра.



Для быстрого определения наличия адреса его делят на три части: тег, индекс и смещение.

Биты смещения зависят от размера строки. Например, при строке в 32 байта последние пять битов адреса задают смещение внутри неё, поскольку $2^5 = 32$.

Индекс указывает множество или строку, где может находиться запись. Рассмотрим кэш из 256 строк с прямым отображением. Поскольку данные могут находиться только в одной строке, следующие восемь битов после смещения указывают, какую из строк от 0 до 255 проверить: $2^8 = 256$.

Теперь разделим тот же кэш на два пути. Получатся две группы по 128 строк, и адрес может находиться в любой из них. Для индекса внутри каждого пути достаточно семи битов. При фиксированном размере кэша увеличение числа путей уменьшает число битов индекса, поскольку каждый путь становится меньше.

Каталогу всё ещё нужно проверить, что сохранённая строка соответствует интересующему адресу. Оставшиеся биты образуют тег, который сравнивается с тегом входящего адреса для определения попадания. Эта зависимость показана на рисунке 3.5.

При нескольких путях сравнение выполняется параллельно во всех путях, после чего мультиплексор выдаёт итоговый результат. Чем выше ассоциативность, тем меньше битов индекса и больше битов тега. В предельном случае полностью ассоциативного кэша биты индекса не нужны. Параллельное сравнение тегов — дорогостоящая часть конструкции, которая ограничивает число строк и размер кэша.

> **Примечание переводчика.** Подпись к рисунку в оригинале противоречит основному тексту относительно связи ассоциативности и числа битов тега. При фиксированных размере кэша и длине строки больше путей означает меньше битов индекса и больше битов тега.

Периферийные устройства и шины

Периферия — многочисленные внешние устройства, подключаемые к компьютеру. Очевидно, процессору нужен способ общаться с ними, иначе они не принесут пользы. Канал связи между процессором и периферийными устройствами называется шиной.

Основные понятия периферийных шин

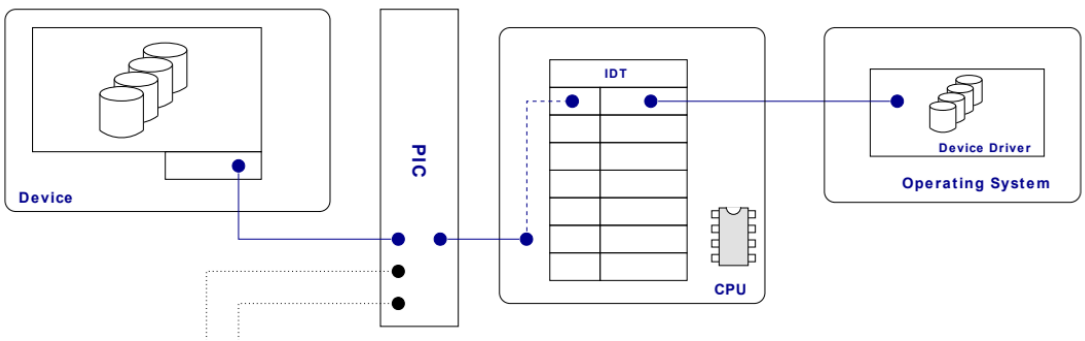
Чтобы устройство было полезным, ему необходимы ввод и вывод. Для связи с периферией используется несколько общих понятий.

Прерывания

Прерывание позволяет устройству буквально прервать процессор, чтобы сообщить некоторую информацию. Например, при нажатии клавиши возникает прерывание, передающее событие операционной системе. Каждому устройству назначается прерывание совместными средствами операционной системы и BIOS.

Устройства обычно подключаются к программируемому контроллеру прерываний (PIC) — отдельной микросхеме материнской платы, которая буферизует сведения о прерываниях и передаёт их процессору. От каждого устройства к одному из контроллеров системы идёт физическая линия. Когда устройство хочет вызвать прерывание, оно изменяет напряжение на линии.

В самом общем виде PIC принимает сигнал и преобразует его в сообщение для процессора. Хотя точная процедура зависит от архитектуры, общий принцип таков: операционная система настраивает таблицу дескрипторов прерываний, связывающую каждое возможное прерывание с адресом кода, к которому нужно перейти. Это показано на рисунке 3.6. Написание обработчика — задача автора драйвера устройства, работающего с учётом требований операционной системы.



Устройство подаёт сигнал контроллеру прерываний, который передаёт информацию процессору. Процессор обращается к таблице дескрипторов, заполненной операционной

системой, и находит код обработчика.

Большинство драйверов разделяют обработку на верхнюю и нижнюю половины. Одна часть быстро подтверждает прерывание, ставит действия в очередь и возвращает процессор к прежней работе; другая запускается позднее и выполняет более трудоёмкую обработку. Это не позволяет обработчику надолго занять процессор.

> **Примечание переводчика.** В оригинале названия верхней и нижней половин переставлены местами. В распространённой терминологии Linux быстрая непосредственная обработка — верхняя половина, а отложенная — нижняя. Современные контроллеры и способы доставки прерываний разнообразнее описанной схемы PIC.

Сохранение состояния

Поскольку прерывание может произойти в любой момент, после его обработки важно вернуться к выполнявшейся операции. Обычно операционная система сохраняет состояние, например регистры, при входе в обработчик и восстанавливает его при выходе. Так, не считая потраченного времени, прерывание остаётся прозрачным для прерванной программы.

Прерывания, ловушки и исключения

Хотя прерывание обычно связано с внешним событием физического устройства, тот же механизм полезен для внутренних операций системы. Например, обнаружив обращение к недопустимой памяти, деление на ноль или недопустимую инструкцию, процессор может создать исключение для обработки операционной системой. Этот механизм также используется для входа в операционную систему при системных вызовах и для реализации виртуальной памяти, рассматриваемой в главе 6. Хотя источник здесь внутренний, а не внешний, принцип передачи управления обработчику похож.

> **Примечание переводчика.** Внешние прерывания обычно асинхронны относительно инструкций программы, тогда как исключения, вызванные конкретной инструкцией, обычно синхронны.

Виды сигналов прерываний

Есть два основных способа сигнализировать прерывание по линии: по уровню и по фронту.

При сигнализации по уровню высокий уровень напряжения на линии указывает на ожидающее прерывание. При сигнализации по фронту обнаруживается изменение напряжения, например переход от низкого уровня к высокому. В этом случае PIC воспринимает импульс как сигнал о возникшем прерывании.

Различие особенно заметно, когда несколько устройств разделяют линию. При сигнализации по уровню она остаётся активной, пока все устройства, вызвавшие прерывание, не будут обслужены и не снимут сигнал.

При сигнализации по фронту импульс сообщает PIC о прерывании, которое передаётся операционной системе для обработки. Автор далее отмечает проблему дополнительных импульсов на уже активной общей линии, но оставляет это предложение незавершённым.

По описанию автора, при обработке прерывания по уровню линия может долго оставаться активной, затрудняя определение других устройств, и задержка обслуживания становится непредсказуемой. При сигнализации по фронту долго обрабатываемое прерывание можно поставить в очередь, а другие устройства могут сформировать новые переходы. Однако если два устройства подадут сигнал одновременно, один из переходов может быть пропущен; помехи также могут создать ложное прерывание, которое нужно игнорировать.

Немаскируемые прерывания

Системе важно иметь возможность временно маскировать, то есть запрещать, прерывания. Как правило, их можно отложить, но особый класс — немаскируемые прерывания (NMI) — является исключением. Типичный пример, приведённый автором, — сигнал сброса.

NMI полезны, например, для сторожевого таймера. Он периодически вызывает NMI и устанавливает флаг, который операционная система должна подтвердить. Если до следующего сигнала подтверждение не получено, можно считать, что система перестала продвигаться. Другой пример — профилирование: периодический NMI позволяет определить, какой код выполняет процессор; со временем это создаёт полезную картину производительности.

> **Примечание переводчика.** Сигнал аппаратного сброса и NMI — разные механизмы; их конкретное поведение зависит от архитектуры.

Пространство ввода-вывода

Для связи с периферийным устройством процессор выполняет операции ввода-вывода. Распространённый способ — ввод-вывод с отображением в память, при котором регистры устройства отображаются на адреса памяти.

Чтобы обратиться к устройству, достаточно прочитать данные по определённому адресу или записать их туда. В оригинале этот раздел помечен автором как требующий расширения.

Прямой доступ к памяти

Периферийные устройства значительно медленнее процессора, поэтому нужен способ не заставлять CPU ждать поступления данных.

Прямой доступ к памяти (DMA) позволяет передавать данные непосредственно между периферийным устройством и оперативной памятью системы.

Драйвер настраивает передачу, указывая устройству область RAM для данных, затем запускает DMA и позволяет процессору заниматься другими задачами. Завершив передачу, устройство вызывает прерывание и сообщает драйверу об окончании. После этого данные

— например, файл с диска или кадры платы видеозахвата — находятся в памяти и готовы к использованию.

Другие шины

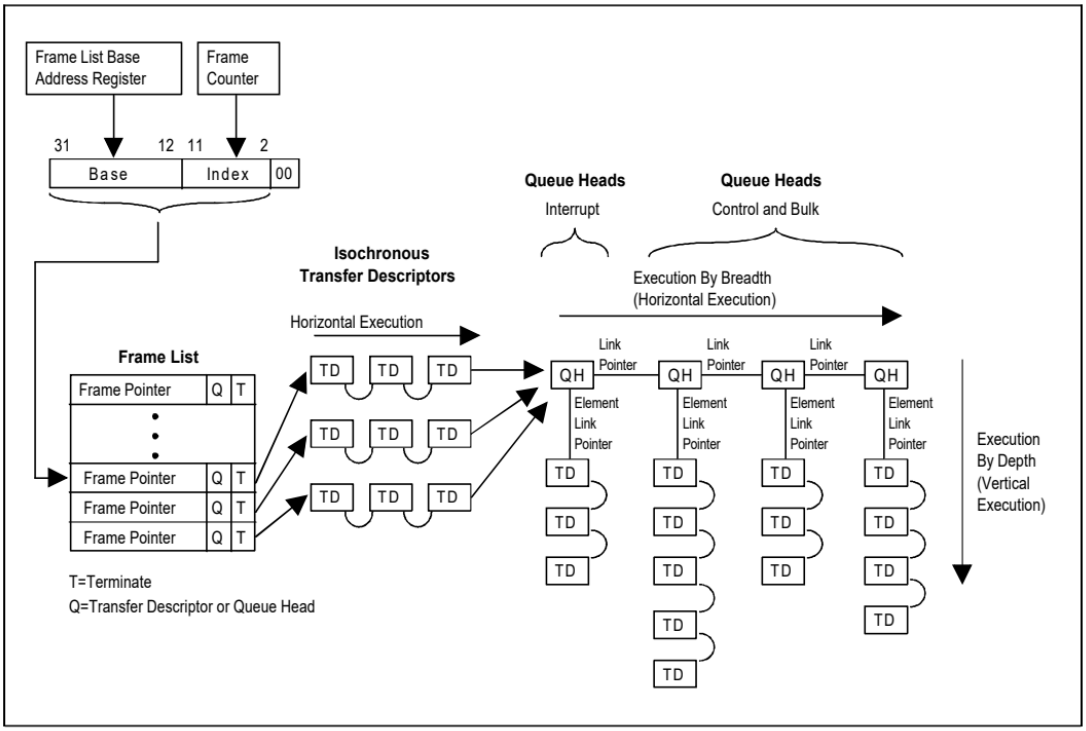
Другие шины связывают шину PCI с внешними устройствами.

USB

С точки зрения операционной системы USB-устройство состоит из конечных точек, объединённых в интерфейс. Конечная точка бывает входной или выходной и поэтому передаёт данные только в одном направлении. Существуют разные виды конечных точек:

- **Управляющие** используются для настройки устройства и подобных задач.
- **Прерываний** передают небольшие объёмы данных и имеют более высокий приоритет, чем массовые.
- **Массовые** передают большие объёмы данных, но не имеют гарантированных временных ограничений.
- **Изохронные передачи** имеют высокий приоритет и предназначены для работы в реальном времени, но пропущенные данные не передаются повторно. Это подходит для потокового видео или аудио, где повторная отправка уже не имеет смысла.

Интерфейсов, состоящих из нескольких конечных точек, может быть много. Интерфейсы объединяются в конфигурации, хотя у большинства устройств конфигурация одна.



На рисунке показана общая схема работы универсального интерфейса хост-контроллера UHCI, взятая из документации Intel. Она иллюстрирует, как программное и аппаратное

обеспечение совместно передают USB-данные из системы. По сути, программа создаёт данные определённого формата, которые хост-контроллер считывает и отправляет по шине USB.

В верхней левой части схемы находится регистр кадра со счётчиком, который увеличивается каждую миллисекунду. Это значение используется как индекс созданного программой списка кадров. Каждая запись таблицы указывает на очередь дескрипторов передачи. Программа размещает данные в памяти, а хост-контроллер — отдельная микросхема, управляющая USB-шиной, — считывает их. Программное обеспечение должно планировать очереди так, чтобы 90% времени кадра выделялось изохронным данным, а 10% оставалось для прерываний, управления и массовой передачи.

Как видно на схеме, дескрипторы изохронных передач связаны только с одним указателем кадра, то есть с одним временным интервалом, и затем отбрасываются. Данные прерываний, управления и массовой передачи ставятся в очередь после изохронных и, если не отправлены в одном кадре, могут передаваться в следующем.

Уровень USB обменивается запросами через блоки USB-запросов, или URB. URB содержит сведения о конечной точке, данные и связанные атрибуты, а также функцию обратного вызова, запускаемую по завершении. Драйверы USB передают URB фиксированного формата ядру USB, которое управляет ими совместно с хост-контроллером. Ядро отправляет ваши данные устройству USB и после завершения вызывает заданную функцию.

От небольших систем к большим

Как предсказывает закон Мура, вычислительная мощность долгое время быстро росла. Высокопроизводительные серверы редко содержат лишь один процессор. Несколько процессоров можно объединить различными способами.

Симметричная многопроцессорность

Симметричная многопроцессорность, обычно сокращаемая до SMP, — распространённая конфигурация с несколькими процессорами в одной системе.

Название «симметричная» означает, что процессоры системы одинаковы, например имеют одинаковую архитектуру и тактовую частоту. Они совместно используют остальные ресурсы системы: память, диски и так далее.

Когерентность кэша

По большей части процессоры работают независимо: у каждого свой набор регистров, счётчик инструкций и так далее. Но один компонент требует строгой синхронизации — кэш процессора. Как мы помним, кэш — небольшая быстрая память, отражающая значения в основной системной памяти. Если один процессор изменит данные, а в кэше другого останется старая копия, состояние системы окажется несогласованным. Проблема возникает при записи; если данные только читаются, они остаются согласованными.

Для поддержания когерентности кэшей SMP-система может использовать слежение за шиной (*snooping*). Процессор наблюдает за событиями кэша на общей для всех процессоров шине и соответствующим образом обновляет свой кэш.

Один из протоколов — MOESI: Modified (изменённая), Owner (владелец), Exclusive (исключительная), Shared (разделяемая), Invalid (недействительная). Это возможные состояния строки кэша процессора. Существуют и другие протоколы, но их идеи похожи. Рассмотрим MOESI, чтобы понять общий процесс.

Когда процессору нужна строка из основной памяти, он сначала выясняет, есть ли информация об этой области у других процессоров, например есть ли она в их кэше. Если ни у кого её нет, процессор загружает данные в кэш и помечает строку как исключительную. После записи строка становится изменённой. Дальнейшее зависит от политики кэша: при сквозной записи изменение сразу передаётся в память, а при обратной записи изменённое значение остаётся в кэше до вытеснения, например когда кэш заполнится.

В другом случае процессор обнаруживает значение в чужом кэше. Если оно изменено, он копирует данные к себе и помечает как разделяемые, а другому процессору сообщает, что его строка теперь находится в состоянии владельца.

Представим, что ту же память хочет использовать третий процессор. Он обнаружит разделяемую копию и копию владельца и получит данные от владельца. Пока все остальные процессоры только читают значение, строка остаётся разделяемой. Но когда одному из них потребуется её изменить, он отправит сообщение об аннулировании. Остальные процессоры должны пометить свои строки недействительными, поскольку те больше не отражают актуальное значение. Записывающий процессор помечает свою строку изменённой. Если она была исключительной, он знает, что других копий нет, и может обойтись без аннулирования.

Далее процесс повторяется. Ответственность за запись актуального изменённого значения в RAM при вытеснении лежит на процессоре, который им владеет. Если проследить протокол, станет видно, как он поддерживает согласованность строк между процессорами.

По мере увеличения числа процессоров возникают сложности. При небольшом числе затраты на проверку чужих копий и отправку аннулирований приемлемы, но с ростом системы увеличивается трафик шины. Поэтому автор отмечает, что SMP-системы с общей шиной обычно масштабировались примерно до восьми процессоров.

Общая шина создаёт и физические проблемы. Проводники имеют ограничения по длине и расположению. При частотах в несколько гигагерц скорость света становится существенным фактором времени распространения сообщений по системе.

Как правило, системное программное обеспечение непосредственно в этом процессе не участвует, но программисту полезно понимать происходящее в оборудовании, чтобы повышать производительность своих программ.

> **Примечание переводчика.** Ограничение примерно восемью процессорами относится к обсуждаемым автором системам с общей шиной, а не ко всем современным SMP-

> системам. Когерентность также может поддерживаться каталогами и другими механизмами.

Инклюзивные и эксклюзивные кэши в SMP

Ранее мы рассмотрели инклюзивные и эксклюзивные кэши. При инклюзивной организации данные L1 также содержатся в L2. В многопроцессорной системе это позволяет использовать L2 для отслеживания обращений к памяти и поддержания когерентности, поскольку изменения на нижнем уровне могут быть отражены и в L1. Это уменьшает сложность L1 и освобождает его от части работы по слежению.

Автор также отмечает распространённую в рассматриваемых им высокопроизводительных процессорах политику: сквозная запись для L1 и обратная для более низких уровней. Поскольку L2 в таких системах обычно находится на кристалле и достаточно быстр, стоимость сквозной записи L1 может быть приемлемой. Кроме того, небольшой L1 не заполняется изменёнными данными, которые, возможно, больше не будут прочитаны. Наконец, при сквозной записи L1 не нужно хранить грязные данные, и дополнительную логику когерентности можно перенести в L2.

> **Примечание переводчика.** Политика записи и инклюзивность зависят от конкретного процессора; современные L1-кэши нередко используют обратную запись. Инклюзивным обычно называют нижний, более крупный уровень, содержащий данные верхнего уровня.

Hyper-Threading

Современный процессор значительную часть времени ждёт, пока более медленные уровни иерархии памяти доставят данные. Поэтому важны способы поддерживать заполненность конвейера.

Один из них — включить достаточно регистров и логики состояния, чтобы одновременно обрабатывать два потока инструкций. Из-за этого один физический процессор выглядит как два логических.

Хотя каждый логический процессор имеет собственное архитектурное состояние, они совместно используют основные вычислительные блоки, кэш и пропускную способность связи с памятью. Два потока могут лучше загрузить ядро, но прирост обычно меньше, чем от двух независимых физических процессоров. Величина улучшения сильно зависит от нагрузки; приведённая автором оценка менее 20% помечена им как требующая проверки.

Многоядерные процессоры

Увеличение числа транзисторов на кристалле позволило разместить два или более процессорных ядра в одном корпусе. Распространённый вариант — двухъядерный процессор. В отличие от Hyper-Threading, ядра представляют собой полноценные процессоры и выглядят для системы как отдельные процессоры в SMP. Каждое ядро при этом может дополнительно поддерживать аппаратную многопоточность.

Обычно у ядер собственные кэши L1, но они разделяют связь с основной памятью и другими устройствами. Поэтому производительность зависит от общих ресурсов и не обязательно равна производительности полностью независимых систем.

У многоядерных процессоров есть и другие преимущества. Внешние шины между процессорами имеют физические ограничения; размещая ядра очень близко на одном кристалле, некоторые из этих проблем можно обойти. Энергопотребление может быть ниже, чем у двух отдельных процессоров, а значит, требуется отводить меньше тепла — важное преимущество для плотно заполненных дата-центров. Благодаря общему корпусу многопроцессорность становится практичной там, где раньше была неудобна, например в ноутбуках. Производство одного кристалла вместо двух также может быть дешевле.

Кластеры

Многим приложениям нужны системы крупнее, чем позволяет масштабирование SMP. Один из способов дальнейшего роста — кластер.

Кластер — набор отдельных компьютеров, способных обмениваться данными. На аппаратном уровне эти системы не знают друг о друге; объединение компьютеров оставлено программному обеспечению.

Например, MPI позволяет распределять части программы по другим компьютерам. Представим цикл, выполняющий тысячи независимых действий: ни одна итерация не влияет на другую. В кластере из четырёх компьютеров работа может быть поделена между ними поровну.

Соединение компьютеров может быть медленным, например через интернет, или быстрым и специализированным, например InfiniBand. Но оно всё равно обычно находится дальше по иерархии и намного медленнее локальной RAM. Поэтому кластер плохо подходит для ситуации, когда каждому процессору постоянно нужны данные из памяти другой машины: программному обеспечению придётся запрашивать копию, передавать её по сравнительно медленному каналу и загружать в локальную память, прежде чем процессор сможет продолжить работу.

Однако многим приложениям постоянное копирование не требуется. Крупный пример — SETI@Home, где данные радиотелескопа анализируются в поисках признаков внеземной жизни. Каждому компьютеру можно выдать несколько минут данных; после обработки он возвращает только сводку результатов. По сути, SETI@Home — очень большой специализированный кластер.

Другой пример — рендеринг изображений, особенно спецэффектов в кино. Каждому компьютеру можно поручить отдельный кадр с каркасными моделями, текстурами и источниками света, которые нужно объединить в готовое изображение. Поскольку кадр статичен, после получения исходных данных связь не нужна до отправки результата для включения в фильм. Например, спецэффекты «Властелина колец» создавались на большом кластере под Linux.

Неоднородный доступ к памяти

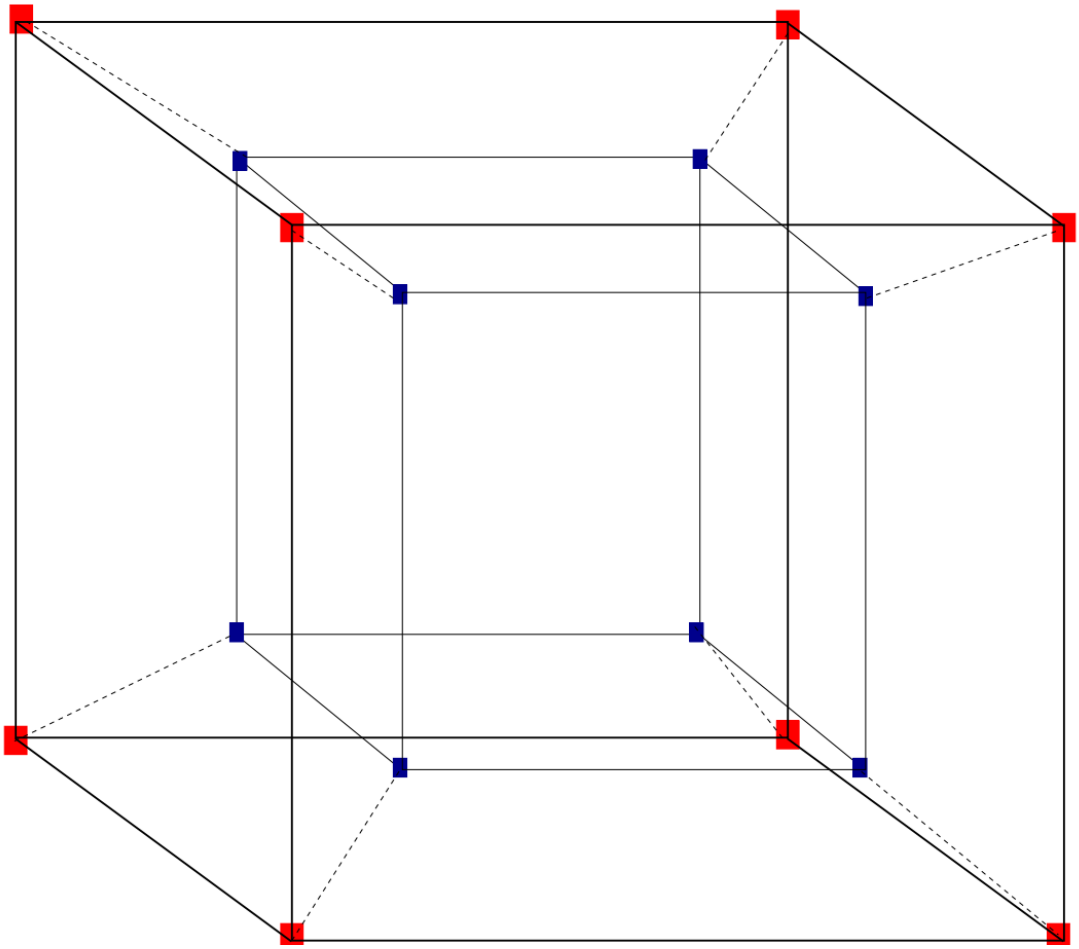
NUMA — архитектура с неоднородным доступом к памяти — почти противоположна описанному выше кластеру. Как и кластер, она состоит из отдельных связанных узлов, но связь между ними специализированная и дорогая. В кластере оборудование отдельных машин не знает об общем объединении; в NUMA большую часть работы по соединению узлов выполняет оборудование, поэтому программам требуется меньше знаний о расположении ресурсов.

Название означает, что RAM может находиться не рядом с процессором: за данными приходится обращаться к удалённому узлу. Это занимает больше времени, в отличие от упрощённой модели однопроцессорной или SMP-системы с одинаковым временем доступа к памяти.

Топология NUMA

Когда узлов много, крайне важно уменьшить расстояние между ними. Прямое соединение каждого узла со всеми остальными минимизировало бы расстояние, но становится непрактичным при сотнях и тысячах узлов крупных суперкомпьютеров. Для n узлов число попарных соединений равно `n(n – 1)/2`.

Для компромисса между расстоянием и числом соединений используют другие топологии. Одна из них — гиперкуб. У него есть строгое математическое определение; подобно тому как куб является трёхмерным аналогом квадрата, четырёхмерный гиперкуб является аналогом куба в четырёх измерениях.



На рисунке внешний куб содержит восемь узлов; между любыми двумя нужно пройти не более трёх рёбер. Добавив второй куб и соответствующие соединения, получаем вдвое больше узлов, а максимальная длина пути возрастает лишь до четырёх. Таким образом, при числе узлов 2^n максимальная длина пути растёт линейно с n.

> **Примечание переводчика.** Число связей в полном графе растёт квадратично, а не экспоненциально. Гиперкубы определяются не только в четырёх измерениях: размерность может быть произвольной.

Когерентность кэша NUMA

Когерентность кэшей можно поддерживать и в NUMA; такую систему называют ccNUMA. Как уже говорилось, широковещательная схема SMP плохо масштабируется до сотен и тысяч процессоров. Распространённая альтернатива — модель на основе каталогов. Процессоры обращаются к специальному аппаратному каталогу, который поддерживает согласованную картину данных и скрывает от процессоров детали NUMA.

В схеме каталога Сансие и Фотрие для каждого блока памяти хранится бит действительности для каждого процессора и один бит изменения. Когда процессор считывает блок в свой кэш, каталог устанавливает соответствующий ему бит действительности.

Если процессор хочет записать строку кэша, каталог устанавливает бит изменения и отправляет сообщение об аннулировании тем процессорам, которые используют строку. Поскольку сообщения направляются только процессорам с установленными битами, широковещательный трафик не требуется.

Если после этого другой процессор попытается прочитать блок, каталог увидит бит изменения. Он получит обновлённую строку от процессора, у которого установлен бит действительности, запишет изменённые данные в основную память и передаст их запросившему процессору, установив для него бит действительности. Для запрашивающего процессора эта работа прозрачна: данные могут поступить как из близкого, так и из далёкого места.

Один каталог, к которому обращаются тысячи процессоров, тоже плохо масштабируется. Поэтому схемы расширяют иерархией каталогов, обменивающихся данными по отдельному протоколу. Они могут использовать сеть общего назначения вместо процессорной шины, что позволяет строить гораздо более крупные системы.

Применение NUMA

NUMA хорошо подходит для задач с интенсивным взаимодействием процессоров и памяти. Например, при моделировании погоды среду делят на небольшие ячейки с разными свойствами: океан и суша по-разному отражают и сохраняют тепло. В модель вводят небольшие изменения и наблюдают общий результат. Каждая ячейка влияет на соседние, поэтому требуется много обмена данными. Это отличается от рендеринга отдельных кадров, которые друг на друга не влияют. Аналогичная ситуация возникает при моделировании автомобильной аварии: каждая ячейка модели автомобиля деформируется и поглощает часть энергии.

Хотя программа может не знать напрямую, что работает в NUMA-системе, для максимальной производительности разработчикам важно учитывать устройство системы. Лучше держать память близко к процессору, который её использует. Профилирование помогает изучить выполняемые пути кода и последствия его работы для системы.

Порядок операций с памятью, блокировки и атомарные операции

Многоуровневый кэш и суперскалярная многопроцессорная архитектура создают интересные вопросы о том, как программист видит выполнение кода процессором.

Представим, что код одновременно выполняется на двух процессорах, разделяющих большую область памяти. Если один процессор выполняет инструкцию записи значения регистра в память, когда можно быть уверенным, что другой процессор при чтении того же адреса увидит правильное значение?

В простейшем случае система могла бы гарантировать: после инструкции записи любое последующее чтение увидит это значение. Это называют строгим порядком памяти,

поскольку правила не оставляют места для перестановок. Нетрудно понять, почему такая схема может серьёзно снижать производительность.

Часто настолько строгий порядок не нужен. Программист может определить точки, в которых необходимо убедиться, что все ожидающие операции видны глобально, а между ними выполнять множество инструкций без таких ограничений.

Рассмотрим следующую ситуацию.

Пример 3.1. Порядок операций с памятью

```
typedef struct {
 int a;
 int b;
} a_struct;

/*
 * Pass in a pointer to be allocated as a new structure
 */
void get_struct(a_struct *new_struct)
{
 void *p = malloc(sizeof(a_struct));

 /* We don't particularly care what order the following two
  * instructions end up acutally executing in */
 p->a = 100;
 p->b = 150;

 /* However, they must be done before this instruction.
  * Otherwise, another processor who looks at the value of p
  * could find it pointing into a structure whose values have
  * not been filled out.
  */
 new_struct = p;
}
```

Здесь две записи можно выполнить в любом порядке, удобном процессору. Однако указатель должен обновляться только после того, как обе предыдущие записи действительно выполнены. Иначе другой процессор может увидеть указатель, обратиться по нему к памяти и получить неверные значения.

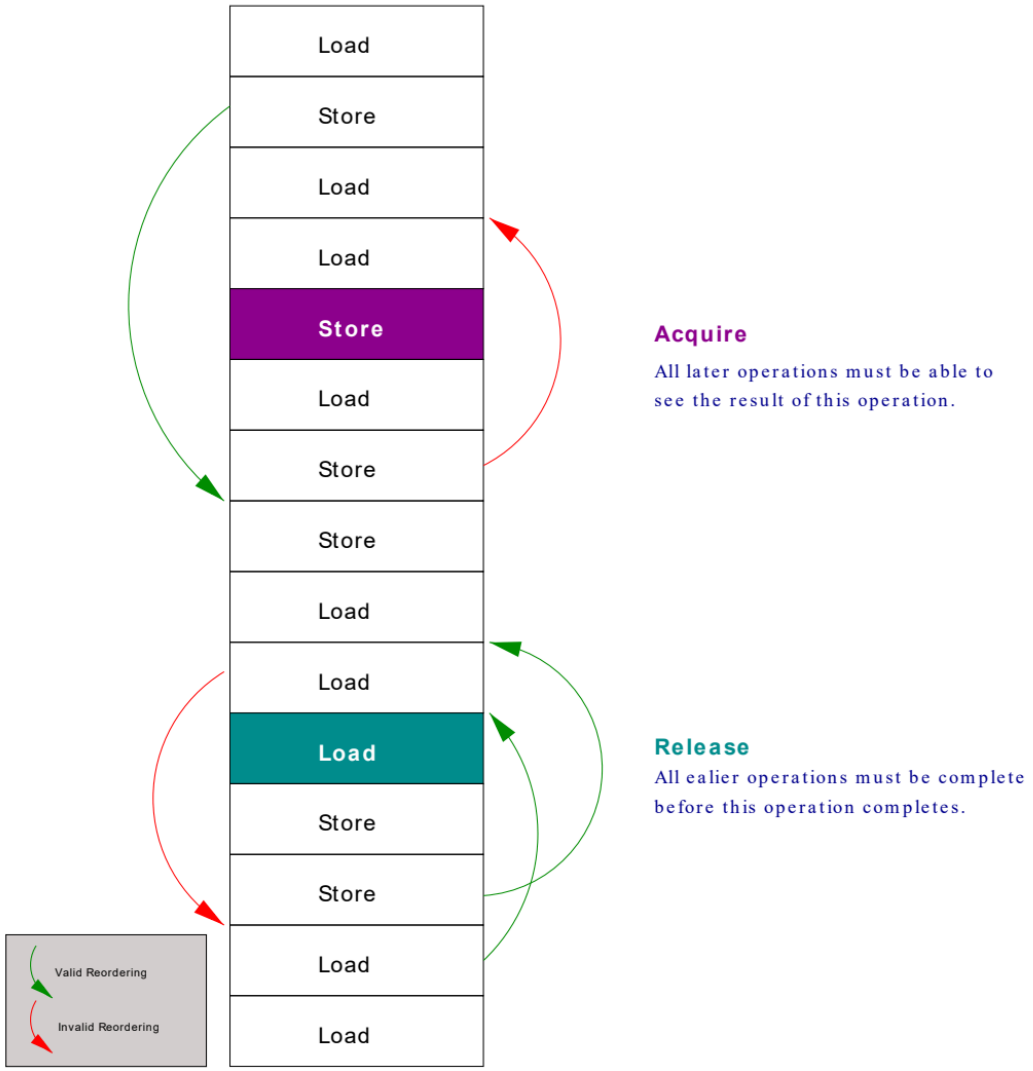
> **Примечание переводчика.** Код в оригинале лишь иллюстрирует замысел и как программа на C некорректен: `p` имеет тип `void *`, поэтому обращения `p->a` и `p->b` недопустимы; присваивание локальному параметру `new_struct` не передаёт указатель вызывающей стороне. Без корректной межпоточной синхронизации пример также не обеспечивает безопасной публикации данных.

Чтобы выразить требуемый порядок, операции чтения и записи снабжаются семантикой, описывающей их поведение. Её объясняют через барьеры, определяющие, как операции могут переупорядочиваться относительно данной операции.

В отсутствие дополнительных ограничений операции могут переупорядочиваться в пределах, разрешённых моделью памяти и обеспечением наблюдаемого поведения программы.

Семантика **acquire** похожа на односторонний барьер: последующие операции чтения и записи не должны перемещаться перед ней. Это позволяет коду после успешного приобретения наблюдать данные, опубликованные соответствующей операцией release.

Семантика **release** действует в противоположном направлении: предыдущие операции не должны перемещаться после неё. При публикации значения release позволяет сделать предшествующие записи видимыми операции acquire, которая с ней синхронизируется.



Полный барьер памяти сочетает ограничения в обоих направлениях: операции чтения и записи не должны пересекать его в запрещённом порядке. В строгой модели барьер можно было бы использовать для каждой операции; в слабой многие операции допускают переупорядочивание.

> **Примечание переводчика.** Само наличие acquire или release ещё не гарантирует, что произвольный поток увидит любые изменения: нужна соответствующая связь синхронизации по правилам применяемой модели памяти.

Процессоры и модели памяти

Разные процессоры реализуют разные модели памяти. Автор описывает x86 и AMD64 как архитектуры со сравнительно строгой моделью и отмечает, что префикс `lock` обеспечивает барьер памяти. Itanium допускает более свободное упорядочивание и требует явно указывать необходимые ограничения. В оригинале эти замечания оставлены краткими и помечены для доработки.

> **Примечание переводчика.** Точные гарантии зависят от инструкции, архитектурной спецификации и модели памяти языка. Для переносимого кода следует использовать стандартные атомарные операции и средства синхронизации, а не полагаться на упрощённое описание отдельных процессоров.

Блокировки

Знать требования к порядку памяти для каждой архитектуры непрактично для всех программистов; кроме того, такие программы было бы трудно переносить и отлаживать на разных процессорах.

Чтобы программы могли работать одновременно на нескольких процессорах, используют более высокий уровень абстракции — блокировки.

Когда поток захватил блокировку, другой поток не может получить её до освобождения. Перед входом в защищённый критический участок поток должен попытаться захватить блокировку; если это не удаётся, он не продолжает выполнение этого участка.

Здесь видна связь с названиями семантик памяти. При захвате нужно не допустить перемещения операций защищённого участка до блокировки — это роль acquire. При освобождении нужно обеспечить упорядочение выполненных под блокировкой операций перед её освобождением — это роль release.

Многие программные библиотеки позволяют не заботиться о деталях семантики памяти и использовать высокоуровневые операции вроде `lock()` и `unlock()`.

Трудности с блокировками

Блокировки усложняют программирование, поскольку могут приводить к взаимной блокировке, или deadlock. Представим, что один процессор удерживает блокировку данных и ждёт другую. Если второй процессор не освобождает свою блокировку, пока не получит первую, возникает взаимная блокировка: каждый ждёт другого и никто не может продолжить.

Часто такая ситуация возникает из-за тонкого состояния гонки — одной из самых трудных для поиска ошибок. Если два процессора рассчитывают на определённый временной порядок операций, возможна гонка. Даже случайная задержка может нарушить ожидаемый порядок. В результате может возникнуть описанная взаимная блокировка. Именно поэтому порядок должен обеспечиваться соответствующими гарантиями, а не ожиданием, что события всегда будут происходить с определённой скоростью.

Похожая ситуация — активная взаимная блокировка, или livelock. Например, можно придумать «вежливую» блокировку, которую поток уступает каждому попросившему. Два потока могут бесконечно уступать её друг другу, так и не удержав достаточно долго для выполнения работы. В жизни это похоже на двух людей в дверях, каждый из которых настаивает: «Нет, сначала вы!» В итоге никто не проходит.

Способы реализации блокировок

На нижнем уровне существует много способов реализовать поведение блокировок.

Простую блокировку с двумя состояниями, «занята» и «свободна», называют мьютексом — сокращением от *mutual exclusion*, взаимного исключения: если ресурс занят одним участником, другой не может его одновременно захватить.

Мьютекс можно реализовать по-разному. Простейший вариант — спинлок: процессор в тесном цикле постоянно проверяет, не освободилась ли блокировка. Это похоже на ребёнка, который бесконечно спрашивает родителя: «А теперь можно?»

Недостаток в том, что время тратится впустую: пока процессор постоянно проверяет блокировку, полезной работы он не выполняет. Для блокировок, удерживаемых совсем недолго, это может быть оправданно, но во многих случаях ожидание длится значительно дольше.

Другой подход — заснуть в ожидании блокировки. Если получить её не удалось, процессор выполняет другую работу, пока ожидающий поток не получит уведомление об освобождении. В следующих главах мы увидим, как операционная система переключает процессы и находит для процессора другую работу.

Мьютекс можно рассматривать как частный случай семафора — известного механизма, придуманного нидерландским учёным Эдсгером Дейкстрой. Когда имеется несколько одинаковых ресурсов, семафор может учитывать число доступных обращений к ним. При одном ресурсе получается поведение, похожее на мьютекс. Подробнее работа семафоров описывается в книгах по алгоритмам.

У этих схем остаются сложности. Часто многие участники лишь читают данные, которые обновляются редко. Если все читатели должны захватывать одну блокировку, возникает конкуренция за неё: все ждут доступа к общим данным, а полезной работы выполняется меньше.

> **Примечание переводчика.** Двоичный семафор и мьютекс похожи, но не всегда взаимозаменяемы: например, у мьютекса обычно есть понятие владельца.

Атомарные операции

В исходнике этот раздел содержит только авторскую пометку «объяснить, что это такое» и не содержит готового объяснения.

Глава 4. Операционная система

Роль операционной системы

Операционная система лежит в основе всей работы современного компьютера.

Абстрагирование оборудования

Главная задача операционной системы (ОС) — скрыть особенности оборудования от программиста и пользователя. Она предоставляет общие интерфейсы к возможностям аппаратуры.

В мире без операционных систем каждому программисту пришлось бы знать мельчайшие подробности устройства оборудования, чтобы запустить хоть что-нибудь. Более того, программы не работали бы на другом оборудовании, даже если бы различия были невелики.

Многозадачность

Мы ожидаем, что современные компьютеры делают много разных вещей одновременно, поэтому нужен способ согласовывать работу всех запущенных программ. Операционная система должна обеспечивать это незаметно для пользователя.

ОС отвечает за распределение ресурсов системы. Множество задач соперничают за процессорное время, память, диск и пользовательский ввод. Операционная система распределяет ресурсы между задачами и обеспечивает упорядоченный доступ к ним. Вы, вероятно, сталкивались с последствиями ошибки в этом механизме: компьютер обычно аварийно завершает работу, например показывает знаменитый «синий экран смерти».

Стандартизированные интерфейсы

Программисты хотят создавать программы, работающие на как можно большем количестве аппаратных платформ. Поддержка стандартизированных интерфейсов операционной системой помогает им в этом.

Например, если функция открытия файла в одной системе называется `open()`, в другой — `open_file()`, а в третьей — `openf()`, программистам приходится помнить особенности каждой системы, а программы не могут работать сразу на всех.

Portable Operating System Interface (POSIX) — очень важный стандарт, реализуемый UNIX-подобными системами. В Microsoft Windows существуют похожие проприетарные стандарты.

> **Примечание автора.** Буква X происходит от Unix, из которого вырос стандарт. На момент написания POSIX соответствует Single UNIX Specification Version 3 и ISO/IEC 9945:2002; стандарт свободно доступен в интернете. Ранее Single UNIX Specification публиковалась консорциумом Open Group бесплатно, а стандарты IEEE POSIX серии IEEE 1003 распространялись отдельно. IEEE 1003.1-2001 эквивалентен SUS Version 3; в начале 2002 года стандарты были объединены и также стандартизированы ISO как ISO/IEC 9945:2002.

Безопасность

В многопользовательских системах безопасность особенно важна. Управляя доступом к системе, ОС должна гарантировать, что ресурсы доступны только обладателям соответствующих прав.

Например, если файл принадлежит одному пользователю, другой не должен иметь возможность открыть и прочитать его без разрешения. При этом нужны и механизмы безопасного совместного доступа, если пользователи этого хотят.

Операционные системы — большие и сложные программы, в которых обнаруживаются проблемы безопасности. Вирус или червь может использовать такую ошибку для доступа к недоступным ему ресурсам, например к вашим файлам или сетевому соединению. Для защиты необходимо устанавливать исправления и обновления поставщика ОС.

Производительность

Операционная система предоставляет компьютеру множество услуг, поэтому её производительность критична. Многие её части выполняются чрезвычайно часто: даже дополнительные несколько тактов процессора могут в сумме значительно снизить общую скорость.

ОС должна использовать особенности оборудования, чтобы операции выполнялись как можно быстрее. Поэтому системным программистам необходимо глубоко понимать архитектуру, под которую они разрабатывают.

Часто работа системного программиста состоит в выборе политики работы системы. Ускорение одной части ОС может замедлить другую или снизить её эффективность. При разработке необходимо понимать эти компромиссы.

Устройство операционной системы

Общая организация показана на рисунке.

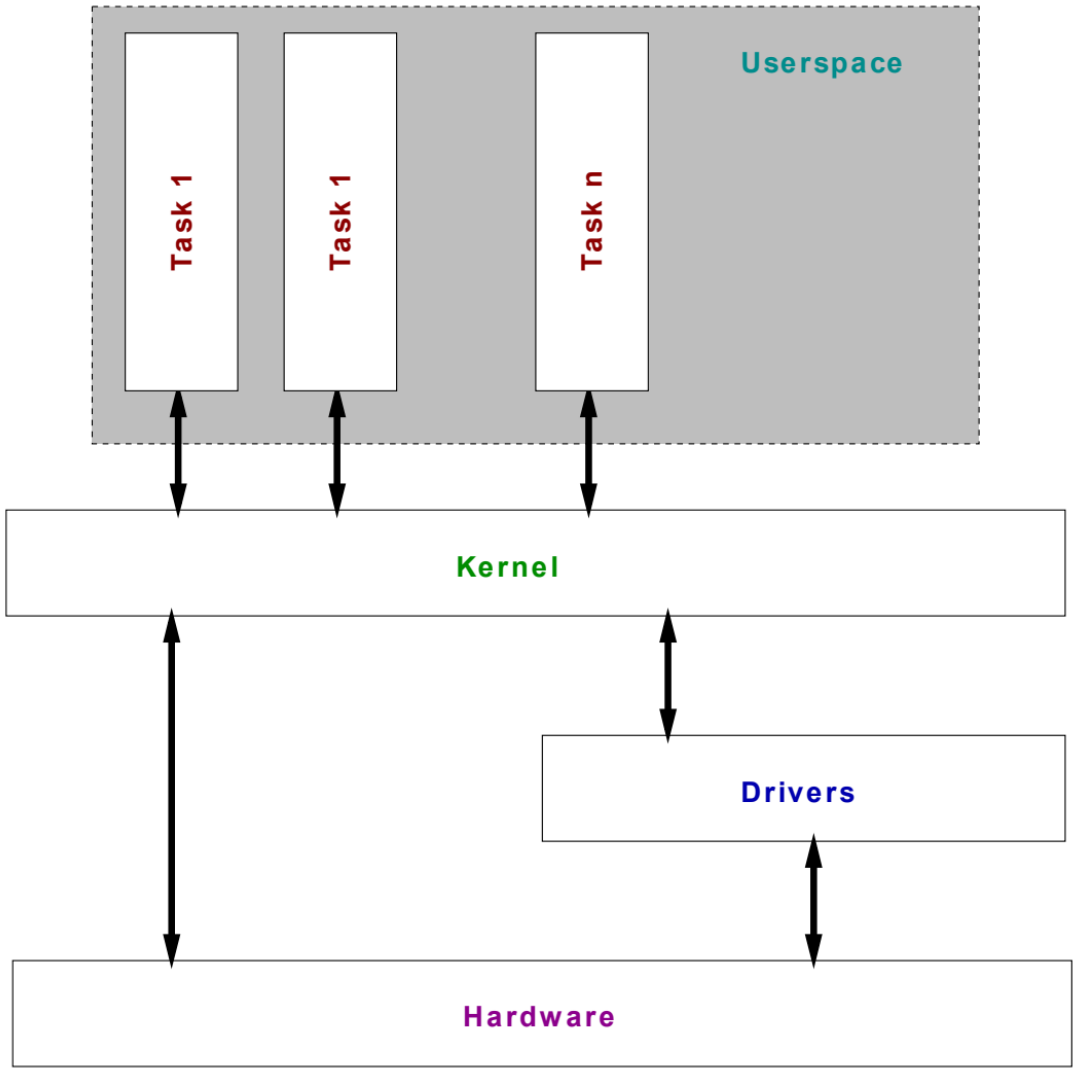


Рисунок 4.1. Устройство операционной системы. Процессы работают в пользовательском пространстве; ядро взаимодействует с оборудованием напрямую и через драйверы.

Ядро

Ядро — это операционная система. Как показано на рисунке, оно взаимодействует с оборудованием непосредственно и через драйверы.

Подобно тому как ядро предоставляет пользовательским программам абстракцию оборудования, драйверы предоставляют её ядру. Например, существует много видеокарт с немного различающимися возможностями. Если ядро предоставляет API, разработчики, имеющие спецификацию устройства, могут написать реализующий этот API драйвер. Так ядро получает доступ к множеству разных устройств.

Ядро обычно работает в *привилегированном* режиме. Аппаратура играет важную роль в выполнении нескольких задач и защите системы, но ограничения, применяемые к обычным программам, на ядро не распространяются. Ядро должно справляться с аварийным завершением программ: ОС распределяет ресурсы между несколькими программами, и нет гарантии, что те будут вести себя правильно. Но если откажет внутренняя часть самой ОС,

скорее всего вся система станет неработоспособной. Аналогично уязвимость может позволить пользовательскому процессу получить привилегии ядра и затем бесконтрольно обращаться к любой части системы.

Монолитные ядра и микроядра

Часто обсуждается вопрос, должно ли ядро быть монолитным или микроядром.

Монолитный подход распространённее и используется большинством популярных Unix-систем, включая Linux. В этой модели привилегированное ядро велико: оно включает драйверы, управление доступом к файловым системам, проверку прав и службы вроде Network File System (NFS).

Поскольку ядро всегда привилегировано, авария любой его части может остановить всю систему. Ошибка одного драйвера способна беспрепятственно перезаписать любую память и в конечном счёте привести к падению.

Архитектура микроядра старается уменьшить риск, делая привилегированную часть как можно меньше. Большинство компонентов работают как непривилегированные программы, поэтому ущерб от сбоя отдельного компонента ограничен. Например, драйверы могут выполняться в отдельных процессах: сбойный драйвер не сможет перезаписать память за пределами выделенной ему области.

Хотя такой подход кажется очевидным, возникают две основные проблемы:

1. Снижается производительность: взаимодействие множества отдельных компонентов стоит времени.
2. Программирование становится несколько сложнее.

Причина обеих проблем в том, что для разделения компонентов большинство микроядер применяет передачу сообщений — межпроцессное взаимодействие, или IPC. Сообщение нужно упаковать, отправить другому компоненту, распаковать, обработать, снова упаковать, отправить обратно и распаковать результат.

Для сравнительно простого запроса это много шагов. Один запрос может вызвать дальнейшие запросы к другим компонентам, и проблема умножается. Медленная передача сообщений во многом объясняла низкую производительность ранних микроядер; сама модель также несколько сложнее для программистов. Дополнительной защиты, которую давало разделение компонентов, оказалось недостаточно, чтобы компенсировать эти недостатки, и ранние микроядра вышли из моды.

В монолитном ядре взаимодействие компонентов происходит посредством обычных, знакомых каждому программисту вызовов функций.

Окончательного ответа на вопрос о лучшей организации нет; он породил множество споров как в научной среде, так и за её пределами. Надеюсь, изучив операционные системы подробнее, вы сможете составить собственное мнение!

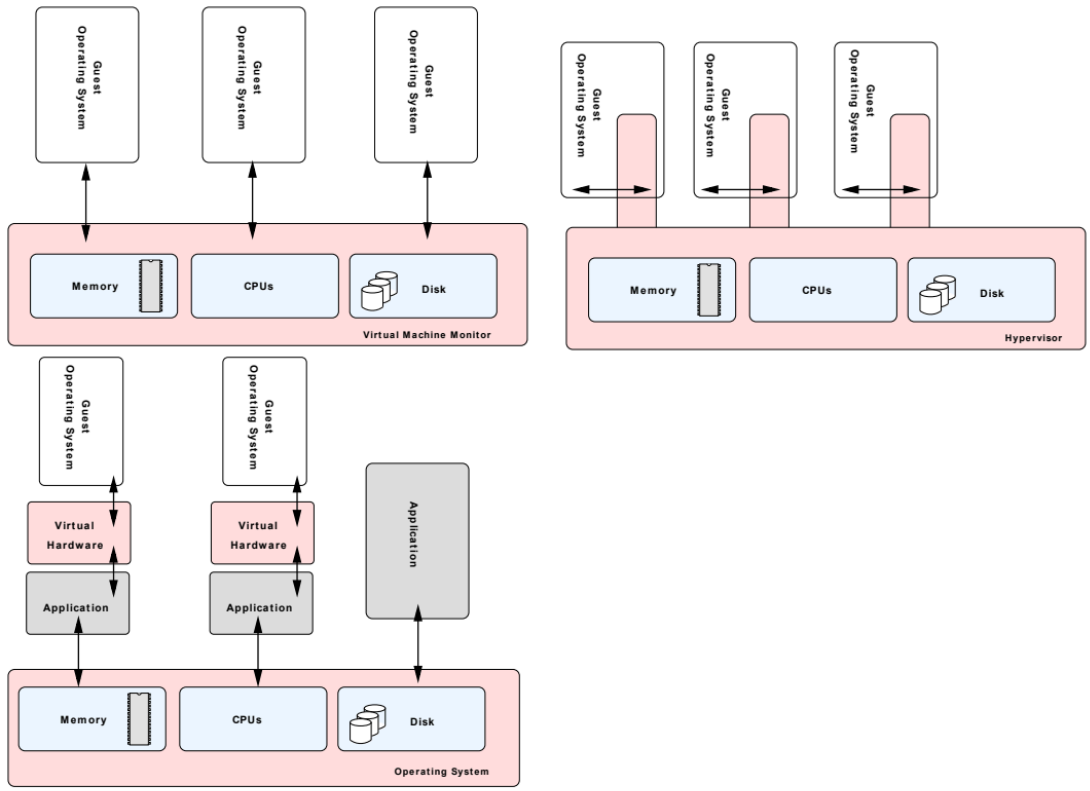
Модули

В ядре Linux реализована система модулей: драйверы могут загружаться в работающее ядро «на лету», по мере необходимости. Это удобно, поскольку драйверы, составляющие большую часть кода ОС, не загружаются для отсутствующих устройств. Разработчик наиболее универсального ядра, работающего на множестве аппаратных платформ, как в Red Hat или Debian, может включить большинство драйверов в виде модулей, загружаемых только при наличии соответствующего оборудования.

Однако модули загружаются непосредственно в привилегированное ядро и работают с теми же правами, поэтому ядро по-прежнему считается монолитным.

Виртуализация

С ядром тесно связано понятие виртуализации оборудования. Современные компьютеры очень мощны, и иногда полезно рассматривать один физический компьютер как несколько отдельных виртуальных машин. Каждая выглядит совершенно самостоятельной, хотя физически все они находятся в одном корпусе.



Some different virtualisation methods.

Рисунок 4.2. Различные способы виртуализации.

Это можно организовать по-разному. В простейшем случае небольшой монитор виртуальных машин работает непосредственно на оборудовании и предоставляет интерфейс гостевым ОС. Такой монитор часто называется *гипервизором* (от слова supervisor). Гостевая ОС может даже не знать о его существовании, поскольку он представляет ей видимость полноценной

системы. Гипервизор перехватывает взаимодействие между гостевой ОС и оборудованием и предоставляет каждой лишь часть ресурсов.

> Гипервизор во многом похож на микроядро: оба стремятся быть небольшим слоем, безопасно предоставляющим оборудование вышележащим уровням.

Такой подход часто используется на больших машинах с множеством процессоров и большим объёмом ОЗУ для разделения на виртуальные машины. Нередко работающим системам можно добавлять ресурсы на лету, когда возникает потребность. Гипервизоры многих больших машин IBM весьма сложны, содержат миллионы строк кода и предоставляют множество служб управления системой.

Другой вариант — ОС знает о гипервизоре и запрашивает ресурсы через него. Такой промежуточный подход иногда называют паравиртуализацией. Он похож на работу ранних версий Xen и является компромиссом. Ожидается более высокая производительность, поскольку ОС явно запрашивает ресурсы, а гипервизору не приходится определять потребности динамически.

Наконец, приложение в существующей ОС может представлять виртуальную систему с процессором, памятью, BIOS и диском, на которой запускается обычная ОС. Приложение преобразует запросы к оборудованию в обращения к реальному оборудованию через основную ОС, как в VMware. Такой подход создаёт значительные накладные расходы: процесс приложения эмулирует целую систему и преобразует все запросы. Однако он позволяет эмулировать совершенно другую архитектуру, динамически переводя инструкции одного процессора в инструкции другого, как это делала система Rosetta при переходе программ Apple с PowerPC на Intel.

Производительность важна при любом способе виртуализации: операции, прежде выполнявшиеся непосредственно на оборудовании, проходят через слои абстракции.

Intel обсуждала аппаратную поддержку виртуализации в новых процессорах. Эти расширения вызывают специальное исключение при операциях, требующих вмешательства монитора виртуальных машин. Для приложения процессор выглядит обычным, но при запросе ресурсов, которые могут разделяться между гостевыми ОС, вызывается монитор.

Производительность повышается, поскольку монитору не нужно проверять каждую операцию: он ждёт уведомления процессора о потенциально небезопасном действии.

Скрытые каналы

Это небольшое отступление об интересной проблеме безопасности виртуальных машин. Если разделение ресурсов системы динамическое, а не статическое, возникает потенциальная уязвимость.

В динамической системе ресурсы выделяются гостевым ОС по потребности. Если одна выполняет вычислительно тяжёлую задачу, пока другая ждёт данные с диска, первая получит больше процессорного времени. При статическом разделении каждая получила бы по 50%, а неиспользованная доля пропала бы.

Динамическое распределение открывает канал связи между двумя ОС. Везде, где можно различить два состояния, возможна передача двоичных данных. Представим две полностью защищённые системы, между которыми никогда не должна проходить информация. Два человека, имеющие к ним доступ, могут сговориться и написать программы, одновременно запрашивающие большое количество ресурсов.

Когда одна программа занимает много памяти, другой остаётся меньше. Отслеживая максимальный объём выделяемой памяти, можно передать бит. Допустим, программы договорились каждую секунду проверять возможность выделить большой блок: успех означает двоичный ноль, а неудача — единицу, поскольку другая машина заняла память. Скорость один бит в секунду невелика, но информация всё же передаётся.

Такой механизм называется скрытым каналом. Хотя пример выглядит надуманным, подобные механизмы действительно приводили к нарушениям безопасности. Жизнь системного программиста никогда не бывает простой!

Пользовательское пространство

Пользовательским пространством называют условную среду, в которой выполняются пользовательские программы. Каждая взаимодействует с ядром посредством системных вызовов, описанных ниже.

Как уже говорилось, пользовательское пространство непривилегированно. Программы могут выполнять лишь ограниченный набор действий и не должны приводить к отказу других программ, даже если сами аварийно завершаются.

Системные вызовы

Обзор

Системные вызовы — способ взаимодействия программ пользовательского пространства с ядром. Ниже описан общий принцип их работы.

Номера системных вызовов

Каждый системный вызов имеет номер, известный и пользовательскому пространству, и ядру. Например, обе стороны могут знать, что номер 10 обозначает `open()`, номер 11 — `read()` и так далее.

Двоичный интерфейс приложения (ABI) похож на API, но описывает соглашения на уровне двоичного кода и оборудования. ABI определяет, в какой регистр нужно поместить номер системного вызова, чтобы ядро могло его найти.

Аргументы

Без аргументов системные вызовы мало полезны: например, `open()` должен сообщить ядру, какой файл открыть. ABI также определяет регистры, в которые помещаются аргументы.

Ловушка

Чтобы выполнить системный вызов, необходимо сообщить ядру о намерении программы. Каждая архитектура определяет инструкцию, обычно называемую `break` или подобным образом, которая подаёт аппаратуре такой сигнал.

Эта инструкция изменяет указатель команд так, чтобы он указывал на обработчик системных вызовов ядра. При запуске ОС сообщает аппаратуре адрес обработчика. После инструкции `break` пользовательская программа теряет управление, и оно передаётся ядру.

Далее ядро читает номер вызова из заранее определённого регистра, находит соответствующую функцию в таблице и вызывает её. Функция выполняет необходимую работу и помещает результат в регистр возврата, определённый ABI.

На последнем шаге ядро передаёт управление обратно в пользовательскую программу, чтобы та продолжила работу с прежнего места. Программа получает данные из регистра возврата и спокойно продолжает выполнение. Несмотря на возможную сложность подробностей, в основе системного вызова лежит именно это.

libc

Хотя все описанные действия можно выполнять вручную для каждого системного вызова, системные библиотеки обычно делают большую часть работы за вас. В UNIX-подобных системах за это отвечает стандартная библиотека `libc`; подробнее о её роли мы поговорим позже.

Разбор системного вызова

Поскольку системные библиотеки обычно выполняют вызовы за нас, для демонстрации их устройства придётся немного поработать на низком уровне. Рассмотрим, вероятно, самый простой вызов — `getpid()`. Он не принимает аргументов и возвращает идентификатор текущей программы, или процесса; подробнее о процессах поговорим позже.

Пример 4.1. Вызов `getpid()`

```
                              #include <stdio.h>

                 /* for syscall() */
#include <sys/syscall.h>
                 #include <unistd.h>

                 /* system call numbers */
                 #include <asm/unistd.h>
                 void function(void)
                 {
                  int pid;

pid = __syscall(__NR_getpid);
                 }
```

Начнём с небольшой программы на C, которая поможет показать механизм системных вызовов. Системные библиотеки предоставляют `syscall` для непосредственного выполнения вызова. Программист может использовать её, не зная точной последовательности ассемблерных инструкций для своей аппаратуры. Зачем же нужен `getpid()`? Во-первых, понятнее использовать в коде символическое имя функции. Во-вторых, на разных системах `getpid()` может работать по-разному. Например, в описываемой автором версии Linux результат мог кэшироваться: при повторном вызове библиотеке не требовалось снова обращаться к ядру за тем же значением.

По соглашению Linux номера системных вызовов определяются в файле исходного кода ядра `asm/unistd.h`. Поскольку файл находится в подкаталоге `asm`, он различается между архитектурами. Имена констант определяются через `#define` с префиксом `__NR_`. Наш код выполняет системный вызов `getpid` и сохраняет значение в `pid`.

Рассмотрим, как несколько архитектур реализуют этот код. Перед нами реальный код, поэтому местами будет сложно. Но не сдавайтесь — именно так работает ваша система!

PowerPC

PowerPC — архитектура RISC, использовавшаяся в старых компьютерах Apple и лежащая в основе некоторых версий Xbox.

Пример 4.2. Системный вызов PowerPC

```
                /* On powerpc a system call basically clobbers the same registers like a
                 * function call, with the exception of LR (which is needed for the
* "sc; bnslr" sequence) and CR (where only CR0.SO is clobbered to signal




     * an error return status).
     */

    #define __syscall_nr(nr, type, name, args...)    \
unsigned long __sc_ret, __sc_err;    \
     {        \
      register unsigned long __sc_0  __asm__ ("r0");  \
      register unsigned long __sc_3  __asm__ ("r3");  \
      register unsigned long __sc_4  __asm__ ("r4");  \
register unsigned long __sc_5  __asm__ ("r5");  \
      register unsigned long __sc_6  __asm__ ("r6");  \
      register unsigned long __sc_7  __asm__ ("r7");  \
             \
      __sc_loadargs_##nr(name, args);    \
__asm__ __volatile__     \
       ("sc           \n\t"    \
        "mfcr %0      "    \
       : "=&r" (__sc_0),    \
         "=&r" (__sc_3),  "=&r" (__sc_4),  \
"=&r" (__sc_5),  "=&r" (__sc_6),  \
         "=&r" (__sc_7)    \
       : __sc_asm_input_##nr    \
       : "cr0", "ctr", "memory",   \
         "r8", "r9", "r10","r11", "r12");  \
__sc_ret = __sc_3;     \
      __sc_err = __sc_0;     \
```

```
     }        \
     if (__sc_err & 0x10000000)     \
     {        \
errno = __sc_ret;     \
      __sc_ret = -1;      \
     }        \
     return (type) __sc_ret

#define __sc_loadargs_0(name, dummy...)     \
     __sc_0 = __NR_##name
    #define __sc_loadargs_1(name, arg1)     \
     __sc_loadargs_0(name);      \
     __sc_3 = (unsigned long) (arg1)
#define __sc_loadargs_2(name, arg1, arg2)    \
     __sc_loadargs_1(name, arg1);     \
     __sc_4 = (unsigned long) (arg2)
    #define __sc_loadargs_3(name, arg1, arg2, arg3)    \
     __sc_loadargs_2(name, arg1, arg2);    \
__sc_5 = (unsigned long) (arg3)
    #define __sc_loadargs_4(name, arg1, arg2, arg3, arg4)   \
     __sc_loadargs_3(name, arg1, arg2, arg3);   \
     __sc_6 = (unsigned long) (arg4)
    #define __sc_loadargs_5(name, arg1, arg2, arg3, arg4, arg5)  \
__sc_loadargs_4(name, arg1, arg2, arg3, arg4);   \
     __sc_7 = (unsigned long) (arg5)

    #define __sc_asm_input_0 "0" (__sc_0)
    #define __sc_asm_input_1 __sc_asm_input_0, "1" (__sc_3)




#define __sc_asm_input_2 __sc_asm_input_1, "2" (__sc_4)
    #define __sc_asm_input_3 __sc_asm_input_2, "3" (__sc_5)
    #define __sc_asm_input_4 __sc_asm_input_3, "4" (__sc_6)
    #define __sc_asm_input_5 __sc_asm_input_4, "5" (__sc_7)

#define _syscall0(type,name)      \
    type name(void)        \
    {         \
     __syscall_nr(0, type, name);     \
    }
    #define _syscall1(type,name,type1,arg1)     \
    type name(type1 arg1)       \
    {         \
     __syscall_nr(1, type, name, arg1);    \
}

    #define _syscall2(type,name,type1,arg1,type2,arg2)   \
    type name(type1 arg1, type2 arg2)     \
    {         \
__syscall_nr(2, type, name, arg1, arg2);   \
    }

    #define _syscall3(type,name,type1,arg1,type2,arg2,type3,arg3)  \
    type name(type1 arg1, type2 arg2, type3 arg3)    \
{         \
     __syscall_nr(3, type, name, arg1, arg2, arg3);   \
    }

    #define _syscall4(type,name,type1,arg1,type2,arg2,type3,arg3,type4,arg4) \
type name(type1 arg1, type2 arg2, type3 arg3, type4 arg4)  \
    {         \
     __syscall_nr(4, type, name, arg1, arg2, arg3, arg4);  \
    }

#define _syscall5(type,name,type1,arg1,type2,arg2,type3,arg3,type4,arg4,type5,arg5) \
```

```
type name(type1 arg1, type2 arg2, type3 arg3, type4 arg4, type5 arg5) \
{         \
 __syscall_nr(5, type, name, arg1, arg2, arg3, arg4, arg5); \
}
```

Этот фрагмент заголовочного файла ядра `asm/unistd.h` показывает реализацию системных вызовов на PowerPC. Он выглядит сложно, но его можно разобрать по шагам.

Перейдём к концу примера, где определены макросы `_syscallN`. Каждый следующий принимает на один аргумент больше. Начнём с простейшего `_syscall0`: он принимает тип возвращаемого значения, например C `int` или `char`, и имя вызова. Для `getpid` получаем `_syscall0(int,getpid)`.

Пока просто! Теперь разберём макрос `__syscall_nr`. Его первый параметр — число аргументов, затем следуют тип, имя и сами аргументы.

Сначала объявляются имена регистров: например, `__sc_0` обозначает `r0`, то есть регистр 0. Компилятор обычно использует регистры по своему усмотрению, поэтому необходимо задать ограничения, чтобы он случайно не занял нужный нам регистр.

Затем вызывается `__sc_loadargs` с интересным оператором `##`, который склеивает части токенов; вместо `nr` подставляется соответствующее значение. В нашем примере получается `__sc_loadargs_0(name, args);`. Как видно ниже, этот макрос помещает номер системного вызова в `__sc_0`; обратите внимание на оператор склеивания с префиксом `__NR_` и переменную, связанную с конкретным регистром.

Итак, весь этот сложный на вид код лишь помещает номер вызова в регистр 0! Другие макросы помещают аргументы в регистры с `r3` по `r7`: системный вызов здесь может иметь не более пяти аргументов.

Теперь рассмотрим блок `__asm__`. Это *встроенный ассемблер* — ассемблерный код, смешанный с исходным кодом C. Его синтаксис слишком сложен для подробного разбора здесь, но отметим главное.

Пока пропустим `__volatile__`: он сообщает компилятору, что код имеет особые эффекты и его не следует произвольно оптимизировать. Снова начнём с конца. Записи после двоеточий сообщают компилятору, как встроенный ассемблер использует регистры процессора. Это необходимо, чтобы компилятор не использовал их несовместимым образом и не вызвал сбой.

Интереснее две ассемблерные инструкции в первом аргументе. Вся основная работа выполняется инструкцией `sc`. Это всё, что нужно для системного вызова!

При её выполнении процессор передаёт управление коду, адрес которого был задан при загрузке системы для обработки прерываний. Существует множество прерываний; системные вызовы — лишь один пример. Обработчик находит номер вызова в регистре 0, ищет в таблице нужную функцию и переходит к ней. Аргументы она получает в регистрах с 3 по 7.

Когда обработчик завершает работу, управление возвращается к инструкции после `sc`. Автор описывает её как барьер памяти, обеспечивающий завершение записей в память с учётом конвейера процессора; конкретный листинг выше содержит `mfcr`, поэтому детали следует сверять с документацией соответствующей версии архитектуры и кода.

Почти готово! Осталось вернуть результат системного вызова. Значение `__sc_ret` берётся из `r3`, а `__sc_err` — из `r0`. Зачем два значения?

Одно представляет результат, другое — ошибку. Системные вызовы, как и любые функции, могут завершаться ошибкой. Но возвращаемым значением может быть любое число: отрицательное значение не всегда означает ошибку.

Поэтому функция возвращает результат в `r3`, а признак ошибки — через состояние, считанное в `r0`. Проверяется соответствующий бит; при ошибке глобальная переменная `errno` получает код ошибки, а возвращаемое значение становится `-1`. Если ошибки нет, результат возвращается непосредственно. Вызывающая функция должна проверить результат на `-1` и при необходимости узнать причину из `errno`.

Вот и весь системный вызов PowerPC!

Системные вызовы x86

Ниже показан тот же интерфейс для процессора x86.

Пример 4.3. Системный вызов x86

```
                            /* user-visible error numbers are in the range -1 - -124: see <asm-
i386/errno.h> */

              #define __syscall_return(type, res)    \
do {        \
                      if ((unsigned long)(res) >= (unsigned long)(-125)) { \
                              errno = -(res);     \
                              res = -1;     \
                      }       \
return (type) (res);     \
              } while (0)

              /* XXX - _foo needs to be __foo, while __NR_bar could be _NR_bar. */
              #define _syscall0(type,name)   \
type name(void)     \
              {      \
              long __res;     \
              __asm__ volatile ("int $0x80"   \
                      : "=a" (__res)    \
: "0" (__NR_##name));   \
              __syscall_return(type,__res);
              }

              #define _syscall1(type,name,type1,arg1)   \
type name(type1 arg1)     \
              {       \
              long __res;      \
              __asm__ volatile ("int $0x80"    \
                      : "=a" (__res)     \
: "0" (__NR_##name),"b" ((long)(arg1))); \
              __syscall_return(type,__res);
              }
```

```
              #define _syscall2(type,name,type1,arg1,type2,arg2)   \
type name(type1 arg1,type2 arg2)     \
              {         \
              long __res;        \
              __asm__ volatile ("int $0x80"      \
                      : "=a" (__res)       \
: "0" (__NR_##name),"b" ((long)(arg1)),"c" ((long)(arg2))); \
              __syscall_return(type,__res);
              }

              #define _syscall3(type,name,type1,arg1,type2,arg2,type3,arg3)  \
type name(type1 arg1,type2 arg2,type3 arg3)    \
              {         \
              long __res;        \
              __asm__ volatile ("int $0x80"      \




            : "=a" (__res)       \
: "0" (__NR_##name),"b" ((long)(arg1)),"c" ((long)(arg2)), \
                      "d" ((long)(arg3)));     \
    __syscall_return(type,__res);      \
    }

#define _syscall4(type,name,type1,arg1,type2,arg2,type3,arg3,type4,arg4) \
    type name (type1 arg1, type2 arg2, type3 arg3, type4 arg4)   \
    {          \
    long __res;         \
    __asm__ volatile ("int $0x80"       \
: "=a" (__res)        \
            : "0" (__NR_##name),"b" ((long)(arg1)),"c" ((long)(arg2)),  \
              "d" ((long)(arg3)),"S" ((long)(arg4)));    \
    __syscall_return(type,__res);       \
    }
    #define _syscall5(type,name,type1,arg1,type2,arg2,type3,arg3,type4,arg4, \
              type5,arg5)        \
    type name (type1 arg1,type2 arg2,type3 arg3,type4 arg4,type5 arg5)  \
    {          \
long __res;         \
    __asm__ volatile ("int $0x80"       \
            : "=a" (__res)        \
            : "0" (__NR_##name),"b" ((long)(arg1)),"c" ((long)(arg2)),  \
              "d" ((long)(arg3)),"S" ((long)(arg4)),"D" ((long)(arg5)));  \
__syscall_return(type,__res);       \
    }

    #define _syscall6(type,name,type1,arg1,type2,arg2,type3,arg3,type4,arg4,   \
              type5,arg5,type6,arg6)        \
type name (type1 arg1,type2 arg2,type3 arg3,type4 arg4,type5 arg5,type6 arg6)   \
    {           \
    long __res;           \
    __asm__ volatile ("push %%ebp ; movl %%eax,%%ebp ; movl %1,%%eax ; int $0x80 ; pop %%ebp" \
            : "=a" (__res)          \
: "i" (__NR_##name),"b" ((long)(arg1)),"c" ((long)(arg2)),    \
              "d" ((long)(arg3)),"S" ((long)(arg4)),"D" ((long)(arg5)),    \
              "0" ((long)(arg6)));         \
    __syscall_return(type,__res);         \
    }
```

Архитектура x86 сильно отличается от рассмотренной PowerPC. В отличие от RISC-процессора PowerPC, x86 относится к CISC и имеет значительно меньше регистров.

Начните с простейшего макроса `_syscall0`. Он вызывает инструкцию `int` со значением `0x80`. Процессор возбуждает прерывание `0x80`, которое передаёт управление обработчику системных вызовов ядра.

Передачу аргументов можно изучить по более длинным макросам. В реализации PowerPC макросы вызывали друг друга, каждый раз добавляя аргумент. Здесь код повторяется чаще, зато его немного легче проследить.

Имена регистров x86 основаны на буквах, а не числах, как у PowerPC. Макрос без аргументов загружает только регистр A; следовательно, номер системного вызова ожидается в `EAX`. В остальных макросах сокращённые имена регистров видны в аргументах `__asm__`.

В макросе для шести аргументов есть интересная деталь: инструкции `push` и `pop`. Они работают со стеком x86, помещая значение на вершину стека в памяти и извлекая его обратно. Для шестого аргумента необходимо сохранить `ebp`, поместить в него аргумент инструкцией `mov`, выполнить системный вызов и восстановить исходное значение. Это недостаток малого числа регистров: обращения к памяти дороги, поэтому их желательно избегать.

Также здесь нет аналога описанного выше барьера памяти PowerPC. Автор отмечает, что в x86 эффекты завершённых инструкций гарантированно видимы; это облегчает работу компилятору и программисту, хотя уменьшает гибкость.

Наконец, различается возврат результата. В PowerPC использовались два регистра: значение и признак ошибки. На x86 одно возвращаемое значение передаётся в `__syscall_return`. Макрос приводит его к `unsigned long` и сравнивает с диапазоном отрицательных значений, зависящим от архитектуры и ядра, которые обозначают ошибки. Значение `errno` положительно, поэтому отрицательный результат ядра меняет знак. Из-за этого системные вызовы не могут непосредственно возвращать небольшие отрицательные значения: они были бы неотличимы от ошибок. Некоторые вызовы, например `getpriority()`, добавляют смещение, делая результат положительным; пользовательская библиотека должна вычесть его и восстановить настоящее значение.

Привилегии

Аппаратная поддержка

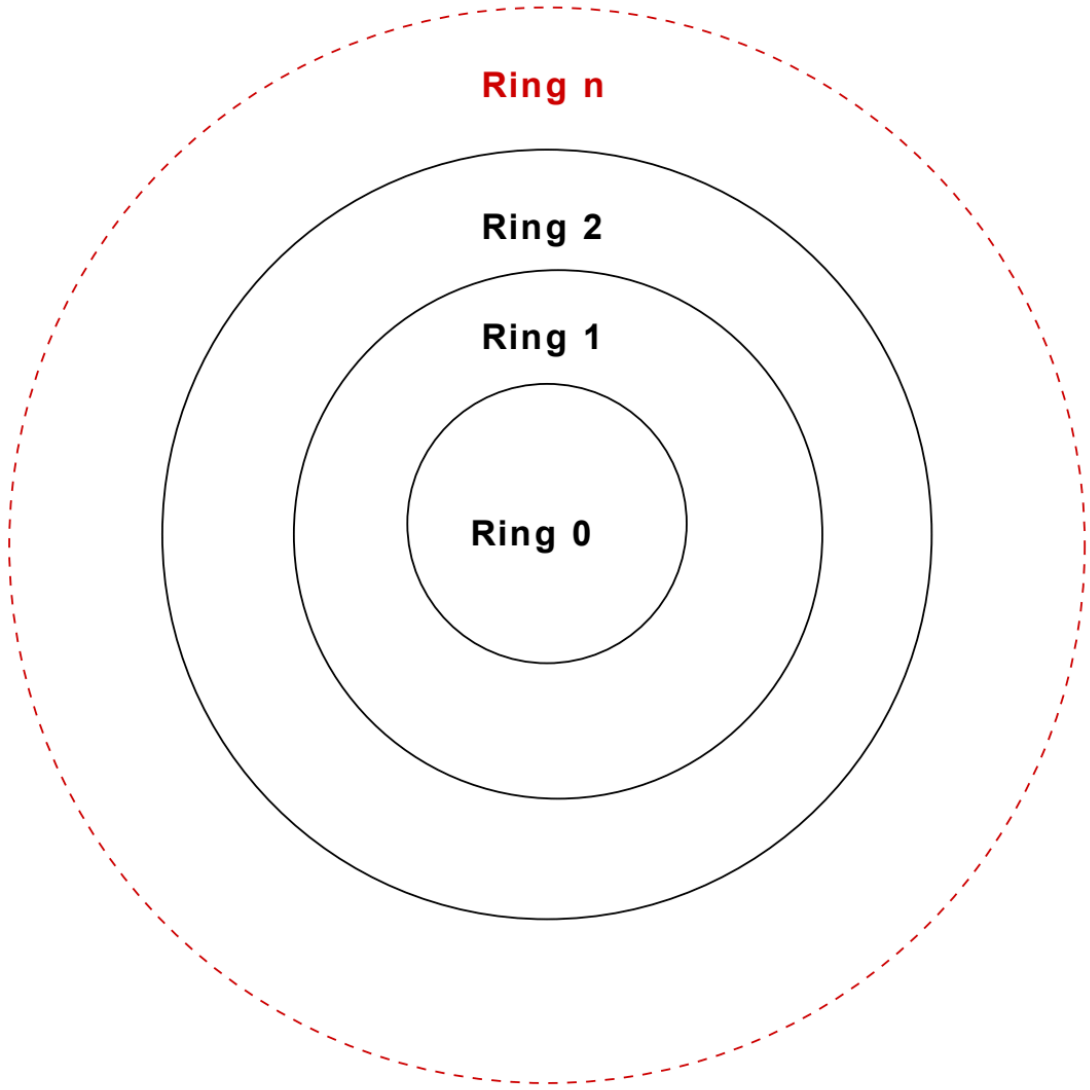
Одна из главных задач ОС — безопасность: приложения и пользователи не должны мешать друг другу. Приложения не должны перезаписывать чужую память или файлы и получают доступ к ресурсам согласно политике системы.

Однако во время выполнения приложение единолично использует процессор; подробнее об этом — в следующей главе. Ограничение доступа к памяти реализуется системой виртуальной памяти, рассмотренной через главу. Существенно то, что соблюдение правил обеспечивает аппаратура.

Интерфейс системных вызовов служит шлюзом доступа к ресурсам. Вынуждая приложение обращаться к ядру, система может контролировать разрешения. Например, при вызове `open()` ядро сравнивает права пользователя с правами файла и разрешает или запрещает доступ.

Уровни привилегий

Аппаратную защиту обычно можно представить концентрическими кольцами вокруг набора основных операций.



Privilege levels on x86

Рисунок 4.3. Уровни привилегий x86.

Во внутреннем кольце находятся наиболее защищённые инструкции, доступные только ядру. Например, пользовательское приложение не должно выполнять `HLT`, останавливающую процессор, иначе весь компьютер перестанет работать. Но ядру она нужна при корректном выключении.

> Если недобросовестная программа всё же вызовет такую инструкцию, аппаратура обычно возбудит исключение и передаст управление обработчику ОС, подобно системному

> вызову. ОС, вероятно, завершит программу и сообщит пользователю об ошибке.

Каждое внутреннее кольцо имеет доступ к инструкциям внешних колец, но не более внутренних. Не все архитектуры предоставляют несколько таких уровней, однако большинство различает хотя бы режимы ядра и пользователя.

Модель защиты 386

В модели 386 четыре кольца, хотя большинство ОС, включая Linux и Windows, используют только два для совместимости с архитектурами, имеющими меньше уровней защиты.

У каждого выполняющегося фрагмента кода есть небольшой дескриптор кода, описывающий в том числе уровень привилегий. Если программа переходит к коду за пределами области своего дескриптора, проверяется уровень назначения. Если он выше текущего, аппаратура запрещает переход и приложение аварийно завершается.

Повышение привилегий

Приложения могут повышать привилегии лишь через специально разрешённые вызовы, например инструкцию системного вызова. Такой механизм называют *шлюзом вызова*: это узкий проход в непроницаемой стене. При выполнении инструкции аппаратура полностью останавливает приложение и передаёт управление ядру.

Ядро выступает привратником и проверяет, что через шлюз не проходит ничего опасного. Аргументы системного вызова нужно тщательно проверять, чтобы ядро нельзя было обманом заставить выполнить запрещённое действие: такая возможность была бы уязвимостью. Ядро работает во внутреннем кольце и может выполнять любые операции. Закончив, оно возвращает управление приложению с его более низким уровнем привилегий.

Быстрые системные вызовы

Описанные ловушки дороги для процессора: перед переключением контекста нужно сохранить много состояния. Современные процессоры стараются уменьшить эти затраты.

Чтобы понять механизм шлюза, нужно разобраться в изобретательной, но сложной схеме сегментации. Изначально она позволяла адресовать больше, чем 16 бит, помещавшихся в регистр, как показано на рисунке 4.4.

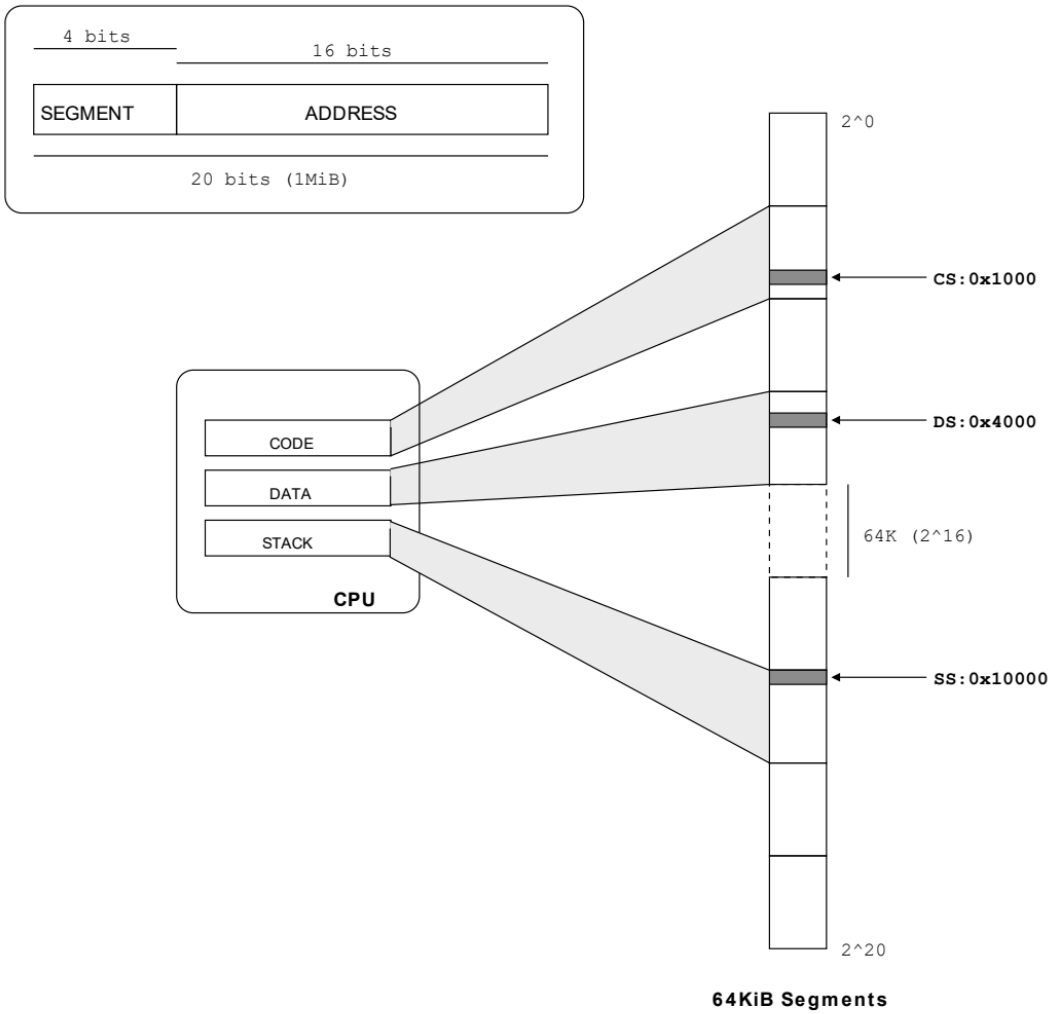


Рисунок 4.4. Сегментная адресация x86. Сегментация расширяет адресное пространство, разделяя его на участки. Процессор хранит специальные сегментные регистры; адрес задаётся сочетанием сегмента и смещения, а их значения используются для вычисления итогового адреса.

При переходе x86 к 32-битным регистрам сегментация сохранилась, но изменилась: вместо фиксированного размера сегменты получили произвольный. Процессор отслеживает их через дескрипторы. Общедоступные дескрипторы хранятся в глобальной таблице дескрипторов — GDT. Регистры каждого процесса указывают на записи GDT, определяя доступные ему сегменты. Существуют также локальные таблицы и сегменты состояния задачи, но сейчас это несущественно. Общая схема показана на рисунке 4.5.

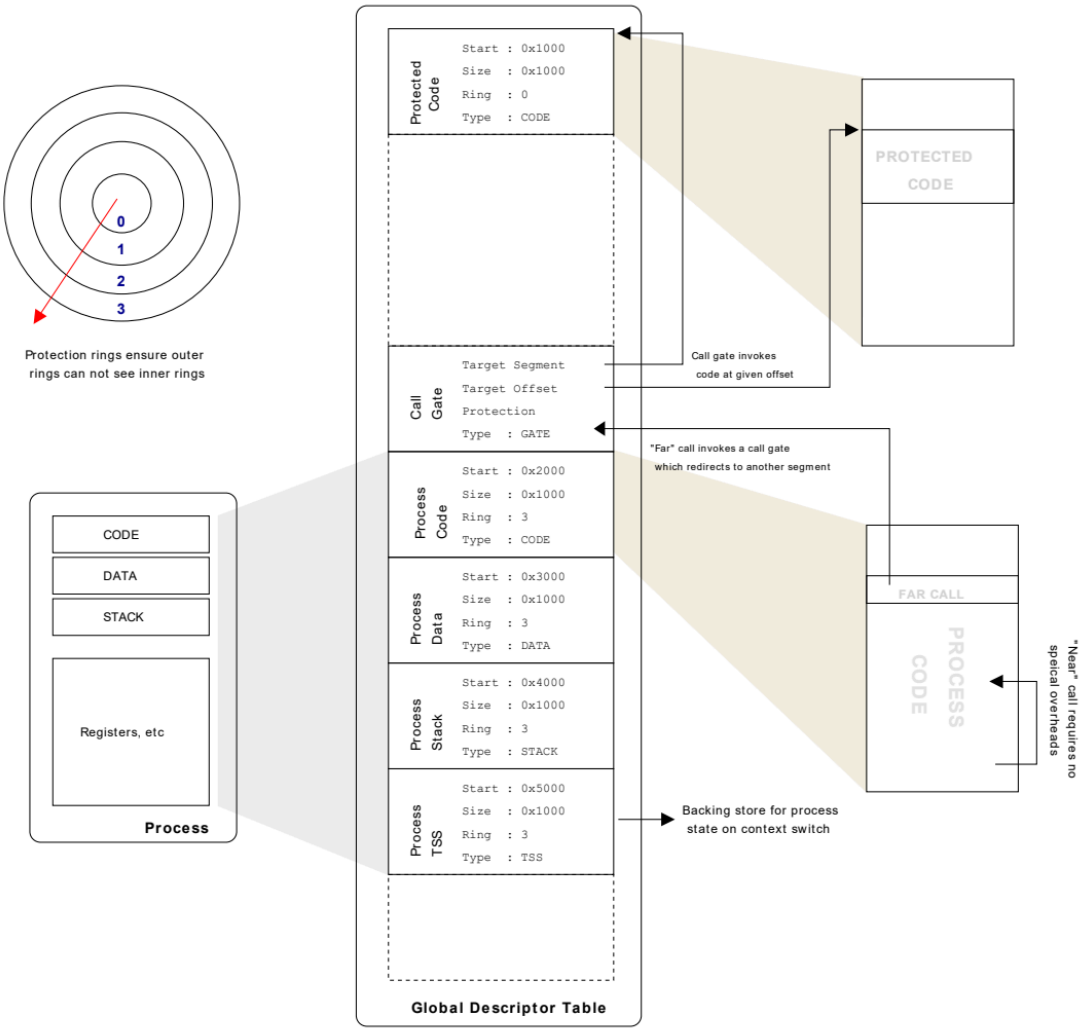


Рисунок 4.5. Сегменты x86. Дальний вызов проходит через шлюз, который перенаправляет его в сегмент кода с другим уровнем привилегий. Выбор нового дескриптора и возможное изменение привилегий происходят через известную точку входа.

Поскольку ОС задаёт сегментные регистры как часть состояния процесса, процессор знает, к каким сегментам памяти тот может обращаться, и обеспечивает защиту. При выходе за границы возникает знакомая многим программистам ошибка сегментации.

Ситуация интереснее, когда код вызывает другой код в другом сегменте. Как обсуждалось выше, x86 применяет кольца: 0 — максимальные права, 3 — минимальные; внутренние кольца имеют доступ к внешним, но не наоборот.

Когда код кольца 3 хочет перейти к коду кольца 0, фактически он меняет селектор своего сегмента кода. Для этого нужна специальная инструкция дальнего вызова, которая обязательно проходит через шлюз. Иного способа выбрать новый дескриптор нет. Процессор начинает выполнение с известного смещения в сегменте кольца 0; этот код отвечает за целостность системы и, например, не должен выполнять произвольный потенциально вредоносный код. Разумеется, злоумышленники всегда ищут способы заставить программу сделать непредусмотренное!

Так можно построить целую иерархию сегментов и прав. Межсегментный вызов напоминает системный вызов. В ассемблере Linux x86 стандартный способ системного вызова — `int 0x80`, возбуждающий прерывание. Оно останавливает текущий код и обращается к шлюзу прерывания, который действует подобно шлюзу вызова: меняет уровень привилегий и направляет выполнение в другую область кода.

Недостаток этой схемы — скорость. Все проверки требуют работы, при переходе приходится сохранять множество регистров, а при возврате — восстанавливать их.

На современных для автора системах x86 виртуальная память, рассмотренная в главе 6, делает обычную сегментацию и четыре кольца ненужными. Основное применение переключения сегментов — системные вызовы: переход из режима 3 пользовательского пространства в режим 0 и к обработчику ядра. Поэтому процессор предоставляет быстрые инструкции `sysenter` и `sysexit` для возврата. Они ускоряют работу по сравнению с `int 0x80`, отказываясь от общего механизма дальнего вызова, допускающего переход в любой сегмент и кольцо, и ограничивая переход известным сегментом и смещением кода кольца 0, сохранёнными в регистрах.

Благодаря заранее известной информации весь процесс ускоряется. Кроме того, при получении управления ядром состояние не сохраняется автоматически. Ядро должно позаботиться о его сохранности, но может сохранять только необходимый минимум, повышая эффективность. Это характерная для RISC философия, показывающая, как стирается граница между RISC и CISC.

Подробнее о реализации в Linux см. раздел «Библиотека ядра».

Другие способы взаимодействия с ядром

ioctl

В исходнике здесь оставлена краткая пометка об `ioctl` без дальнейшего объяснения.

Файловые системы

В исходнике здесь оставлена пометка о `proc`, `sysfs`, `debugfs` и других файловых системах без дальнейшего объяснения.

Глава 5. Процесс

Что такое процесс?

Нам всем знакома многозадачность современной операционной системы, которая одновременно выполняет множество задач.

Каждый процесс можно представить как набор элементов, которые ядро хранит для учёта всех работающих задач.

Элементы процесса

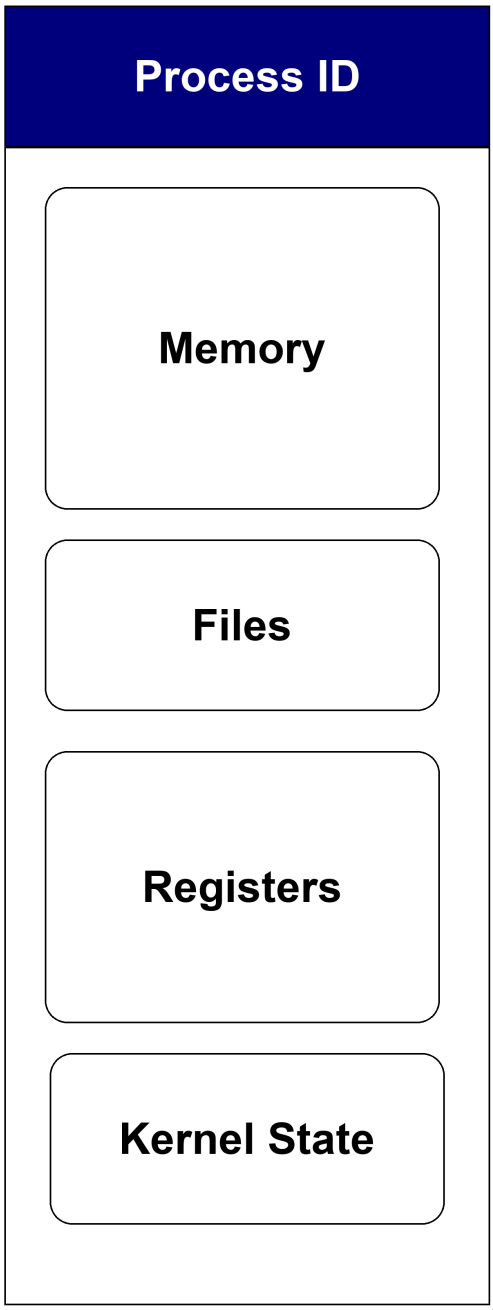


Рисунок 5.1. Элементы процесса: идентификатор процесса, память, файлы, регистры и состояние ядра.

Идентификатор процесса

Идентификатор процесса, или PID, назначается операционной системой и уникален для каждого работающего процесса.

Память

В следующих главах мы подробно узнаем, как процесс получает память: это один из важнейших механизмов ОС. Пока достаточно знать, что каждый процесс получает собственную область памяти.

В ней хранятся код программы, переменные и другие выделенные данные.

Некоторые области могут разделяться между процессами; это называется разделяемой памятью. Часто встречается название System V Shared Memory, или SysV SHM, по имени первоначальной реализации в старой операционной системе.

Другая важная возможность — отображение файла на диске в память с помощью `mmap`. Вместо открытия файла и вызовов `read()` и `write()` к нему можно обращаться как к памяти. Отображённые области имеют права на чтение, запись и выполнение, которые необходимо отслеживать. ОС отвечает за безопасность и стабильность, поэтому она должна выявлять попытку записи в область только для чтения и сообщать об ошибке.

Код и данные

Память процесса можно разделить на области кода и данных. Их следует хранить отдельно, поскольку им нужны разные права; кроме того, разделение облегчает совместное использование кода, как мы увидим позже. Коду обычно требуются чтение и выполнение, но не запись. Данным, например переменным, нужны чтение и запись, но не выполнение.

> Не все архитектуры поддерживают такое разделение прав, что привело ко многим проблемам безопасности.

Стек

Другая очень важная область памяти процесса — стек. Его можно считать частью данных процесса; он тесно связан с выполнением любой программы.

Стек — общая структура данных, работающая как стопка тарелок: можно положить элемент сверху (*push*), и он станет верхним, или снять верхний элемент (*pop*), открыв предыдущий.

Стек лежит в основе вызовов функций. При каждом вызове создаётся новый кадр стека — область памяти, обычно содержащая как минимум адрес возврата, аргументы и место для локальных переменных.

По соглашению стек обычно растёт вниз: он начинается с высокого адреса и постепенно переходит к более низким. Некоторые архитектуры, например HP PA-RISC, используют растущий вверх стек. В других, например IA-64, существуют дополнительные области хранения регистров, растущие снизу навстречу стеку.

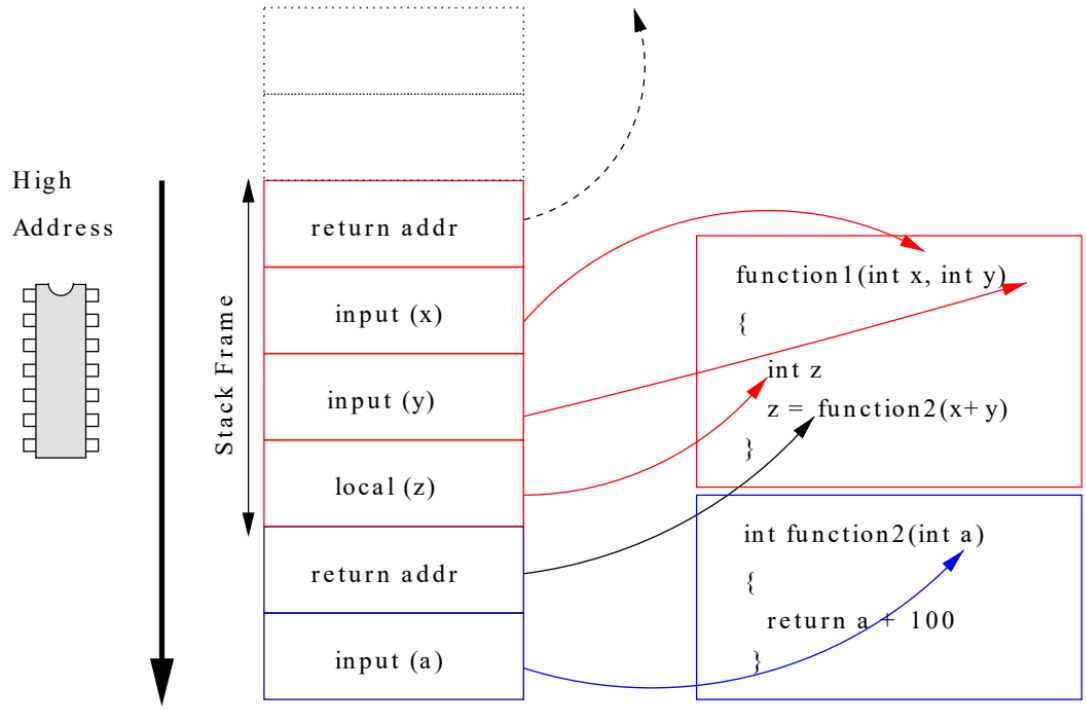


Рисунок 5.2. Стек и кадры функций.

Стек объясняет многие свойства функций:

- Каждая функция получает собственные копии аргументов, поскольку ей выделяется новый кадр в отдельной области памяти.
- Локальная переменная одной функции не видна другим. Глобальные переменные, доступные всем функциям, находятся в отдельной области данных.
- Возможна рекурсия: функция может снова вызвать себя, поскольку для нового вызова создаётся собственный кадр локальных переменных.
- Каждый кадр содержит адрес возврата. В C функция возвращает одно значение; по соглашению оно передаётся вызывающей функции в определённом регистре, а не через стек.
- Если кадры связаны друг с другом, отладчик может пройти по ним назад и построить трассировку стека — список вызовов, приведших к текущей функции. Это чрезвычайно полезно при отладке.

Порядок работы функций соответствует природе стека: любая функция может вызвать другую, которая становится верхней; завершившись, она возвращает управление вызвавшей её функции и убирает свой кадр.

- Использование стека может замедлять вызовы, поскольку значения приходится переносить из регистров в память. Некоторые архитектуры передают аргументы

непосредственно в регистрах; для сохранения отдельных значений вызовов регистры могут переключаться или сохраняться.

- Вы могли слышать о переполнении стека. Один из способов атаки использует запись некорректных данных за пределы локального буфера. Если программа принимает произвольный ввод с клавиатуры или из сети в переменную стека, она обязана проверять его размер. Иначе данные перезапишут соседнюю память. Обычно это вызывает сбой, но специально подобранный ввод может перезаписать адрес возврата и направить выполнение на введённый машинный код, например запускающий терминал с правами администратора. В описанной схеме стек растёт вниз, а данные записываются от меньших адресов к большим.
- Защита включает обязательную проверку длины ввода программистом и запрет выполнения кода в области стека со стороны ОС и процессора. Современные системы поддерживают такую защиту.
- Управление стеком в конечном счёте осуществляет компилятор, генерирующий код. Для ОС стек выглядит как обычная область памяти процесса.

Для отслеживания вершины стека аппаратура предоставляет регистр — указатель стека. Компилятор или программист, пишущий на ассемблере, использует его для контроля текущего положения вершины.

Пример 5.1. Указатель стека

```
                $ cat sp.c
    void function(void)
    {
int i = 100;
            int j = 200;
            int k = 300;
    }

$ gcc -fomit-frame-pointer -S sp.c

    $ cat sp.s
            .file   "sp.c"
            .text
.globl function
            .type   function, @function
    function:
            subl    $16, %esp
            movl    $100, 4(%esp)
movl    $200, 8(%esp)
            movl    $300, 12(%esp)
            addl    $16, %esp
            ret
            .size   function, .-function
.ident  "GCC: (GNU) 4.0.2 20050806 (prerelease) (Debian 4.0.1-4)"
            .section        .note.GNU-stack,"",@progbits
```

Выше показана простая функция, выделяющая в стеке три переменные. Полученный ассемблерный код показывает использование указателя стека на x86. Сначала выделяется место для локальных переменных. Поскольку стек растёт вниз, из указателя вычитается число. Значение 16 достаточно для переменных, хотя может превышать их суммарный

размер: трём четырёхбайтовым `int` нужно всего 12 байтов. Дополнительное место обеспечивает требуемое компилятором выравнивание.

Затем значения записываются в стек, где реальная функция их использовала бы. Перед возвратом в вызывающую функцию значения «снимаются» со стека восстановлением прежнего указателя.

> Флаг GCC `-fomit-frame-pointer` указывает не занимать дополнительный регистр указателем на начало кадра. Такой указатель помогает отладчикам проходить по кадрам, но уменьшает число регистров, доступных для других целей.

Куча

Куча — область памяти, которой процесс управляет для динамического выделения. Она предназначена, в частности, для данных, размер которых неизвестен во время компиляции.

Граница кучи называется `brk` по имени изменяющего её системного вызова. Изменяя границу, процесс может запросить у ядра дополнительную память.

Чаще всего кучей управляет библиотечная функция `malloc`. Она позволяет программисту легко выделять память и освобождать её через `free`. Внутри `malloc` может использовать различные схемы распределения, например buddy allocator. Она также может запрашивать дополнительную память через анонимное отображение `mmap`: вместо файла в адресное пространство отображается область системной памяти. Это может быть эффективнее. Из-за сложности правильного управления памятью современным программам редко требуется напрямую вызывать `brk`.

Размещение памяти

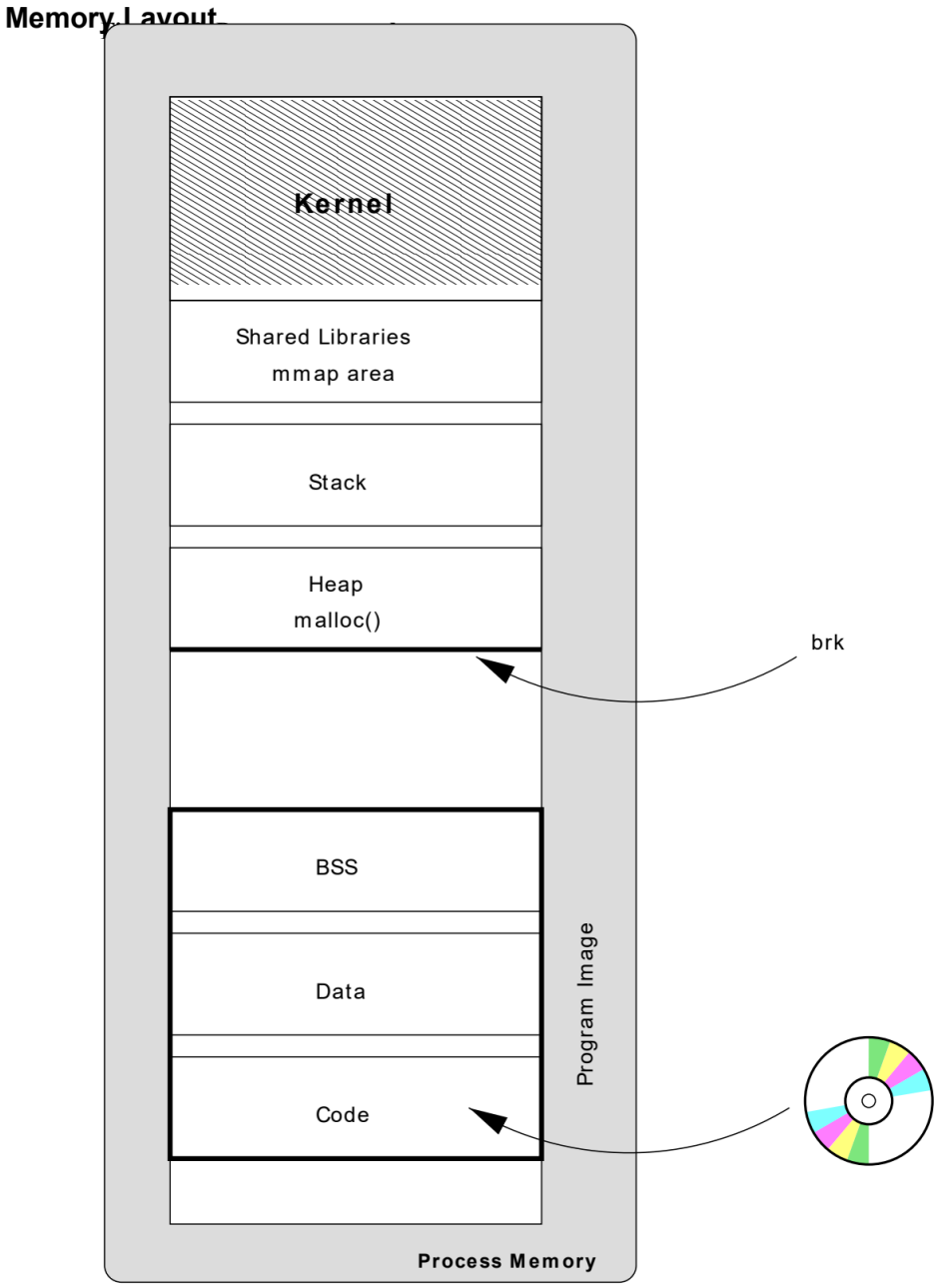


Рисунок 5.3. Пример расположения областей памяти процесса.

Как мы видели, процессу выделяются области памяти с разным назначением. На рисунке показан пример их размещения ядром. В верхней части адресного пространства ядро резервирует память для себя; в главе о виртуальной памяти мы увидим, как эта память разделяется между процессами.

Ниже находится место для отображённых файлов и библиотек, затем стек, а под ним куча. Внизу расположен образ программы, загруженный из исполняемого файла на диске. Подробно процесс загрузки рассматривается в следующих главах.

Файловые дескрипторы

В первой главе мы познакомились с `stdin`, `stdout` и `stderr` — стандартными файлами процесса. Их дескрипторы обычно имеют номера 0, 1 и 2 соответственно. Ядро ведёт

отдельный набор файловых дескрипторов для каждого процесса.

Дескрипторы связаны с разрешёнными операциями. Например, файл может быть открыт для чтения, но не для записи. При открытии ОС сохраняет соответствующую информацию и не позволяет процессу выполнять запрещённые действия.

Регистры

Из предыдущей главы мы знаем, что процессор выполняет в основном простые операции над значениями в регистрах. Значения читаются из памяти и записываются в неё; ядро отслеживает выделенную каждому процессу память.

Необходимо также отслеживать регистры. Когда работающий процесс уступает процессор другому, его состояние нужно сохранить, чтобы позже восстановить. ОС копирует значения регистров процессора в память. Когда процесс снова получает время на процессоре, она восстанавливает регистры, и процесс продолжает работу с прежнего места.

Состояние в ядре

Внутри ядра для каждого процесса хранится ряд дополнительных сведений.

Состояние процесса

ОС должна знать состояние процесса. Если он исполняется, естественно отметить его как работающий. Но если он запросил чтение с диска, операция может занять много времени. Процесс уступает процессор другому и не должен запускаться вновь, пока данные не окажутся в памяти. Ядро может отметить его как ожидающий диск до готовности данных.

Приоритет

Одни процессы важнее других и получают более высокий приоритет. Планировщик рассматривается ниже.

Статистика

Ядро может вести статистику поведения каждого процесса, помогающую принимать решения: например, процесс в основном читает диск или выполняет вычисления на процессоре?

Иерархия процессов

Хотя ОС может одновременно выполнять много процессов, при загрузке ядро запускает начальный процесс `init`. Остальные процессы можно рассматривать как его потомков. У процессов есть семейное дерево: каждый имеет родителя и может иметь множество братьев — процессов, созданных одним родителем. Дочерние процессы могут создавать собственных потомков.

> В исходной книге указан PID 0 для `init`; в обычной системе Linux пользовательский `init` имеет PID 1. Процесс с PID 0 связан с ядром. Термин *spawn* часто обозначает создание дочернего процесса.

Пример 5.2. Вывод `pstree`

```
                          init-+-apmd
                     |-atd
                     |-cron
...
                     |-dhclient
                     |-firefox-bin-+-firefox-bin---2*[firefox-bin]
                     |             |-java_vm---java_vm---13*[java_vm]
                     |             `-swf_play
```

Fork и Exec

Новые процессы создаются с помощью связанных интерфейсов `fork` и `exec`.

Fork

На развилке дороги (*fork*) обычно можно выбрать один из двух путей, и решение влияет на будущее. Программа достигает такой развилки при системном вызове `fork()`.

ОС создаёт новый процесс, первоначально подобный родительскому. Копируется описанное выше состояние, включая открытые файлы, регистры и выделенную память с кодом программы.

По возвращаемому значению вызова программа определяет, исполняется ли она в родительском или дочернем процессе. Родитель получает PID потомка, а потомок — ноль. Теперь говорят, что процесс разветвился, и между ними установлена связь родителя и потомка.

Exec

`fork` позволяет существующему процессу создать новый, но что если новая программа не является частью родительской? Именно так работает оболочка: введённая команда должна выполняться отдельным процессом, но не относится к самой оболочке.

Для этого используется семейство вызовов `exec`, заменяющих содержимое текущего процесса образом исполняемой программы. Запуская команду, оболочка сначала вызывает `fork`, а затем дочерний процесс загружает и запускает нужную программу через `exec`.

Как Linux реализует fork и exec

clone

В описанной реализации ядра `fork` строится на системном вызове `clone`, который предоставляет более общий способ создания процессов Linux. С помощью `clone` можно явно

указать, какие части состояния нового процесса копируются, а какие разделяются с родителем. Поначалу это может показаться странным, но такой простой интерфейс позволяет реализовать потоки.

Потоки

Представим, что при создании нового процесса копируются все перечисленные атрибуты, кроме памяти. Тогда родитель и потомок разделяют память, в том числе код и данные программы.

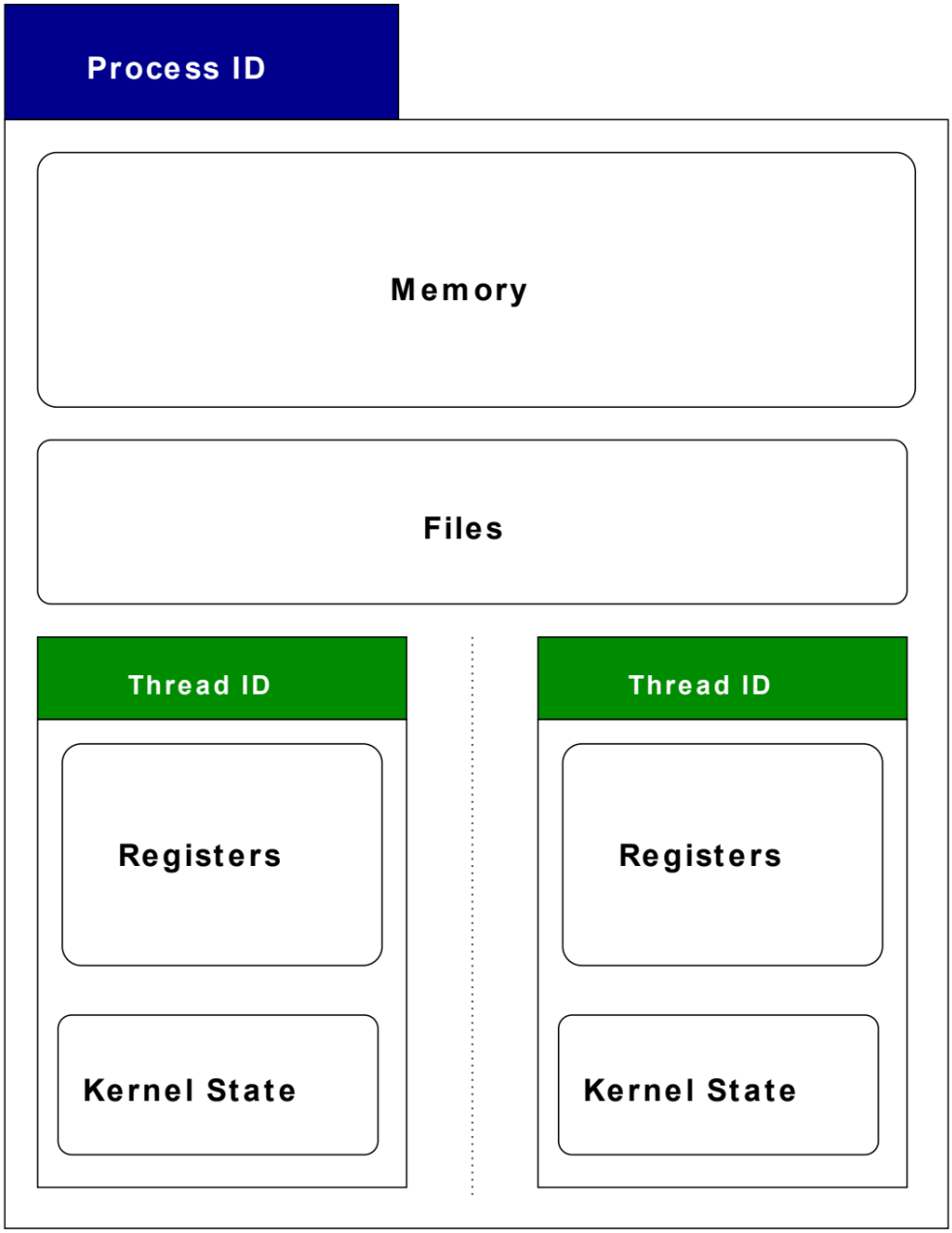


Рисунок 5.4. Потоки разделяют память и другие ресурсы, но имеют собственные регистры и состояние.

Такой гибридный потомок называется потоком. По сравнению с отдельными процессами, создаваемыми через `fork`, у потоков есть преимущества:

1. Отдельные процессы не видят память друг друга и обычно обмениваются данными через системные механизмы. Потоки разделяют память и могут обмениваться данными через неё. Однако они легко мешают друг другу: один поток может увеличить переменную, пока другой уменьшает её. Такие разнообразные трудности называются проблемами конкурентного доступа. Существуют пользовательские библиотеки, помогающие правильно работать с потоками; самая известная — POSIX threads, или `pthreads`.
2. Переключение процессов относительно дорого; значительная часть затрат связана с учётом используемой каждым памяти. При общей памяти часть этих расходов устраняется, что может заметно повысить производительность.

Потоки можно реализовать разными способами. Пользовательская библиотека может организовать их внутри процесса без ведома ядра: для ядра все они выглядят как один процесс. Это неоптимально, поскольку ядро не знает обо всей выполняемой работе и может не лучшим образом распределять ресурсы.

Другой подход — предоставить ядру полную информацию о потоках. В Linux это достигается возможностью процессов разделять ресурсы через `clone`. У каждого потока остаются связанные с ним ресурсы ядра, поэтому планировщик учитывает его при распределении ресурсов.

Другие ОС используют гибридный подход: некоторые потоки работают только в пользовательском пространстве и скрыты от ядра, а другие представлены лёгкими процессами, сообщающими ядру об их принадлежности к группе.

Копирование при записи

Копирование всей памяти процесса при `fork` дорого. Оптимизация *copy-on-write* сначала разделяет память между родителем и потомком. Пока оба процесса только читают, настоящая копия не нужна.

Когда один процесс записывает данные, ему нужна собственная копия соответствующей памяти. Как следует из названия, копирование происходит лишь при записи.

Это особенно выгодно перед `exec`: поскольку новая программа заменит содержимое памяти, предварительное копирование старой программы напрасно тратило бы время.

Процесс init

Мы уже обсуждали общую задачу `init` и теперь можем понять её работу. При загрузке ядро запускает `init`, который с помощью `fork` и `exec` запускает загрузочные сценарии. Те запускают другие программы и в конечном счёте процесс входа в систему.

Другая обязанность `init` — сбор завершённых процессов. Когда процесс вызывает `exit` с кодом завершения, родитель обычно хочет проверить, завершился ли потомок успешно. Но код относится к уже завершившемуся процессу: тот больше не выполняется, однако должен сохраняться до получения кода. Такой процесс называется зомби.

Зомби существует, пока родитель не получит его код вызовом `wait`. Если родитель завершается раньше, потомок передаётся под опеку `init`, специальный обработчик которого забирает статус. После этого запись процесса можно удалить из таблицы ядра.

Пример процесса-зомби

Пример 5.3. Процесс-зомби

```
                            $ cat zombie.c
                #include <stdio.h>
                #include <stdlib.h>
                int main(void)
                {
                        pid_t pid;

printf("parent : %d\n", getpid());

                        pid = fork();

                        if (pid == 0) {
printf("child : %d\n", getpid());
                                sleep(2);
                                printf("child exit\n");
                                exit(1);
                        }
                        /* in parent */
                        while (1)
                        {
                                sleep(1);
}
                }

                ianw@lime:~$ ps ax | grep [z]ombie
                16168 pts/9    S      0:00 ./zombie
16169 pts/9    Z      0:00 [zombie] <defunct>
```

В этом примере родитель спит бесконечно, а потомок завершается через несколько секунд. Вывод показывает родительский процесс `16168` в состоянии `S` — сон — и потомка в состоянии `Z` — зомби. Описание `ps` также помечает его как `defunct`.

> Квадратные скобки вокруг `z` в `grep [z]ombie` — приём, исключающий сам процесс `grep` из вывода. Выражение соответствует слову `zombie`, но не строке имени команды `grep [z]ombie`, в которой присутствуют скобки.

Переключение контекста

Переключение контекста — действия ядра при переходе от выполнения одного процесса к другому. В исходнике на этом месте оставлена пометка для дальнейшего дополнения.

Планирование

В работающей системе может быть много процессов — сотни или даже тысячи. Часть ядра, отслеживающая их и выбирающая следующий для выполнения, называется планировщиком.

Алгоритмы планирования многочисленны и разнообразны. Цели зависят от назначения компьютера. На настольном ПК желательно выделять достаточно времени графическим приложениям, пусть даже системные процессы займут немного больше времени: пользователь получит более быстрый отклик. На сервере приоритет может быть важен для веб-сервера.

Новые алгоритмы придумываются постоянно; вы, вероятно, легко сможете придумать и свой. Но у планирования есть несколько общих аспектов.

Кооперативное и вытесняющее планирование

Стратегии можно разделить на две категории:

1. При кооперативном планировании текущий процесс добровольно уступает управление другому. Очевидный недостаток: процесс может никогда этого не сделать, например из-за бесконечного цикла, и остальные не смогут работать.
2. При вытесняющем планировании процесс прерывается, чтобы запустить другой. Каждому выделяется квант времени; при переключении контекста устанавливается таймер, вызывающий прерывание по истечении кванта. Аппаратура обрабатывает прерывание независимо от процесса и возвращает управление ОС, после чего планировщик выбирает следующую задачу. Этот подход применяется современными ОС.

Реальное время

Некоторым процессам необходимо знать длительность своего кванта и максимальное время ожидания следующего. Например, в аппарате искусственного кровообращения следующий импульс не должен задержаться из-за другой задачи!

Системы жёсткого реального времени гарантируют ограничения планирования, например максимальную задержку до возобновления процесса. Они применяются там, где важна жизнь людей: в медицине, авиации и военной технике.

Мягкое реальное время предполагает менее строгие гарантии, но предсказуемое поведение в целом. Linux может использоваться так, например для аудио и видео. При записи звука длительное прерывание приведёт к потере данных, которые уже не восстановить.

Значение nice

В UNIX каждому процессу назначается значение `nice`, учитываемое планировщиком. Большая «уступчивость» обычно соответствует более низкому приоритету процесса, позволяя другим получить больше времени.

Краткий взгляд на планировщик Linux

Планировщик Linux постоянно менялся и меняется по мере того, как разработчики улучшают его поведение. В рассматриваемой автором версии использовался планировщик O(1):

независимо от количества процессов выбор следующего занимал постоянное время.

Ранее планировщик оценивал пригодность (*goodness*) каждой задачи. Готовые к запуску задачи хранились в очереди — связанном списке процессов, не ожидающих диск и не находящихся в другом состоянии сна. Чтобы выбрать следующую задачу, приходилось вычислять оценку каждой; побеждала наибольшая. Чем больше задач, тем дольше выбор.

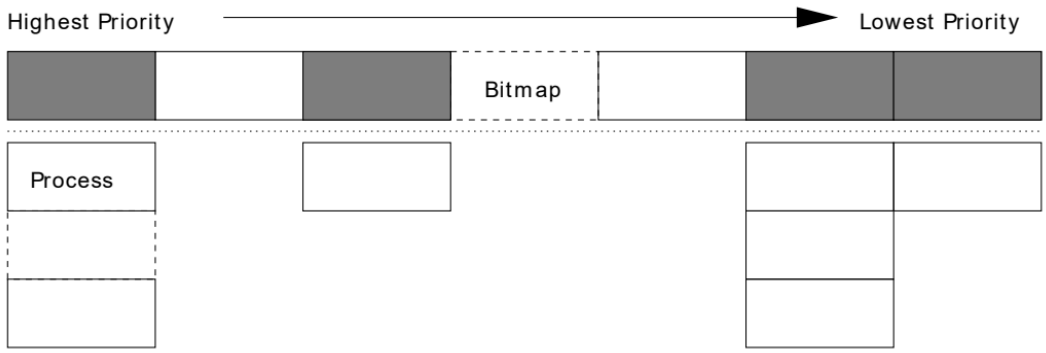


Рисунок 5.5. Планировщик O(1).

В отличие от этого планировщик O(1) использует очереди по приоритетам и битовую карту, показывающую, какие очереди непусты. Он находит по карте первую непустую очередь и берёт из неё первый процесс. Используются два таких массива: активный для готовых процессов и исчерпанный для тех, кто уже использовал квант. Когда все получили время процессора, массивы можно поменять местами простой перестановкой указателей.

Интереснее всего выбор очереди для процесса. Учитываются `nice`, привязка к процессору — перенос между CPU в SMP-системе может быть дорогим — и распознавание интерактивных программ. Например, графическое приложение может долго спать в ожидании ввода, но должно быстро реагировать, когда пользователь начинает с ним работать.

> Нотация O описывает изменение времени работы алгоритма при росте входных данных. Если при удвоении входа время удваивается, зависимость линейная; постоянное время не зависит от размера входа. Обычно чем медленнее растут затраты, тем лучше. Подробно анализ алгоритмов рассматривается в учебниках информатики.

Командная оболочка

В UNIX оболочка — стандартный интерфейс управления процессами. Когда-то она была основным интерфейсом; современные системы Linux обычно имеют графическую среду и предоставляют оболочку через терминальное приложение. Её главная задача — помогать пользователю запускать, останавливать и контролировать процессы.

Получив команду, оболочка создаёт свою копию через `fork`, а затем запускает указанную программу через `exec`. По умолчанию она ждёт завершения процесса и только после этого вновь показывает приглашение.

Команду можно запустить в фоне, обычно добавив `&` после неё. Это указывает оболочке создать процесс и выполнить команду, но не ждать её завершения перед следующим

приглашением. Новый процесс работает в фоне, а оболочка готова к следующей команде. Обычно процесс можно вернуть на передний план, чтобы снова ждать его завершения.

В исходнике здесь оставлена пометка о возможном дополнении истории оболочки Bourne.

Сигналы

Процессам нужен способ узнавать о событиях, которые на них влияют. В UNIX для уведомления о важных событиях служат сигналы между ядром и процессами.

При получении сигнала ядро может вызвать обработчик, который процесс заранее зарегистрировал: это функция, специально написанная для данного события. Сигнал может исходить от ядра или от другого процесса, что является одной из форм межпроцессного взаимодействия. Обработчик вызывается асинхронно: обычное выполнение программы прерывается для обработки события.

Например, при нажатии Ctrl+C процессу направляется сигнал прерывания `SIGINT`. Ядро проверяет, зарегистрирован ли обработчик; если да, выполняет его, иначе применяет действие по умолчанию — завершает процесс.

Некоторые сигналы процесс может игнорировать, другие — нет. `SIGKILL` безусловно завершает процесс; программа не может попросить ядро игнорировать или обработать этот сигнал. Ядро контролирует, кому разрешено посылать сигналы: обычно пользователь может управлять своими процессами, а администратор — чужими.

Команда `kill -9` использует `SIGKILL`, номер которого обычно равен 9. Это крайняя мера: у программы нет возможности освободить ресурсы и корректно завершиться. Лучше сначала послать `SIGTERM` и лишь при отсутствии результата использовать `SIGKILL`. По соглашению многие программы устанавливают обработчик `SIGHUP` — сигнала, оставшегося со времён последовательных терминалов и модемов, — для повторной загрузки конфигурации.

Программистам Unix знакома ошибка сегментации при обращении к недоступной памяти. Ядро посылает процессу `SIGSEGV`; обычно специального обработчика нет и процесс завершается. Иногда обработчик устанавливают, хотя причин для этого немного.

После завершения обработчика сигнала управление обычно возвращается в прерванный процесс, который продолжает работу с прежнего места.

Пример

Следующая простая программа показывает работу нескольких сигналов.

Пример 5.4. Сигналы

```
                              $ cat signal.c
                #include <stdio.h>
                #include <unistd.h>
#include <signal.h>

                void sigint_handler(int signum)
                {
```

```
                        printf("got SIGINT\n");
}

                int main(void)
                {
                        signal(SIGINT, sigint_handler);
printf("pid is %d\n", getpid());
                        while (1)
                                sleep(1);
                }
                $ gcc -Wall -o signal signal.c
$ ./signal
                pid is 2859
                got SIGINT # press ctrl-c
                           # press ctrl-z
                [1]+  Stopped                 ./signal
                $ kill -SIGINT 2859
                $ fg




    ./signal
    got SIGINT
Quit # press ctrl-\

    $
```

Программа определяет обработчик `SIGINT`, посылаемого при Ctrl+C. Сигналы и функция `signal` для регистрации обработчика объявлены в `signal.h`. Программа бездействует в цикле до завершения. Нажав Ctrl+C, мы получаем вызов обработчика и сообщение вместо действия по умолчанию.

Затем Ctrl+Z приостанавливает процесс: он исключается из очереди готовых задач и остаётся в системе. В обычной терминальной настройке Ctrl+Z посылает `SIGTSTP`; в исходном тексте ошибочно указан `SIGSTOP`.

Командой `kill` из другого терминала можно послать процессу сигнал. Программа `kill` использует одноимённый системный вызов с номером сигнала и PID. Название не вполне точно: не все сигналы уничтожают процесс. Пока процесс остановлен, сигнал ставится в очередь и будет доставлен позже.

Команда `fg` возвращает процесс на передний план и посылает `SIGCONT`, который возобновляет его. Ядро вновь ставит процесс в очередь выполнения, и ожидающий сигнал доставляется.

Наконец, Ctrl+\ завершает программу сигналом `SIGQUIT` — в исходнике здесь указан `SIGABRT`. Сообщение `Quit` выводит оболочка: когда дочерний процесс завершается, родитель получает `SIGCHLD` и может узнать через `wait` код и сигнал завершения. Подобным образом оболочка сообщает об ошибке сегментации при `SIGSEGV`.

Даже в этой простой программе около пяти сигналов обеспечивают связь процессов и ядра. Существуют и многие другие; большинство имеют системное назначение, а `SIGUSR` зарезервированы для пользовательских целей.

Глава 6. Виртуальная память

Чем виртуальная память не является

Виртуальную память часто упрощённо описывают как расширение ОЗУ с использованием диска в качестве дополнительной, более медленной памяти: когда ОЗУ заканчивается, данные якобы переходят на диск и становятся «виртуальной» памятью.

В современных ОС такой механизм обычно называется подкачкой (*swap*): неиспользуемые части памяти переносятся на диск, чтобы освободить основную память. Напомним, что программы выполняются из основной памяти.

Подкачка на диск действительно важна, но, как мы увидим, она не является назначением виртуальной памяти — это лишь полезное следствие её организации!

Что такое виртуальная память

В основе виртуальной памяти лежит использование адресного пространства.

Адресное пространство процессора — диапазон адресов, которые он может использовать при чтении и записи памяти. Он связан с разрядностью регистров, содержащих адреса. Например, 32-битный регистр может содержать адреса от `0x00000000` до `0xFFFFFFFF`. Поскольку 2^{32} = 4 ГиБ, 32-битный адрес охватывает до 4 ГиБ памяти.

64-битные вычисления

Новые процессоры обычно 64-битные: их регистры имеют ширину 64 бита. В качестве упражнения вычислите доступное адресное пространство. Подсказка: оно велико!

У 64-битных вычислений есть компромиссы. Программы в 64-битном режиме используют восьмибайтовые указатели, что увеличивает размер кода и данных и может влиять на эффективность кэшей команд и данных. С другой стороны, у 64-битных процессоров часто больше регистров, поэтому компилятору реже приходится сохранять временные значения в память при нехватке регистров.

Канонические адреса

Хотя регистры 64-битных процессоров имеют ширину 64 бита, системы обычно не реализуют все 64 бита адресации: использовать все теоретические 16 эксабайт физической памяти невозможно.

Поэтому архитектуры определяют нереализованную область адресного пространства, адреса в которой считаются недопустимыми. В x86-64 и Itanium определяется старший реализованный бит адреса, значение которого должно быть распространено на старшие разряды, чтобы получить допустимый адрес. В результате адресное пространство разделяется на верхнюю и нижнюю части, а адреса между ними недопустимы. Допустимые адреса называются *каноническими*, остальные — неканоническими.

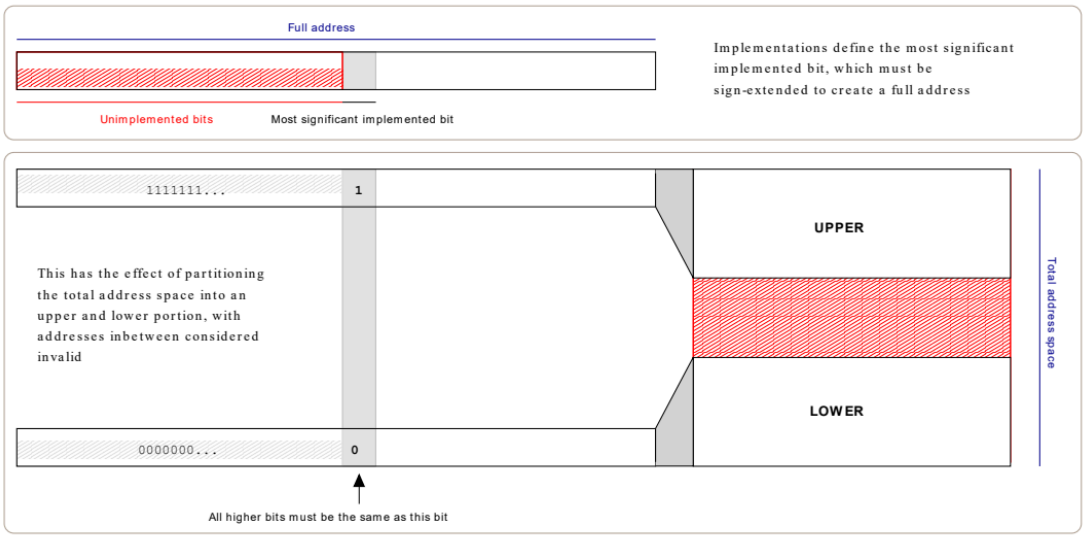


Рисунок 6.1. Канонические адреса: старшие нереализованные биты повторяют старший реализованный бит.

Количество реализованных битов обычно можно узнать с помощью информационных инструкций процессора. Оно зависит от реализации; типичный для описываемых систем пример — 48 бит, или 2^{48} = 256 ТиБ адресного пространства.

Ограничение адресного пространства позволяет упростить логику адресации процессора и связанных компонентов. Поскольку старшие биты обязаны повторять знак, переносимые ОС не должны использовать их для хранения дополнительных признаков: так сохраняется совместимость при расширении адресного пространства будущими реализациями.

Использование адресного пространства

Подобно многим компонентам ОС, виртуальная память создаёт абстракцию между адресным пространством и физической памятью. Адрес, который использует программа, не обозначает напрямую конкретную физическую ячейку.

Все адреса программы называются виртуальными. ОС отслеживает их соответствие физическим адресам. При чтении или записи процессор и ОС совместно преобразуют виртуальный адрес в адрес ячейки памяти оборудования.

Страницы

Всё адресное пространство разделено на страницы. Размеры страниц различаются; часто это 4 КиБ, но бывают и значительно большие страницы. Страница — наименьшая единица памяти, которой совместно управляют ОС и аппаратные средства.

У каждой страницы есть атрибуты, задаваемые ОС, обычно права чтения, записи и выполнения. Например, страницы кода процесса можно пометить исполняемыми, а процессор не будет выполнять код на страницах без соответствующего признака.

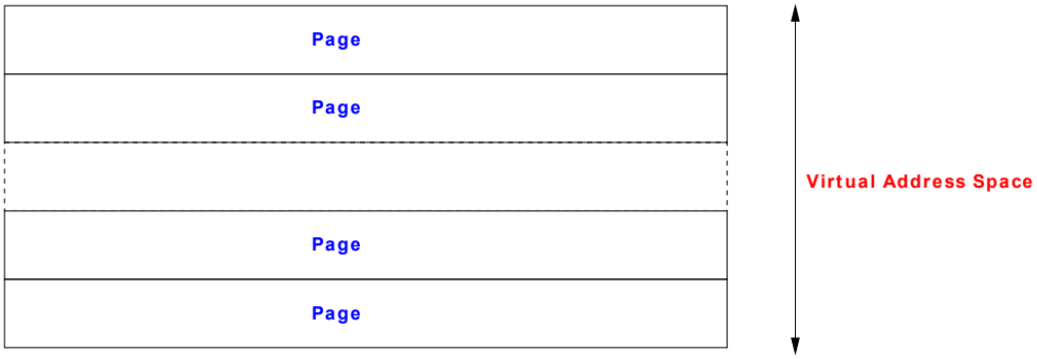


Рисунок 6.2. Страницы виртуальной памяти.

Программист может возразить, что через `malloc` легко выделить гораздо меньше 4 КиБ. На самом деле куча обеспечивается выделениями размером со страницу, а реализация `malloc` эффективно делит и распределяет эти области.

Физическая память

Подобно тому как ОС делит адресное пространство на страницы, физическая память делится на *кадры*. Кадр — обычное название блока физической памяти размером со страницу системы.

ОС ведёт таблицу кадров, в которой отмечено, какие из них свободны. При выделении памяти процессу кадр помечается занятым. Так ОС отслеживает выделения физической памяти.

Сведения о расположении памяти, её объёме и свойствах передаются ОС во время инициализации, например BIOS.

Страницы + кадры = таблицы страниц

ОС должна отслеживать, какой виртуальной странице соответствует какой физический кадр. Эти сведения хранятся в таблице страниц. Простейшая линейная таблица могла бы содержать по одной строке с соответствующим кадром для каждой возможной страницы. При 32-битном адресном пространстве и страницах по 4 КиБ понадобится 2^{32} / 4096 = 1 048 576 записей.

Таблицы страниц имеют разные структуры и тщательно оптимизируются, поскольку поиск записи может занимать значительное время. Подробнее мы рассмотрим их позже.

Таблица страниц процесса находится под исключительным контролем ОС. При выделении памяти ядро находит свободный физический кадр и записывает соответствие виртуального и физического адресов. Когда процесс освобождает память, соответствие удаляется, а кадр может быть выделен другому процессу.

Виртуальные адреса

Обращаясь к памяти, программа не знает и не обязана знать, где физически находятся данные. Поиск соответствующего адреса — задача ОС и оборудования. Поэтому используемый программой адрес называется виртуальным. Он состоит из номера страницы и смещения внутри неё.

Страница

Поскольку всё адресное пространство разделено на страницы фиксированного размера, каждый возможный адрес принадлежит странице. Номер страницы используется как индекс таблицы. Маленькие страницы требуют большого числа записей, а большие могут приводить к потере неиспользуемой памяти; приходится выбирать компромисс.

Смещение

Младшие биты виртуального адреса задают смещение нужного байта от начала страницы. Битов должно хватать для адресации каждого байта: для страницы 4 КиБ = 4096 = 2^{12} байтов необходимо 12 бит. На уровне ОС и аппаратуры все 4096 байтов обрабатываются как одна страница.

Преобразование виртуального адреса

Преобразование позволяет определить, какой физический кадр соответствует виртуальной странице. Номер страницы ищется в таблице, по записи находится физический кадр, а исходное смещение прибавляется к его адресу. Так получается реальное местоположение данных.

Поскольку таблицами управляет ОС, при отсутствии допустимого соответствия или прав система может запретить доступ к невыделенной памяти.

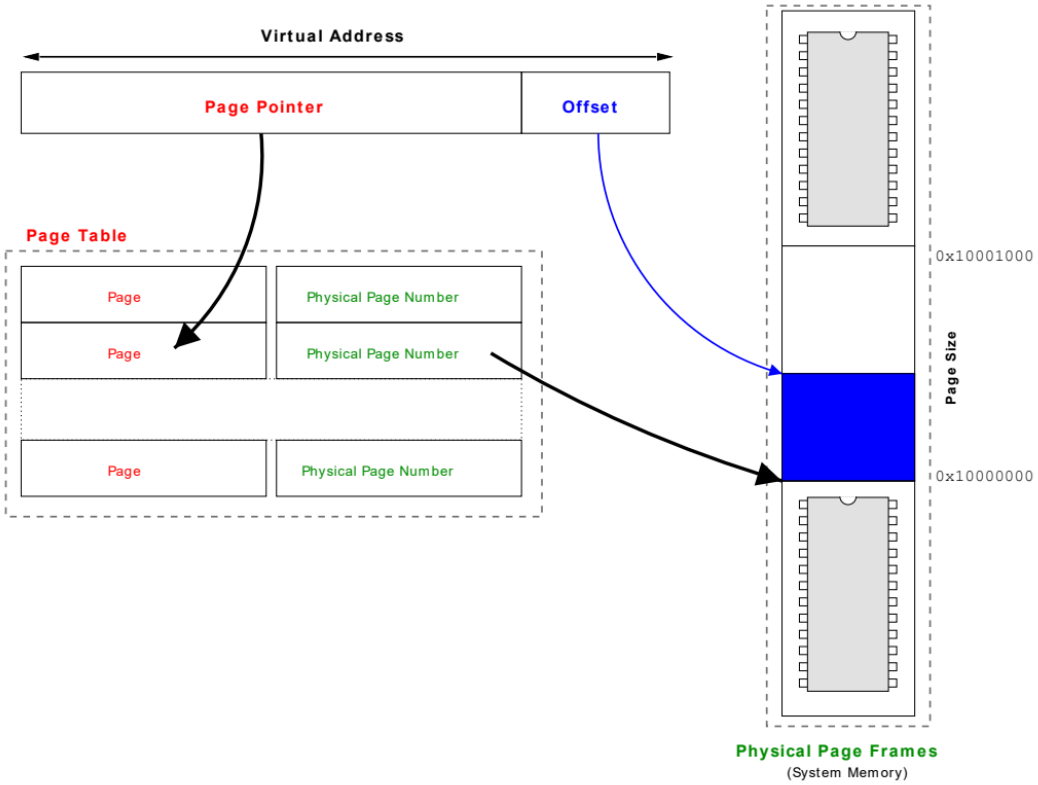


Рисунок 6.3. Преобразование виртуального адреса.

Вернёмся к линейной таблице из 1 048 576 записей для 32-битного пространства и страниц 4 КиБ. Для адреса `0x80001234` отбрасываем 12 бит смещения сдвигом вправо и получаем номер страницы `0x80001`, или 524289. В соответствующей строке таблицы находится физический кадр.

Линейная таблица должна учитывать каждую возможную страницу, используемую или нет, поэтому при 64-битном пространстве она практически неприменима. Даже при страницах 64 КиБ получается $2^{64} / 2^{16} = 2^{48}$ записей. Если запись занимает 8 байтов, потребуется 2^{51} байтов, то есть 2 ПиБ непрерывной памяти. В исходном тексте приведён неверный расчёт.

Следствия виртуальной адресации

Виртуальные адреса, страницы и таблицы страниц лежат в основе современных ОС и многих привычных возможностей компьютера.

Отдельные адресные пространства

Предоставив каждому процессу собственную таблицу страниц, ОС создаёт видимость доступа ко всему адресному пространству процессора. Два процесса могут использовать один и тот же виртуальный адрес: разные таблицы сопоставят его разным физическим кадрам.

Со временем свободная физическая память фрагментируется. Было бы неудобно требовать, чтобы при `malloc` 8 КиБ два физических кадра по 4 КиБ обязательно располагались рядом. Виртуальная адресация позволяет предоставить программе

непрерывные 8 КиБ, даже если физические кадры находятся далеко друг от друга. Программист оставляет решение проблемы фрагментации ОС.

Защита

В системе без изоляции виртуальной памятью процесс может обращаться ко всей физической памяти и перезаписать данные другого процесса, вызвав сбой или, что хуже, неверные результаты — например, при обработке банковского счёта.

При виртуальной адресации ОС контролирует отображения: обращение к недоступной странице вызывает исключение. Атрибуты страниц позволяют ограничивать чтение, запись и выполнение. Попытка записи на страницу только для чтения будет остановлена.

При корректной ОС такая изоляция повышает стабильность: процесс обычно может повредить только себя. Разумеется, ОС пишут люди, и ошибки ядра всё ещё могут привести к отказу всей системы.

Подкачка

Теперь видно, как реализуется подкачка: информация о странице может указывать, что её содержимое находится на диске. При обращении ОС возвращает страницу в физическую память. Если память заполнена, для освобождения места может понадобиться вытеснить другую страницу. Если она вскоре снова понадобится, процесс повторится.

Это серьёзная проблема подкачки: диск гораздо медленнее памяти. Многие видели компьютер, который долго обращается к диску и при этом почти не реагирует.

mmap

Родственный механизм — отображение файла в память, `mmap`, названное по системному вызову. Страницы сопоставляются содержимому файла на диске. Вместо последовательных операций чтения и записи через файловый дескриптор отображённый файл доступен подобно обычной памяти.

Разделение памяти

Обычно у каждого процесса собственная таблица страниц, но ОС может направить записи двух таблиц на один физический кадр. Тогда изменения одного процесса будут видны другому.

Так можно понять устройство потоков: `clone()` позволяет разделять различные ресурсы. Если новый процесс разделяет таблицу страниц с родителем, оба видят одну физическую память.

Аналогично реализуется копирование при записи. Страница помечается только для чтения; при попытке записи ОС получает исключение. Зная, что это страница copy-on-write, она создаёт копию, изменяет запись таблицы и разрешает запись в новую страницу. Теперь у процесса собственная копия.

Дисковый кэш

В современной системе часто имеется больше свободной памяти, чем сейчас требуется программам. Поскольку диск намного медленнее памяти, полезно держать в памяти данные прочитанных файлов.

Linux и другие системы кэшируют содержимое файлов: даже если программа сначала запросила лишь часть, она с большой вероятностью обратится и к остальному. При чтении или записи ОС сначала проверяет кэш. Когда свободной памяти становится мало, пригодные для освобождения кэшированные страницы могут быть вытеснены.

Кэш страниц

Кэш страниц — набор страниц, с помощью которого ядро управляет данными, связанными с файлами и хранилищем, и быстро находит их при запросах чтения и записи. В исходнике этот раздел содержит незавершённую авторскую пометку.

Аппаратная поддержка

До сих пор мы говорили, что оборудование совместно с ОС реализует виртуальную память, не вдаваясь в подробности. Реализация сильно зависит от архитектуры, но есть и общие элементы.

Физический и виртуальный режимы

Процессоры обычно различают физическую и виртуальную адресацию. В физическом режиме адрес относится непосредственно к физической памяти; в виртуальном он должен быть преобразован.

В Itanium режимы называются физическим и виртуальным. В x86 исторически используются названия реального и защищённого режима. Защищённый режим появился в 386; последующие x86 сохраняют и реальный режим для совместимости. В реальном режиме применяется сегментация.

Проблемы сегментации

Сегментация интересна прежде всего исторически: виртуальная память уменьшила её значение. У неё есть недостатки, в том числе сложность для начинающих.

Сегментные регистры задают начала сегментов, а адрес указывается как смещение относительно регистра. Размер сегмента ограничен числом битов смещения. В исходной 16-битной схеме x86 смещение составляет 16 бит, то есть максимум 64 КиБ. При росте памяти до мегабайт и гигабайт необходимость обращаться дальше этого предела стала серьёзной проблемой.

> Если бы смещение имело 32 бита, всё адресное пространство можно было бы представить относительно сегмента с началом `0x00000000`, получив почти плоскую

> модель. Историческое ограничение в 16 бит связано с ранними процессорами Intel и обратной совместимостью.

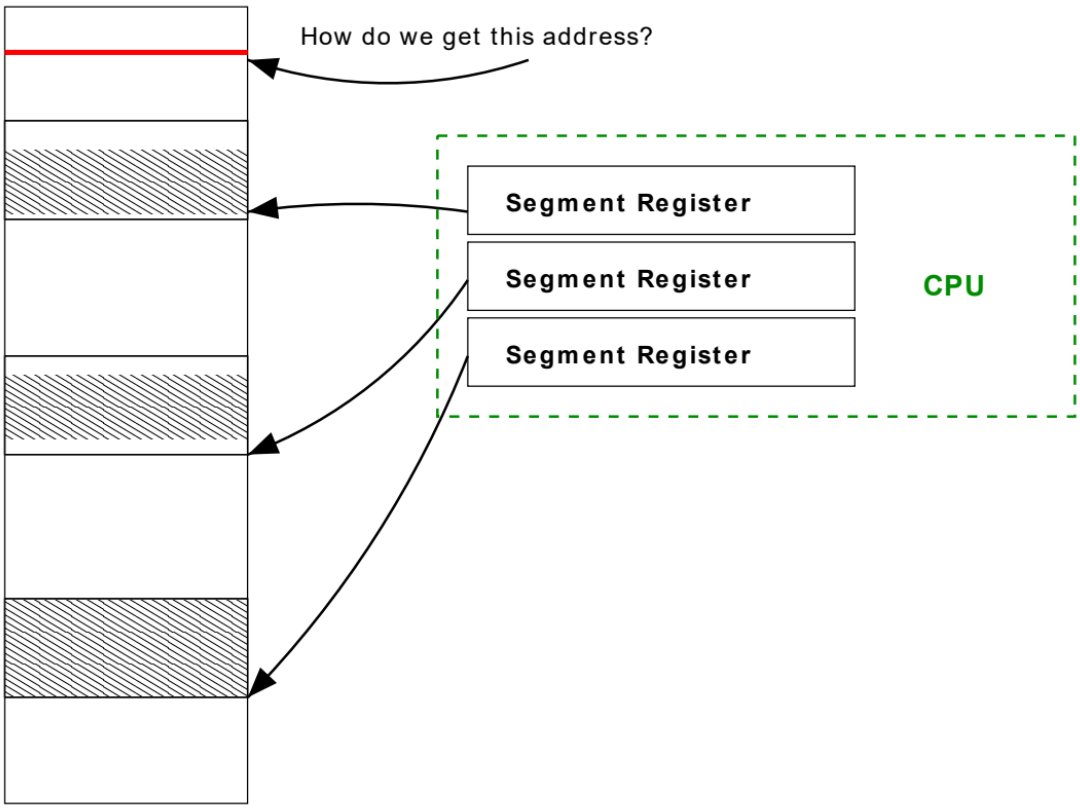


Рисунок 6.4. Сегментация: ограниченный диапазон смещений от сегментных регистров.

На рисунке три регистра указывают на сегменты; допустимый диапазон смещения заштрихован. Для доступа за его пределами регистры приходится перенастраивать, что быстро становится неудобным. При виртуальной памяти программа задаёт адрес, а преобразование выполняют ОС и аппаратура.

TLB

Буфер ассоциативной трансляции, или TLB (*Translation Lookaside Buffer*), — кэш соответствий виртуальных страниц физическим кадрам внутри процессора. ОС и аппаратура совместно управляют им.

Промахи TLB и ошибки страниц

При обращении к виртуальному адресу процессор ищет преобразование в TLB. Найдя действительную запись, он соединяет адрес физического кадра со смещением и выполняет чтение или запись.

Если записи нет, способ обработки зависит от архитектуры. При программном заполнении TLB возникает исключение, и ОС ищет соответствие в таблице страниц. Если TLB заполнен, одну из записей приходится вытеснить. Желательно не удалять ту, которая скоро

понадобится; могут использоваться стратегии, приближённые к LRU, вытесняющие давно не использовавшиеся записи. После заполнения TLB доступ повторяется.

Если подходящего отображения нет или права недостаточны, ОС обрабатывает ошибку страницы: в зависимости от ситуации она может загрузить страницу либо сообщить процессу об ошибке, например привести к `segmentation fault`. Сам по себе промах TLB не всегда означает ошибку страницы: на ряде архитектур аппаратный механизм сначала обходит таблицу страниц.

Поиск таблицы страниц

Адрес основания таблицы страниц хранится в регистре, связанном с текущим контекстом процесса. Его часто называют базовым регистром таблицы страниц. По нему и номеру виртуальной страницы можно найти нужную запись.

Другие признаки страницы

Важны признаки обращения и изменения страницы. Они обычно представлены отдельными битами.

Обращённая страница — та, к которой обращались. ОС может периодически сбрасывать признак и затем определять, какие страницы используются. При нехватке памяти страницы, к которым давно не обращались, становятся хорошими кандидатами на вытеснение.

> Загрузка преобразования без немедленного обращения может быть спекулятивной: например, последовательное чтение позволяет заранее подготовить перевод следующей страницы.

Грязная страница была изменена и больше не соответствует копии на диске. Прежде чем вытеснить такую страницу, её изменения, если требуется, нужно сохранить. Чистую неизменённую страницу можно удалить без обратной записи.

Признаки помогают ОС управлять памятью. В описанной схеме отсутствие бита обращения или изменения может вызывать исключение, чтобы ОС установила соответствующий признак и продолжила выполнение. В других реализациях аппаратура устанавливает эти биты самостоятельно.

Управление TLB

Аппаратура использует TLB, а ОС отвечает за корректность отображений и удаление устаревших записей.

Очистка TLB

Удаление записей называется очисткой (*flush*). Это важно для изоляции: разные процессы могут использовать один виртуальный адрес, и старая запись не должна направить новый процесс в память предыдущего. При переключении между потоками одного адресного пространства нужды в полной очистке по этой причине нет.

Некоторые процессоры полностью очищают TLB при каждом переключении контекста. Это дорого: новому процессу приходится заново заполнять кэш преобразований.

Другие используют идентификатор адресного пространства ASID. Он хранится вместе с каждой записью TLB. Разные адресные пространства имеют разные ASID, а потоки одного пространства — общий. После переключения можно сохранить старые записи: совпасть должна не только страница, но и ASID. Это уменьшает число очисток и повышает производительность ценой дополнительных аппаратных битов.

ASID обычно хранится в отдельном регистре состояния процесса. При поиске TLB учитывает только записи с текущим идентификатором. Ширина регистра определяет количество доступных ASID и влияет на производительность.

Аппаратное и программное заполнение TLB

При программном заполнении промах TLB передаётся ОС, которая обходит таблицу страниц и устанавливает преобразование. Есть и другой способ — аппаратное заполнение.

В этом случае архитектура задаёт определённый формат таблиц. Если преобразования нет в TLB, процессор сам обходит таблицы и загружает найденную запись. Исключение передаётся ОС, если допустимого преобразования найти не удалось.

Специализированная аппаратура ускоряет поиск, но ограничивает свободу разработчика ОС в выборе структуры таблиц. Архитектуры можно приблизительно разделить на эти две группы; далее рассмотрим примеры.

Особенности Linux

Общие принципы виртуальной памяти постоянны, но подробности сильно зависят от ОС и оборудования.

Расположение адресного пространства

Linux разделяет адресное пространство на общую область ядра и частные области пользователей. Адреса ядра отображаются в одну и ту же физическую память для разных процессов, а адреса пользовательского пространства принадлежат конкретному процессу. В описанной 32-битной схеме x86 разделение проходит на отметке 3 ГиБ: верхний 1 ГиБ из 4 ГиБ отведён ядру.

> Это упрощённая схема; специальные расширения и режимы high memory позволяют работать с большим объёмом физической памяти. Современные конфигурации могут использовать другие схемы.

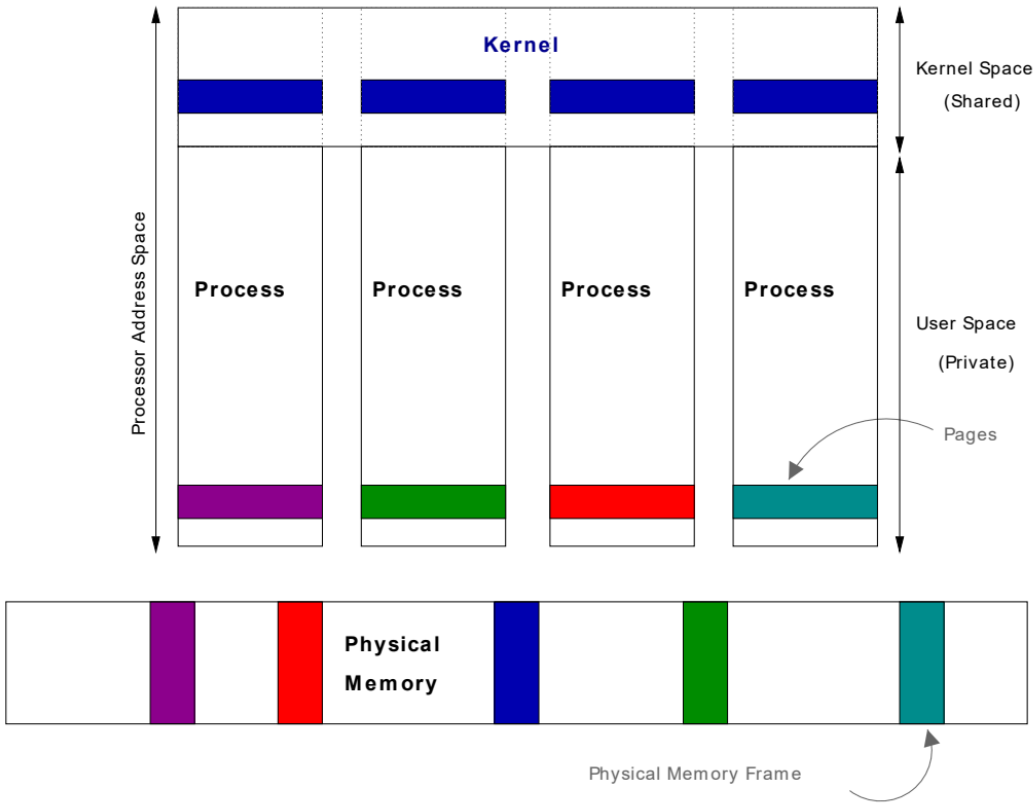


Рисунок 6.5. Схема адресного пространства Linux.

Трёхуровневая таблица страниц

Таблицы можно организовать по-разному; Linux использует иерархический подход. В описываемой автором схеме было три уровня, поэтому она называлась трёхуровневой. Такой выбор доказал свою надёжность, хотя и вызывал критику. Универсальная схема должна приспосабливаться к большим различиям аппаратных реализаций.

Принцип прост: виртуальный адрес содержит номер страницы и смещение, но номер дополнительно делится на части для разных уровней. В одноуровневой таблице запись непосредственно указывает на кадр. В многоуровневой запись верхнего уровня указывает на кадр с таблицей следующего уровня.

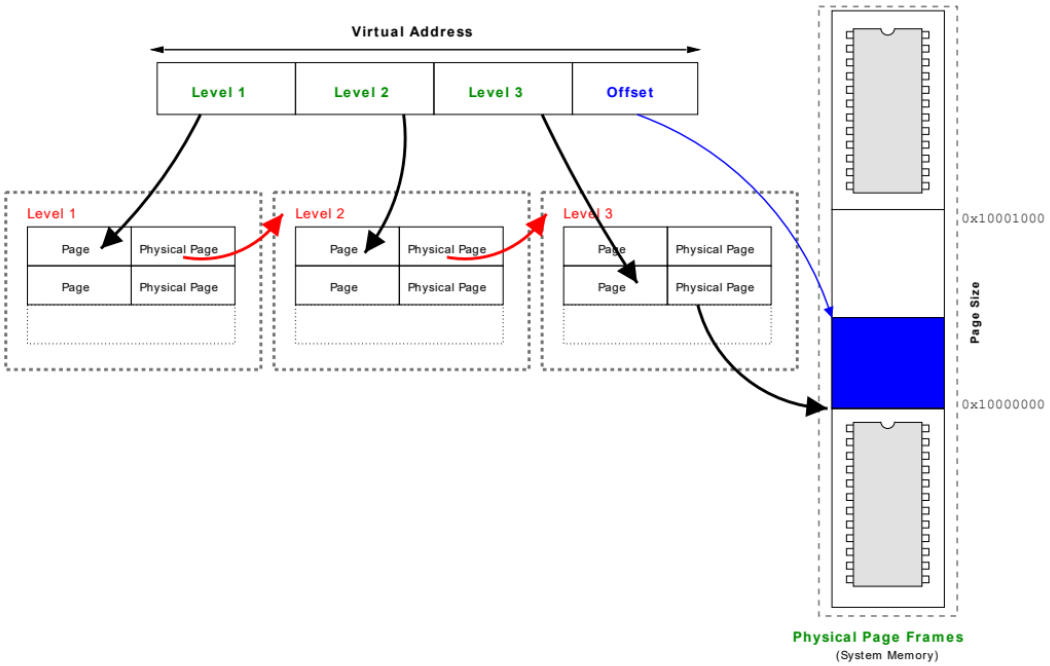


Рисунок 6.6. Трёхуровневая таблица страниц Linux.

При обращении сначала ищется запись верхней таблицы, по ней находится таблица следующего уровня, затем следующая — и так до физического кадра данных.

На первый взгляд схема излишне сложна, но она экономит память. Представим процесс с одной отображённой страницей у самого конца адресного пространства. В линейной таблице пришлось бы выделить весь непрерывный массив до этой записи. В трёхуровневой достаточно выделить нужные таблицы вдоль одного пути, то есть лишь небольшую часть такого объёма.

Недостаток очевиден: поиск одного адреса требует дополнительных обращений к памяти. Для архитектур, которым столько уровней не требуется, Linux может сворачивать уровни; в описываемой 32-битной реализации x86 использовались два. Современные варианты Linux и процессоров могут иметь другое число уровней.

Аппаратная поддержка виртуальной памяти

Как мы видели при обсуждении TLB, аппаратура связывает виртуальные адреса с физическими. Архитектуры используют разные способы управления буфером со своими достоинствами и недостатками. Компонент процессора, занимающийся виртуальной памятью, обычно называется блоком управления памятью — MMU.

x86-64

В исходном тексте этот раздел оставлен незаполненным.

Itanium

MMU Itanium предоставляет ОС несколько интересных возможностей для работы с виртуальной памятью.

Адресные пространства

Идентификаторы адресного пространства помогают уменьшить накладные расходы на очистку TLB. Потоки одного процесса используют общий ASID и разделяют записи, повышая производительность. Но общий ASID сам по себе делает разделение по принципу «всё или ничего»: даже ради совместного доступа к нескольким байтам потоки теряют аппаратную изоляцию друг от друга.

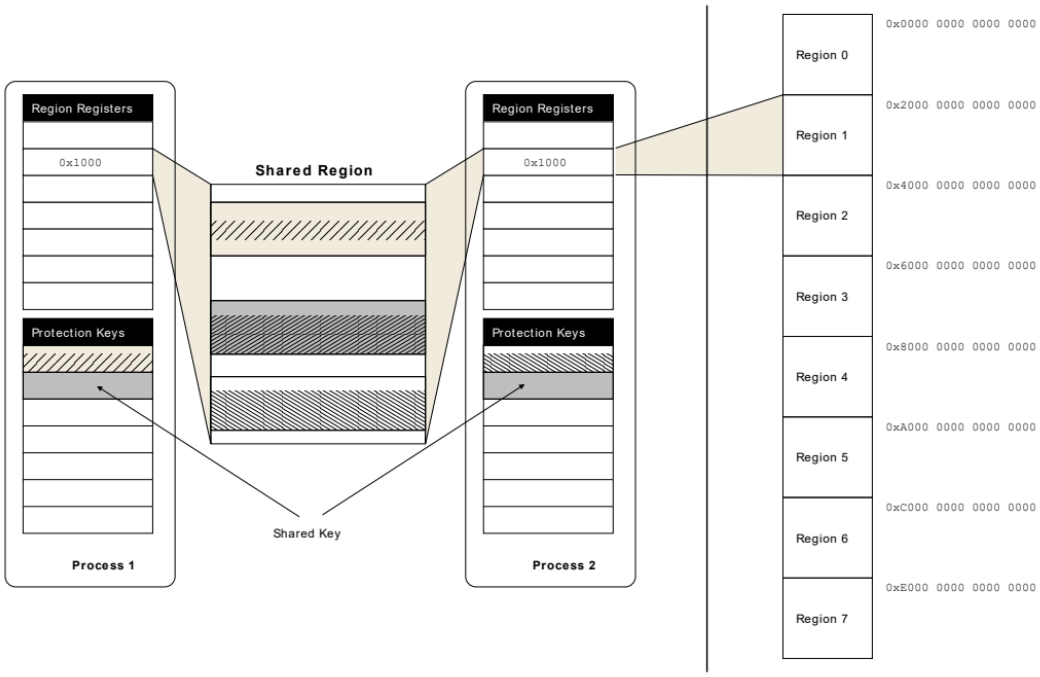


Рисунок 6.7. Регионы и ключи защиты Itanium.

Itanium поддерживает более мелкое разделение адресного пространства и записей TLB с сохранением аппаратной защиты. Его 64-битное пространство поделено на восемь регионов. В состоянии процесса есть восемь 24-битных регистров регионов с идентификаторами RID. Записи TLB помечаются RID и совпадают только при наличии соответствующего идентификатора у процесса.

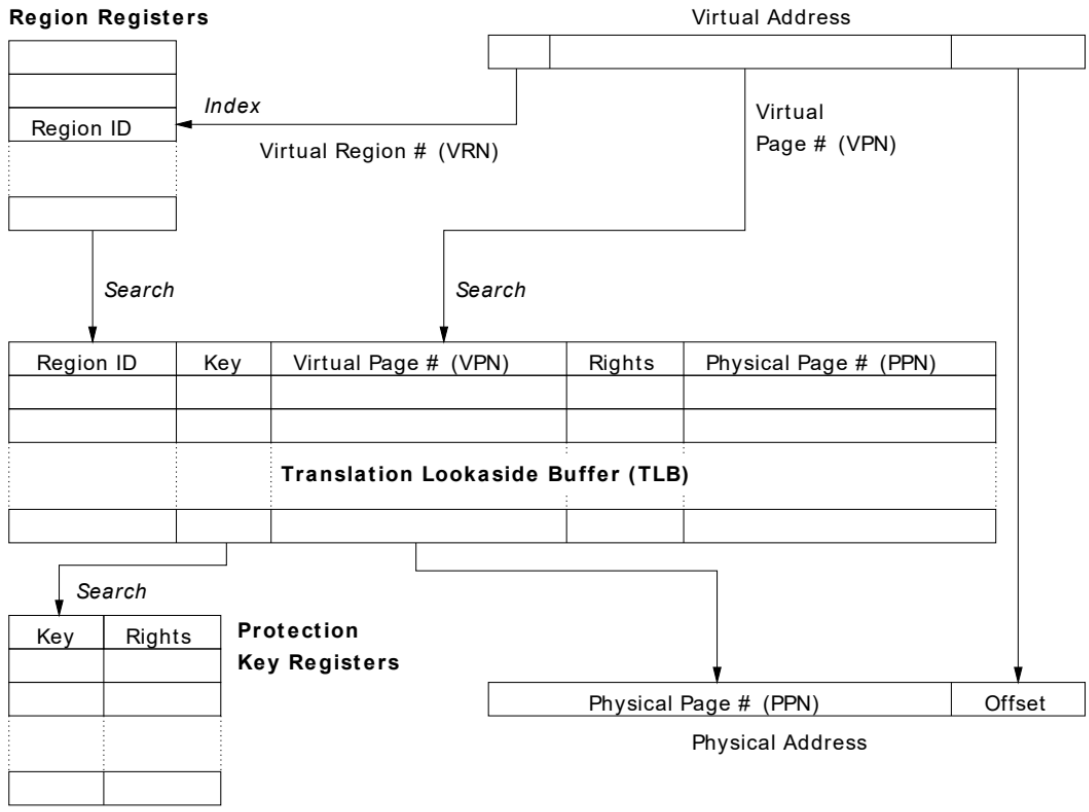


Рисунок 6.8. Преобразование адреса в TLB Itanium.

Старшие три бита, выбирающие регион, не участвуют в самом преобразовании. Поэтому два процесса с одинаковым RID могут иметь разные отображения одного региона. Например, если процесс A хранит RID `0x100` в регистре региона 3, а процесс B — в регистре региона 5, регион 3 первого соответствует региону 5 второго. Такое ограниченное разделение позволяет использовать общие записи TLB без доступа ко всему адресному пространству друг друга.

Ключи защиты

Для ещё более точного контроля каждая запись TLB Itanium также помечена ключом защиты. У процесса есть набор регистров ключей, управляемых ОС.

Когда страницы должны быть общими, например страницы кода системной библиотеки, им назначается ключ, который ОС предоставляет допущенным процессам. При обращении TLB сравнивает ключ записи с ключами процесса: совпадение разрешает доступ, иначе ОС получает исключение защиты.

Ключи могут ограничивать и права: одному процессу разрешена запись, другому — только чтение. Это позволяет разделять преобразования вплоть до одной страницы и потенциально значительно повышает эффективность TLB.

Аппаратный обход таблиц страниц Itanium

Передача управления ОС при промахе TLB создаёт значительные накладные расходы. Itanium может использовать встроенный аппаратный механизм обхода таблицы страниц, или HPW, который сам загружает преобразования в TLB. Он избегает дорогого перехода к ОС, но требует понятного аппаратуре фиксированного формата записей.

В документации Intel этот механизм называется VHPT walker — обходчик виртуально хешированной таблицы страниц. Предусмотрены две взаимоисключающие реализации: виртуальная линейная таблица и хеш-таблица.

Можно работать и без аппаратного обхода: тогда каждый промах обрабатывает ОС, как в архитектуре с программной загрузкой TLB. Однако снижение производительности настолько велико, что польза такого отключения маловероятна.

Виртуальная линейная таблица страниц

Эта реализация называется коротким форматом виртуально хешированной таблицы страниц — SF-VHPT. В описанной версии Linux на Itanium она используется по умолчанию.

Обычное решение — многоуровневая таблица, где биты номера страницы индексируют промежуточные уровни. Неиспользуемые области адресного пространства в иерархии просто отсутствуют. Для разреженного пространства с компактными группами отображений это требует гораздо меньше памяти, чем линейная таблица. Главный недостаток — несколько обращений к памяти при поиске.

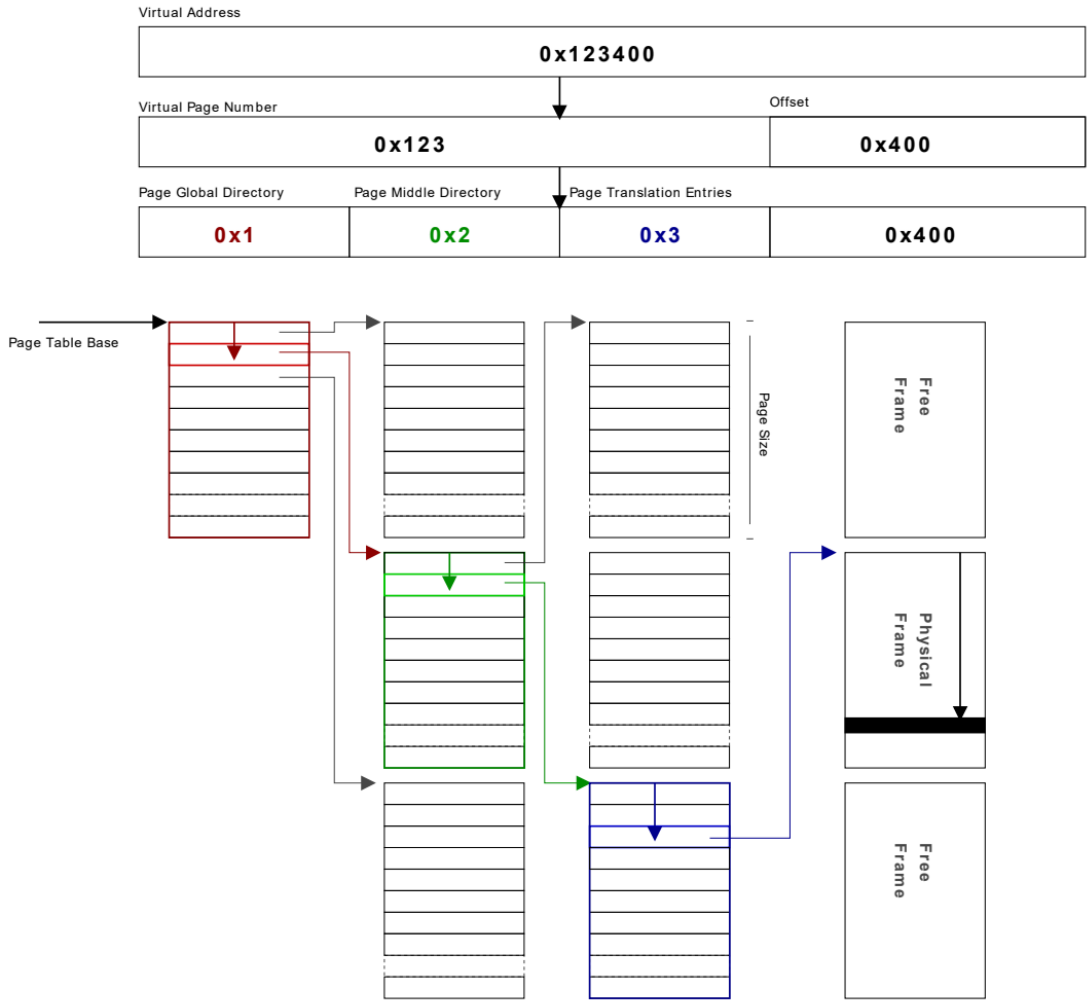


Рисунок 6.9. Иерархическая таблица страниц.

В 64-битном адресном пространстве даже огромная линейная таблица занимает лишь малую долю теоретических 16 эксабайт. Поэтому виртуальную линейную таблицу страниц (VLPT) можно разместить в непрерывной области виртуального пространства, не выделяя сразу всю соответствующую физическую память.

При промахе TLB аппаратный механизм, как и для физически линейной таблицы, использует номер виртуальной страницы как смещение от основания таблицы. Если запись действительна, она загружается в TLB. Но адрес самой записи VLPT тоже виртуальный, и нужной страницы таблицы может не оказаться в TLB. Тогда возникает вложенная ошибка, которую обрабатывает ОС, отображая страницу с записью в VLPT.

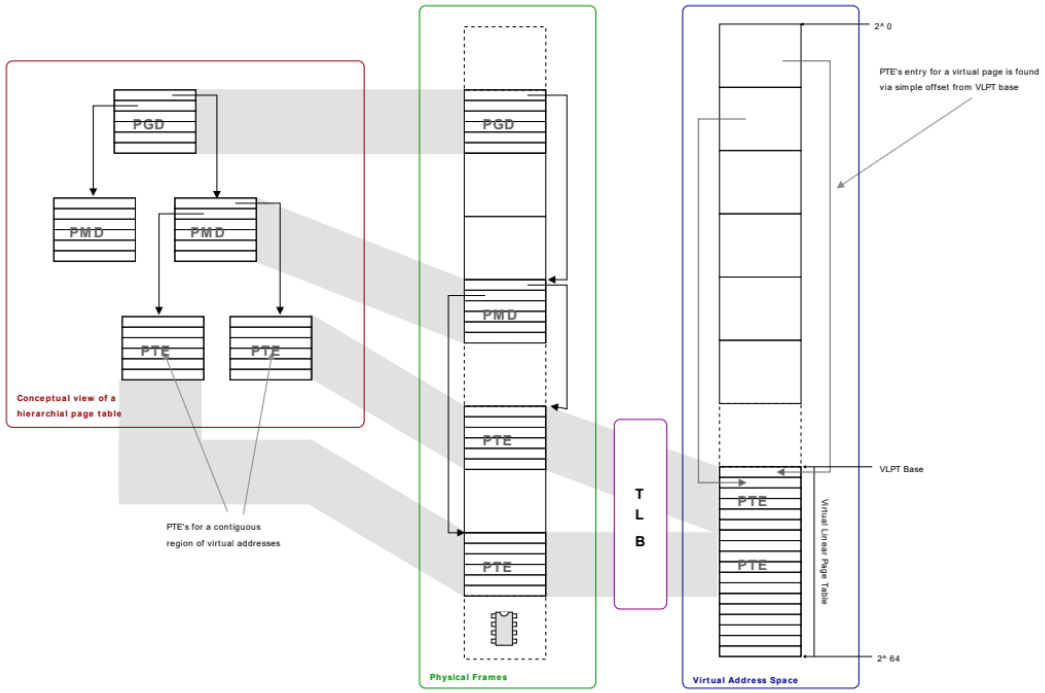


Рисунок 6.10. Короткий формат VHPT Itanium.

Если ОС ведёт иерархическую таблицу, процесс упрощается: её листовая страница содержит записи для непрерывного участка виртуальных адресов и может отображаться через TLB как часть VLPT.

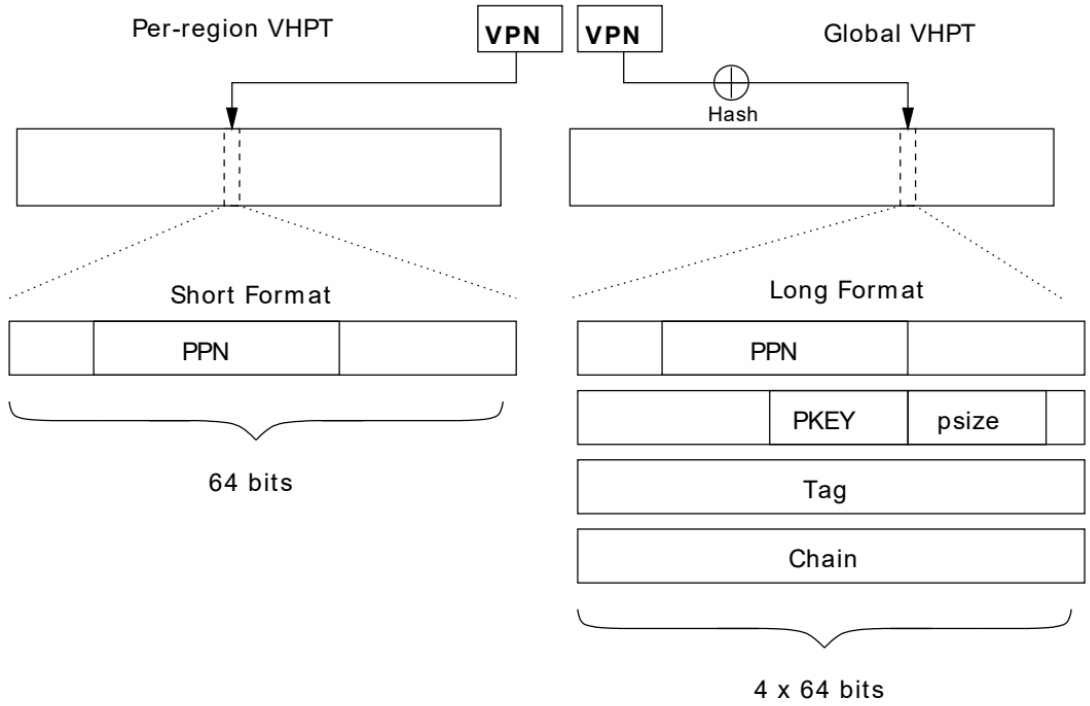


Рисунок 6.11. Форматы записей таблицы страниц Itanium.

Преимущество VLPT проявляется при повторных или последовательных обращениях. Первое исключение отображает страницу с множеством записей трансляции. Следующее обращение к соседней виртуальной странице получает её запись непосредственно из VLPT

быстро и без вызова ОС. Выигрыш возникает, если стоимость первого вложенного исключения окупается последующими попаданиями аппаратного обходчика.

Главный недостаток — самой VLPT нужны записи TLB, что увеличивает нагрузку на буфер. Поскольку каждому адресному пространству нужна таблица, накладные расходы растут с активностью системы. Однако дополнительные промахи могут компенсироваться более дешёвым аппаратным заполнением. Патологический шаблон доступа мог бы каждый раз перескакивать на `размер страницы / размер записи` элементов и вызывать вложенную ошибку, но это маловероятно.

Аппаратный обходчик ожидает определённый формат, показанный слева на рисунке 6.11. Для VLPT нужен восьмибайтовый короткий формат. Если ОС использует свои таблицы как основу VLPT, она должна соблюдать его. Архитектура оставляет некоторые игнорируемые биты для программных целей, но существенные изменения формата невозможны.

Линейная таблица предполагает фиксированный размер страницы: при смешанных размерах преобразование уже не находится по постоянному смещению. Поэтому у каждого из восьми регионов есть отдельная VLPT, отображающая только его адреса. Для каждого региона задаётся размер страницы по умолчанию; например, Linux HugeTLB может выделять регион для больших страниц. Однако размеры внутри одного региона смешивать нельзя.

Виртуальная хеш-таблица

Использование записей TLB для снижения стоимости его заполнения, как в SF-VHPT, не всегда выгодно. Itanium также поддерживает хешированную таблицу страниц, потенциально уменьшающую нагрузку на TLB. Процессор хеширует виртуальный адрес и получает смещение в непрерывной таблице.

Физически линейную таблицу можно считать хеш-таблицей с идеальной функцией без коллизий, но она требует непрактично большого непрерывного объёма физической памяти. Ограничение размера таблицы допускает коллизии: разные виртуальные адреса могут получить одно смещение. Тогда записи объединяются указателями в связный список, а нужная запись определяется по метке, полученной из виртуального адреса.

Из-за дополнительной информации этот вариант называется длинным форматом VHPT — LF-VHPT. Запись занимает 32 байта, как показано справа на рисунке 6.11.

Основное преимущество: глобальную хеш-таблицу можно закрепить одной записью TLB. Поскольку её разделяют все процессы, масштабирование может быть лучше, чем у SF-VHPT, где каждому процессу требуется всё больше записей для страниц VLPT. Но крупные записи хуже используют кэш: вместо одной 32-байтовой записи помещаются четыре короткие по 8 байтов. Большие кэши Itanium могут уменьшить этот недостаток.

Достоинство SF-VHPT в том, что ОС может хранить преобразования в иерархической таблице и, соблюдая аппаратный формат, непосредственно отображать листовые страницы в VLPT. При LF-VHPT она должна либо использовать хеш-таблицу как основной источник преобразований, либо поддерживать её как кэш собственных данных. Второй вариант

добавляет накладные расходы на критическом пути обработки исключений, хотя необходимость лишь одной записи TLB даёт преимущество.

Глава 7. Инструментальная цепочка

Компилируемые и интерпретируемые программы

Компилируемые программы

До сих пор мы рассматривали, как программа загружается в виртуальную память, запускается как процесс, за которым следит операционная система, и взаимодействует с ней посредством системных вызовов.

Программа, которую можно непосредственно загрузить в память, должна находиться в двоичном формате. Процесс преобразования исходного кода, написанного на языке вроде C, в готовый к исполнению двоичный файл называется компиляцией. Выполняет её компилятор; один из самых распространённых примеров — gcc™.

Интерпретируемые программы

У компилируемых программ есть некоторые недостатки для современной разработки. Каждый раз, когда разработчик вносит изменение, нужно снова вызвать компилятор и создать исполняемый файл. Логичным продолжением этой идеи является создание компилируемой программы, которая читает текст другой программы и выполняет её код строка за строкой.

Такую скомпилированную программу мы называем интерпретатором: она интерпретирует каждую строку входного файла и выполняет её как код. Благодаря этому саму программу компилировать не нужно, а все изменения вступают в силу при следующем запуске её кода интерпретатором.

За это удобство обычно приходится платить скоростью: интерпретируемые программы работают медленнее аналогичных компилируемых. Затраты на чтение и интерпретацию кода у компилируемой программы возникают только один раз, тогда как у интерпретируемой — при каждом запуске.

Однако у интерпретируемых языков есть много достоинств. Многие из них работают на виртуальной машине, абстрагированной от нижележащего оборудования. Python и Perl 6 — примеры языков, реализующих виртуальную машину для выполнения интерпретируемого кода.

Виртуальные машины

Скомпилированная программа полностью зависит от оборудования машины, для которой она собрана: её должно быть возможно просто скопировать в память и выполнить. Виртуальная машина представляет собой программную абстракцию оборудования.

Например, Java — гибридный язык, частично компилируемый и частично интерпретируемый. Код Java компилируется в программу, выполняемую внутри виртуальной машины Java, чаще

называемой JVM. Это означает, что скомпилированная программа может работать на любом оборудовании, для которого написана JVM: так реализуется принцип «написать один раз, запускать где угодно».

Построение исполняемого файла

Когда мы говорим о компиляторе, создание исполняемого файла на самом деле включает три отдельных этапа:

1. Компиляция.
2. Ассемблирование.
3. Компоновка.

Участвующие в этом процессе компоненты вместе называются инструментальной цепочкой: выход каждого инструмента поступает на вход следующего, пока не получится конечный результат. Каждое звено последовательно приближает исходный код к двоичному коду, пригодному для исполнения.

Компиляция

Процесс компиляции

Первый шаг на пути от исходного файла к исполняемому — преобразование кода с высокоуровневого языка, понятного человеку, в код ассемблера. Из предыдущих глав мы знаем, что код ассемблера непосредственно использует инструкции и регистры процессора.

Компилятор — самый сложный этап этого процесса по нескольким причинам. Во-первых, люди весьма непредсказуемы и записывают исходный код множеством способов. Компилятору интересен только сам код, тогда как людям нужны комментарии и пробельные символы — пробелы, табуляции, отступы и так далее, — чтобы его понимать. Преобразование написанного человеком исходного текста во внутреннее представление компилятора называется синтаксическим анализом, или разбором.

Код C

В случае C перед синтаксическим анализом выполняется ещё один шаг — препроцессорная обработка. По своей сути препроцессор представляет собой программу замены текста. Например, после директивы `#define variable text` вхождения `variable` будут заменены на `text`. Затем обработанный код передаётся компилятору.

Синтаксис

У каждого вычислительного языка есть определённый синтаксис, описывающий его правила. И вы, и компилятор знаете эти правила и, если всё идёт хорошо, понимаете друг друга. Но людям свойственно забывать и нарушать правила, из-за чего компилятор не может понять

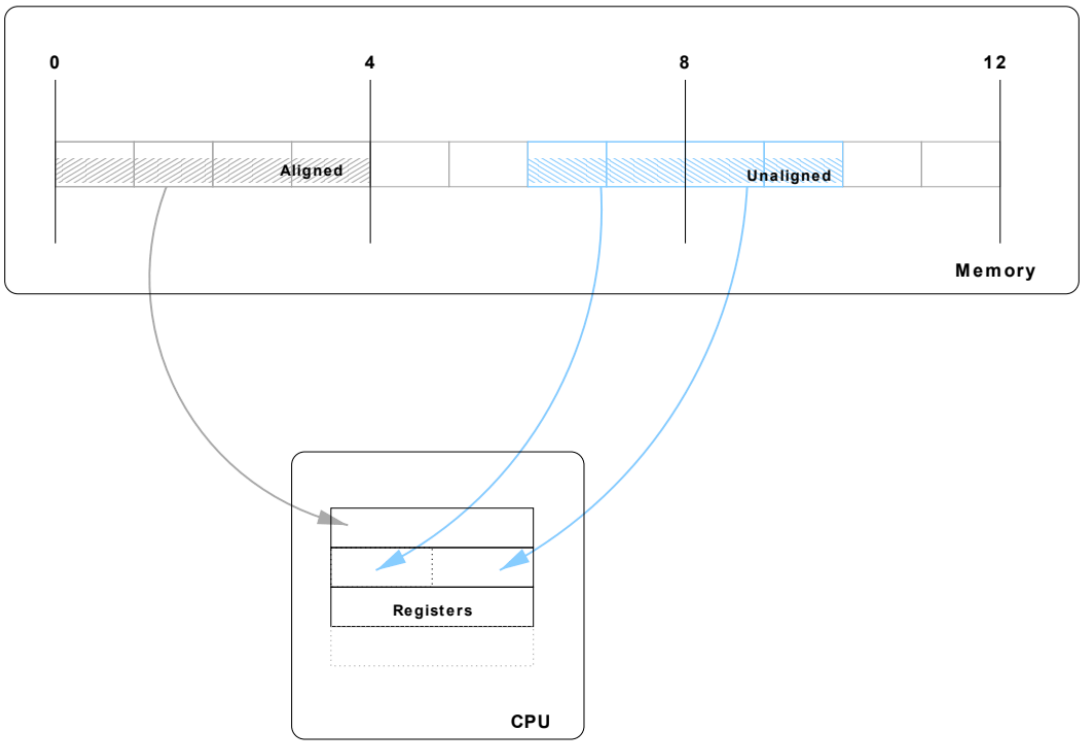
ваши намерения. Например, если в условии `if` пропустить закрывающую скобку, компилятор не поймёт, где заканчивается условие.

Синтаксис чаще всего описывается формой Бэкуса — Наура (BNF) — языком для описания языков! На практике особенно распространена расширенная форма Бэкуса — Наура, EBNF, поскольку она допускает дополнительные правила, лучше подходящие для современных языков.

Генерация кода ассемблера

Работа компилятора состоит в преобразовании высокоуровневого языка в ассемблерный код, подходящий для целевой платформы. Очевидно, разные архитектуры имеют разные наборы инструкций, разное число регистров и разные правила корректной работы.

Выравнивание



Выравнивание переменных в памяти важно для компилятора. Системным программистам следует знать об ограничениях на выравнивание, чтобы помочь компилятору создавать наиболее эффективный код.

Процессоры обычно не могут загрузить значение в регистр из произвольного адреса памяти: требуется выравнивать переменные по определённым границам. На рисунке показано, как 32-битное значение (четыре байта) загружается в регистр на машине, требующей выравнивания переменных по четырём байтам.

Первую переменную можно загрузить в регистр напрямую: она расположена между четырёхбайтовыми границами. Вторая же пересекает такую границу. Поэтому, чтобы

поместить её в один регистр, потребуется как минимум два чтения: сначала младшей, затем старшей части.

Некоторые архитектуры, например x86, обрабатывают невыровненные загрузки аппаратно; единственным заметным следствием будет снижение производительности из-за дополнительной работы оборудования. Другие архитектуры не допускают нарушения правил выравнивания и возбуждают исключение. Его обычно перехватывает операционная система, которой приходится вручную загружать регистр по частям, что создаёт ещё большие накладные расходы.

Дополнение структур

Программистам особенно важно учитывать выравнивание при создании структур. Хотя компилятор знает правила выравнивания целевой архитектуры, иногда программист может непреднамеренно получить неоптимальное поведение.

Стандарт C99 гарантирует лишь, что поля структуры располагаются в памяти в порядке их объявления и что все элементы массива структур имеют одинаковый размер.

Пример 7.1. Дополнение структуры

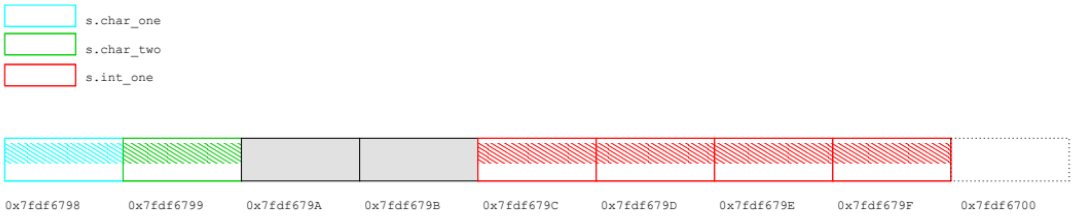
```
$ cat struct.c
#include <stdio.h>

struct a_struct {
    char char_one;
    char char_two;
    int int_one;
};

int main(void)
{
    struct a_struct s;
    printf("%p : s.char_one\n" \
           "%p : s.char_two\n" \
           "%p : s.int_one\n", &s.char_one,
           &s.char_two, &s.int_one);
    return 0;
}

$ gcc -o struct struct.c
$ gcc -fpack-struct -o struct-packed struct.c
$ ./struct
0x7fdf6798 : s.char_one
0x7fdf6799 : s.char_two
0x7fdf679c : s.int_one
$ ./struct-packed
0x7fcd2778 : s.char_one
0x7fcd2779 : s.char_two
0x7fcd277a : s.int_one
```

В этом примере мы создали структуру с двумя байтами (`char`), за которыми следует четырёхбайтовое целое число. Компилятор добавляет в структуру промежуточные байты, как показано ниже.



В другом варианте мы указали компилятору не добавлять отступы, и целое число начинается непосредственно после двух полей `char`.

Выравнивание по строкам кэша

Ранее мы говорили об алиасинге в кэше и о том, как нескольким адресам может соответствовать одна строка кэша. Программистам следует избегать постоянного вытеснения строк кэша.

Такая ситуация возникает, когда программа постоянно обращается к двум областям памяти, отображаемым в одну строку кэша. Строка используется неэффективно: её загружают, немного используют, затем вытесняют и загружают на её место другую.

Если это повторяется, производительность заметно снижается. Проблему можно уменьшить, организовав конфликтующие данные несколько иначе, чтобы избежать такого конфликта.

Один из способов обнаружения — профилирование: наблюдение за программой для анализа путей выполнения и затрачиваемого времени. При оптимизации с использованием профиля (PGO) компилятор может добавить в первую сборку специальный код, который во время работы записывает сведения о выбранных ветвях и других событиях. Затем программу можно перекомпилировать с учётом этих данных и, возможно, получить более быстрый результат. Либо программист сам изучает профиль и обнаруживает, например, постоянное вытеснение строк кэша.

Компромисс между объёмом памяти и скоростью

В предыдущем примере компилятор потратил немного дополнительной памяти ради повышения скорости выполнения. Он знает правила архитектуры и способен выбрать способ выравнивания, пожертвовав небольшим объёмом памяти ради повышения производительности или даже обеспечения правильной работы.

Поэтому программисту не следует предполагать, как именно компилятор расположит переменные и данные. Такой код непереносим: на другой архитектуре правила могут отличаться, а компилятор может принять иные решения в зависимости от явных параметров и уровня оптимизации.

Допущения

Программисту на C нужно знать, какие предположения о работе компилятора допустимы, а что может меняться. Подробные правила изложены в стандарте C99. Если вы

программируете на C, стоит ознакомиться с ними, чтобы не писать непереносимый или ошибочный код.

Пример 7.2. Выравнивание стека

```
$ cat stack.c
#include <stdio.h>
struct a_struct {
    int a;
    int b;
};
int main(void)
{
    int i;
    struct a_struct s;
    printf("%p\n%p\ndiff %ld\n", &i, &s, (unsigned long)&s - (unsigned long)&i);
    return 0;
}
$ gcc-3.3 -Wall -o stack-3.3 ./stack.c
$ gcc-4.0 -o stack-4.0 stack.c
$ ./stack-3.3
0x60000fffffc2b510
0x60000fffffc2b520
diff 16
$ ./stack-4.0
0x60000fffff89b520
0x60000fffff89b524
diff 4
```

В этом примере с машины Itanium видно, что дополнение и выравнивание стека значительно изменились между версиями gcc. Такие изменения ожидаемы, и программист должен их учитывать. В общем случае не следует делать предположения о размерах типов и правилах выравнивания.

Типичные приёмы C для выравнивания

Существует несколько распространённых последовательностей кода для работы с выравниванием; большинство программ так или иначе его учитывают. Такие приёмы встречаются не только в ядре, но и во многих программах, работающих с блоками данных, поэтому стоит их изучить.

Примеры можно взять из ядра Linux, которому часто приходится работать с выравниванием страниц памяти системы.

Пример 7.3. Операции выравнивания страниц

```
[ include/asm-ia64/page.h ]
/*
 * PAGE_SHIFT determines the actual kernel page size.
 */
#if defined(CONFIG_IA64_PAGE_SIZE_4KB)
# define PAGE_SHIFT     12
#elif defined(CONFIG_IA64_PAGE_SIZE_8KB)
# define PAGE_SHIFT     13
#elif defined(CONFIG_IA64_PAGE_SIZE_16KB)
# define PAGE_SHIFT     14
#elif defined(CONFIG_IA64_PAGE_SIZE_64KB)
# define PAGE_SHIFT     16
```

```
#else
# error Unsupported page size!
#endif
#define PAGE_SIZE       (__IA64_UL_CONST(1) << PAGE_SHIFT)
#define PAGE_MASK       (~(PAGE_SIZE - 1))
#define PAGE_ALIGN(addr) (((addr) + PAGE_SIZE - 1) & PAGE_MASK)
```

Выше показано несколько возможных размеров страницы ядра — от 4 до 64 КБ.

Назначение макроса `PAGE_SIZE` достаточно очевидно: он выдаёт выбранный размер страницы, сдвигая единицу на заданное число разрядов. Напомним, что это эквивалентно 2^n, где n равно `PAGE_SHIFT`.

Затем определяется `PAGE_MASK`. Автор указывает, что эта маска позволяет выделить биты внутри текущей страницы, то есть смещение адреса в ней.

> Примечание переводчика: формула `PAGE_MASK` в примере, напротив, обнуляет биты смещения; сами биты смещения можно получить с помощью `PAGE_SIZE - 1`.

Далее в оригинале оставлена пометка автора о необходимости продолжить обсуждение.

Оптимизация

Когда компилятор построил внутреннее представление кода, начинается самое интересное. Его цель — получить для заданного входного кода наиболее оптимизированный ассемблерный вывод. Это обширная и разнообразная задача, требующая знаний от эффективных алгоритмов информатики до глубокого понимания конкретного процессора, на котором будет выполняться код.

Ниже приведены некоторые распространённые оптимизации, которые компилятор может применять при генерации кода. Существует множество других стратегий, а поиск лучших способов генерации остаётся активной областью исследований.

Общая оптимизация

Компилятор нередко может определить, что некоторый участок кода никогда не используется, и исключить его, либо заменить конструкцию языка более короткой, но дающей тот же результат.

Развёртывание циклов

Если код содержит цикл `for` или `while` и компилятору известно примерное число его повторений, может быть эффективнее развернуть цикл в последовательность команд. Вместо выполнения тела с последующим переходом к началу тело дублируется и выполняется повторно без перехода.

Это увеличивает размер кода, но может позволить процессору эффективнее обрабатывать инструкции: переходы способны снижать эффективность конвейера инструкций.

Встраивание функций

Подобно развёртыванию цикла, код вызываемой функции можно встроить в вызывающую. Программист может попросить компилятор попытаться сделать это, указав `inline` в определении функции. И здесь увеличение размера кода обменивается на более последовательное выполнение инструкций.

Предсказание переходов

У условия `if` два возможных исхода: истина и ложь. Процессор стремится как можно полнее загрузить свои конвейеры, поэтому он не может ждать результата проверки, прежде чем подавать в них команды.

Компилятор может предположить, какой исход вероятнее. Для таких предположений применяются простые правила: например, условие `if (val == -1)`, вероятно, ложно, поскольку `-1` обычно обозначает код ошибки, а ошибки, как мы надеемся, возникают не слишком часто.

Некоторые компиляторы позволяют собрать программу, запустить её и записать, как выбирались ветви в реальных условиях, а затем собрать её заново с учётом наблюдений.

Ассемблер

Ассемблерный код, выданный компилятором, остаётся читаемым для человека, знакомого с особенностями ассемблера данного процессора. Разработчики часто просматривают его, чтобы вручную проверить оптимальность кода или найти ошибки компилятора. Последнее встречается чаще, чем можно подумать, особенно при агрессивной оптимизации.

Работа ассемблера более механическая: он переводит ассемблерный код в двоичную форму. По сути, ассемблер хранит большую таблицу всех возможных инструкций и их двоичных эквивалентов — кодов операций, или опкодов. Сочетая их с указанными регистрами, он создаёт двоичный выходной файл.

Такой код называется объектным и на этом этапе ещё не является исполняемым. Это двоичное представление конкретного исходного файла. Хорошая практика программирования советует не «класть все яйца в одну корзину», помещая весь исходный код в один файл.

Компоновщик

В большой программе код часто разделяют на несколько файлов, группируя связанные функции. Каждый из файлов можно скомпилировать в объектный код, но конечная цель — один исполняемый файл. Для объединения объектных файлов используется компоновка.

Даже если программа умещается в одном файле, для правильной работы её всё равно нужно скомпоновать с некоторыми системными библиотеками. Например, вызов `printf` находится в библиотеке, которую необходимо объединить с исполняемым файлом. Поэтому, хотя вам не приходится явно заботиться о компоновке, при создании программы она обязательно происходит.

В следующих разделах объясняются необходимые для понимания компоновки термины.

Символы

Переменные и функции имеют имена в исходном коде, по которым мы к ним обращаемся. Объявление `int a` можно понимать как указание компилятору: «Выдели `sizeof(int)` байтов памяти, и отныне имя `a` будет обозначать эту область». Аналогично функция означает: «Сохрани этот код в памяти, а при вызове `function()` перейди к нему и выполни его».

В этом случае `a` и `function` называются символами: они являются символическим представлением области памяти.

Символы помогают людям понимать программы. Можно сказать, что главная задача компиляции — удалить символы: процессор не знает, что означает `a`, он знает лишь, что по некоторому адресу памяти находятся данные. Компиляция должна преобразовать `a += 2` во что-то вроде «увеличить на 2 значение по адресу 0xABCDE».

Видимость символов

В программах на C вы могли видеть модификаторы переменных `static` и `extern`. Они влияют на так называемую видимость символов.

Представьте, что программа разделена на два файла, а некоторым функциям нужна общая переменная. Для неё нужно только одно определение, то есть одна область памяти — иначе переменная не была бы общей, — но ссылаться на неё должны оба файла.

Для этого переменную определяют в одном файле, а в другом объявляют переменную с тем же именем и префиксом `extern`. Слово означает «внешний» и сообщает человеку, что переменная определена где-то ещё.

Для компилятора `extern` означает, что память для переменной выделять не нужно, а символ следует оставить в объектном коде для последующего разрешения. Компилятор не может знать, где именно определён символ, зато это может выяснить компоновщик: его работа — рассмотреть объектные файлы вместе и объединить их в один исполняемый файл. Увидев символ во втором файле, он может сопоставить его с уже известным определением из первого и подставить адрес этой переменной.

`static` почти противоположен `extern`: он ограничивает видимость указанного символа. При объявлении переменной с `static` компилятору сообщается, что символ не должен быть доступен для связывания с другими объектными файлами. Поэтому компоновщик не сможет связать его с одноимённым символом из другого файла. Это помогает разделять код и уменьшать конфликты: имя статической переменной можно повторно использовать в других файлах. В отличие от неё, нестатический символ доступен компоновщику извне.

Процесс компоновки

По существу, компоновка состоит из двух действий: объединения объектных файлов в один исполняемый и разрешения символов из каждого объектного файла. Обычно для этого

нужны два прохода: сначала считываются определения и отмечаются неразрешённые символы, затем оставшиеся ссылки исправляются на нужные адреса.

В конечном исполняемом файле не должно оставаться неразрешённых символов: иначе компоновщик выдаст ошибку. Здесь речь идёт о статической компоновке. Динамическая компоновка — похожая операция во время выполнения программы; она рассматривается далее.

Практический пример

Пройдём пошагово весь процесс построения простого приложения.

Когда вы вводите `gcc`, фактически запускается управляющая программа, скрывающая большинство шагов. Обычно именно этого и хочется: точные команды и параметры для создания работающего исполняемого файла могут быть довольно сложными и зависеть от архитектуры.

Мы покажем процесс на двух исходных файлах C. В одном определена функция `main()` — начальная точка входа программы, а в другом — вспомогательная функция. Для иллюстрации добавлена и глобальная переменная.

Пример 7.4. Hello World

```
#include <stdio.h>

/* We need a prototype so the compiler knows what types function() takes */
int function(char *input);

/* Since this is static, we can define it in both hello.c and function.c */
static int i = 100;

/* This is a global variable */
int global = 10;

int main(void)
{
    /* function() should return the value of global */
    int ret = function("Hello, World!");
    exit(ret);
}
```

Пример 7.5. Вспомогательная функция

```
#include <stdio.h>

static int i = 100;

/* Declard as extern since defined in hello.c */
extern int global;

int function(char *input)
{
    printf("%s\n", input);
    return global;
}
```

Компиляция примера

У всех компиляторов есть возможность выполнить только первый этап компиляции. Обычно для этого применяется параметр `-S`, а результат записывается в файл с тем же именем, что и исходный, но с расширением `.s`. Поэтому первый этап можно показать с помощью `gcc -S`, как в примере ниже.

Пример 7.6. Компиляция

```
ianw@lime:~/programs/csbu/wk7/code$ gcc -S hello.c
ianw@lime:~/programs/csbu/wk7/code$ gcc -S function.c
    ianw@lime:~/programs/csbu/wk7/code$ cat function.s
.file   "function.c"
.pred.safe_across_calls p1-p5,p16-p63
.section        .sdata,"aw",@progbits
.align 4
.type   i#, @object
.size   i#, 4
    i:
data4   100
.section        .rodata
.align 8
.LC0:
stringz "%s\n"
.text
.align 16
.global function#
.proc function#
    function:
.prologue 14, 33
.save ar.pfs, r34
alloc r34 = ar.pfs, 1, 4, 2, 0
.vframe r35
mov r35 = r12
adds r12 = -16, r12
mov r36 = r1
.save rp, r33
mov r33 = b0
.body
;;
st8 [r35] = r32
addl r14 = @ltoffx(.LC0), r1
;;
ld8.mov r37 = [r14], .LC0
ld8 r38 = [r35]
br.call.sptk.many b0 = printf#
mov r1 = r36
;;
addl r15 = @ltoffx(global#), r1
;;
ld8.mov r14 = [r15], global#
;;
ld4 r14 = [r14]
;;
mov r8 = r14
mov ar.pfs = r34
mov b0 = r33
.restore sp
mov r12 = r35
br.ret.sptk.many b0
;;
.endp function#
.ident  "GCC: (GNU) 3.3.5 (Debian 1:3.3.5-11)"
```

Ассемблерный код несколько сложен для полного разбора, но вы должны увидеть, где `i` определяется как `data4` — четыре байта, или 32 бита, размер `int`, — где определена `function` (`function:`) и где вызывается `printf()`.

Теперь у нас есть два ассемблерных файла, готовых к преобразованию в машинный код.

Ассемблирование

Ассемблирование — довольно простой процесс. Ассемблер обычно называется `as` и принимает аргументы примерно так же, как `gcc`.

Пример 7.7. Ассемблирование

```
ianw@lime:~/programs/csbu/wk7/code$ as -o function.o function.s
ianw@lime:~/programs/csbu/wk7/code$ as -o hello.o hello.s
ianw@lime:~/programs/csbu/wk7/code$ ls
function.c  function.o  function.s  hello.c  hello.o  hello.s
```

После ассемблирования у нас есть объектный код, готовый к компоновке в исполняемый файл. Обычно можно не вызывать ассемблер вручную: вызов компилятора с `-c` непосредственно преобразует исходный файл в объектный и запишет результат в файл с тем же именем и расширением `.o`.

Непосредственно прочитать объектный код нельзя, поскольку он представлен в двоичном формате; этот формат мы изучим позже. Однако можно исследовать объектные файлы с помощью инструментов: например, `readelf --symbols` покажет их символы.

Пример 7.8. Readelf

```
ianw@lime:~/programs/csbu/wk7/code$ readelf --symbols ./hello.o

Symbol table '.symtab' contains 15 entries:
Num:    Value          Size Type    Bind   Vis      Ndx Name
0: 0000000000000000     0 NOTYPE  LOCAL  DEFAULT  UND
1: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS hello.c
2: 0000000000000000     0 SECTION LOCAL  DEFAULT    1
3: 0000000000000000     0 SECTION LOCAL  DEFAULT    3
4: 0000000000000000     0 SECTION LOCAL  DEFAULT    4
5: 0000000000000000     0 SECTION LOCAL  DEFAULT    5
6: 0000000000000000     4 OBJECT  LOCAL  DEFAULT    5 i
7: 0000000000000000     0 SECTION LOCAL  DEFAULT    6
8: 0000000000000000     0 SECTION LOCAL  DEFAULT    7
9: 0000000000000000     0 SECTION LOCAL  DEFAULT    8
10: 0000000000000000     0 SECTION LOCAL  DEFAULT   10
11: 0000000000000004     4 OBJECT  GLOBAL DEFAULT    5 global
12: 0000000000000000    96 FUNC    GLOBAL DEFAULT    1 main
13: 0000000000000000     0 NOTYPE  GLOBAL DEFAULT  UND function
14: 0000000000000000     0 NOTYPE  GLOBAL DEFAULT  UND exit

ianw@lime:~/programs/csbu/wk7/code$ readelf --symbols ./function.o
Symbol table '.symtab' contains 14 entries:
Num:    Value          Size Type    Bind   Vis      Ndx Name
0: 0000000000000000     0 NOTYPE  LOCAL  DEFAULT  UND
1: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS function.c
2: 0000000000000000     0 SECTION LOCAL  DEFAULT    1
3: 0000000000000000     0 SECTION LOCAL  DEFAULT    3
4: 0000000000000000     0 SECTION LOCAL  DEFAULT    4
```

```
5: 0000000000000000     0 SECTION LOCAL  DEFAULT    5
6: 0000000000000000     4 OBJECT  LOCAL  DEFAULT    5 i
7: 0000000000000000     0 SECTION LOCAL  DEFAULT    6
8: 0000000000000000     0 SECTION LOCAL  DEFAULT    7
9: 0000000000000000     0 SECTION LOCAL  DEFAULT    8
10: 0000000000000000     0 SECTION LOCAL  DEFAULT   10
11: 0000000000000000   128 FUNC    GLOBAL DEFAULT    1 function
12: 0000000000000000     0 NOTYPE  GLOBAL DEFAULT  UND printf
13: 0000000000000000     0 NOTYPE  GLOBAL DEFAULT  UND global
```

Хотя вывод снова выглядит довольно сложным, многое в нём должно быть понятно:

- В выводе для `hello.o` найдите символ `i`. Он помечен `LOCAL`, поскольку мы объявили его `static` и тем самым указали, что он локален для этого объектного файла.
- В том же выводе глобальная переменная помечена `GLOBAL`, то есть видна за пределами файла. Аналогично вовне видна функция `main()`.
- Символ `function`, соответствующий вызову `function()`, имеет отметку `UND`, то есть не определён. Это означает, что адрес функции должен найти компоновщик.
- Рассмотрите символы в файле `function.c` и то, как они представлены в выводе.

Компоновка примера

Непосредственный вызов компоновщика `ld` в реальной системе — очень сложный процесс. Поэтому мы поручаем компоновку `gcc`.

Но, конечно, с помощью параметра `-v` (подробный вывод) можно подсмотреть, что `gcc` делает внутри.

Пример 7.9. Компоновка

```
/usr/lib/gcc-lib/ia64-linux/3.3.5/collect2 -static
/usr/lib/gcc-lib/ia64-linux/3.3.5/../../../crt1.o
/usr/lib/gcc-lib/ia64-linux/3.3.5/../../../crti.o
/usr/lib/gcc-lib/ia64-linux/3.3.5/crtbegin.o
-L/usr/lib/gcc-lib/ia64-linux/3.3.5
-L/usr/lib/gcc-lib/ia64-linux/3.3.5/../../..
hello.o
function.o
--start-group
-lgcc
-lgcc_eh
-lunwind
-lc
--end-group
/usr/lib/gcc-lib/ia64-linux/3.3.5/crtend.o
/usr/lib/gcc-lib/ia64-linux/3.3.5/../../../crtn.o
```

Первое, что бросается в глаза, — вызов программы `collect2`. Это простая обёртка над `ld`, используемая внутри `gcc`.

Затем видно, что компоновщику передаются объектные файлы, начинающиеся с `crt`. Они предоставляются `gcc` и системными библиотеками и содержат код, необходимый для запуска программы. В действительности первой при запуске вызывается не `main()`, а функция

`_start`, находящаяся в объектных файлах `crt`. Она выполняет общую подготовку, о которой разработчику приложения обычно не нужно заботиться.

Иерархия путей довольно сложна, но по существу на последнем этапе дополнительно подключаются следующие объектные файлы:

- `crt1.o`: предоставляется системной библиотекой `libc` и содержит функцию `_start`, вызываемую в программе первой;
- `crti.o`: предоставляется системными библиотеками;
- `crtbegin.o`;
- `crtsaveres.o`;
- `crtend.o`;
- `crtn.o`.

Позже мы обсудим, как они используются для запуска программы.

Далее подключаются наши объектные файлы `hello.o` и `function.o`, а затем с помощью параметров `-l` указываются дополнительные библиотеки. Они зависят от системы и нужны каждой программе. Главная из них — `-lc`, подключающая библиотеку C с распространёнными функциями, такими как `printf()`.

После этого подключаются ещё некоторые системные объектные файлы, выполняющие очистку после завершения программы.

Хотя детали сложны, сама идея проста: все объектные файлы объединяются в один готовый к запуску исполняемый файл.

Исполняемый файл

Скоро мы рассмотрим исполняемые файлы подробнее, но пока можно исследовать их подобно объектным файлам и посмотреть, что произошло.

Пример 7.10. Исполняемый файл

```
ianw@lime:~/programs/csbu/wk7/code$ gcc -o program hello.c function.c
ianw@lime:~/programs/csbu/wk7/code$ readelf --symbols ./program

Symbol table '.dynsym' contains 11 entries:
Num:    Value          Size Type    Bind   Vis      Ndx Name
0: 0000000000000000     0 NOTYPE  LOCAL  DEFAULT  UND
         1: 6000000000000de0     0 OBJECT  GLOBAL DEFAULT  ABS _DYNAMIC
         2: 0000000000000000   176 FUNC    GLOBAL DEFAULT  UND printf@GLIBC_2.2 (2)
3: 600000000000109c     0 NOTYPE  GLOBAL DEFAULT  ABS __bss_start
         4: 0000000000000000   704 FUNC    GLOBAL DEFAULT  UND exit@GLIBC_2.2 (2)
         5: 600000000000109c     0 NOTYPE  GLOBAL DEFAULT  ABS _edata
         6: 6000000000000fe8     0 OBJECT  GLOBAL DEFAULT  ABS _GLOBAL_OFFSET_TABLE_
         7: 60000000000010b0     0 NOTYPE  GLOBAL DEFAULT  ABS _end
         8: 0000000000000000     0 NOTYPE  WEAK   DEFAULT  UND _Jv_RegisterClasses
9: 0000000000000000   544 FUNC    GLOBAL DEFAULT  UND __libc_start_main@GLIBC_2.2 (2)
        10: 0000000000000000     0 NOTYPE  WEAK   DEFAULT  UND __gmon_start__

    Symbol table '.symtab' contains 127 entries:
       Num:    Value          Size Type    Bind   Vis      Ndx Name
```

```
0: 0000000000000000     0 NOTYPE  LOCAL  DEFAULT  UND
         1: 40000000000001c8     0 SECTION LOCAL  DEFAULT    1
         2: 40000000000001e0     0 SECTION LOCAL  DEFAULT    2
         3: 4000000000000200     0 SECTION LOCAL  DEFAULT    3
         4: 4000000000000240     0 SECTION LOCAL  DEFAULT    4
5: 4000000000000348     0 SECTION LOCAL  DEFAULT    5
         6: 40000000000003d8     0 SECTION LOCAL  DEFAULT    6
         7: 40000000000003f0     0 SECTION LOCAL  DEFAULT    7
         8: 4000000000000410     0 SECTION LOCAL  DEFAULT    8
         9: 4000000000000440     0 SECTION LOCAL  DEFAULT    9
10: 40000000000004a0     0 SECTION LOCAL  DEFAULT   10
        11: 40000000000004e0     0 SECTION LOCAL  DEFAULT   11
        12: 40000000000005e0     0 SECTION LOCAL  DEFAULT   12
        13: 4000000000000b00     0 SECTION LOCAL  DEFAULT   13
        14: 4000000000000b40     0 SECTION LOCAL  DEFAULT   14
15: 4000000000000b60     0 SECTION LOCAL  DEFAULT   15
        16: 4000000000000bd0     0 SECTION LOCAL  DEFAULT   16
        17: 4000000000000ce0     0 SECTION LOCAL  DEFAULT   17
        18: 6000000000000db8     0 SECTION LOCAL  DEFAULT   18
        19: 6000000000000dd0     0 SECTION LOCAL  DEFAULT   19
20: 6000000000000dd8     0 SECTION LOCAL  DEFAULT   20
        21: 6000000000000de0     0 SECTION LOCAL  DEFAULT   21
        22: 6000000000000fc0     0 SECTION LOCAL  DEFAULT   22
        23: 6000000000000fd0     0 SECTION LOCAL  DEFAULT   23
        24: 6000000000000fe0     0 SECTION LOCAL  DEFAULT   24
25: 6000000000000fe8     0 SECTION LOCAL  DEFAULT   25
        26: 6000000000001040     0 SECTION LOCAL  DEFAULT   26
        27: 6000000000001080     0 SECTION LOCAL  DEFAULT   27
        28: 60000000000010a0     0 SECTION LOCAL  DEFAULT   28
        29: 60000000000010a8     0 SECTION LOCAL  DEFAULT   29
30: 0000000000000000     0 SECTION LOCAL  DEFAULT   30
        31: 0000000000000000     0 SECTION LOCAL  DEFAULT   31
        32: 0000000000000000     0 SECTION LOCAL  DEFAULT   32
        33: 0000000000000000     0 SECTION LOCAL  DEFAULT   33
        34: 0000000000000000     0 SECTION LOCAL  DEFAULT   34
35: 0000000000000000     0 SECTION LOCAL  DEFAULT   35
        36: 0000000000000000     0 SECTION LOCAL  DEFAULT   36
        37: 0000000000000000     0 SECTION LOCAL  DEFAULT   37
        38: 0000000000000000     0 SECTION LOCAL  DEFAULT   38
        39: 0000000000000000     0 SECTION LOCAL  DEFAULT   39
40: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS /build/buildd/glibc-2.3.2
        41: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS /build/buildd/glibc-2.3.2
        42: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS /build/buildd/glibc-2.3.2
        43: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS /build/buildd/glibc-2.3.2
        44: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS /build/buildd/glibc-2.3.2
45: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS /build/buildd/glibc-2.3.2
        46: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS <command line>
        47: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS /build/buildd/glibc-2.3.2
        48: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS <command line>
        49: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS <built-in>
50: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS abi-note.S
        51: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS /build/buildd/glibc-2.3.2
        52: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS abi-note.S
        53: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS /build/buildd/glibc-2.3.2
        54: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS abi-note.S
55: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS <command line>
        56: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS /build/buildd/glibc-2.3.2
        57: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS <command line>
        58: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS <built-in>
        59: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS abi-note.S
60: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS init.c
        61: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS /build/buildd/glibc-2.3.2
        62: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS /build/buildd/glibc-2.3.2
        63: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS initfini.c
        64: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS /build/buildd/glibc-2.3.2
65: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS <command line>
        66: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS /build/buildd/glibc-2.3.2
```

```
        67: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS <command line>
        68: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS <built-in>
        69: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS /build/buildd/glibc-2.3.2
70: 4000000000000670   128 FUNC    LOCAL  DEFAULT   12 gmon_initializer
        71: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS /build/buildd/glibc-2.3.2
        72: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS /build/buildd/glibc-2.3.2
        73: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS initfini.c
        74: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS /build/buildd/glibc-2.3.2
75: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS <command line>
        76: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS /build/buildd/glibc-2.3.2
        77: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS <command line>
        78: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS <built-in>
        79: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS /build/buildd/glibc-2.3.2
80: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS auto-host.h
        81: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS <command line>
        82: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS <built-in>
        83: 6000000000000fc0     0 NOTYPE  LOCAL  DEFAULT   22 __CTOR_LIST__
        84: 6000000000000fd0     0 NOTYPE  LOCAL  DEFAULT   23 __DTOR_LIST__
85: 6000000000000fe0     0 NOTYPE  LOCAL  DEFAULT   24 __JCR_LIST__
        86: 6000000000001088     8 OBJECT  LOCAL  DEFAULT   27 dtor_ptr
        87: 40000000000006f0   128 FUNC    LOCAL  DEFAULT   12 __do_global_dtors_aux
        88: 4000000000000770   128 FUNC    LOCAL  DEFAULT   12 __do_jv_register_classes
        89: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS hello.c
90: 6000000000001090     4 OBJECT  LOCAL  DEFAULT   27 i
        91: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS function.c
        92: 6000000000001098     4 OBJECT  LOCAL  DEFAULT   27 i
        93: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS auto-host.h
        94: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS <command line>
95: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS <built-in>
        96: 6000000000000fc8     0 NOTYPE  LOCAL  DEFAULT   22 __CTOR_END__
        97: 6000000000000fd8     0 NOTYPE  LOCAL  DEFAULT   23 __DTOR_END__
        98: 6000000000000fe0     0 NOTYPE  LOCAL  DEFAULT   24 __JCR_END__
        99: 6000000000000de0     0 OBJECT  GLOBAL DEFAULT  ABS _DYNAMIC
100: 4000000000000a70   144 FUNC    GLOBAL HIDDEN   12 __do_global_ctors_aux
       101: 6000000000000dd8     0 NOTYPE  GLOBAL DEFAULT  ABS __fini_array_end
       102: 60000000000010a8     8 OBJECT  GLOBAL HIDDEN   29 __dso_handle
       103: 40000000000009a0   208 FUNC    GLOBAL DEFAULT   12 __libc_csu_fini
       104: 0000000000000000   176 FUNC    GLOBAL DEFAULT  UND printf@@GLIBC_2.2
105: 40000000000004a0    32 FUNC    GLOBAL DEFAULT   10 _init
       106: 4000000000000850   128 FUNC    GLOBAL DEFAULT   12 function
       107: 40000000000005e0   144 FUNC    GLOBAL DEFAULT   12 _start
       108: 6000000000001094     4 OBJECT  GLOBAL DEFAULT   27 global
       109: 6000000000000dd0     0 NOTYPE  GLOBAL DEFAULT  ABS __fini_array_start
110: 40000000000008d0   208 FUNC    GLOBAL DEFAULT   12 __libc_csu_init
       111: 600000000000109c     0 NOTYPE  GLOBAL DEFAULT  ABS __bss_start
       112: 40000000000007f0    96 FUNC    GLOBAL DEFAULT   12 main
       113: 6000000000000dd0     0 NOTYPE  GLOBAL DEFAULT  ABS __init_array_end
       114: 6000000000000dd8     0 NOTYPE  WEAK   DEFAULT   20 data_start
115: 4000000000000b00    32 FUNC    GLOBAL DEFAULT   13 _fini
       116: 0000000000000000   704 FUNC    GLOBAL DEFAULT  UND exit@@GLIBC_2.2
       117: 600000000000109c     0 NOTYPE  GLOBAL DEFAULT  ABS _edata
       118: 6000000000000fe8     0 OBJECT  GLOBAL DEFAULT  ABS _GLOBAL_OFFSET_TABLE_
       119: 60000000000010b0     0 NOTYPE  GLOBAL DEFAULT  ABS _end
120: 6000000000000db8     0 NOTYPE  GLOBAL DEFAULT  ABS __init_array_start
       121: 6000000000001080     4 OBJECT  GLOBAL DEFAULT   27 _IO_stdin_used
       122: 60000000000010a0     8 OBJECT  GLOBAL DEFAULT   28 __libc_ia64_register_back
       123: 6000000000000dd8     0 NOTYPE  GLOBAL DEFAULT   20 __data_start
       124: 0000000000000000     0 NOTYPE  WEAK   DEFAULT  UND _Jv_RegisterClasses
125: 0000000000000000   544 FUNC    GLOBAL DEFAULT  UND __libc_start_main@@GLIBC_
       126: 0000000000000000     0 NOTYPE  WEAK   DEFAULT  UND __gmon_start__
```

Обратите внимание на несколько вещей:

- Исполняемый файл был собран «простым» способом.

- Здесь две таблицы символов: `dynsym` и `symtab`. Скоро мы объясним, как работают символы `dynsym`; пока обратите внимание, что у некоторых после знака `@` указана версия.
- Добавлено много символов из дополнительных объектных файлов. Многие начинаются с `__`, чтобы избежать конфликта с именами, выбранными программистом. Просмотрите список, найдите упомянутые ранее символы объектных файлов и посмотрите, изменилось ли что-нибудь.

Глава 8. Что стоит за процессом

Повторение: исполняемые файлы

Мы знаем, что работающая в памяти программа состоит из двух основных частей: кода, который по историческим причинам также часто называют текстом, и данных. Однако исполняемый файл существует не только в памяти: большую часть времени он проводит на диске. Этот файл имеет так называемый двоичный формат, поскольку его биты и байты предназначены для непосредственной интерпретации процессором.

Представление исполняемых файлов

Три стандартные секции

Любой формат исполняемого файла должен указывать, где в двоичном файле расположены код и данные.

Ещё один компонент, о котором мы до сих пор не говорили, — место для неинициализированных глобальных переменных. Если переменная объявлена с начальным значением, его необходимо сохранить в исполняемом файле, чтобы при выполнении оно было правильным. Но многие переменные при запуске программы не инициализированы или равны нулю. Выделять для них место в файле и записывать нули или `NULL` расточительно: это без нужды увеличит размер файла. Поэтому исполняемый файл может определить секцию BSS, задающую только размер неинициализированных данных; при загрузке программы дополнительная память будет выделена и заполнена нулями.

BSS, вероятно, означает *Block Started by Symbol* — название ассемблерной команды старого компьютера IBM.

Двоичный формат

Инструментальная цепочка создаёт исполняемый файл из исходного кода. Формат файла должен быть точно определён, чтобы компилятор мог его создать, а операционная система — распознать и загрузить в память, превратив в управляемый работающий процесс. Такой формат может зависеть от операционной системы: обычно мы не ожидаем, что программа, скомпилированная для одной системы, будет выполняться в другой. Например, программы Windows обычно не запускаются в Linux, а программы Linux — в OS X.

Общая черта всех форматов исполняемых файлов — заранее определённый стандартный заголовок, описывающий расположение кода и данных в остальной части файла. Условно он может сообщать: «Код программы начинается через 20 байтов от начала файла и занимает 50 килобайтов. За ним следуют данные длиной 20 килобайтов».

В последнее время фактическим стандартом представления исполняемых файлов современных UNIX-подобных систем стал формат Executable and Linker Format, сокращённо ELF. Скоро мы рассмотрим его подробнее.

История двоичных форматов

a.out

ELF существовал не всегда: в первоначальных системах UNIX использовался формат `a.out`. Его следы можно увидеть и сегодня. Если компилировать программу без параметра `-o`, задающего имя выходного файла, исполняемый файл по умолчанию получит имя `a.out`.

Строго говоря, `a.out` — имя выходного файла компоновщика по умолчанию. Для промежуточных ассемблерных и объектных файлов компилятор обычно использует случайно сформированные имена.

Заголовок формата `a.out` очень прост и допускает только по одной секции данных, кода и BSS. Как мы увидим далее, для современных систем с динамическими библиотеками этого недостаточно.

COFF

Предшественником ELF был Common Object File Format, или COFF. Его заголовок был более гибким и допускал больше секций, хотя их число оставалось ограниченным.

COFF также испытывал трудности с изящной поддержкой разделяемых библиотек, и для Linux в качестве альтернативы был выбран ELF.

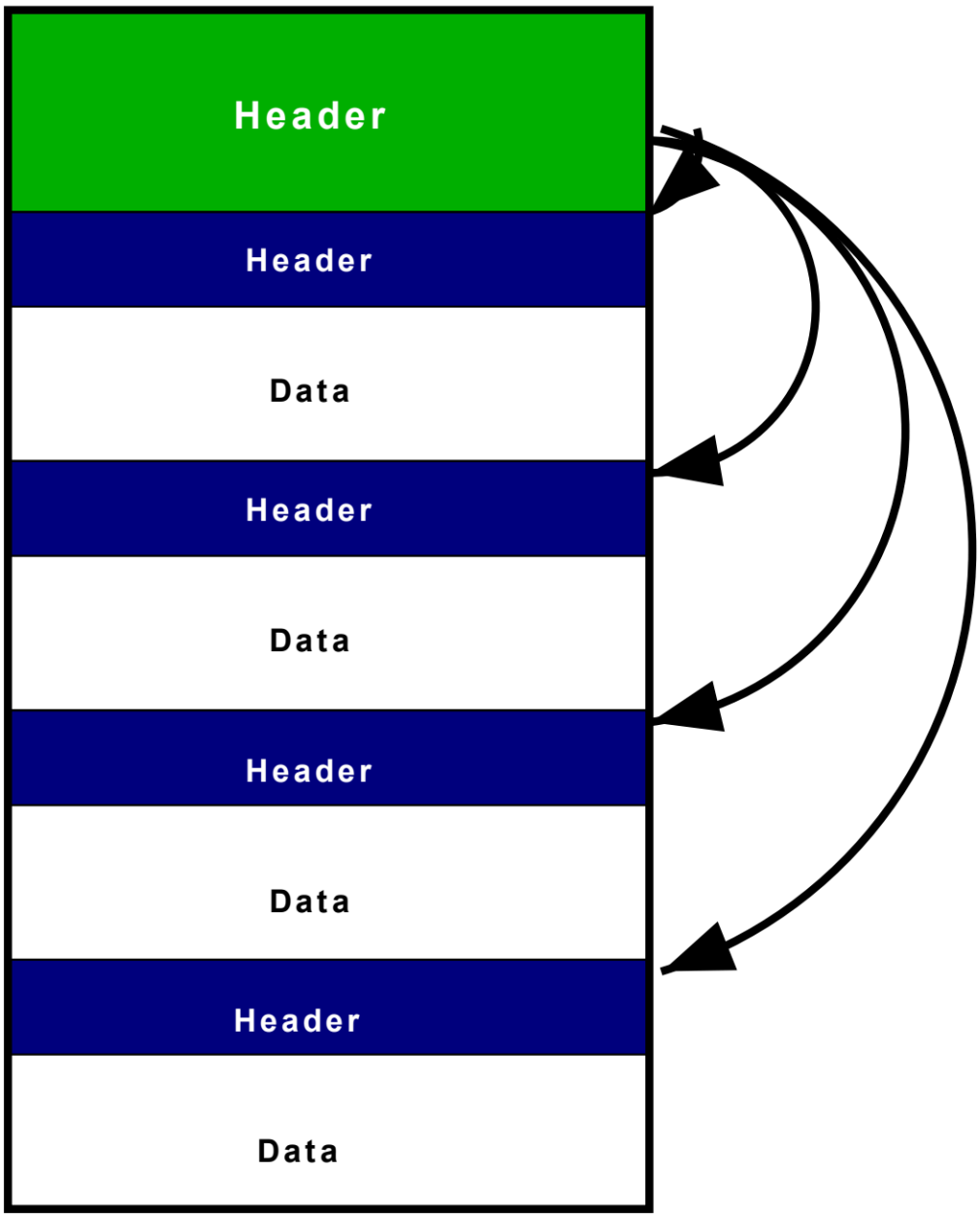
Однако COFF продолжает жить в Microsoft Windows в форме Portable Executable, или PE. PE относится к Windows так же, как ELF — к Linux.

ELF

ELF — чрезвычайно гибкий формат представления двоичного кода в системе. Следуя стандарту ELF, одинаково легко можно представить двоичный файл ядра, обычный исполняемый файл или системную библиотеку. Одни и те же инструменты подходят для исследования и обработки всех ELF-файлов, а разработчик, понимающий этот формат, сможет применить свои знания в большинстве современных UNIX-систем.

ELF подробнее

ELF развивает COFF: его заголовок достаточно гибок, чтобы описывать произвольное число секций, каждая со своими свойствами. Это упрощает динамическую компоновку и отладку.



Заголовок файла ELF

В целом файл имеет заголовок, описывающий его общие свойства и содержащий указатели на отдельные секции, из которых состоит файл.

Пример 8.1. Заголовок ELF

```
typedef struct {
    unsigned char e_ident[EI_NIDENT];
    Elf32_Half    e_type;
    Elf32_Half    e_machine;
    Elf32_Word    e_version;
    Elf32_Addr    e_entry;
    Elf32_Off     e_phoff;
    Elf32_Off     e_shoff;
    Elf32_Word    e_flags;
    Elf32_Half    e_ehsize;
```

```
    Elf32_Half    e_phentsize;
    Elf32_Half    e_phnum;
    Elf32_Half    e_shentsize;
    Elf32_Half    e_shnum;
    Elf32_Half    e_shstrndx;
} Elf32_Ehdr;
```

Выше приведено описание из документации API: так выглядит структура C, определяющая заголовок ELF.

Пример 8.2. Заголовок ELF в выводе readelf

```
$ readelf --header /bin/ls

ELF Header:
  Magic:   7f 45 4c 46 01 02 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF32
  Data:                              2's complement, big endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              EXEC (Executable file)
  Machine:                           PowerPC
  Version:                           0x1
  Entry point address:               0x10002640
  Start of program headers:          52 (bytes into file)
  Start of section headers:          87460 (bytes into file)
  Flags:                             0x0
  Size of this header:               52 (bytes)
  Size of program headers:           32 (bytes)
  Number of program headers:         8
  Size of section headers:           40 (bytes)
  Number of section headers:         29
  Section header string table index: 28
  [...]
```

Это более удобное для человека представление, выдаваемое программой `readelf` из состава GNU binutils.

Массив `e_ident` находится в самом начале любого ELF-файла и всегда начинается с нескольких «магических» байтов. Первый байт равен `0x7F`, за ним следуют три байта `ELF`. Это можно проверить самостоятельно, например с помощью команды `hexdump`.

Пример 8.3. Проверка магического числа ELF

```
ianw@mingus:~$ hexdump -C /bin/ls | more
00000000  7f 45 4c 46 01 02 01 00  00 00 00 00 00 00 00 00  |.ELF............|
... (rest of the program follows) ...
```

Обратите внимание на начальный `0x7F` и записанную в ASCII строку `ELF`. Изучите стандарт, чтобы узнать, что определяет остальная часть массива и какие значения присутствуют в двоичном файле.

Далее следуют признаки типа машины, для которой создан файл. ELF определяет варианты с разными размерами типов: 32- и 64-битный; здесь рассматривается 32-битный. Главное различие состоит в том, что на 64-битных машинах адреса должны храниться в 64-битных переменных. В примере файл создан для машины с прямым порядком байтов и

представлением отрицательных чисел в дополнительном коде. Поле `Machine` ниже указывает, что это файл для PowerPC.

Адрес точки входа на первый взгляд прост: это адрес в памяти, с которого начинается код программы.

Начинающим программистам на C говорят, что первой вызывается функция `main()`. Используя адрес точки входа, можно проверить, что это не так.

Пример 8.4. Исследование точки входа

```
$ cat test.c
#include <stdio.h>

int main(void)
{
    printf("main is : %p\n", &main);
    return 0;
}
$ gcc -Wall -o test test.c
$ ./test
main is : 0x10000430
$ readelf --headers ./test | grep 'Entry point'
  Entry point address:               0x100002b0
$ objdump --disassemble ./test | grep 100002b0
100002b0 <_start>:
100002b0:       7c 29 0b 78     mr      r9,r1
```

Здесь видно, что точкой входа на самом деле является функция `_start`. Наша программа её не определяла, а начальное подчёркивание подсказывает, что она находится в отдельном пространстве имён. Ниже мы рассмотрим запуск программы.

После этого заголовок содержит указатели на места, с которых начинаются другие важные части ELF-файла, подобно оглавлению.

Символы и релокации

Спецификация ELF предусматривает таблицы символов, которые просто сопоставляют строки — символы — с местами в файле. Символы необходимы для компоновки. Например, чтобы присвоить значение переменной `foo`, объявленной как `extern int foo;`, компоновщик должен найти её адрес: для этого он ищет `foo` в таблице символов.

С символами тесно связаны релокации. Релокация — по сути, место, оставленное для последующего исправления. Пока адрес `foo` неизвестен, его нельзя использовать. Но на 32-битной системе известно, что адрес занимает четыре байта, поэтому каждый раз, когда компилятору нужен этот адрес, например для присваивания значения, он может оставить четыре пустых байта и создать запись релокации, которая сообщает компоновщику: «Помести фактический адрес `foo` в четыре байта по этому месту». Для этого, как уже говорилось, символ `foo` должен быть разрешён. Подробнее релокации рассматриваются в соответствующем разделе.

Секции и сегменты

Формат ELF предусматривает два «взгляда» на файл: один используется при компоновке, другой — при выполнении. Это даёт разработчикам систем значительную гибкость.

О секциях говорят применительно к объектному коду, ожидающему компоновки в исполняемый файл. Одна или несколько секций отображаются в сегмент исполняемого файла.

Сегменты

Как и прежде, иногда проще сначала рассмотреть более высокий уровень абстракции — сегменты, — а затем нижние уровни.

Как уже говорилось, заголовок ELF описывает общую структуру файла. Он указывает на другую группу заголовков — программные заголовки. Они сообщают операционной системе всё необходимое для загрузки файла в память и его выполнения. Программные заголовки описывают не только сегменты, но и некоторые другие данные, необходимые для запуска.

Пример 8.5. Программный заголовок

```
typedef struct {
    Elf32_Word p_type;
    Elf32_Off  p_offset;
    Elf32_Addr p_vaddr;
    Elf32_Addr p_paddr;
    Elf32_Word p_filesz;
    Elf32_Word p_memsz;
    Elf32_Word p_flags;
    Elf32_Word p_align;
}
```

Выше приведено определение программного заголовка. В определении заголовка ELF можно было заметить поля `e_phoff`, `e_phnum` и `e_phentsize`: это соответственно смещение начала программных заголовков в файле, их количество и размер каждого. По этим трём значениям заголовки легко найти и прочитать.

Как уже говорилось, программные заголовки описывают не только сегменты. Поле `p_type` определяет назначение заголовка. Например, `PT_INTERP` обозначает указатель на строку с именем интерпретатора двоичного файла. Ранее мы различали компилируемые и интерпретируемые языки и говорили, что компилятор создаёт двоичный файл, способный выполняться самостоятельно. Зачем ему интерпретатор? Как обычно, реальная картина немного сложнее. Современной системе требуется гибкость при загрузке исполняемых файлов, а некоторые сведения можно получить только непосредственно при подготовке программы к запуску. Мы увидим это в следующих главах при рассмотрении динамической компоновки. Поэтому для правильной работы файла во время выполнения могут понадобиться небольшие изменения. В роли такого интерпретатора обычно выступает динамический загрузчик, который завершает загрузку исполняемого файла и подготавливает его образ к запуску.

Сегменты обозначаются значением `PT_LOAD` в поле `p_type`. Остальные поля программного заголовка описывают каждый сегмент. `p_offset` указывает смещение данных сегмента в файле на диске. `p_vaddr` задаёт виртуальный адрес, по которому данные должны находиться в памяти; `p_paddr` описывает физический адрес, что полезно прежде всего в небольших встроенных системах без виртуальной памяти. Поля `p_filesz` и `p_memsz` задают размеры сегмента на диске и в памяти. Если размер в памяти больше, дополнительная область заполняется нулями. Так можно значительно сократить размер файлов, не занимая дисковое место под пустые глобальные переменные. Наконец, `p_flags` задаёт права сегмента. Права исполнения, чтения и записи допускают разные сочетания: например, сегмент кода должен быть доступен только для чтения и исполнения, а данные — для чтения и записи, но не исполнения.

В программных заголовках определены и некоторые другие типы сегментов; подробнее они описаны в спецификации стандарта. В оригинале здесь также оставлена авторская пометка `XXX`.

Секции

Как уже говорилось, секции составляют сегменты. Они позволяют организовать двоичный файл в логические области и передавать сведения между компилятором и компоновщиком. В некоторых особых двоичных файлах, например в ядре Linux, секции применяются более специфическими способами.

Как мы видели, сегмент в конечном счёте представляет собой блок данных в файле на диске вместе с описанием места загрузки и прав доступа. У секций есть заголовок, похожий на заголовок сегментов.

Пример 8.6. Секции

```
typedef struct {
    Elf32_Word sh_name;
    Elf32_Word sh_type;
    Elf32_Word sh_flags;
    Elf32_Addr sh_addr;
    Elf32_Off  sh_offset;
    Elf32_Word sh_size;
    Elf32_Word sh_link;
    Elf32_Word sh_info;
    Elf32_Word sh_addralign;
    Elf32_Word sh_entsize;
}
```

Для поля секции `sh_type` определено несколько типов. Например, `SH_PROGBITS` означает секцию, содержащую двоичные данные для использования программой. Другие значения указывают, что секция является таблицей символов, используемой, например, компоновщиком или отладчиком, либо содержит данные для динамического загрузчика.

Есть и другие атрибуты, например признак выделения памяти, показывающий, что для секции потребуется память.

Лучше всего рассмотреть секции на примере их использования в следующей программе.

Пример 8.7. Секции

```
#include <stdio.h>

int big_big_array[10*1024*1024];
char *a_string = "Hello, World!";
int a_var_with_value = 0x100;

int main(void)
{
    big_big_array[0] = 100;
    printf("%s\n", a_string);
    a_var_with_value += 20;
}
```

Пример 8.8. Вывод readelf для секций

```
$ readelf --all ./sections
    ELF Header:
     ...
Size of section headers:           40 (bytes)
      Number of section headers:         37
      Section header string table index: 34

    Section Headers:
[Nr] Name              Type            Addr     Off    Size   ES Flg Lk Inf Al

      [ 0]                   NULL            00000000 000000 000000 00      0   0  0
      [ 1] .interp           PROGBITS        10000114 000114 00000d 00   A  0   0  1
      [ 2] .note.ABI-tag     NOTE            10000124 000124 000020 00   A  0   0  4
      [ 3] .hash             HASH            10000144 000144 00002c 04   A  4   0  4
[ 4] .dynsym           DYNSYM          10000170 000170 000060 10   A  5   1  4
      [ 5] .dynstr           STRTAB          100001d0 0001d0 00005e 00   A  0   0  1
      [ 6] .gnu.version      VERSYM          1000022e 00022e 00000c 02   A  4   0  2
      [ 7] .gnu.version_r    VERNEED         1000023c 00023c 000020 00   A  5   1  4
      [ 8] .rela.dyn         RELA            1000025c 00025c 00000c 0c   A  4   0  4
[ 9] .rela.plt         RELA            10000268 000268 000018 0c   A  4  25  4
      [10] .init             PROGBITS        10000280 000280 000028 00  AX  0   0  4
      [11] .text             PROGBITS        100002b0 0002b0 000560 00  AX  0   0 16
      [12] .fini             PROGBITS        10000810 000810 000020 00  AX  0   0  4
      [13] .rodata           PROGBITS        10000830 000830 000024 00   A  0   0  4
[14] .sdata2           PROGBITS        10000854 000854 000000 00   A  0   0  4
      [15] .eh_frame         PROGBITS        10000854 000854 000004 00   A  0   0  4
      [16] .ctors            PROGBITS        10010858 000858 000008 00  WA  0   0  4
      [17] .dtors            PROGBITS        10010860 000860 000008 00  WA  0   0  4
      [18] .jcr              PROGBITS        10010868 000868 000004 00  WA  0   0  4
[19] .got2             PROGBITS        1001086c 00086c 000010 00  WA  0   0  1
      [20] .dynamic          DYNAMIC         1001087c 00087c 0000c8 08  WA  5   0  4
      [21] .data             PROGBITS        10010944 000944 000008 00  WA  0   0  4
      [22] .got              PROGBITS        1001094c 00094c 000014 04 WAX  0   0  4
      [23] .sdata            PROGBITS        10010960 000960 000008 00  WA  0   0  4
[24] .sbss             NOBITS          10010968 000968 000000 00  WA  0   0  1
      [25] .plt              NOBITS          10010968 000968 000060 00 WAX  0   0  4
      [26] .bss              NOBITS          100109c8 000968 2800004 00  WA  0   0  4
      [27] .comment          PROGBITS        00000000 000968 00018f 00      0   0  1
      [28] .debug_aranges    PROGBITS        00000000 000af8 000078 00      0   0  8
[29] .debug_pubnames   PROGBITS        00000000 000b70 000025 00      0   0  1
      [30] .debug_info       PROGBITS        00000000 000b95 0002e5 00      0   0  1
      [31] .debug_abbrev     PROGBITS        00000000 000e7a 000076 00      0   0  1
      [32] .debug_line       PROGBITS        00000000 000ef0 0001de 00      0   0  1
      [33] .debug_str        PROGBITS        00000000 0010ce 0000f0 01  MS  0   0  1
[34] .shstrtab         STRTAB          00000000 0011be 00013b 00      0   0  1
      [35] .symtab           SYMTAB          00000000 0018c4 000c90 10     36  65  4
      [36] .strtab           STRTAB          00000000 002554 000909 00      0   0  1
    Key to Flags:
```

```
      W (write), A (alloc), X (execute), M (merge), S (strings)
I (info), L (link order), G (group), x (unknown)
      O (extra OS processing required) o (OS specific), p (processor specific)

    There are no section groups in this file.
     ...
    Symbol table '.symtab' contains 201 entries:
       Num:    Value  Size Type    Bind   Vis      Ndx Name
    ...
        99: 100109cc 0x2800000 OBJECT  GLOBAL DEFAULT   26 big_big_array
...
       110: 10010960     4 OBJECT  GLOBAL DEFAULT   23 a_string
    ...
       130: 10010964     4 OBJECT  GLOBAL DEFAULT   23 a_var_with_value
    ...

144: 10000430    96 FUNC    GLOBAL DEFAULT   11 main
```

Для ясности часть вывода `readelf` опущена. Проанализируем каждую часть простой программы и посмотрим, что с ней происходит.

Сначала рассмотрим `big_big_array` — как следует из имени, довольно большой глобальный массив. В таблице символов видно, что переменная находится по адресу `0x100109cc`. Его можно сопоставить с секцией `.bss` в списке: она начинается чуть раньше, по адресу `0x100109c8`. Обратите внимание на значительный размер массива. Мы говорили, что BSS — стандартная часть двоичного образа: было бы нелепо требовать десять мегабайт дискового пространства для области, целиком заполненной нулями. Тип секции `NOBITS` означает, что она не занимает соответствующие байты на диске.

Таким образом, секция `.bss` предназначена для глобальных переменных, которые при запуске должны быть равны нулю. При обсуждении сегментов мы видели, что размер в памяти может отличаться от размера на диске; присутствие переменной в `.bss` означает её обнуление при запуске.

Переменная `a_string` находится в секции `.sdata`, что означает «малые данные». К данным в `.sdata` и соответствующей секции `.sbss` можно обратиться по смещению от известного указателя. Это ускоряет доступ: не нужны дополнительные поиски и загрузки адресов в память. С другой стороны, большинство архитектур ограничивает размер непосредственного значения, которое можно прибавить к регистру. Например, в `r1 = add r2, 70` число 70 является непосредственным значением, в отличие от сложения значений двух регистров `r1 = add r2,r3`. Поэтому от адреса можно отступить лишь на определённое «небольшое» расстояние. В исходном тексте здесь стоит пометка `XXX`.

Переменная `a_var_with_value` находится там же. А функция `main`, как и ожидалось, находится в секции `.text`: напомним, что применительно к программе в памяти слова «текст» и «код» взаимозаменяемы.

Секции и сегменты вместе

Пример 8.9. Секции и сегменты

```
$ readelf --segments /bin/ls
```

```
Elf file type is EXEC (Executable file)
Entry point 0x100026c0
There are 8 program headers, starting at offset 52

Program Headers:
Type           Offset   VirtAddr   PhysAddr   FileSiz MemSiz  Flg Align
PHDR           0x000034 0x10000034 0x10000034 0x00100 0x00100 R E 0x4
INTERP         0x000154 0x10000154 0x10000154 0x0000d 0x0000d R   0x1
[Requesting program interpreter: /lib/ld.so.1]
LOAD           0x000000 0x10000000 0x10000000 0x14d5c 0x14d5c R E 0x10000
LOAD           0x014d60 0x10024d60 0x10024d60 0x002b0 0x00b7c RWE 0x10000
DYNAMIC        0x014f00 0x10024f00 0x10024f00 0x000d8 0x000d8 RW  0x4
NOTE           0x000164 0x10000164 0x10000164 0x00020 0x00020 R   0x4
GNU_EH_FRAME   0x014d30 0x10014d30 0x10014d30 0x0002c 0x0002c R   0x4
GNU_STACK      0x000000 0x00000000 0x00000000 0x00000 0x00000 RWE 0x4

Section to Segment mapping:
Segment Sections...
00
01     .interp
02     .interp .note.ABI-tag .hash .dynsym .dynstr .gnu.version .gnu.version_r .rela.dyn .rela.plt .init
.text .fini .rodata .eh_frame_hdr
03     .data .eh_frame .got2 .dynamic .ctors .dtors .jcr .got .sdata .sbss .plt .bss
04     .dynamic
05     .note.ABI-tag
06     .eh_frame_hdr
07
```

Команда `readelf` показывает сегменты ELF-файла `/bin/ls` и соответствие секций сегментам.

В нижней части вывода видно, какие секции помещены в какие сегменты. Например, секция `.interp` находится в сегменте с признаком `INTERP`. Программа `readelf` сообщает, что запрошен интерпретатор `/lib/ld.so.1`: это динамический компоновщик, запускаемый для подготовки файла к выполнению.

В двух сегментах `LOAD` видно различие между кодом и данными. У первого есть только права чтения и исполнения, у второго — чтения, записи и исполнения. Они описывают соответственно сегменты кода и данных.

Но зачем данным право исполнения? Действительно, на большинстве архитектур, например на распространённой x86, секция данных не помечается исполняемой. Однако приведённый вывод получен на PowerPC, где немного другая модель программирования — ABI, см. ниже, — требует права исполнения для секции данных. Такова жизнь системного программиста: правила иногда нарушаются!

В ABI PowerPC заглушки функций динамических библиотек вызываются непосредственно из GOT, а не через отдельный элемент PLT. Поэтому для секции GOT нужно право исполнения; как видно в выводе, она находится внутри сегмента данных. После главы о динамической компоновке это должно стать понятнее.

Ещё одна интересная деталь: размер сегмента кода в файле равен его размеру в памяти, а размер сегмента данных в памяти больше. Причина — секция BSS с обнуляемыми глобальными переменными.

Отладка

Традиционно основной способ посмертной отладки называется дампом памяти, или *core dump*. Название связано с физическими свойствами старой памяти на магнитных сердечниках: состояние хранилось в ориентации небольших магнитных колец.

Таким образом, core dump — это полный снимок работающей программы в конкретный момент. Отладчик может исследовать его и восстановить состояние программы. В примере 8.10 программа записывает данные по случайному адресу памяти, чтобы вызвать аварийное завершение. Процесс останавливается, а снимок его состояния сохраняется.

Пример 8.10. Создание дампа памяти и его исследование с помощью gdb™

```
$ cat coredump.c


int main(void) {
char *foo = (char*)0x12345;
*foo = 'a';

return 0;
}

$ gcc -Wall -g -o coredump coredump.c

$ ./coredump
Segmentation fault (core dumped)

$ file ./core
./core: ELF 32-bit LSB core file Intel 80386, version 1 (SYSV), SVR4-style, from './coredump'

$ gdb ./coredump
...
(gdb) core core
[New LWP 31614]
Core was generated by `./coredump'.
Program terminated with signal 11, Segmentation fault.
#0  0x080483c4 in main () at coredump.c:3
3  *foo = 'a';
(gdb)
```

Символы и отладочная информация

Как показывает пример, для отладки `gdb` нужны исходный исполняемый файл и дамп памяти. Обратите внимание, что файл был собран с параметром `-g`, который указывает компилятору включить отладочную информацию. Она создаётся компилятором и хранится в специальных секциях ELF. В ней подробно описано, например, какие значения регистров соответствуют переменным кода, размеры переменных, длины массивов и так далее. Обычно используется стандартный формат DWARF — игра слов с названием ELF.

Отладочная информация может существенно увеличить размер исполняемых файлов и библиотек. Хотя она не должна постоянно находиться в памяти для выполнения программы, на диске она занимает заметное место. Поэтому обычно её удаляют из ELF-файла. Можно распространять и очищенную, и полную версии, но большинство современных способов распространения двоичных файлов помещает отладочные сведения в отдельные файлы.

Утилита `objcopy` может извлечь их (`--only-keep-debug`), а затем добавить в исходный файл ссылку на них (`--add-gnu-debuglink`). После этого в исполняемом файле появится секция `.gnu_debuglink` с контрольным значением, по которому отладчик при запуске сеанса проверяет соответствие данных файлу.

Пример 8.11. Выделение отладочной информации в отдельный файл с помощью objcopy™

```
$ gcc -g -shared -o libtest.so libtest.c
$ objcopy --only-keep-debug libtest.so libtest.debug
$ objcopy --add-gnu-debuglink=libtest.debug libtest.so
$ objdump -s -j .gnu_debuglink libtest.so

libtest.so:     file format elf32-i386

Contents of section .gnu_debuglink:
0000 6c696274 6573742e 64656275 67000000  libtest.debug...
0010 52a7fd0a                             R...
```

Символы занимают гораздо меньше места, но их тоже могут удалять из окончательного файла. После объединения объектных файлов в один образ большинство символов обычно больше не требуется. Как обсуждалось в разделе о символах и релокациях, они нужны для исправления записей релокации, но после этого не строго необходимы для выполнения. В Linux удалять символы позволяет утилита `strip` из набора GNU. Некоторые символы всё же разрешаются во время выполнения при динамической компоновке — теме главы 9. Они помещаются в отдельные таблицы динамических символов, поэтому не удаляются вместе с остальными.

Что внутри дампа памяти

Дамп памяти — это ещё один ELF-файл, что показывает гибкость ELF как двоичного формата.

Пример 8.12. Исследование дампа с помощью readelf™ и eu-readelf™

```
$ readelf --all ./core
ELF Header:
Magic:   7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00 00
Class:                             ELF32
Data:                              2's complement, little endian
Version:                           1 (current)
OS/ABI:                            UNIX - System V
ABI Version:                       0
Type:                              CORE (Core file)
Machine:                           Intel 80386
Version:                           0x1
Entry point address:               0x0
Start of program headers:          52 (bytes into file)
Start of section headers:          0 (bytes into file)
Flags:                             0x0
Size of this header:               52 (bytes)
Size of program headers:           32 (bytes)
Number of program headers:         15
Size of section headers:           0 (bytes)
Number of section headers:         0
Section header string table index: 0
```

```
There are no sections in this file.
There are no sections to group in this file.

Program Headers:
Type           Offset   VirtAddr   PhysAddr   FileSiz MemSiz  Flg Align
NOTE           0x000214 0x00000000 0x00000000 0x0022c 0x00000     0

      LOAD           0x001000 0x08048000 0x00000000 0x01000 0x01000 R E 0x1000
      LOAD           0x002000 0x08049000 0x00000000 0x01000 0x01000 RW  0x1000
      LOAD           0x003000 0x489fc000 0x00000000 0x01000 0x1b000 R E 0x1000
      LOAD           0x004000 0x48a17000 0x00000000 0x01000 0x01000 R   0x1000
LOAD           0x005000 0x48a18000 0x00000000 0x01000 0x01000 RW  0x1000
      LOAD           0x006000 0x48a1f000 0x00000000 0x01000 0x153000 R E 0x1000
      LOAD           0x007000 0x48b72000 0x00000000 0x00000 0x01000     0x1000
      LOAD           0x007000 0x48b73000 0x00000000 0x02000 0x02000 R   0x1000
      LOAD           0x009000 0x48b75000 0x00000000 0x01000 0x01000 RW  0x1000
LOAD           0x00a000 0x48b76000 0x00000000 0x03000 0x03000 RW  0x1000
      LOAD           0x00d000 0xb771c000 0x00000000 0x01000 0x01000 RW  0x1000
      LOAD           0x00e000 0xb774d000 0x00000000 0x02000 0x02000 RW  0x1000
      LOAD           0x010000 0xb774f000 0x00000000 0x01000 0x01000 R E 0x1000
      LOAD           0x011000 0xbfeac000 0x00000000 0x22000 0x22000 RW  0x1000
    There is no dynamic section in this file.

    There are no relocations in this file.

There are no unwind sections in this file.

    No version information found in this file.

    Notes at offset 0x00000214 with length 0x0000022c:
Owner                Data size Description
      CORE                 0x00000090 NT_PRSTATUS (prstatus structure)
      CORE                 0x0000007c NT_PRPSINFO (prpsinfo structure)
      CORE                 0x000000a0 NT_AUXV (auxiliary vector)
      LINUX                0x00000030 Unknown note type: (0x00000200)
    $ eu-readelf -n ./core

    Note segment of 556 bytes at offset 0x214:
      Owner          Data size  Type
CORE                 144  PRSTATUS
        info.si_signo: 11, info.si_code: 0, info.si_errno: 0, cursig: 11
        sigpend: <>
        sighold: <>
        pid: 31614, ppid: 31544, pgrp: 31614, sid: 31544
utime: 0.000000, stime: 0.000000, cutime: 0.000000, cstime: 0.000000
        orig_eax: -1, fpvalid: 0
        ebx:     1219973108  ecx:     1243440144  edx:              1
        esi:              0  edi:              0  ebp:     0xbfecb828
        eax:          74565  eip:     0x080483c4  eflags:  0x00010286
esp:     0xbfecb818
        ds: 0x007b  es: 0x007b  fs: 0x0000  gs: 0x0033  cs: 0x0073  ss: 0x007b
      CORE                 124  PRPSINFO
        state: 0, sname: R, zomb: 0, nice: 0, flag: 0x00400400
        uid: 1000, gid: 1000, pid: 31614, ppid: 31544, pgrp: 31614, sid: 31544
fname: coredump, psargs: ./coredump
      CORE                 160  AUXV
        SYSINFO: 0xb774f414
        SYSINFO_EHDR: 0xb774f000
        HWCAP: 0xafe8fbff  <fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov clflush dts
acpi mmx fxsr sse sse2 ss tm pbe>

PAGESZ: 4096
CLKTCK: 100
PHDR: 0x8048034
PHENT: 32
PHNUM: 8
```

```
BASE: 0
FLAGS: 0
ENTRY: 0x8048300
UID: 1000
EUID: 1000
GID: 1000
EGID: 1000
SECURE: 0
RANDOM: 0xbfecba1b
EXECFN: 0xbfecdff1
PLATFORM: 0xbfecba2b
NULL
LINUX                 48  386_TLS
index: 6, base: 0xb771c8d0, limit: 0x000fffff, flags: 0x00000051
index: 7, base: 0x00000000, limit: 0x00000000, flags: 0x00000028
index: 8, base: 0x00000000, limit: 0x00000000, flags: 0x00000028
```

В примере 8.12 сначала с помощью `readelf` исследуется дамп, полученный в примере 8.10. Файл не содержит секций, релокаций и других дополнительных данных, необходимых для загрузки исполняемого файла или библиотеки; он состоит из программных заголовков, описывающих сегменты `LOAD`. Эти сегменты — созданные ядром снимки текущих выделенных областей памяти.

Другой компонент дампа — секции `NOTE`, содержащие данные, нужные для отладки, но не обязательно присутствующие непосредственно в снимке памяти. Используемая во второй части примера программа `eu-readelf` декодирует их и показывает более полную картину.

Запись `PRSTATUS` содержит разнообразные сведения о процессе на момент остановки. Например, по `cursig` видно, что программа получила сигнал 11 — ошибку сегментации, как и ожидалось. Помимо номеров процессов, здесь есть снимок всех текущих регистров. По их значениям отладчик может восстановить состояние стека и построить трассировку вызовов; вместе с символами и отладочной информацией исходного файла это позволяет показать, как программа достигла текущей точки.

Другая интересная часть вывода — текущий вспомогательный вектор `AUXV`, рассматриваемый в разделе о передаче данных от ядра программам. Запись `386_TLS` описывает элементы глобальной таблицы дескрипторов, используемые реализацией локального для потока хранилища в x86. Подробнее сегментация и потоки рассматриваются в соответствующих разделах. Для многопоточного приложения такие записи повторяются для каждого потока; отладчик это понимает, и именно так команда `thread` в `gdb` показывает потоки и переключается между ними.

Ядро создаёт дамп с учётом текущих ограничений `ulimit`: программа, использующая много памяти, может создать очень большой файл, заполнить диск и усугубить проблему. Поэтому ограничение обычно мало или равно нулю, тем более что большинству обычных пользователей дамп не нужен. Однако для посмертного исследования неожиданной ситуации он остаётся одним из самых полезных средств отладки.

Исполняемые файлы ELF

Одно из основных применений ELF — исполняемые файлы. Двоичный файл содержит всё необходимое операционной системе, чтобы выполнить код предусмотренным образом.

Поскольку исполняемый файл запускается как процесс с собственным адресным пространством, код может предполагать, где различные части программы будут загружены в память. В примере 8.13 инструмент `readelf` показывает сегменты исполняемого файла. Видны виртуальные адреса размещения сегментов `LOAD`. Один сегмент содержит код и имеет только права чтения и исполнения; другой содержит данные и, как и ожидается, имеет права чтения и записи, но не исполнения. Это важно: даже если ошибка позволит злоумышленнику внедрить произвольные данные, страницы без права исполнения на большинстве процессоров не позволят выполнить содержащийся в них код.

Пример 8.13. Сегменты исполняемого файла

```
$ readelf --segments /bin/ls

Elf file type is EXEC (Executable file)
Entry point 0x4046d4
There are 8 program headers, starting at offset 64

Program Headers:
Type           Offset             VirtAddr           PhysAddr
FileSiz            MemSiz              Flags  Align
PHDR           0x0000000000000040 0x0000000000400040 0x0000000000400040
0x00000000000001c0 0x00000000000001c0  R E    8
INTERP         0x0000000000000200 0x0000000000400200 0x0000000000400200
0x000000000000001c 0x000000000000001c  R      1
[Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]
LOAD           0x0000000000000000 0x0000000000400000 0x0000000000400000
0x0000000000019ef4 0x0000000000019ef4  R E    200000
LOAD           0x000000000001a000 0x000000000061a000 0x000000000061a000
0x000000000000077c 0x0000000000001500  RW     200000
DYNAMIC        0x000000000001a028 0x000000000061a028 0x000000000061a028
0x00000000000001d0 0x00000000000001d0  RW     8
NOTE           0x000000000000021c 0x000000000040021c 0x000000000040021c
0x0000000000000044 0x0000000000000044  R      4
GNU_EH_FRAME   0x0000000000017768 0x0000000000417768 0x0000000000417768
0x00000000000006fc 0x00000000000006fc  R      4
GNU_STACK      0x0000000000000000 0x0000000000000000 0x0000000000000000
0x0000000000000000 0x0000000000000000  RW     8

Section to Segment mapping:
Segment Sections...
00
01     .interp
02     .interp .note.ABI-tag .note.gnu.build-id .hash .gnu.hash .dynsym .dynstr .gnu.version
.gnu.version_r .rela.dyn .rela.plt .init .plt .text .fini .rodata .eh_frame_hdr .eh_frame
03     .ctors .dtors .jcr .dynamic .got .got.plt .data .bss
04     .dynamic
05     .note.ABI-tag .note.gnu.build-id

06     .eh_frame_hdr
07
```

Сегменты программы должны быть загружены по этим адресам. Последний шаг компоновщика — разрешить большинство релокаций и исправить их с учётом предполагаемых абсолютных адресов. Затем данные о релокации отбрасываются из конечного файла, и найти их уже нельзя.

На практике исполняемые файлы обычно зависят от разделяемых библиотек — общих фрагментов кода, используемых всей системой. Почти все сложные на вид части примера

8.13 связаны именно с ними. О библиотеках мы поговорим далее, а о динамических библиотеках — в главе 9.

Библиотеки

Разработчикам быстро надоело каждый раз писать всё с нуля, поэтому одним из первых изобретений информатики стали библиотеки.

Библиотека — это просто набор функций, которые можно вызывать из программы. Преимущества очевидны: повторное использование чужой работы экономит много времени и обычно даёт больше уверенности в том, что ошибок меньше, поскольку библиотекой пользуются многие и помогают находить и исправлять ошибки. Библиотека похожа на исполняемый файл, но её функции не запускаются напрямую, а вызываются с параметрами из вашей программы.

Статические библиотеки

Самый простой способ использовать библиотечную функцию — скомпоновать объектные файлы библиотеки непосредственно с исполняемым файлом, как и собственные скомпилированные файлы. Такая библиотека называется статической: она останется неизменной, пока программа не будет собрана заново.

Это простой способ, поскольку в результате получается один исполняемый файл без зависимостей.

Внутри статических библиотек

Статическая библиотека — просто группа объектных файлов, собранных в архив; отсюда обычное расширение `.a`. Архив можно представить как ZIP-файл без сжатия.

Ниже показано создание простой статической библиотеки и несколько распространённых инструментов для работы с библиотеками.

Пример 8.14. Создание и использование статической библиотеки

```
$ cat library.c
/* Library Function */
int function(int input)
{
return input + 10;
}

$ cat library.h
/* Function Definition */
int function(int);

$ cat program.c
#include <stdio.h>
/* Library header file */
#include "library.h"

int main(void)
{
```

```
int d = function(100);

printf("%d\n", d);
}

$ gcc -c library.c
$ ar rc libtest.a library.o
$ ranlib ./libtest.a
$ nm --print-armap ./libtest.a

Archive index:
function in library.o

library.o:
00000000 T function
$ gcc -L . program.c -ltest -o program

$ ./program
110
```

Сначала мы компилируем библиотеку в объектный файл, как делали в предыдущей главе.

Обратите внимание, что API библиотеки определяется в заголовочном файле. API состоит из объявлений библиотечных функций: они нужны компилятору, чтобы знать типы аргументов при сборке объектных файлов, обращающихся к библиотеке, например `program.c`, включающего этот заголовок.

Библиотека создаётся командой `ar`, название которой сокращает слово *archive*. По соглашению имена файлов статических библиотек начинаются с `lib` и имеют расширение `.a`. Аргумент `c` предписывает создать архив, а операции команды добавляют указанные объектные файлы в библиотеку.

Архивы, созданные `ar`, встречаются в Linux и в других контекстах. Например, формат пакетов `.deb`, применяемый Debian, Ubuntu и некоторыми другими системами, использует архивы для хранения всех файлов приложения в одном пакете. В пакетах Red Hat RPM применяется другой, но похожий формат `cpio`. Классическим способом объединения файлов также является `tar`, часто используемый для распространения исходного кода.

Затем программа `ranlib` создаёт в библиотеке индекс символов входящих в неё объектных файлов. Это помогает быстро находить символы: для одной функции такой шаг может показаться избыточным, но большая библиотека может содержать тысячи символов, и индекс значительно ускоряет поиск ссылок. Новый индекс можно исследовать программой `nm`. Как и ожидалось, символ функции `function()` находится по смещению ноль.

Компилятору библиотека задаётся параметром `-lname`, где `name` — имя файла библиотеки без префикса `lib`. Мы также добавляем каталог поиска библиотек — текущий каталог с помощью `-L .`, поскольку по умолчанию он не просматривается.

В результате получается один исполняемый файл, включающий новую библиотеку.

Недостатки статической компоновки

Статическая компоновка проста, но у неё есть недостатки.

Два главных: при обновлении кода библиотеки, например для исправления ошибки, программу приходится собирать заново; кроме того, каждая программа системы, использующая библиотеку, содержит её собственную копию. Это очень неэффективно, а повторная сборка при исправлении ошибок неудобна.

Например, библиотека C `glibc` используется всеми программами и предоставляет распространённые функции вроде `printf`.

Разделяемые библиотеки

Разделяемые библиотеки позволяют изящно обойти проблемы статических. Такая библиотека динамически загружается во время выполнения для каждого приложения, которому она нужна.

Приложение оставляет указание на требуемую библиотеку, а при вызове функции библиотека загружается в память и выполняется. Если она уже загружена другим приложением, оба могут использовать один и тот же код, что заметно экономит ресурсы для широко используемых библиотек.

Этот процесс называется динамической компоновкой. Это одна из более сложных частей современной операционной системы, поэтому следующая глава посвящена её подробному изучению.

ABI

При системном программировании вы часто будете слышать термин ABI. Мы подробно обсуждали API — интерфейсы, которые видит программист, использующий ваш код.

ABI обозначает интерфейсы более низкого уровня, о которых должны договориться компилятор, операционная система и, в некоторой степени, процессор, чтобы взаимодействовать друг с другом. Ниже представлены несколько важных для понимания ABI понятий.

Порядок байтов

Порядок байтов (*endianness*).

Соглашения о вызовах

Передача параметров

Регистры или стек?

Дескрипторы функций

На многих архитектурах функцию необходимо вызывать через её дескриптор, а не напрямую.

Например, на IA64 дескриптор функции состоит из двух частей: адреса функции — 64-битного, или восьмибайтового, значения — и адреса глобального указателя `gp`. ABI предписывает, чтобы регистр `r1` всегда содержал значение `gp` данной функции. Поэтому при вызове вызывающая сторона сохраняет своё значение `gp`, устанавливает в `r1` новое значение из дескриптора и затем вызывает функцию.

Способ может показаться странным, но, как мы увидим в следующей главе о таблицах глобальных смещений, он даёт полезные практические преимущества. На IA64 инструкция сложения может принимать непосредственное значение длиной не более 22 бит. Непосредственным называют значение, указанное прямо в инструкции, а не находящееся в регистре: например, в `add r1 + 100` число 100 является непосредственным значением.

Технически это связано с тем, как IA64 объединяет инструкции в пакеты: в каждом пакете находятся три инструкции, и для непосредственного значения остаётся только 22 бита.

В 22 битах можно представить 4 194 304 байта, или 4 МБ. Таким образом, каждая функция может непосредственно обращаться со смещением к области памяти размером 4 МБ, не затрачивая время на загрузку дополнительных значений в регистр. Если компилятор, компоновщик и загрузчик согласны, куда указывает глобальный указатель, как предписывает ABI, производительность повышается благодаря уменьшению числа загрузок.

Запуск процесса

Ранее мы упоминали, что утверждение «программа начинается с функции `main()`» не совсем верно. Рассмотрим, что происходит при загрузке и запуске типичной динамически скомпонованной программы. Статически скомпонованные программы запускаются похоже, но с отличиями; в оригинале автор оставил здесь вопрос о необходимости дополнительного объяснения.

Прежде всего, в ответ на системный вызов `exec` ядро выделяет структуры нового процесса и считывает с диска указанный ELF-файл.

Как мы говорили, поле `PT_INTERP` в ELF может задавать интерпретатор программы. Для динамически скомпонованных приложений им служит динамический компоновщик `ld.so`, который позволяет выполнить часть компоновки непосредственно перед запуском.

В этом случае ядро также загружает код динамического компоновщика и начинает выполнение с указанной в нём точки входа. В следующей главе мы подробно рассмотрим его роль. Пока достаточно сказать, что он выполняет подготовку, например загружает библиотеки, требуемые приложением и указанные в динамической секции файла, после чего начинает выполнение программы с её точки входа; автор называет здесь функцию `_init`.

Передача данных от ядра программе

При запуске ядро должно сообщить программе её аргументы, текущие переменные окружения и специальную структуру — вспомогательный вектор (*Auxiliary Vector*, или `auxv`). Чтобы попросить динамический компоновщик показать отладочные сведения об `auxv`, можно установить переменную окружения `LD_SHOW_AUXV=1`.

Аргументы и окружение достаточно понятны; различные варианты системного вызова `exec` позволяют задать их для программы.

Ядро передаёт всю необходимую информацию, помещая её в стек нового процесса. Поэтому после запуска программа может найти эти сведения с помощью указателя стека.

Вспомогательный вектор предназначен для непосредственной передачи информации от ядра новой программе. Он содержит зависящие от системы данные, например размер страницы виртуальной памяти по умолчанию или аппаратные возможности — конкретные особенности оборудования, обнаруженные ядром и доступные программам пользовательского пространства.

Библиотека ядра

Ранее мы отмечали, что системные вызовы медленны и современные системы располагают механизмами, уменьшающими затраты на вызов ловушки процессора.

В Linux это реализовано с помощью взаимодействия динамического загрузчика и ядра через структуру `AUXV`. Ядро добавляет небольшую разделяемую библиотеку в адресное пространство каждого нового процесса. Она содержит функцию выполнения системных вызовов. Если оборудование поддерживает быстрый механизм, ядро как создатель библиотеки может использовать его; иначе — старый способ с ловушкой. Библиотека называется `linux-gate.so.1`, поскольку служит воротами к внутренним механизмам ядра.

Запуская динамический компоновщик, ядро добавляет в `auxv` запись `AT_SYSINFO_EHDR` с адресом этой специальной библиотеки в памяти. Компоновщик ищет указатель и, найдя его, загружает библиотеку для программы. Сама программа о ней не знает: это внутреннее соглашение компоновщика и ядра.

Программисты выполняют системные вызовы косвенно, вызывая функции системной библиотеки `libc`. Она может проверить наличие специального образа ядра и использовать его функции для системных вызовов. Если ядро определило, что оборудование поддерживает быстрый механизм, он и будет использован.

Запуск программы

После загрузки интерпретатора ядро передаёт управление точке входа, указанной в его файле. Сейчас мы не разбираем запуск самого динамического компоновщика; подробное обсуждение динамической компоновки будет в главе 9. Компоновщик переходит к адресу точки входа, заданному в двоичном ELF-файле программы.

Пример 8.15. Дизассемблирование запуска программы

```
$ cat test.c

int main(void)
{
return 0;
}
```

```
$ gcc -o test test.c
$ readelf --headers ./test | grep Entry
Entry point address:               0x80482b0

$ objdump --disassemble ./test
[...]

080482b0 <_start>:
80482b0:       31 ed                   xor    %ebp,%ebp
80482b2:       5e                      pop    %esi
80482b3:       89 e1                   mov    %esp,%ecx
80482b5:       83 e4 f0                and    $0xfffffff0,%esp

     80482b8:       50                      push   %eax
     80482b9:       54                      push   %esp
80482ba:       52                      push   %edx
     80482bb:       68 00 84 04 08          push   $0x8048400
     80482c0:       68 90 83 04 08          push   $0x8048390
     80482c5:       51                      push   %ecx
     80482c6:       56                      push   %esi
80482c7:       68 68 83 04 08          push   $0x8048368
     80482cc:       e8 b3 ff ff ff          call   8048284 <__libc_start_main@plt>
     80482d1:       f4                      hlt
     80482d2:       90                      nop
     80482d3:       90                      nop
    08048368 <main>:
     8048368:       55                      push   %ebp
     8048369:       89 e5                   mov    %esp,%ebp
     804836b:       83 ec 08                sub    $0x8,%esp
804836e:       83 e4 f0                and    $0xfffffff0,%esp
     8048371:       b8 00 00 00 00          mov    $0x0,%eax
     8048376:       83 c0 0f                add    $0xf,%eax
     8048379:       83 c0 0f                add    $0xf,%eax
     804837c:       c1 e8 04                shr    $0x4,%eax
804837f:       c1 e0 04                shl    $0x4,%eax
     8048382:       29 c4                   sub    %eax,%esp
     8048384:       b8 00 00 00 00          mov    $0x0,%eax
     8048389:       c9                      leave
     804838a:       c3                      ret
804838b:       90                      nop
     804838c:       90                      nop
     804838d:       90                      nop
     804838e:       90                      nop
     804838f:       90                      nop
    08048390 <__libc_csu_init>:
     8048390:       55                      push   %ebp
     8048391:       89 e5                   mov    %esp,%ebp
     [...]
    08048400 <__libc_csu_fini>:
     8048400:       55                      push   %ebp
     [...]
```

Здесь рассматривается простейшая программа. С помощью `readelf` видно, что её точка входа — функция `_start`. В дизассемблированном коде видно, как некоторые значения помещаются в стек. Первое значение, `0x8048400`, — адрес функции `__libc_csu_fini`; `0x8048390` — адрес `__libc_csu_init`; наконец, `0x8048368` — адрес `main()`. После этого вызывается `__libc_start_main`.

Функция `__libc_start_main` определена в файле исходного кода glibc `sysdeps/generic/libc-start.c`. Она довольно сложна и скрыта за множеством определений, поскольку должна быть переносимой на огромное число систем и архитектур, поддерживаемых glibc. Она выполняет ряд специфических действий по подготовке библиотеки C, о которых обычному

программисту заботиться не нужно. Следующая точка, где библиотека возвращается к коду программы, — обработка инициализации.

`init` и `fini` — специальные механизмы вызова частей кода разделяемых библиотек соответственно перед началом работы и при выгрузке библиотеки. Они полезны для инициализации переменных при загрузке и очистки при завершении. Первоначально в библиотеке искали функции `_init` и `_fini`, но это ограничивало возможности: всё приходилось помещать в эти функции. Далее мы рассмотрим, как работает этот процесс.

На этом этапе функция `__libc_start_main` получает довольно много входных параметров через стек. Во-первых, ей доступны аргументы программы, переменные окружения и вспомогательный вектор ядра. Кроме того, код запуска помещает в стек адреса функций для `init` и `fini` и, наконец, адрес самой `main`.

Нам нужен способ указать в исходном коде, что функция должна вызываться при инициализации или завершении. В gcc для этого применяются атрибуты, помечающие две функции основной программы как конструктор и деструктор. Эти термины чаще используются в объектно-ориентированных языках для описания жизненного цикла объектов.

Пример 8.16. Конструкторы и деструкторы

```
$ cat test.c
    #include <stdio.h>

void __attribute__((constructor)) program_init(void)  {
      printf("init\n");
    }

    void  __attribute__((destructor)) program_fini(void) {
printf("fini\n");
    }

    int main(void)
    {
return 0;
    }

    $ gcc -Wall  -o test test.c

$ ./test
    init
    fini

    $ objdump --disassemble ./test | grep program_init
08048398 <program_init>:

    $ objdump --disassemble ./test | grep program_fini
    080483b0 <program_fini>:

$ objdump --disassemble ./test

    [...]
    08048280 <_init>:
     8048280:       55                      push   %ebp
8048281:       89 e5                   mov    %esp,%ebp
     8048283:       83 ec 08                sub    $0x8,%esp

     8048286:       e8 79 00 00 00          call   8048304 <call_gmon_start>
```

```
     804828b:       e8 e0 00 00 00          call   8048370 <frame_dummy>
     8048290:       e8 2b 02 00 00          call   80484c0 <__do_global_ctors_aux>
8048295:       c9                      leave
     8048296:       c3                      ret
    [...]

    080484c0 <__do_global_ctors_aux>:
80484c0:       55                      push   %ebp
     80484c1:       89 e5                   mov    %esp,%ebp
     80484c3:       53                      push   %ebx
     80484c4:       52                      push   %edx
     80484c5:       a1 2c 95 04 08          mov    0x804952c,%eax
80484ca:       83 f8 ff                cmp    $0xffffffff,%eax
     80484cd:       74 1e                   je     80484ed <__do_global_ctors_aux+0x2d>
     80484cf:       bb 2c 95 04 08          mov    $0x804952c,%ebx
     80484d4:       8d b6 00 00 00 00       lea    0x0(%esi),%esi
     80484da:       8d bf 00 00 00 00       lea    0x0(%edi),%edi
80484e0:       ff d0                   call   *%eax
     80484e2:       8b 43 fc                mov    0xfffffffc(%ebx),%eax
     80484e5:       83 eb 04                sub    $0x4,%ebx
     80484e8:       83 f8 ff                cmp    $0xffffffff,%eax
     80484eb:       75 f3                   jne    80484e0 <__do_global_ctors_aux+0x20>
80484ed:       58                      pop    %eax
     80484ee:       5b                      pop    %ebx
     80484ef:       5d                      pop    %ebp
     80484f0:       c3                      ret
     80484f1:       90                      nop
80484f2:       90                      nop
     80484f3:       90                      nop


    $ readelf --sections ./test
There are 34 section headers, starting at offset 0xfb0:

    Section Headers:
      [Nr] Name              Type            Addr     Off    Size   ES Flg Lk Inf Al
      [ 0]                   NULL            00000000 000000 000000 00      0   0  0
[ 1] .interp           PROGBITS        08048114 000114 000013 00   A  0   0  1
      [ 2] .note.ABI-tag     NOTE            08048128 000128 000020 00   A  0   0  4
      [ 3] .hash             HASH            08048148 000148 00002c 04   A  4   0  4
      [ 4] .dynsym           DYNSYM          08048174 000174 000060 10   A  5   1  4
      [ 5] .dynstr           STRTAB          080481d4 0001d4 00005e 00   A  0   0  1
[ 6] .gnu.version      VERSYM          08048232 000232 00000c 02   A  4   0  2
      [ 7] .gnu.version_r    VERNEED         08048240 000240 000020 00   A  5   1  4
      [ 8] .rel.dyn          REL             08048260 000260 000008 08   A  4   0  4
      [ 9] .rel.plt          REL             08048268 000268 000018 08   A  4  11  4
      [10] .init             PROGBITS        08048280 000280 000017 00  AX  0   0  4
[11] .plt              PROGBITS        08048298 000298 000040 04  AX  0   0  4
      [12] .text             PROGBITS        080482e0 0002e0 000214 00  AX  0   0 16
      [13] .fini             PROGBITS        080484f4 0004f4 00001a 00  AX  0   0  4
      [14] .rodata           PROGBITS        08048510 000510 000012 00   A  0   0  4
      [15] .eh_frame         PROGBITS        08048524 000524 000004 00   A  0   0  4
[16] .ctors            PROGBITS        08049528 000528 00000c 00  WA  0   0  4

      [17] .dtors            PROGBITS        08049534 000534 00000c 00  WA  0   0  4
      [18] .jcr              PROGBITS        08049540 000540 000004 00  WA  0   0  4
      [19] .dynamic          DYNAMIC         08049544 000544 0000c8 08  WA  5   0  4
      [20] .got              PROGBITS        0804960c 00060c 000004 04  WA  0   0  4
[21] .got.plt          PROGBITS        08049610 000610 000018 04  WA  0   0  4
      [22] .data             PROGBITS        08049628 000628 00000c 00  WA  0   0  4
      [23] .bss              NOBITS          08049634 000634 000004 00  WA  0   0  4
      [24] .comment          PROGBITS        00000000 000634 00018f 00      0   0  1
      [25] .debug_aranges    PROGBITS        00000000 0007c8 000078 00      0   0  8
[26] .debug_pubnames   PROGBITS        00000000 000840 000025 00      0   0  1
      [27] .debug_info       PROGBITS        00000000 000865 0002e1 00      0   0  1
      [28] .debug_abbrev     PROGBITS        00000000 000b46 000076 00      0   0  1
      [29] .debug_line       PROGBITS        00000000 000bbc 0001da 00      0   0  1
```

```
      [30] .debug_str        PROGBITS        00000000 000d96 0000f3 01  MS  0   0  1
[31] .shstrtab         STRTAB          00000000 000e89 000127 00      0   0  1
      [32] .symtab           SYMTAB          00000000 001500 000490 10     33  53  4
      [33] .strtab           STRTAB          00000000 001990 000218 00      0   0  1
    Key to Flags:
      W (write), A (alloc), X (execute), M (merge), S (strings)
I (info), L (link order), G (group), x (unknown)
      O (extra OS processing required) o (OS specific), p (processor specific)

    $ objdump --disassemble-all --section .ctors ./test

./test:     file format elf32-i386

    Contents of section .ctors:
     8049528 ffffffff 98830408 00000000           ............
```

Последнее значение, помещённое в стек для `__libc_start_main`, — функция инициализации `__libc_csu_init`. Если проследить цепочку вызовов, видно, что она выполняет некоторую подготовку и вызывает `_init` исполняемого файла. Та в итоге вызывает `__do_global_ctors_aux`. В дизассемблированном коде видно, что функция начинает с адреса `0x804952c` и в цикле считывает значение и вызывает его. Этот адрес находится в секции `.ctors`; внутри неё находятся первое значение `-1`, адрес функции и ноль.

Адрес функции — `0x08048398`, то есть адрес `program_init`. Таким образом, секция `.ctors` содержит сначала `-1`, затем адреса функций, вызываемых при инициализации, и, наконец, ноль как признак конца списка. Вызывается каждый элемент; в этом случае функция всего одна.

Закончив вызов `_init`, `__libc_start_main` наконец вызывает `main()`! Напомним, что в начале ядро подготовило стек с аргументами и указателями окружения: именно так `main` получает аргументы `argc`, `argv[]` и `envp[]`. Теперь процесс работает, а этап подготовки завершён.

Аналогичным образом при выходе программы обрабатывается секция `.dtors` с деструкторами: `__libc_start_main` вызывает их после завершения `main()`.

Как видите, до начала работы программы происходит очень многое — и даже немного после того, как вы считаете её завершённой!

Глава 9. Динамическая компоновка

Совместное использование кода

Мы знаем, что операционная система рассматривает код как доступный только для чтения и отделяет его от данных. Поэтому логично: если программы не могут изменять код и содержат много общего кода, его следует не копировать в каждый исполняемый файл, а использовать совместно.

С виртуальной памятью это несложно. На физические страницы памяти, в которые загружен код библиотеки, может ссылаться любое число виртуальных страниц из любого числа адресных пространств. Таким образом, в физической памяти системы есть только одна копия кода библиотеки, но каждый процесс может обращаться к ней по удобному ему виртуальному адресу.

Поэтому быстро возникла идея разделяемой библиотеки, которую, как следует из названия, используют несколько исполняемых файлов. Каждый исполняемый файл содержит ссылку, по существу сообщающую: «Мне нужна библиотека foo». При загрузке программы система проверяет, не загрузил ли уже другой процесс код библиотеки foo. Если загрузил, соответствующие физические страницы отображаются в адресное пространство программы; иначе библиотека загружается в память.

Этот процесс называется динамической компоновкой: часть работы компоновщика выполняется «на лету» при запуске программ в системе.

Подробности динамических библиотек

Библиотека очень похожа на программу, которую никогда не запускают целиком. У неё, как и у любого исполняемого файла, есть секции кода и данных — функции и переменные, — но нет места, с которого начинается выполнение. Она просто предоставляет библиотеку функций, которые разработчики могут вызывать.

Поэтому ELF может представлять динамическую библиотеку так же, как исполняемый файл. Есть принципиальные отличия, например отсутствие указателя на начало исполнения, но любая разделяемая библиотека является объектом ELF, как и другие исполняемые файлы.

Два взаимоисключающих значения в заголовке ELF — `ET_EXEC` и `ET_DYN` — обозначают соответственно исполняемый файл и разделяемый объект.

Включение библиотек в исполняемый файл

Компиляция

При компиляции программы, использующей динамическую библиотеку, в объектных файлах остаются ссылки на её функции, как и любые другие внешние ссылки.

Нужно включить заголовочный файл библиотеки, чтобы компилятор знал точные типы вызываемых функций. Ему достаточно знать типы — например, что функция принимает `int` и возвращает `char *`, — чтобы правильно подготовить вызов.

Так было не всегда. Ранее компиляторы C предполагали, что неизвестная функция возвращает `int`. На 32-битной системе указатель обычно имеет тот же размер, что и `int`, поэтому проблемы могло не возникнуть. На 64-битной системе указатель обычно вдвое больше, и если функция на самом деле возвращает указатель, его значение будет испорчено и не укажет на допустимую память. Стандарт C99 требует объявлять типы вызываемых функций.

Компоновка

Хотя значительную часть работы с разделяемыми библиотеками выполняет динамический компоновщик, обычный компоновщик тоже участвует в создании исполняемого файла.

Он должен оставить в исполняемом файле указание, по которому динамический компоновщик во время выполнения определит библиотеку для удовлетворения зависимости.

В динамической секции исполняемого файла требуется запись `NEEDED` для каждой разделяемой библиотеки, от которой он зависит. Эти поля снова можно исследовать с помощью `readelf`. Ниже рассмотрен обычный системный файл `/bin/ls`.

Пример 9.1. Указание динамических библиотек

```
$ readelf --dynamic /bin/ls

Dynamic segment at offset 0x22f78 contains 27 entries:
Tag        Type                         Name/Value
0x0000000000000001 (NEEDED)             Shared library: [librt.so.1]
0x0000000000000001 (NEEDED)             Shared library: [libacl.so.1]
0x0000000000000001 (NEEDED)             Shared library: [libc.so.6.1]
0x000000000000000c (INIT)               0x4000000000001e30
... snip ...
```

В примере указаны три библиотеки. Самая распространённая из них, используемая большинством, если не всеми программами системы, — `libc`. Есть и другие библиотеки, необходимые для правильной работы программы.

Иногда полезно читать ELF напрямую, но обычно динамически скомпонованный исполняемый файл исследуют с помощью `ldd`. Утилита обходит зависимости библиотек: если одна библиотека зависит от другой, она тоже будет показана.

Пример 9.2. Просмотр динамических библиотек

```
$ ldd /bin/ls
librt.so.1 => /lib/tls/librt.so.1 (0x2000000000058000)
libacl.so.1 => /lib/libacl.so.1 (0x2000000000078000)
libc.so.6.1 => /lib/tls/libc.so.6.1 (0x2000000000098000)
libpthread.so.0 => /lib/tls/libpthread.so.0 (0x20000000002e0000)
/lib/ld-linux-ia64.so.2 => /lib/ld-linux-ia64.so.2 (0x2000000000000000)
libattr.so.1 => /lib/libattr.so.1 (0x2000000000310000)
$ readelf --dynamic /lib/librt.so.1
```

```
Dynamic segment at offset 0xd600 contains 30 entries:
Tag        Type                         Name/Value
0x0000000000000001 (NEEDED)             Shared library: [libc.so.6.1]
0x0000000000000001 (NEEDED)             Shared library: [libpthread.so.0]
... snip ...
```

Здесь видно, что откуда-то потребовалась библиотека `libpthread`. Немного поискав, можно выяснить, что зависимость исходит от `librt`.

Динамический компоновщик

Динамический компоновщик — программа, управляющая разделяемыми динамическими библиотеками от имени исполняемого файла. Она загружает библиотеки в память и во время выполнения изменяет программу так, чтобы та вызывала библиотечные функции.

ELF позволяет исполняемому файлу указать интерпретатор — программу, с помощью которой его следует запускать. Компилятор и статический компоновщик указывают для файлов, зависящих от динамических библиотек, в качестве интерпретатора динамический компоновщик.

Пример 9.3. Проверка интерпретатора программы

```
ianw@lime:~/programs/csbu$ readelf --headers /bin/ls

Program Headers:
Type           Offset             VirtAddr           PhysAddr
FileSiz            MemSiz              Flags  Align
PHDR           0x0000000000000040 0x4000000000000040 0x4000000000000040
0x0000000000000188 0x0000000000000188  R E    8
INTERP         0x00000000000001c8 0x40000000000001c8 0x40000000000001c8
0x0000000000000018 0x0000000000000018  R      1
[Requesting program interpreter: /lib/ld-linux-ia64.so.2]
LOAD           0x0000000000000000 0x4000000000000000 0x4000000000000000
0x0000000000022e40 0x0000000000022e40  R E    10000
LOAD           0x0000000000022e40 0x6000000000002e40 0x6000000000002e40
0x0000000000001138 0x00000000000017b8  RW     10000
DYNAMIC        0x0000000000022f78 0x6000000000002f78 0x6000000000002f78
0x0000000000000200 0x0000000000000200  RW     8
NOTE           0x00000000000001e0 0x40000000000001e0 0x40000000000001e0
0x0000000000000020 0x0000000000000020  R      4
IA_64_UNWIND   0x0000000000022018 0x4000000000022018 0x4000000000022018
0x0000000000000e28 0x0000000000000e28  R      8
```

Выше в качестве интерпретатора указан `/lib/ld-linux-ia64.so.2` — динамический компоновщик. Загружая файл для выполнения, ядро проверяет наличие поля `PT_INTERP`; если оно присутствует, загружает указанный объект в память и запускает его.

Мы говорили, что динамически скомпонованные исполняемые файлы оставляют ссылки, которые нужно исправить с помощью сведений, доступных только во время выполнения, например адреса функции в разделяемой библиотеке. Такие ссылки называются релокациями.

Релокации

Существенная часть работы динамического компоновщика — исправление адресов во время выполнения, когда уже точно известно место загрузки в память. Релокацию можно представить как заметку о том, что конкретный адрес нужно исправить при загрузке. Прежде чем код будет готов к выполнению, нужно прочитать все релокации и исправить указанные в них адреса.

Таблица 9.1. Пример релокаций

Адрес	Действие
`0x123456`	Адрес символа `x`
`0x564773`	Функция X

Для каждой архитектуры существует много типов релокаций; точное поведение каждого описано в ABI системы. Определение релокации достаточно просто.

Пример 9.4. Релокация в определении ELF

```
typedef struct {
    Elf32_Addr  r_offset;  /* address to fix */
    Elf32_Word  r_info;    /* symbol table pointer and relocation type */
}

typedef struct {
    Elf32_Addr  r_offset;
    Elf32_Word  r_info;
    Elf32_Sword r_addend;
} Elf32_Rela
```

Поле `r_offset` задаёт смещение в файле, которое требуется исправить. `r_info` определяет тип релокации, описывающий точный способ исправления. Простейшая релокация для архитектуры обычно просто подставляет значение символа. В этом случае адрес символа записывается в указанное место, после чего релокация считается обработанной.

Две формы, с добавкой и без неё, задают разные способы работы. Добавка — это величина, которую нужно прибавить к исправленному адресу, чтобы получить правильный адрес. Например, если релокация относится к символу `i`, потому что исходный код делает что-то вроде `i[8]`, добавка равна 8. Это означает: «Найди адрес `i` и перейди на восемь единиц дальше».

Добавку нужно где-то хранить. Для этого существуют две формы. В форме REL добавка хранится в самом коде программы на месте будущего исправленного адреса. Поэтому для исправления сначала нужно прочитать и сохранить добавку, затем найти фактический адрес, прибавить её и записать результат поверх прежнего значения. В форме RELA добавка содержится непосредственно в записи релокации.

Компромисс очевиден: REL требует дополнительного обращения к памяти за добавкой, но не расходует отдельное место в файле, поскольку использует адрес назначения релокации.

В RELA добавка хранится вместе с релокацией, но занимает дополнительное место в двоичном файле на диске. Большинство современных систем использует RELA.

Релокации в действии

В следующем примере показана работа релокаций. Мы создаём две очень простые разделяемые библиотеки и ссылаемся из одной на другую.

Пример 9.5. Указание динамических библиотек

```
$ cat addendtest.c
extern int i[4];
int *j = i + 2;
$ cat addendtest2.c
int i[4];

$ gcc -nostdlib -shared -fpic -s -o addendtest2.so addendtest2.c
$ gcc -nostdlib -shared -fpic -o addendtest.so addendtest.c ./addendtest2.so

$ readelf -r ./addendtest.so

Relocation section '.rela.dyn' at offset 0x3b8 contains 1 entries:
Offset          Info           Type           Sym. Value    Sym. Name + Addend
0000000104f8  000f00000027 R_IA64_DIR64LSB   0000000000000000 i + 8
```

В `addendtest.so` имеется одна релокация типа `R_IA64_DIR64LSB`. По ABI IA64 её обозначение можно разобрать так:

1. `R_IA64`: этот префикс имеют все релокации архитектуры.
2. `DIR64`: 64-битная прямая релокация.
3. `LSB`: IA64 поддерживает прямой и обратный порядок байтов; здесь используется обратный порядок, с младшим байтом первым.

Согласно ABI эта релокация означает «значение символа, на который она ссылается, плюс добавка». Добавка равна 8, поскольку `sizeof(int) == 4`, а мы сдвинулись на два элемента `int` в массиве (`i + 2`). При выполнении для обработки релокации нужно найти адрес символа `i` и записать его значение плюс 8 по адресу `0x104f8`.

Независимость от расположения

В исполняемом файле сегментам кода и данных задаётся определённый базовый виртуальный адрес. Код исполняемого файла не разделяется, а каждый процесс получает собственное адресное пространство. Поэтому компилятор знает, где будет секция данных, и может ссылаться на неё напрямую.

Для библиотек такой гарантии нет. Они могут знать, что секция данных находится на определённом смещении от базового адреса, но сам адрес становится известен только во время выполнения.

Поэтому библиотеки должны создаваться с кодом, который может выполняться независимо от места размещения в памяти. Его называют позиционно-независимым кодом, или PIC.

Секция данных по-прежнему находится на фиксированном смещении от кода, но для получения её фактического адреса нужно прибавить это смещение к адресу загрузки.

Таблицы глобальных смещений

Рассуждая о целях разделяемой библиотеки, можно заметить важную проблему релокаций. Мы говорили, что главное преимущество разделяемой библиотеки при виртуальной памяти — возможность нескольким программам использовать один код благодаря совместному использованию страниц.

Проблема в том, что библиотеки не знают заранее, где окажутся в памяти. Динамический компоновщик выбирает удобное место в виртуальной памяти для каждой нужной библиотеки и размещает её там. Подумайте, что было бы иначе: каждой библиотеке системы понадобился бы собственный непересекающийся участок виртуальной памяти, который пришлось бы выделять при добавлении библиотеки. Кто-то мог бы написать огромную библиотеку и не оставить места другим, хотя вашей программе она, возможно, и вовсе не нужна.

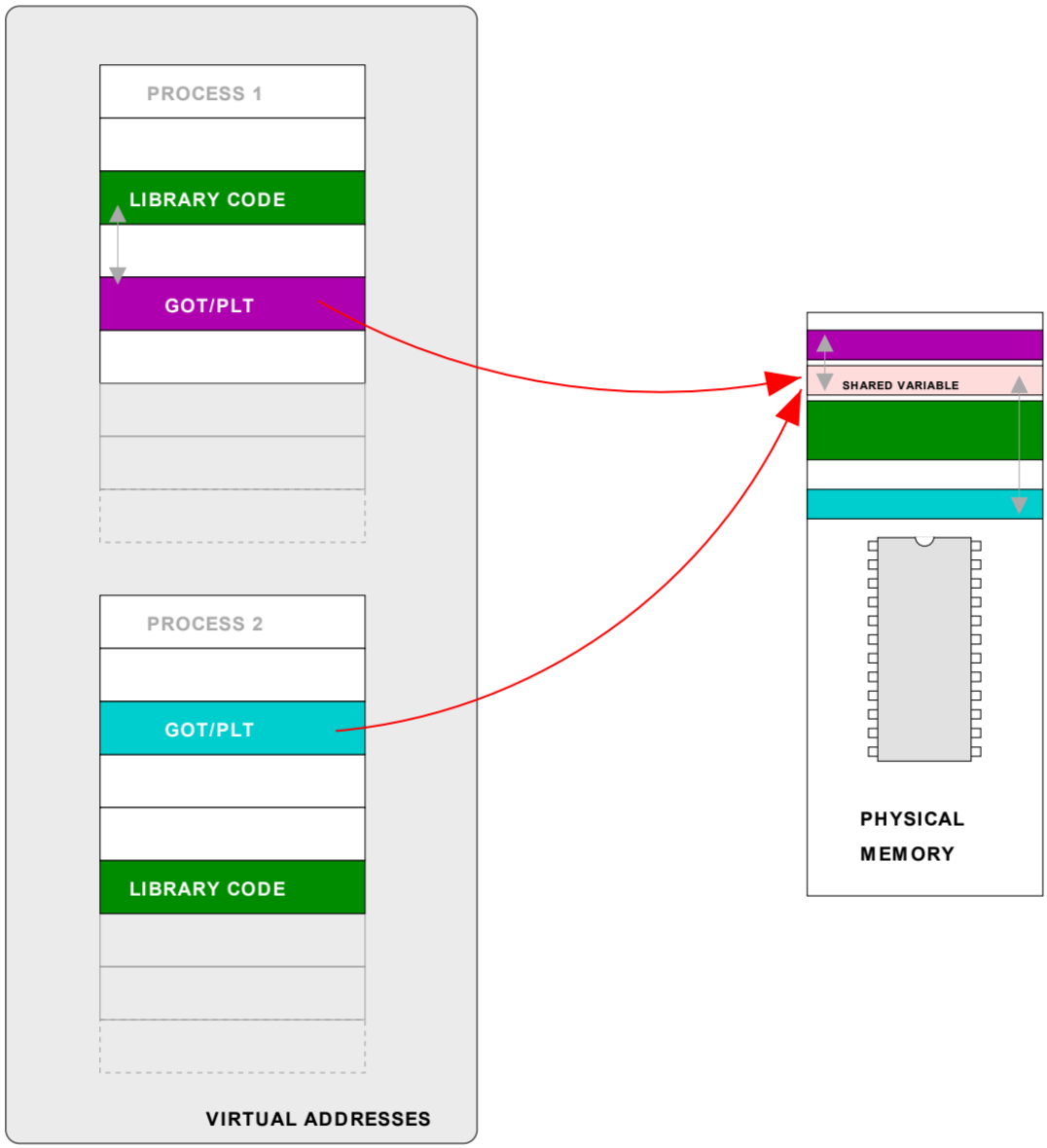
Если изменить код разделяемой библиотеки при обработке релокации, его уже нельзя будет совместно использовать, и главное преимущество библиотеки пропадёт. Ниже объясняется механизм, позволяющий этого избежать.

Глобальная таблица смещений

Представим, что мы получаем значение символа. Имея только релокации, динамический компоновщик нашёл бы адрес символа в памяти и переписал код так, чтобы тот загружал этот адрес.

Простое улучшение — зарезервировать в двоичном файле место для адреса и попросить динамический компоновщик записать адрес туда, а не непосредственно в код. Тогда трогать кодовую часть файла не понадобится.

Эта область называется глобальной таблицей смещений — *Global Offset Table*, или GOT. Она находится в секции ELF-файла `.got`.



GOT принадлежит каждому процессу отдельно, и процесс должен иметь право записи в неё. Напротив, код библиотеки разделяется, и процессу следует предоставить к нему только права чтения и исполнения: возможность изменять код стала бы серьёзным нарушением безопасности.

GOT в действии

Пример 9.6. Использование GOT

```
$ cat got.c
extern int i;

void test(void)

    {
i = 100;
    }

$ gcc -nostdlib  -shared -o got.so ./got.c
```

```
    $ objdump --disassemble ./got.so

    ./got.so:     file format elf64-ia64-little
    Disassembly of section .text:

    0000000000000410 <test>:
     410:   0d 10 00 18 00 21       [MFI]       mov r2=r12
416:   00 00 00 02 00 c0                   nop.f 0x0
     41c:   81 09 00 90                        addl r14=24,r1;;
     420:   0d 78 00 1c 18 10       [MFI]       ld8 r15=[r14]
     426:   00 00 00 02 00 c0                   nop.f 0x0
     42c:   41 06 00 90                        mov r14=100;;
430:   11 00 38 1e 90 11       [MIB]       st4 [r15]=r14
     436:   c0 00 08 00 42 80                  mov r12=r2
     43c:   08 00 84 00                        br.ret.sptk.many b0;;

    $ readelf --sections ./got.so
There are 17 section headers, starting at offset 0x640:

    Section Headers:
      [Nr] Name              Type             Address           Offset
Size              EntSize          Flags  Link  Info  Align
[ 0]                   NULL             0000000000000000  00000000
0000000000000000  0000000000000000           0     0     0
      [ 1] .hash             HASH             0000000000000120  00000120
00000000000000a0  0000000000000004   A       2     0     8
      [ 2] .dynsym           DYNSYM           00000000000001c0  000001c0
00000000000001f8  0000000000000018   A       3     e     8
      [ 3] .dynstr           STRTAB           00000000000003b8  000003b8
000000000000003f  0000000000000000   A       0     0     1
      [ 4] .rela.dyn         RELA             00000000000003f8  000003f8
0000000000000018  0000000000000018   A       2     0     8
[ 5] .text             PROGBITS         0000000000000410  00000410
0000000000000030  0000000000000000  AX       0     0     16
      [ 6] .IA_64.unwind_inf PROGBITS         0000000000000440  00000440
0000000000000018  0000000000000000   A       0     0     8
      [ 7] .IA_64.unwind     IA_64_UNWIND     0000000000000458  00000458
0000000000000018  0000000000000000  AL       5     5     8
      [ 8] .data             PROGBITS         0000000000010470  00000470
0000000000000000  0000000000000000  WA       0     0     1
      [ 9] .dynamic          DYNAMIC          0000000000010470  00000470
0000000000000100  0000000000000010  WA       3     0     8
[10] .got              PROGBITS         0000000000010570  00000570
0000000000000020  0000000000000000 WAp       0     0     8
      [11] .sbss             NOBITS           0000000000010590  00000590
0000000000000000  0000000000000000   W       0     0     1
      [12] .bss              NOBITS           0000000000010590  00000590

0000000000000000  0000000000000000  WA       0     0     1
      [13] .comment          PROGBITS         0000000000000000  00000590
0000000000000026  0000000000000000           0     0     1
      [14] .shstrtab         STRTAB           0000000000000000  000005b6
000000000000008a  0000000000000000           0     0     1
[15] .symtab           SYMTAB           0000000000000000  00000a80
0000000000000258  0000000000000018          16    12     8
      [16] .strtab           STRTAB           0000000000000000  00000cd8
0000000000000045  0000000000000000           0     0     1
    Key to Flags:
W (write), A (alloc), X (execute), M (merge), S (strings)
      I (info), L (link order), G (group), x (unknown)
      O (extra OS processing required) o (OS specific), p (processor specific)
```

Мы создали простую разделяемую библиотеку, ссылающуюся на внешний символ. При компиляции его адрес неизвестен, поэтому мы оставляем динамическому компоновщику задачу исправить ссылку при выполнении.

Но нужно сохранить возможность совместного использования кода, если к библиотеке обратятся другие процессы.

Дизассемблирование показывает, как это делается с помощью `.got`. На IA64 — архитектуре, для которой собрана библиотека, — регистр `r1` называется глобальным указателем и всегда указывает на место загрузки секции `.got` в память.

Из вывода `readelf` видно, что секция `.got` начинается на 0x10570 байтов дальше адреса загрузки библиотеки. Значит, если библиотека загружена по адресу `0x6000000000000000`, таблица `.got` будет находиться по адресу `0x6000000000010570`, и регистр `r1` всегда будет указывать туда.

Разбирая дизассемблированный код с конца, видим, что значение 100 записывается по адресу в регистре `r15`. Ранее в `r15` загружается значение по адресу из регистра `r14`. Ещё раньше этот адрес получается прибавлением небольшого числа к `r1`. GOT — длинный список записей, по одной для каждой внешней переменной. Запись для внешней переменной `i` находится на смещении 24 байта — три 64-битных адреса.

Пример 9.7. Релокации для GOT

```
$ readelf --relocs ./got.so

    Relocation section '.rela.dyn' at offset 0x3f8 contains 1 entries:
Offset          Info           Type           Sym. Value    Sym. Name + Addend
    000000010588  000f00000027 R_IA64_DIR64LSB   0000000000000000 i + 0
```

Можно также исследовать релокацию этой записи. Она говорит: «Замени значение по смещению `0x10588` адресом памяти, где хранится символ `i`».

Из предыдущего вывода известно, что `.got` начинается со смещения `0x10570`. Код загружает адрес на `0x18`, или 24 десятичных байта, дальше: `0x10570 + 0x18 = 0x10588` — именно для него и задана релокация.

Таким образом, до начала работы программы динамический компоновщик исправит релокацию так, что значением по смещению `0x10588` станет адрес глобальной переменной `i`.

Библиотеки

Таблица поиска процедур

Библиотеки могут содержать множество функций, а программа — использовать много библиотек. При этом из каждой могут понадобиться лишь одна-две функции, и в зависимости от пути выполнения некоторые функции будут вызваны, а другие — нет.

Как мы видели, динамическая компоновка требует заметных вычислений, поскольку связана с поиском по многочисленным таблицам. Любое сокращение накладных расходов повышает производительность.

Таблица поиска процедур (*Procedure Lookup Table*, PLT) обеспечивает так называемое ленивое связывание. Связывание здесь означает то же исправление адреса, которое мы рассмотрели для переменных в GOT. Когда запись исправлена, говорят, что она связана со своим фактическим адресом.

Иногда программа подключает функцию из библиотеки, но никогда её не вызывает, например из-за конкретного ввода пользователя. Связывание функции требует загрузки кода, поиска в таблицах и записи в память. Выполнять такую работу для неиспользуемой функции — пустая трата времени.

Ленивое связывание с помощью PLT откладывает эти затраты до фактического вызова функции.

У каждой библиотечной функции есть запись в PLT, которая первоначально указывает на специальную заглушку. При вызове функции программа на самом деле вызывает запись PLT — подобно тому как переменные адресуются через GOT.

Заглушка загружает несколько параметров, необходимых динамическому компоновщику для разрешения функции, и вызывает специальную функцию поиска компоновщика. Тот находит фактический адрес функции и записывает его в вызывающий двоичный файл вместо вызова заглушки.

Поэтому при следующем вызове адрес можно загрузить без повторного обращения к динамическому компоновщику. Если функция никогда не вызывается, её запись PLT не изменится, но и накладных расходов на разрешение не возникнет.

PLT в действии

Дальше всё становится немного сложнее! Но хотя бы станет ясно, сколько работы может требовать разрешение динамического символа.

Рассмотрим простое приложение «Hello World». Оно выполнит всего один библиотечный вызов `printf`, чтобы вывести пользователю строку.

Пример 9.8. Hello World и PLT

```
$ cat hello.c
#include <stdio.h>

int main(void)
{
printf("Hello, World!\n");
return 0;
}

    $ gcc -o hello hello.c

    $ readelf --relocs ./hello

Relocation section '.rela.dyn' at offset 0x3f0 contains 2 entries:
      Offset          Info           Type           Sym. Value    Sym. Name + Addend
    6000000000000ed8  000700000047 R_IA64_FPTR64LSB  0000000000000000 _Jv_RegisterClasses + 0
    6000000000000ee0  000900000047 R_IA64_FPTR64LSB  0000000000000000 __gmon_start__ + 0
```

```
Relocation section '.rela.IA_64.pltoff' at offset 0x420 contains 3 entries:
      Offset          Info           Type           Sym. Value    Sym. Name + Addend
    6000000000000f10  000200000081 R_IA64_IPLTLSB    0000000000000000 printf + 0
    6000000000000f20  000800000081 R_IA64_IPLTLSB    0000000000000000 __libc_start_main + 0
    6000000000000f30  000900000081 R_IA64_IPLTLSB    0000000000000000 __gmon_start__ + 0
```

Здесь для символа `printf` имеется релокация `R_IA64_IPLTLSB`. Она означает: «Помести адрес символа `printf` по адресу памяти `0x6000000000000f10`». Чтобы понять точную последовательность получения функции, придётся копнуть глубже.

Ниже показан дизассемблированный код функции `main()`.

Пример 9.9. Функция main() программы Hello World

```
4000000000000790 <main>:
    4000000000000790:       00 08 15 08 80 05       [MII]       alloc r33=ar.pfs,5,4,0
    4000000000000796:       20 02 30 00 42 60                   mov r34=r12
40000000000079c:       04 08 00 84                         mov r35=r1
    40000000000007a0:       01 00 00 00 01 00       [MII]       nop.m 0x0
    40000000000007a6:       00 02 00 62 00 c0                   mov r32=b0
    40000000000007ac:       81 0c 00 90                         addl r14=72,r1;;
    40000000000007b0:       1c 20 01 1c 18 10       [MFB]       ld8 r36=[r14]
40000000000007b6:       00 00 00 02 00 00                   nop.f 0x0
    40000000000007bc:       78 fd ff 58                         br.call.sptk.many b0=4000000000000520
<_init+0xb0>
    40000000000007c0:       02 08 00 46 00 21       [MII]       mov r1=r35
    40000000000007c6:       e0 00 00 00 42 00                   mov r14=r0;;
    40000000000007cc:       01 70 00 84                         mov r8=r14
40000000000007d0:       00 00 00 00 01 00       [MII]       nop.m 0x0
    40000000000007d6:       00 08 01 55 00 00                   mov.i ar.pfs=r33
    40000000000007dc:       00 0a 00 07                         mov b0=r32
    40000000000007e0:       1d 60 00 44 00 21       [MFB]       mov r12=r34
    40000000000007e6:       00 00 00 02 00 80                   nop.f 0x0
40000000000007ec:       08 00 84 00                         br.ret.sptk.many b0;;
```

Вызов по адресу `0x4000000000000520` должен быть вызовом `printf`. Узнать, где он находится, можно с помощью `readelf`, просмотрев секции.

Пример 9.10. Секции программы Hello World

```
$ readelf --sections ./hello
    There are 40 section headers, starting at offset 0x25c0:

Section Headers:
      [Nr] Name              Type             Address           Offset
Size              EntSize          Flags  Link  Info  Align
      [ 0]                   NULL             0000000000000000  00000000
0000000000000000  0000000000000000           0     0     0
...
      [11] .plt              PROGBITS         40000000000004c0  000004c0
00000000000000c0  0000000000000000  AX       0     0     32
      [12] .text             PROGBITS         4000000000000580  00000580
00000000000004a0  0000000000000000  AX       0     0     32
[13] .fini             PROGBITS         4000000000000a20  00000a20
0000000000000040  0000000000000000  AX       0     0     16
      [14] .rodata           PROGBITS         4000000000000a60  00000a60
000000000000000f  0000000000000000   A       0     0     8
      [15] .opd              PROGBITS         4000000000000a70  00000a70
0000000000000070  0000000000000000   A       0     0     16
      [16] .IA_64.unwind_inf PROGBITS         4000000000000ae0  00000ae0
00000000000000f0  0000000000000000   A       0     0     8
```

```
      [17] .IA_64.unwind     IA_64_UNWIND     4000000000000bd0  00000bd0
00000000000000c0  0000000000000000  AL      12     c     8
[18] .init_array       INIT_ARRAY       6000000000000c90  00000c90
0000000000000018  0000000000000000  WA       0     0     8
      [19] .fini_array       FINI_ARRAY       6000000000000ca8  00000ca8
0000000000000008  0000000000000000  WA       0     0     8
      [20] .data             PROGBITS         6000000000000cb0  00000cb0
0000000000000004  0000000000000000  WA       0     0     4
      [21] .dynamic          DYNAMIC          6000000000000cb8  00000cb8
00000000000001e0  0000000000000010  WA       5     0     8
      [22] .ctors            PROGBITS         6000000000000e98  00000e98
0000000000000010  0000000000000000  WA       0     0     8
[23] .dtors            PROGBITS         6000000000000ea8  00000ea8
0000000000000010  0000000000000000  WA       0     0     8
      [24] .jcr              PROGBITS         6000000000000eb8  00000eb8
0000000000000008  0000000000000000  WA       0     0     8
      [25] .got              PROGBITS         6000000000000ec0  00000ec0
0000000000000050  0000000000000000 WAp       0     0     8
      [26] .IA_64.pltoff     PROGBITS         6000000000000f10  00000f10
0000000000000030  0000000000000000 WAp       0     0     16
      [27] .sdata            PROGBITS         6000000000000f40  00000f40
0000000000000010  0000000000000000 WAp       0     0     8
[28] .sbss             NOBITS           6000000000000f50  00000f50
0000000000000008  0000000000000000  WA       0     0     8
      [29] .bss              NOBITS           6000000000000f58  00000f50
0000000000000008  0000000000000000  WA       0     0     8
      [30] .comment          PROGBITS         0000000000000000  00000f50
00000000000000b9  0000000000000000           0     0     1
      [31] .debug_aranges    PROGBITS         0000000000000000  00001010
0000000000000090  0000000000000000           0     0     16
      [32] .debug_pubnames   PROGBITS         0000000000000000  000010a0
0000000000000025  0000000000000000           0     0     1
[33] .debug_info       PROGBITS         0000000000000000  000010c5

00000000000009c4  0000000000000000           0     0     1
      [34] .debug_abbrev     PROGBITS         0000000000000000  00001a89
0000000000000124  0000000000000000           0     0     1
      [35] .debug_line       PROGBITS         0000000000000000  00001bad
00000000000001fe  0000000000000000           0     0     1
      [36] .debug_str        PROGBITS         0000000000000000  00001dab
00000000000006a1  0000000000000001  MS       0     0     1
      [37] .shstrtab         STRTAB           0000000000000000  0000244c
000000000000016f  0000000000000000           0     0     1
[38] .symtab           SYMTAB           0000000000000000  00002fc0
0000000000000b58  0000000000000018          39    60     8
      [39] .strtab           STRTAB           0000000000000000  00003b18
0000000000000479  0000000000000000           0     0     1
    Key to Flags:
W (write), A (alloc), X (execute), M (merge), S (strings)
      I (info), L (link order), G (group), x (unknown)
      O (extra OS processing required) o (OS specific), p (processor specific)
```

Как и следовало ожидать, этот адрес находится в секции `.plt`. Значит, мы нашли вызов PLT! Но не будем на этом останавливаться: дизассемблируем секцию `.plt` и посмотрим, что делает этот вызов.

Пример 9.11. PLT программы Hello World

```
40000000000004c0 <.plt>:
    40000000000004c0:       0b 10 00 1c 00 21       [MMI]       mov r2=r14;;
    40000000000004c6:       e0 00 08 00 48 00                   addl r14=0,r2
40000000000004cc:       00 00 04 00                         nop.i 0x0;;
    40000000000004d0:       0b 80 20 1c 18 14       [MMI]       ld8 r16=[r14],8;;
    40000000000004d6:       10 41 38 30 28 00                   ld8 r17=[r14],8
```

```
    40000000000004dc:       00 00 04 00                             nop.i 0x0;;
    40000000000004e0:       11 08 00 1c 18 10       [MIB]           ld8 r1=[r14]
40000000000004e6:       60 88 04 80 03 00                       mov b6=r17
    40000000000004ec:       60 00 80 00                             br.few b6;;
    40000000000004f0:       11 78 00 00 00 24       [MIB]           mov r15=0
    40000000000004f6:       00 00 00 02 00 00                       nop.i 0x0
    40000000000004fc:       d0 ff ff 48                             br.few 40000000000004c0 <_init+0x50>;;
4000000000000500:       11 78 04 00 00 24       [MIB]       mov r15=1
    4000000000000506:       00 00 00 02 00 00                       nop.i 0x0
    400000000000050c:       c0 ff ff 48                             br.few 40000000000004c0 <_init+0x50>;;
    4000000000000510:       11 78 08 00 00 24       [MIB]       mov r15=2
    4000000000000516:       00 00 00 02 00 00                       nop.i 0x0
400000000000051c:       b0 ff ff 48                         br.few 40000000000004c0 <_init+0x50>;;
    4000000000000520:       0b 78 40 03 00 24       [MMI]       addl r15=80,r1;;
    4000000000000526:       00 41 3c 70 29 c0                       ld8.acq r16=[r15],8
    400000000000052c:       01 08 00 84                             mov r14=r1;;
    4000000000000530:       11 08 00 1e 18 10       [MIB]       ld8 r1=[r15]
4000000000000536:       60 80 04 80 03 00                       mov b6=r16
    400000000000053c:       60 00 80 00                             br.few b6;;
    4000000000000540:       0b 78 80 03 00 24       [MMI]       addl r15=96,r1;;
    4000000000000546:       00 41 3c 70 29 c0                       ld8.acq r16=[r15],8

    400000000000054c:       01 08 00 84                             mov r14=r1;;
4000000000000550:       11 08 00 1e 18 10       [MIB]       ld8 r1=[r15]
    4000000000000556:       60 80 04 80 03 00                       mov b6=r16
    400000000000055c:       60 00 80 00                             br.few b6;;
    4000000000000560:       0b 78 c0 03 00 24       [MMI]       addl r15=112,r1;;
    4000000000000566:       00 41 3c 70 29 c0                       ld8.acq r16=[r15],8
400000000000056c:       01 08 00 84                         mov r14=r1;;
    4000000000000570:       11 08 00 1e 18 10       [MIB]       ld8 r1=[r15]
    4000000000000576:       60 80 04 80 03 00                       mov b6=r16
    400000000000057c:       60 00 80 00                             br.few b6;;
```

Пройдём по инструкциям. Сначала к значению в `r1` прибавляется 80, результат сохраняется в `r15`. Мы знаем, что `r1` указывает на GOT, то есть в `r15` помещается адрес на 80 байтов дальше начала GOT. Затем в `r16` загружается значение по этому адресу, а `r15` увеличивается на восемь байтов. Далее `r1`, указывающий на GOT, сохраняется в `r14`, а в `r1` загружается следующее восьмибайтовое значение по адресу `r15`. После этого выполняется переход по адресу из `r16`.

В предыдущей главе мы обсуждали вызов функций через дескриптор, содержащий адрес функции и адрес глобального указателя. Здесь запись PLT сначала загружает адрес функции, переходит на восемь байтов ко второй части дескриптора и загружает её в регистр глобального указателя перед вызовом.

Но что именно мы загружаем? Мы знаем, что `r1` указывает на GOT, и переходим на 80 байтов, то есть на `0x50`, от её начала.

Пример 9.12. GOT программы Hello World

```
$ objdump --disassemble-all ./hello
    Disassembly of section .got:

6000000000000ec0 <.got>:
...
    6000000000000ee8:       80 0a 00 00 00 00                   data8 0x02a000000
    6000000000000eee:       00 40 90 0a                         dep r0=r0,r0,63,1
    6000000000000ef2:       00 00 00 00 00 40       [MIB] (p20) break.m 0x1
6000000000000ef8:       a0 0a 00 00 00 00                   data8 0x02a810000
    6000000000000efe:       00 40 50 0f                         br.few 6000000000000ef0
```

```
<_GLOBAL_OFFSET_TABLE_+0x30>
    6000000000000f02:       00 00 00 00 00 60       [MIB] (p58) break.m 0x1
    6000000000000f08:       60 0a 00 00 00 00                   data8 0x029818000
    6000000000000f0e:       00 40 90 06                         br.few 6000000000000f00
<_GLOBAL_OFFSET_TABLE_+0x40>
Disassembly of section .IA_64.pltoff:

    6000000000000f10 <.IA_64.pltoff>:
    6000000000000f10:       f0 04 00 00 00 00       [MIB] (p39) break.m 0x0
    6000000000000f16:       00 40 c0 0e 00 00                   data8 0x03b010000
6000000000000f1c:       00 00 00 60                         data8 0xc000000000
    6000000000000f20:       00 05 00 00 00 00       [MII] (p40) break.m 0x0
    6000000000000f26:       00 40 c0 0e 00 00                   data8 0x03b010000
    6000000000000f2c:       00 00 00 60                         data8 0xc000000000
    6000000000000f30:       10 05 00 00 00 00       [MIB] (p40) break.m 0x0
6000000000000f36:       00 40 c0 0e 00 00                   data8 0x03b010000
    6000000000000f3c:       00 00 00 60                         data8 0xc000000000
```

`0x6000000000000ec0 + 0x50 = 0x6000000000000f10` — адрес секции `.IA_64.pltoff`. Теперь мы начинаем понимать происходящее!

Можно расшифровать вывод `objdump`, чтобы точно увидеть загружаемое значение. Изменив порядок первых восьми байтов `f0 04 00 00 00 00 00 40`, получим `0x40000000000004f0`. Этот адрес выглядит знакомо: он уже встречался в ассемблерном выводе PLT.

Код по адресу `0x40000000000004f0` сначала помещает ноль в `r15`, а затем переходит обратно к `0x40000000000004c0`. Постойте — это начало секции PLT!

Можно проследить и этот код. Сначала сохраняется значение глобального указателя в `r2`, затем три восьмибайтовых значения загружаются в `r16`, `r17` и, наконец, `r1`. После этого выполняется переход по адресу из `r17`. Перед нами фактический вызов динамического компоновщика!

Чтобы понять загружаемые значения, нужно обратиться к ABI. Согласно ему динамически скомпонованная программа должна иметь специальную область `DT_IA_64_PLT_RESERVE` для трёх восьмибайтовых значений. Указатель на неё находится в динамическом сегменте файла.

Пример 9.13. Динамический сегмент

```
Dynamic segment at offset 0xcb8 contains 25 entries:
      Tag        Type                         Name/Value
0x0000000000000001 (NEEDED)             Shared library: [libc.so.6.1]
     0x000000000000000c (INIT)               0x4000000000000470
     0x000000000000000d (FINI)               0x4000000000000a20
     0x0000000000000019 (INIT_ARRAY)         0x6000000000000c90
     0x000000000000001b (INIT_ARRAYSZ)       24 (bytes)
0x000000000000001a (FINI_ARRAY)         0x6000000000000ca8
     0x000000000000001c (FINI_ARRAYSZ)       8 (bytes)
     0x0000000000000004 (HASH)               0x4000000000000200
     0x0000000000000005 (STRTAB)             0x4000000000000330
     0x0000000000000006 (SYMTAB)             0x4000000000000240
0x000000000000000a (STRSZ)              138 (bytes)
     0x000000000000000b (SYMENT)             24 (bytes)
     0x0000000000000015 (DEBUG)              0x0
     0x0000000070000000 (IA_64_PLT_RESERVE)  0x6000000000000ec0 -- 0x6000000000000ed8
     0x0000000000000003 (PLTGOT)             0x6000000000000ec0
0x0000000000000002 (PLTRELSZ)           72 (bytes)
     0x0000000000000014 (PLTREL)             RELA
```

```
     0x0000000000000017 (JMPREL)             0x4000000000000420
     0x0000000000000007 (RELA)               0x40000000000003f0
     0x0000000000000008 (RELASZ)             48 (bytes)
0x0000000000000009 (RELAENT)            24 (bytes)
     0x000000006ffffffe (VERNEED)            0x40000000000003d0
     0x000000006fffffff (VERNEEDNUM)         1
     0x000000006ffffff0 (VERSYM)             0x40000000000003ba
     0x0000000000000000 (NULL)               0x0
```

Замечаете особенность? Это тот же адрес, что и у GOT. Значит, первые три восьмибайтовые записи GOT и составляют зарезервированную область; следовательно, глобальный указатель всегда указывает на неё.

При запуске динамический компоновщик должен заполнить эти значения. ABI задаёт уникальный идентификатор модуля и значения, необходимые для перехода к функции, которая находит и исправляет символ, включая глобальный указатель компоновщика.

Пример 9.14. Код динамического компоновщика для установки специальных значений, из libc `sysdeps/ia64/dl-machine.h`

```
/* Set up the loaded object described by L so its unrelocated PLT
   entries will jump to the on-demand fixup code in dl-runtime.c.  */

static inline int __attribute__ ((unused, always_inline))
elf_machine_runtime_setup (struct link_map *l, int lazy, int profile)
{
  extern void _dl_runtime_resolve (void);
  extern void _dl_runtime_profile (void);

  if (lazy)
    {
      register Elf64_Addr gp __asm__ ("gp");
      Elf64_Addr *reserve, doit;

      /*
       * Careful with the typecast here or it will try to add l-l_addr
       * pointer elements
       */
      reserve = ((Elf64_Addr *)
                 (l->l_info[DT_IA_64 (PLT_RESERVE)]->d_un.d_ptr + l->l_addr));
      /* Identify this shared object.  */
      reserve[0] = (Elf64_Addr) l;

      /* This function will be called to perform the relocation.  */
      if (!profile)
        doit = (Elf64_Addr) ((struct fdesc *) &_dl_runtime_resolve)->ip;
      else
        {
          if (GLRO(dl_profile) != NULL
              && _dl_name_match_p (GLRO(dl_profile), l))
            {
              /* This is the object we are looking for.  Say that we really
                 want profiling and the timers are started.  */
              GL(dl_profile_map) = l;
            }
          doit = (Elf64_Addr) ((struct fdesc *) &_dl_runtime_profile)->ip;
        }

      reserve[1] = doit;
      reserve[2] = gp;
    }
```

```
  return lazy;
}
```

По этой функции видно, как компоновщик выполняет настройку. Переменная `reserve` получает адрес из указателя `PLT_RESERVE` двоичного файла. Уникальное значение в `reserve[0]` — адрес карты связей (*link map*) объекта; в glibc так внутренне представляются разделяемые объекты. Во второе значение записывается адрес `_dl_runtime_resolve`, если профилирование не используется. Наконец, `reserve[2]` получает `gp`, извлечённый из регистра с помощью `__asm__`.

ABI предписывает поместить индекс релокации записи в `r15`, а уникальный идентификатор передать в `r16`.

Значение `r15` уже было установлено кодом заглушки до перехода к началу PLT. Посмотрите на записи: каждая загружает в `r15` следующее значение. Если взглянуть на релокации, неудивительно, что релокация `printf` имеет номер ноль.

Значение `r16` загружается из данных, заранее установленных динамическим компоновщиком. После этого можно загрузить адрес функции и глобальный указатель и перейти к функции.

В этот момент выполняется функция динамического компоновщика `_dl_runtime_resolve`. Она находит релокацию; напомним, что та содержит имя символа. По имени находится нужная функция: для этого может понадобиться загрузить библиотеку с диска или использовать уже находящийся в памяти разделяемый код.

Запись релокации сообщает компоновщику адрес, который нужно исправить. Он находится в GOT и загружается первоначальной заглушкой PLT. Поэтому после первого вызова следующий вызов сразу получит прямой адрес функции, минуя динамический компоновщик.

Итоги

Теперь вы увидели точный механизм PLT и внутреннюю работу динамического компоновщика. Важно запомнить следующее:

- Библиотечный вызов в программе на самом деле вызывает заглушку в PLT исполняемого файла.
- Заглушка загружает адрес и переходит по нему.
- Сначала этот адрес указывает на функцию динамического компоновщика, способную найти настоящую функцию по сведениям из записи релокации.
- Компоновщик перезаписывает адрес, считываемый заглушкой, так что при следующем вызове переход выполняется непосредственно к нужной функции.

Работа с библиотеками и компоновщиком

Динамический компоновщик предоставляет полезные возможности, но также создаёт дополнительные вопросы, которые необходимо решить для работы системы.

Версии библиотек

Одна из возможных проблем — существование разных версий библиотек. При использовании только статических библиотек проблем меньше: весь библиотечный код включается непосредственно в исполняемый файл. Чтобы использовать новую версию, программу необходимо перекомпоновать и заменить старый исполняемый файл новым.

Для общих библиотек это непрактично. Наиболее распространённый пример — `libc`, используемая почти всеми приложениями. Если бы она существовала только как статическая библиотека, каждое её изменение требовало бы пересборки всех программ системы.

Однако изменения динамической библиотеки тоже могут создавать проблемы. В лучшем случае они полностью совместимы и не меняют внешнего поведения. В худшем приложение аварийно завершится, например если функция, раньше принимавшая `int`, теперь ожидает `int *`. Ещё хуже, если новая версия изменит смысл операции и начнёт незаметно возвращать другие, возможно неверные значения. Такую ошибку трудно найти: при аварии можно определить место сбоя отладчиком, а повреждение данных может проявиться в совершенно другой части программы.

Динамическому компоновщику нужен способ определять версии системных библиотек и находить подходящие редакции. Современные системы применяют для этого несколько схем.

Имена soname

Имя `soname` добавляет библиотеке информацию, помогающую определить её версию.

Как мы видели, приложение перечисляет необходимые библиотеки в полях `DT_NEEDED` динамической секции исполняемого файла. Сами библиотеки находятся на диске, обычно в `/lib` для основных системных библиотек и `/usr/lib` для дополнительных.

Чтобы хранить на диске несколько версий одной библиотеки, нужны разные имена файлов. Схема `soname` создаёт иерархию имён с помощью файлов и символических ссылок.

Для этого вводятся понятия основной (*major*) и дополнительной (*minor*) версий. Дополнительная версия полностью обратно совместима с предыдущей и обычно содержит исправления ошибок. Основная версия меняется при несовместимом изменении, например изменении аргументов функции или её поведения.

Каждая редакция хранится в отдельном файле. По соглашению имя имеет вид `libNAME.so.MAJOR.MINOR`. После дополнительного номера иногда добавляют номер выпуска. Но если каждое приложение будет напрямую ссылаться на такой файл, возникнет прежняя проблема: при любом изменении дополнительной версии придётся пересобирать приложение.

На самом деле приложению достаточно ссылаться на основную версию. Если она меняется, разумно пересобрать программу и проверить её совместимость с новой библиотекой.

Поэтому `soname` имеет вид `libNAME.so.MAJOR` и записывается в поле `DT_SONAME` динамической секции общей библиотеки. Автор задаёт его при сборке. Разные совместимые файлы с одним номером основной версии могут содержать один и тот же `DT_SONAME`; так динамический компоновщик узнаёт, какую версию API и ABI реализует библиотека.

Обычно запускается программа `ldconfig`, создающая символическую ссылку с именем основной версии на последнюю подходящую дополнительную версию. Она просматривает библиотеки соответствующей основной версии, выбирает последнюю и создаёт ссылку `libNAME.so.MAJOR` на фактический файл `libNAME.so.MAJOR.MINOR`.

> В исходнике здесь оставлена пометка о версиях `libtool` без дальнейшего пояснения.

Последнее звено иерархии — имя для компоновки программы. Флаг `-lNAME` заставляет компоновщик искать файл `libNAME.so` в пути поиска библиотек. Номер версии при этом не указывается: обычно нужна текущая библиотека для разработки. Ссылку `libNAME.so` на нужную библиотеку создаёт процедура её установки, как правило менеджер пакетов `dpkg` или `rpm`. Это не всегда автоматически последняя установленная библиотека: например, новая экспериментальная версия может не подходить для повседневной сборки.

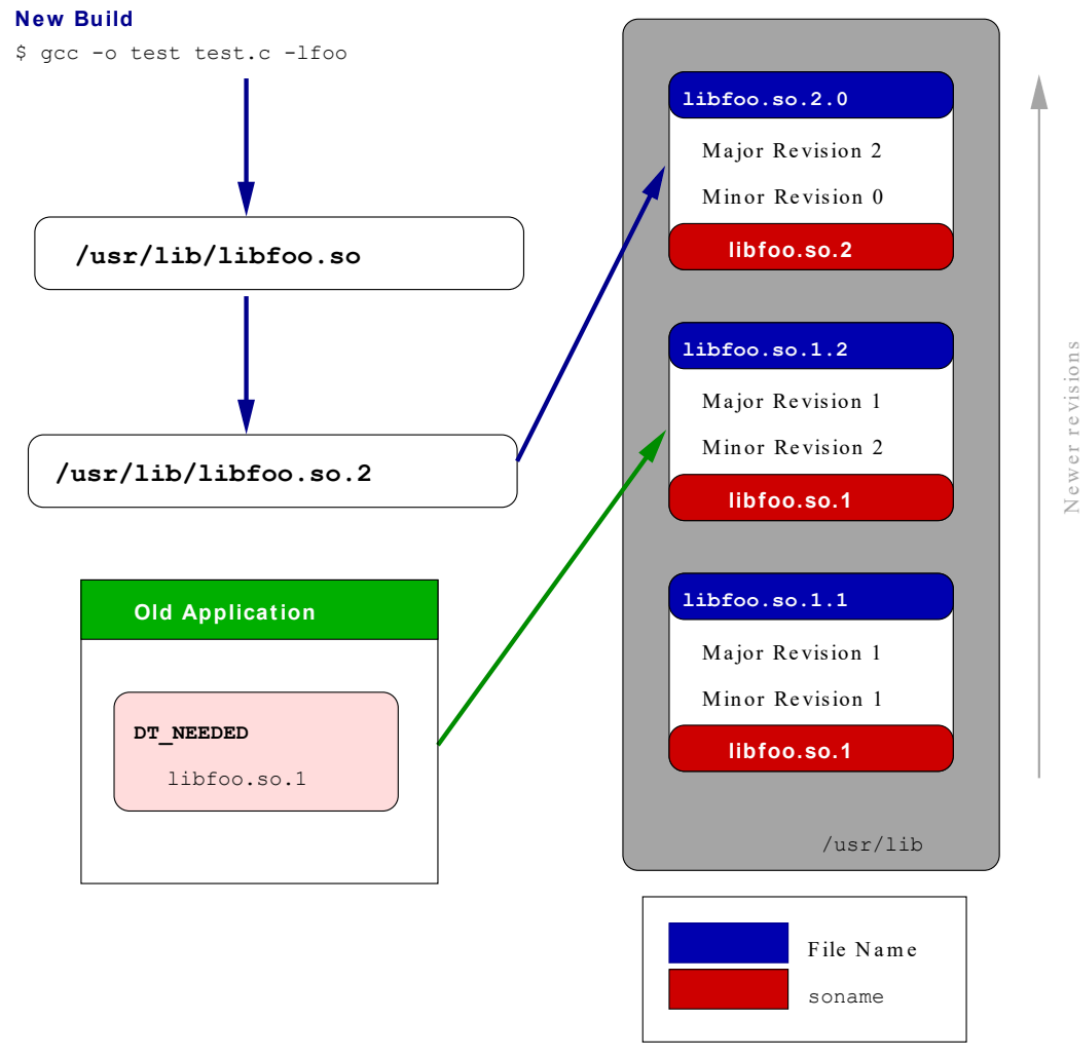


Рисунок 9.2. Имена файлов, soname и символические ссылки разных версий библиотеки.

Как динамический компоновщик ищет библиотеки

При запуске приложения компоновщик изучает `DT_NEEDED`, чтобы узнать необходимые библиотеки. Поле обычно содержит `soname`; затем компоновщик ищет соответствующий файл в своих путях поиска.

В концептуальной схеме сначала нужно найти все библиотеки с нужным `soname`, затем сравнить дополнительные версии и выбрать последнюю. Но созданная `ldconfig` ссылка уже ведёт от имени основной версии к подходящему файлу, поэтому при каждом запуске нет необходимости заново открывать все библиотеки и сравнивать их.

Поскольку доступ к файловой системе медленен, `ldconfig` также создаёт кэш установленных библиотек: сопоставление доступных имён и файлов помогает не перебирать каталоги при каждом запуске. Посмотреть его можно командой `/sbin/ldconfig -p`. Обычно файл кэша находится в `/etc/ld.so.cache`; в исходной книге его путь указан ошибочно. Порядок поиска и использование кэша зависят от динамического загрузчика и настроек системы. После установки библиотек кэш при необходимости обновляют запуском `ldconfig`.

Поиск символов

Мы уже обсуждали, как динамический компоновщик получает адрес библиотечной функции и помещает его в PLT для использования программой. Но как он находит этот адрес? Процесс называется *связыванием* (*binding*): имя символа связывается с обозначаемым им адресом.

Компоновщик знает искомый символ и список библиотек, в которых он может находиться, из записей `DT_NEEDED`.

Каждая общая библиотека содержит секцию `.dynsym` типа `SHT_DYNSYM` — минимальный набор символов, необходимых для динамической компоновки, в частности символов, которые могут вызываться извне.

Таблица динамических символов

В описании динамических символов участвуют три секции. Сначала рассмотрим определение символа из спецификации ELF.

Пример 9.15. Определение символа ELF

```
                            typedef struct {
                          Elf32_Word    st_name;
                          Elf32_Addr    st_value;
Elf32_Word    st_size;
                          unsigned char st_info;
                          unsigned char st_other;
                          Elf32_Half    st_shndx;
                } Elf32_Sym;
```

Таблица 9.2. Поля символа ELF

Поле	Значение
`st_name`	Индекс в таблице строк.
`st_value`	Значение символа; в перемещаемом общем объекте описывается относительно соответствующей секции.
`st_size`	Размер, связанный с символом, если применимо.
`st_info`	Информация о связывании и типе символа: функция, объект и т. д.
`st_other`	Дополнительная информация; в рассматриваемом автором описании не используется.
`st_shndx`	Индекс секции, в которой находится символ.

Сама строка имени хранится отдельно, в `.dynstr`; запись `.dynsym` содержит лишь индекс в таблицу строк. Это создаёт дополнительные расходы: компоновщику нужно читать записи символов и по индексу находить имена для сравнения.

Для ускорения используется третья секция `.hash` с хеш-таблицей, связывающей имена с записями. Она вычисляется при сборке библиотеки и позволяет находить символы значительно быстрее, обычно за одно-два обращения. В зависимости от формата и инструментов могут применяться и другие таблицы хеширования.

Связывание символов

Хотя поиском адреса символа обычно называют его связывание, у термина *binding* применительно к самому символу есть отдельное значение.

Тип связывания определяет видимость символа при компоновке. Локальный символ не виден за пределами объектного файла, в котором определён. Глобальный виден другим объектным файлам и может удовлетворять их неопределённым ссылкам.

Слабая ссылка или слабое определение — специальный вариант с пониженным приоритетом, предназначенный для возможного переопределения, как мы увидим далее.

Рассмотрим небольшую программу на C и исследуем типы связывания её символов.

Пример 9.16. Типы связывания символов

```
                                $ cat test.c
                static int static_variable;

extern int extern_variable;

                int external_function(void);

                int function(void)
{
                        return external_function();
                }

                static int static_function(void)
{
                        return 10;
```

```
}

#pragma weak weak_function
int weak_function(void)
{
        return 10;
}

$ gcc -c test.c
$ objdump --syms test.o

test.o:     file format elf32-powerpc

SYMBOL TABLE:
00000000 l    df *ABS*  00000000 test.c
00000000 l    d  .text  00000000 .text
00000000 l    d  .data  00000000 .data
00000000 l    d  .bss   00000000 .bss
00000038 l     F .text  00000024 static_function
00000000 l    d  .sbss  00000000 .sbss
00000000 l     O .sbss  00000004 static_variable
00000000 l    d  .note.GNU-stack        00000000 .note.GNU-stack
00000000 l    d  .comment       00000000 .comment
00000000 g     F .text  00000038 function
00000000         *UND*  00000000 external_function
0000005c  w    F .text  00000024 weak_function

$ nm test.o
U external_function
00000000 T function
00000038 t static_function
00000000 s static_variable
0000005c W weak_function
```

Обратите внимание на `#pragma`, объявляющую слабый символ. Директива pragma передаёт компилятору дополнительную информацию; она применяется нечасто, но иногда нужна для нестандартных действий.

Символы показаны двумя инструментами. В выводе тип обозначается буквенными кодами; их описание можно найти в справочных страницах соответствующих инструментов.

Переопределение символов

Иногда программисту полезно переопределить библиотечный символ, подставив вместо обычного определения другое.

Переменная окружения `LD_PRELOAD` задаёт общие библиотеки, которые динамический загрузчик подгружает заранее. Их символы могут перехватывать обращения до определения из обычных зависимостей. В исходной книге порядок загрузки ошибочно описан как обратный.

Пример 9.17. Использование `LD_PRELOAD`

```
                $ cat override.c
#define _GNU_SOURCE 1
#include <stdio.h>
```

```
#include <stdlib.h>
                #include <unistd.h>
                #include <sys/types.h>
                #include <dlfcn.h>

pid_t getpid(void)
                {
                        pid_t (*orig_getpid)(void) = dlsym(RTLD_NEXT, "getpid");
                        printf("Calling GETPID\n");

return orig_getpid();
                }

                $ cat test.c
                #include <stdio.h>
#include <stdlib.h>
                #include <unistd.h>

                int main(void)
                {
printf("%d\n", getpid());
                }

                $ gcc -shared -fPIC -o liboverride.so override.c -ldl
                $ gcc -o test test.c
$ LD_PRELOAD=./liboverride.so ./test
                Calling GETPID
```

Здесь мы переопределяем `getpid`, чтобы при вызове печаталось сообщение. Функция `dlsym` с аргументом `RTLD_NEXT` ищет следующее определение `getpid`, позволяя обёртке вызвать исходную функцию.

История слабых символов

Слабый символ имеет более низкий приоритет и может быть переопределён другим. Если подходящего более сильного определения нет, используется слабое.

Можно предположить, что динамический загрузчик должен просмотреть все библиотеки и предпочесть сильное определение любому ранее найденному слабому. Именно так первоначально работала обработка слабых символов в Linux с glibc.

Однако такое поведение не соответствовало букве действовавшего тогда стандарта Unix SysVr4: различие слабых и сильных символов должно было обрабатываться статическим компоновщиком, а не менять обычный порядок поиска динамического загрузчика.

Реализация Linux тогда совпадала с SGI IRIX, но отличалась от Solaris и AIX. Обнаружив несоответствие стандарту, разработчики изменили поведение; для прежней схемы понадобилась специальная переменная окружения `LD_DYNAMIC_WEAK`.

Порядок связывания

Как мы видели, предварительно загруженная библиотека может переопределить функцию другой библиотеки. Порядок обхода областей поиска влияет на то, какое определение будет найдено. Обычный загрузчик ELF, как правило, ищет символы в порядке области поиска и использует первое подходящее определение; `LD_PRELOAD` помещает свои библиотеки впереди

обычных зависимостей. Оригинал описывает этот порядок в обратном направлении, поэтому для конкретной системы следует сверяться с документацией загрузчика.

На зависимости `DT_NEEDED` влияет порядок библиотек при сборке, но точный порядок поиска также определяется правилами конкретного загрузчика и областью видимости объекта.

Некоторым общим библиотекам нужно изменить обычный порядок и сначала искать свои собственные определения. Для этого библиотека может содержать флаг `DT_SYMBOLIC` в динамической секции; его обычно задают при сборке параметром статического компоновщика `-Bsymbolic`.

Это влияет на связывание обращений внутри библиотеки: её собственные определения получают преимущество перед внешним перехватом. Однако такая настройка обычно применяется широко, а не к каждому имени отдельно. Более гибкий механизм позволил бы сделать часть символов внутренними, а часть — публичными.

Версионирование символов

Такой механизм предоставляет управление видимостью и версиями символов. Дополнительные указания статическому компоновщику описывают, какие символы общей библиотеки экспортировать и как их версионировать.

Пример 9.18. Управление символами и версиями

```
                                $ cat Makefile
                all: test testsym

clean:
                        rm -f *.so test testsym

                liboverride.so : override.c
                        $(CC) -shared -fPIC -o liboverride.so override.c
                libtest.so : libtest.c
                        $(CC) -shared -fPIC -o libtest.so libtest.c

                libtestsym.so : libtest.c
$(CC) -shared -fPIC -Wl,-Bsymbolic -o libtestsym.so libtest.c

                test : test.c libtest.so liboverride.so
                        $(CC) -L. -ltest -o test test.c

testsym : test.c libtestsym.so liboverride.so
                        $(CC) -L. -ltestsym -o testsym test.c
```

```
    $ cat libtest.c
    #include <stdio.h>
    int foo(void) {
            printf("libtest foo called\n");
            return 1;
    }
    int test_foo(void)
    {
            return foo();
    }
```

```
    $ cat override.c
    #include <stdio.h>

    int foo(void)
{
            printf("override foo called\n");
            return 0;
    }

$ cat test.c
    #include <stdio.h>

    int main(void)
    {
printf("%d\n", test_foo());
    }

    $ cat Versions
    {global: test_foo;  local: *; };
    $ gcc -shared -fPIC -Wl,-version-script=Versions -o libtestver.so libtest.c

    $ gcc -L. -ltestver -o testver test.c

$ LD_LIBRARY_PATH=. LD_PRELOAD=./liboverride.so ./testver
    libtest foo called

    100000574 l     F .text 00000054              foo
    000005c8 g     F .text 00000038              test_foo
```

В простейшем случае, показанном выше, мы указываем, какие символы глобальные, а какие локальные. Например, `foo` может быть вспомогательной функцией для `test_foo`: мы допускаем переопределение общего интерфейса `test_foo`, но при использовании библиотечной реализации хотим сохранить её внутреннюю вспомогательную функцию неизменной.

Это помогает упорядочить пространство имён. Многие библиотеки могли бы реализовать функции с распространёнными именами, например `read` или `write`. Если все экспортировать, программа рискует получить неподходящее определение. Объявляя внутренние символы локальными, разработчик предотвращает конфликты как внутри библиотеки, так и с другими программами.

Дальнейшее развитие этого подхода — версионирование символов. Одна библиотека может содержать несколько версий одного имени. Статический компоновщик добавляет к имени сведения о версии, например `@VER`.

Если разработчик меняет реализацию функции так, что она становится бинарно или программно несовместимой, он может увеличить версию. Новые приложения при сборке получают последнюю версию символа; ранее собранные продолжают запрашивать старую и получают прежнюю реализацию. В исходном тексте автор оставил пометку о будущем примере.

Глава 10. Основы ввода-вывода

Основы файловых систем

В исходном издании этот раздел не содержит текста.

Основы сетей

В исходном издании этот раздел не содержит текста.

Глоссарий

Интерфейс прикладного программирования (API). Набор переменных и функций, используемых для взаимодействия разных частей программ. См. также интерфейс двоичного взаимодействия приложений.

Интерфейс двоичного взаимодействия приложений (ABI). В исходном глоссарии определён как техническое описание взаимодействия операционной системы с оборудованием. Более широко ABI устанавливает соглашения двоичного взаимодействия, например соглашения о вызовах и форматах данных. См. также API.

Расширяемый язык разметки (XML). Язык разметки. См. также SGML. В исходном издании здесь оставлена незавершённая заметка об определении.

Стандартный обобщённый язык разметки (SGML). Предшественник других языков разметки документов. См. также XML.

Взаимоисключающие состояния. Если варианты взаимно исключают друг друга, истинным или действительным может быть только один из них.

Блок управления памятью (MMU). Компонент аппаратной архитектуры, управляющий памятью.

Открытое программное обеспечение. Программы, распространяемые в исходном виде по лицензиям, которые предоставляют всем право свободно использовать, изменять и распространять код.

Оболочка (shell). Интерфейс взаимодействия с операционной системой.